

PROIECTAREA ALGORITMILOR

Conf. univ. dr. COSTEL BĂLCĂU

2025

Tematica

1	Elemente de complexitatea algoritmilor	8
1.1	Notății asimptotice. Ordine de complexitate	8
1.2	Determinarea maximului și minimului dintr-un vector	11
1.3	Determinarea valorii unui polinom dat într-un punct dat	13
1.4	Problema votului majoritar	16
1.5	Determinarea ultimei cifre nenule a factorialului unui număr natural	22
2	Metoda Greedy	34
2.1	Descrierea metodei. Algoritmi generali	34
2.2	Aplicații ale Inegalității rearanjamentelor	37
2.2.1	Inegalitatea rearanjamentelor	37
2.2.2	Produs scalar maxim/minim	39
2.2.3	Memorarea optimă a textelor pe benzi	43
2.3	Problema rucsacului, varianta continuă	45
2.4	Problema planificării spectacolelor	52
2.5	Arbori parțiali de cost minim	59
2.6	Distanțe și drumuri minime. Algoritmul lui Dijkstra	64
2.7	Fluxuri în rețele	72
3	Recursivitate	88
3.1	Introducere	88
3.2	Recurența liniară de ordinul I cu termen liber constant	90
3.3	Recurența liniară omogenă de ordinul al II-lea	92
3.4	Recurența liniară de ordinul al II-lea cu termen liber constant	94
3.5	Recurența liniară de ordinul I cu termen liber liniar	98
3.6	Recurența liniară de ordinul I cu termen liber pătratic	101
3.7	Recurența liniară de ordinul al II-lea cu termen liber liniar	102

4	Metoda Backtracking	105
4.1	Descrierea metodei. Algoritmi generali	105
4.2	Problema celor n dame pe tabla de șah	113
4.3	Problema nebunilor pe tabla de șah	117
4.4	Generarea obiectelor combinatoriale	126
4.4.1	Preliminarii	126
4.4.2	Produs cartezian	127
4.4.3	Submulțimi	128
4.4.4	Aranjamente cu repetiție	130
4.4.5	Aranjamente	131
4.4.6	Permutări	134
4.4.7	Combinări	136
4.4.8	Combinări cu repetiție	139
4.4.9	Permutări cu repetiție	141
4.4.10	Compuneri ale unui număr natural	146
4.4.11	Partiții ale unui număr natural	152
5	Metoda Divide et Impera	157
5.1	Descrierea și analiza metodei. Algoritmi generali	157
5.2	O problemă de acoperire	162
5.3	Arbori binari stricți. Proprietăți	165
5.4	Algoritmi de căutare	173
5.4.1	Problema căutării	173
5.4.2	Căutarea secvențială	174
5.4.3	Căutarea binară	175
5.5	Algoritmi de sortare	187
5.5.1	Problema sortării	187
5.5.2	Complexitatea algoritmilor de sortare	187
5.5.3	Sortarea prin inserție directă	193
5.5.4	Sortarea prin inserție binară	195
5.5.5	Sortarea prin interclasare (mergesort)	199
5.5.6	Sortarea rapidă (quicksort)	211
6	Metoda programării dinamice	223
6.1	Descrierea metodei. Algoritmi generali	223
6.2	Șirul numerelor lui Fibonacci	225
6.3	Problema subșirului crescător de lungime maximă	227
6.4	Problema subșirului comun de lungime maximă	234
6.5	Problema drumului de sumă maximă/minimă într-o matrice	242
6.6	Matricea drumurilor. Algoritmul Roy-Warshall	259
6.7	Distanțe și drumuri minime. Algoritmul Roy-Floyd	265

<i>TEMATICA</i>	3
7 Metoda Branch and Bound	269
8 Algoritmi euristici	270
8.1 Introducere	270
8.2 Problema rucsacului, varianta discretă	270
8.3 Arbori Steiner. Algoritmul Kou-Markowsky-Berman	272

Evaluare

- Activitate laborator: 30% (Programe și probleme din Temele de laborator)
- Teme de casă: 20% (Programe și probleme suplimentare)
- Examen final: 50% (Probă scrisă: algoritmi -cu implementare- și probleme)

Bibliografie

- [1] A.V. Aho, J.E. Hopcroft, J.D. Ullman, *Data Structures and Algorithms*, Addison-Wesley, Massachusetts, 2009.
- [2] Gh. Barbu, V. Păun, *Programarea în limbajul C/C++*, Editura Matrix Rom, București, 2011.
- [3] Gh. Barbu, V. Păun, *Calculatoare personale și programare în C/C++*, Editura Didactică și Pedagogică, București, 2005.
- [4] Gh. Barbu, I. Văduva, M. Boloșteanu, *Bazele informaticii*, Editura Tehnică, București, 1997.
- [5] C. Bălcău, *Combinatorică și teoria grafurilor*, Editura Universității din Pitești, Pitești, 2007.
- [6] O. Bâscă, L. Livovschi, *Algoritmi euristici*, Editura Universității din București, București, 2003.
- [7] E. Cerchez, M. Șerban, *Programarea în limbajul C/C++ pentru liceu. Vol. 2: Metode și tehnici de programare*, Editura Polirom, Iași, 2005.
- [8] E. Ciurea, L. Ciupală, *Algoritmi. Introducere în algoritmica fluxurilor în rețele*, Editura Matrix Rom, București, 2006.
- [9] T.H. Cormen, *Algorithms Unlocked*, MIT Press, Cambridge, 2013.
- [10] T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein, *Introduction to Algorithms*, MIT Press, Cambridge, 2009.
- [11] C. Croitoru, *Tehnici de bază în optimizarea combinatorie*, Editura Universității "Al. I. Cuza", Iași, 1992.
- [12] N. Dale, C. Weems, *Programming and problem solving with JAVA*, Jones & Bartlett Publishers, Sudbury, 2008.
- [13] P. Deitel, H. Deitel, *Java SE 8 for programmers*, Deitel & Associates, Boston, 2014.
- [14] D. Du, X. Hu, *Steiner Tree Problems in Computer Communication Networks*, World Scientific Publishing Co. Pte. Ltd., Hackensack, 2008.
- [15] S. Even, *Graph Algorithms*, Cambridge University Press, Cambridge, 2012.

- [16] F. Gebali, *Algorithms and parallel computing*, John Wiley & Sons, New Jersey, 2011.
- [17] H. Georgescu, *Tehnici de programare*, Editura Universității din București, București, 2005.
- [18] C.A. Giumale, *Introducere în analiza algoritmilor. Teorie și aplicații*, Editura Polirom, Iași, 2004.
- [19] M. Goodrich, R. Tamassia, *Algorithm Design. Foundations, Analysis and Internet Examples*, Wiley, New Delhi, 2011.
- [20] R. Harper, *Practical foundations for programming languages*, Cambridge University Press, Cambridge, 2013.
- [21] F.V. Jensen, T.D. Nielsen, *Bayesian Networks and Decision Graphs*, Springer, New York, 2007.
- [22] D. Jungnickel, *Graphs, Networks and Algorithms*, Springer, Heidelberg, 2013.
- [23] D.E. Knuth, *The Art Of Computer Programming. Vol. 4A: Combinatorial Algorithms*, Addison-Wesley, Massachusetts, 2011.
- [24] B. Korte, J. Vygen, *Combinatorial Optimization. Theory and Algorithms*, Springer, Heidelberg, 2012.
- [25] R. Lafore, *Data Structures and Algorithms in Java*, Sams Publishing, Indianapolis, 2002.
- [26] A. Levitin, *Introduction to The Design and Analysis of Algorithms*, Pearson, Boston, 2012.
- [27] L. Livovschi, H. Georgescu, *Sinteza și analiza algoritmilor*, Editura Științifică și Enciclopedică, București, 1986.
- [28] D. Logofătu, *Algoritmi fundamentali în C++: Aplicații*, Editura Polirom, Iași, 2007.
- [29] D. Logofătu, *Algoritmi fundamentali în Java: Aplicații*, Editura Polirom, Iași, 2007.
- [30] D. Lucanu, M. Craus, *Proiectarea algoritmilor*, Editura Polirom, Iași, 2008.
- [31] S. Miller, *Mathematics of Optimization: How to do Things Faster*, AMS, Providence, 2017.
- [32] D.A. Popescu, *Bazele programării - JAVA după C++*, ebooks.infobits.ro, 2019.
- [33] D.R. Popescu, *Combinatorică și teoria grafurilor*, Societatea de Științe Matematice din România, București, 2005.
- [34] N. Popescu, *Data structures and algorithms using Java*, Editura Politehnica Press, București, 2008.
- [35] C.P. Popovici, H. Georgescu, L. State, *Bazele informaticii. Vol. I, II*, Editura Universității din București, București, 1990, 1991.

- [36] V. Preda, C. Bălcău, *Entropy optimization with applications*, Editura Academiei Române, București, 2010.
- [37] O.A. Schipor, S.G. Pentiuc, F. Gîză-Belciug, *Limbaajul C - Tehnici de programare eficientă*, Editura Matrix Rom, București, 2014.
- [38] R. Sedgewick, P. Flajolet, *An Introduction to the Analysis of Algorithms*, Addison-Wesley, New Jersey, 2013.
- [39] R. Sedgewick, K. Wayne, *Algorithms*, Addison-Wesley, Massachusetts, 2011.
- [40] R. Stephens, *Essential Algorithms: A Practical Approach to Computer Algorithms*, Wiley, Indianapolis, 2013.
- [41] Ș. Tănăsă, C. Olaru, Ș. Andrei, *Java de la 0 la expert*, Editura Polirom, Iași, 2007.
- [42] T. Toadere, *Grafe. Teorie, algoritmi și aplicații*, Editura Albastră, Cluj-Napoca, 2002.
- [43] I. Tomescu, *Combinatorică și teoria grafurilor*, Tipografia Universității din București, București, 1978.
- [44] I. Tomescu, *Probleme de combinatorică și teoria grafurilor*, Editura Didactică și Pedagogică, București, 1981.
- [45] I. Tomescu, *Data structures*, Editura Universității din București, București, 2004.
- [46] M.A. Weiss, *Data Structures and Algorithm Analysis in Java*, Addison-Wesley, New Jersey, 2012.
- [47] ***, *Handbook of combinatorics*, edited by R.L. Graham, M. Grötschel and L. Lovász, Elsevier, Amsterdam, 1995.
- [48] ***, *Handbook of discrete and combinatorial mathematics*, edited by K.H. Rosen, J.G. Michaels, J.L. Gross, J.W. Grossman and D.R. Shier, CRC Press, Boca Raton, 2000.
- [49] ***, *Revista MATINF. Publicație bianuală de matematică și informatică pentru elevi și profesori*, editată de Departamentul Matematică-Informatică, Universitatea din Pitești, Editura Universității din Pitești.

Tema 1

Elemente de complexitatea algoritmilor

1.1 Notății asimptotice. Ordine de complexitate

Timpul de execuție al unui algoritm depinde, în general, de setul datelor de intrare, iar pentru fiecare astfel de set el este bine determinat de numărul de operații executate și de tipul acestora. Astfel timpul de execuție al unui algoritm poate fi interpretat și analizat drept o funcție pozitivă ce are ca argument dimensiunea datelor de intrare.

Definiția 1.1.1. Pentru orice algoritm $\mathcal{A}$, notăm cu $T_{\mathcal{A}}(n)$ **timpul de execuție** pentru algoritmul $\mathcal{A}$ corespunzător unui set de date de intrare având dimensiunea totală n .

Observația 1.1.1. Pentru simplificarea calculelor, de cele mai multe ori sunt analizate numai anumite operații, semnificative pentru algoritmi respectivi, evaluarea timpului de execuție rezumându-se astfel la numărarea sau estimarea acestor operații.

Definiția 1.1.2. Un algoritm $\mathcal{A}$ este considerat **optim** dacă (se demonstrează că) nu există un algoritm având un timp de execuție mai bun pentru rezolvarea problemei date, adică pentru orice algoritm $\mathcal{A}'$ care rezolvă problema dată avem $T_{\mathcal{A}}(n) \leq T_{\mathcal{A}'}(n)$, pentru orice n .

Obținerea de algoritmi pur optimi - în sensul definiției anterioare - este posibilă în puține situații, iar demonstrarea optimalității acestora este de obicei dificilă. Mult mai des se întâlnesc algoritmi cu o comportare apropiată de cea optimă, pentru valori suficient de mari ale dimensiunii setului datelor

de intrare. Prezentăm în continuare câteva noțiuni prin care se cuantifică această apropiere.

Definiția 1.1.3. O funcție **asimptotic pozitivă** (prescurtat **a.p.**) este o funcție $f : \mathbb{N} \setminus A \rightarrow \mathbb{R}$ a.î.

- $A \subset \mathbb{N}$ este o mulțime finită;
- $\exists n_0 \in \mathbb{N} \setminus A$ astfel încât $f(n) > 0, \forall n \geq n_0$.

Observația 1.1.2. De cele mai multe ori, mulțimea A este de forma

$$A = \underbrace{\{0, 1, 2, \dots, k\}}_{\text{primele numere naturale}}, \text{ unde } k \in \mathbb{N}.$$

Exemplul 1.1.1. Funcția $f : D \rightarrow \mathbb{R}, f(n) = \frac{(3n^4 + n + 3)\sqrt{n-5}}{(5n+1)(n-8)}$, unde $D = \{n \in \mathbb{N} \mid n \geq 5, n \neq 8\}$, este asimptotic pozitivă, deoarece $D = \mathbb{N} \setminus A$ cu $A = \{0, 1, 2, 3, 4, 8\}$ (mulțime finită) și $f(n) > 0, \forall n \geq 9$.

Exemplul 1.1.2. Funcția $g : \mathbb{N} \setminus \{1, 6\} \rightarrow \mathbb{R}, g(n) = \frac{\ln(n^5 + 1) - n}{(n-1)(n-6)}$, nu este asimptotic pozitivă, deoarece $(n-1)(n-6) > 0, \forall n \geq 7$, dar $\lim_{n \rightarrow \infty} [\ln(n^5 + 1) - n] = -\infty$, deci $\exists n_0 \in \mathbb{N}, n_0 \geq 7$ a.î. $g(n) < 0, \forall n \geq n_0$.

Următorul rezultat este o consecință a definiției anterioare.

Lema 1.1.1. O funcție polinomială $f : \mathbb{N} \rightarrow \mathbb{R}$, de grad p ,

$$f(n) = a_p \cdot n^p + a_{p-1} \cdot n^{p-1} + \dots + a_1 \cdot n + a_0, \quad a_0, a_1, \dots, a_p \in \mathbb{R}, \quad a_p \neq 0,$$

este asimptotic pozitivă dacă și numai dacă $a_p > 0$.

Definiția 1.1.4. Fie f și g două funcții asimptotic pozitive.

a) f și g se numesc **asimptotic echivalente** și notăm $f(n) \sim g(n)$ dacă $\exists \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 1$.

b) Spunem că f este **asimptotic mărginită superior** de g , iar g este **asimptotic mărginită inferior** de f și notăm $f(n) = \mathcal{O}(g(n))$ și $g(n) = \Omega(f(n))$ dacă $\exists c > 0, \exists n_0 \in \mathbb{N}$ astfel încât $f(n) \leq c \cdot g(n), \forall n \geq n_0$.

c) Spunem că f și g **au același ordin de creștere** și notăm $f(n) = \Theta(g(n))$ dacă $f(n) = \mathcal{O}(g(n))$ și $f(n) = \Omega(g(n))$.

Următorul rezultat este o consecință a definiției anterioare.

Propoziția 1.1.1. Fie f și g două funcții asimptotic pozitive. Presupunem că există $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = L$. Atunci:

- a) $f(n) = \mathcal{O}(g(n))$ dacă și numai dacă $L \in [0, +\infty)$;
- b) $f(n) = \Omega(g(n))$ dacă și numai dacă $L \in (0, +\infty]$;
- c) $f(n) = \Theta(g(n))$ dacă și numai dacă $L \in (0, +\infty)$.

Corolarul 1.1.1. Fie f și g două funcții asimptotic pozitive. Dacă $f(n) \sim g(n)$, atunci $f(n) = \Theta(g(n))$.

Următorul rezultat este o consecință a propoziției anterioare.

Propoziția 1.1.2. Fie $f : \mathbb{N} \rightarrow \mathbb{R}$ o funcție polinomială de grad p ,

$$f(n) = a_p \cdot n^p + a_{p-1} \cdot n^{p-1} + \cdots + a_1 \cdot n + a_0, \quad a_0, a_1, \dots, a_p \in \mathbb{R},$$

astfel încât $a_p > 0$.

Atunci

- a) $f(n) = \mathcal{O}(n^k)$, $\forall k \geq p$;
- b) $f(n) = \Omega(n^k)$, $\forall k \leq p$;
- c) $f(n) = \Theta(n^p)$;
- d) $f(n) \sim a_p \cdot n^p$.

Definiția 1.1.5. Fie $\mathcal{A}$ un algoritm și f o funcție asimptotic pozitivă. Spunem că algoritmul $\mathcal{A}$ are **ordinul de complexitate (complexitatea)** $\mathcal{O}(f(n))$, $\Omega(f(n))$, respectiv $\Theta(f(n))$ dacă $T_{\mathcal{A}}(n) = \mathcal{O}(f(n))$, $T_{\mathcal{A}}(n) = \Omega(f(n))$, respectiv $T_{\mathcal{A}}(n) = \Theta(f(n))$.

Definiția 1.1.6. Fie $\mathcal{A}$ un algoritm, n dimensiunea datelor de intrare și $T(n)$ timpul de execuție estimat pentru algoritmul $\mathcal{A}$.

- 1) Spunem că algoritmul $\mathcal{A}$ are **complexitate (comportare) polinomială** (sau că este **polinomial** sau că **aparține clasei P**) dacă $\exists p > 0$ astfel încât $T(n) = \mathcal{O}(n^p)$.
- 2) În particular, dacă $T(n) = \mathcal{O}(n)$ atunci spunem că algoritmul $\mathcal{A}$ are **complexitate (comportare) liniară** (sau că este **liniar**).

Observația 1.1.3. Algoritmii polinomiali sunt, în general, acceptabili în practică. Algoritmii care necesită un timp de calcul exponențial sunt utilizați numai în cazuri excepționale și doar pentru date de intrare de dimensiuni relativ mici.

Observația 1.1.4. Notăția $\mathcal{O}$ se utilizează pentru a exprima complexitatea unui algoritm corespunzătoare *timpului de execuție în cazul cel mai defavorabil*, fiind astfel cea mai adecvată analizei algoritmilor. Notăția Ω este corespunzătoare *timpului de execuție în cazul cel mai favorabil*, caz practic irelevant, fiind astfel mai puțin utilizată. Notățiile $\sim$ și Θ se utilizează atunci când se constată că timpii de execuție corespunzători cazurilor cel mai defavorabil și cel mai favorabil fie sunt chiar asimptotic echivalenți (notația $\sim$, deci și notația Θ), cazul cel mai simplu fiind acela al algoritmilor a căror executare depinde doar de dimensiunea setului de date de intrare, nu și de valorile acestor date, fie au măcar același ordin de creștere (notația Θ). Tot aceste notații se utilizează și atunci când se poate determina *timpul mediu de execuție* al algoritmului, calculat ca medie aritmetică ponderată a timpilor de execuție pentru toate seturile de date de intrare posibile, ponderile fiind frecvențele de apariție ale acestor seturi.

Definiția 1.1.7. *Un algoritm $\mathcal{A}$ este considerat **asimptotic-optim** dacă (se demonstrează că) nu există un algoritm având un ordin de complexitate mai bun pentru rezolvarea problemei date, adică pentru orice algoritm $\mathcal{A}'$ care rezolvă problema dată avem $T_{\mathcal{A}}(n) = \mathcal{O}(T_{\mathcal{A}'}(n))$.*

Observația 1.1.5. Evident, orice algoritm optim este asimptotic-optim. Reciproca acestei afirmații nu este adevărată, în continuare fiind prezentat un exemplu în acest sens.

1.2 Determinarea maximului și minimului dintr-un vector

Problema determinării maximului și minimului dintr-un vector este următoarea: Se consideră un vector $A = (a_1, a_2, \dots, a_n)$, $n \geq 1$. Se cere să se determine maximul și minimul dintre elementele $a_1, a_2, \dots, a_n$, adică perechea (M, m) , unde $M = \max\{a_i \mid 1 \leq i \leq n\}$, $m = \min\{a_i \mid 1 \leq i \leq n\}$.

Un algoritm uzual de rezolvare este următorul.

MAX-MIN(A, n, M, m):

$M \leftarrow a_1; m \leftarrow a_1;$

for $i = \overline{2, n}$ **do**

if $a_i > M$ **then**

$M \leftarrow a_i;$

else

if $a_i < m$ **then**

$m \leftarrow a_i;$

Pentru evaluarea complexității algoritmilor care rezolvă problema dată, vom analiza numai comparațiile în care intervin elemente ale vectorului sau valorile M și m , numite *comparații de chei*. Evident, algoritmul MAX-MIN efectuează cel puțin $n - 1$ și cel mult $2n - 2$ astfel de comparații, iar celelalte operații nu depășesc ordinul de creștere al acestora, deci are complexitatea $\Theta(n)$.

Vom utiliza următorul rezultat.

Lema 1.2.1. *Pentru determinarea maximului dintre n numere, $n \in \mathbb{N}^*$, sunt necesare $n - 1$ comparații.*

Demonstrație. Notăm proprietatea din enunț cu $P(n)$ și îi demonstrăm valabilitatea prin inducție după n .

$P(1)$: Pentru $n = 1$ sunt necesare $0 = 1 - 1$ comparații, deci $P(1)$ este adevărată.

$P(k - 1) \Rightarrow P(k)$: Fie $k \in \mathbb{N}$, $k \geq 2$. Presupunem afirmația adevărată pentru orice $k - 1$ numere și o demonstrăm pentru k numere.

Fie $a_1, a_2, \dots, a_{k-1}, a_k$ aceste numere.

Fie a_i și a_j , $i \neq j$, numerele care sunt supuse primei comparații.

Presupunem că $a_i \geq a_j$ (raționamentul este similar în cazul când $a_j \geq a_i$).

Atunci

$$\max\{a_1, a_2, \dots, a_{k-1}, a_k\} = \max\{a_1, a_2, \dots, a_{j-1}, a_{j+1}, \dots, a_{k-1}, a_k\},$$

adică avem de determinat în continuare maximul dintre $k - 1$ numere (celelalte $k - 2$ numere și a_i). Pentru aceasta, conform ipotezei de inducție, sunt necesare încă $k - 2$ comparații. Deci obținem un total de $1 + (k - 2) = k - 1$ comparații.

Demonstrația prin inducție este astfel încheiată. $\square$

Conform lemei anterioare, orice algoritm $\mathcal{A}$ care calculează maximul dintre n elemente, bazat pe comparații de chei, necesită cel puțin $n - 1$ astfel de comparații, deci are complexitatea $\Omega(n)$. Astfel $T_{\mathcal{A}}(n) = \Omega(T_{\text{MAX-MIN}}(n))$, sau, echivalent, $T_{\text{MAX-MIN}}(n) = \mathcal{O}(T_{\mathcal{A}}(n))$, deci am obținut următorul rezultat:

Propoziția 1.2.1. *Algoritmul MAX-MIN este asimptotic-optimal (în clasa algoritmilor bazați pe comparații de chei).*

Pe de altă parte, avem:

Propoziția 1.2.2. *Din punct de vedere al timpului de execuție în cazul cel mai defavorabil, algoritmul MAX-MIN nu este optim.*

Demonstrație. Există algoritmi (bazați pe comparații de chei) care în cazul cel mai defavorabil efectuează mai puțin de $2n - 2$ comparații de chei, cât efectuează algoritmul MAX-MIN. Un astfel de exemplu este următorul algoritm, ce compară a_1 cu a_2 , a_3 cu a_4 , $\dots$, și calculează maximumul dintre maximele acestor perechi și minimumul dintre minimele perechilor.

MAX-MIN-PER(A, n, M, m):

```

 $M \leftarrow a_n; m \leftarrow a_n;$ 
for  $i = 1, \lfloor n/2 \rfloor$  do
    if  $a_{2i-1} > a_{2i}$  then
        if  $a_{2i-1} > M$  then
             $M \leftarrow a_{2i-1};$ 
        if  $a_{2i} < m$  then
             $m \leftarrow a_{2i};$ 
    else
        if  $a_{2i} > M$  then
             $M \leftarrow a_{2i};$ 
        if  $a_{2i-1} < m$  then
             $m \leftarrow a_{2i-1};$ 

```

($\lfloor x \rfloor$ reprezintă partea întreagă a numărului real x). Evident, algoritmul MAX-MIN-PER efectuează exact $3 \cdot \lfloor \frac{n}{2} \rfloor$ comparații de chei, indiferent de ordinea dintre elementele vectorului A . $\square$

Observația 1.2.1. Algoritmul MAX-MIN-PER are tot complexitatea $\Theta(n)$, deoarece efectuează $3 \cdot \lfloor \frac{n}{2} \rfloor$ comparații de chei, iar celelalte operații nu depășesc ordinul de creștere al acestora.

1.3 Determinarea valorii unui polinom dat într-un punct dat

Problema determinării valorii unui polinom dat într-un punct dat este următoarea:

Se consideră un polinom

$$f = a_n \cdot X^n + a_{n-1} \cdot X^{n-1} + \dots + a_1 \cdot X + a_0, \quad n \in \mathbb{N}, \quad a_0, a_1, \dots, a_n \in \mathbb{R}, \quad a_n \neq 0$$

și un punct (valoare) $x \in \mathbb{R}$.

Se cere să se determine valoarea polinomului f în x , adică valoarea

$$f(x) = a_n \cdot x^n + a_{n-1} \cdot x^{n-1} + \dots + a_1 \cdot x + a_0 \in \mathbb{R}.$$

Exemplul 1.3.1. De exemplu, pentru polinomul

$$f = 2X^4 - 4X^3 + 5X^2 + 7X - 8$$

și $x = 3$ avem

$$\begin{aligned} f(3) &= 2 \cdot 3^4 - 4 \cdot 3^3 + 5 \cdot 3^2 + 7 \cdot 3 - 8 \\ &= 2 \cdot 81 - 4 \cdot 27 + 5 \cdot 9 + 7 \cdot 3 - 8 \\ &= 162 - 108 + 45 + 21 - 8 \\ &= 112. \end{aligned}$$

Metoda 1) Calculăm valoarea $v = f(x)$ ca *sumă a monoamelor* componente $a_0, a_1 \cdot x, a_2 \cdot x^2, \dots, a_{n-1} \cdot x^{n-1}, a_n \cdot x^n$ (ca în exemplul de mai sus).

Descrierea în pseudocod a algoritmului are următoarea formă.

Algoritmul 1.3.1.

VALPOL1(A, n, x, v) :

$v \leftarrow A[0];$

for $i = \overline{1, n}$ **do**

$p \leftarrow A[i];$ $// p = a_i \cdot x^i$
for $j = \overline{1, i}$ **do**
 $\quad \perp p \leftarrow p \cdot x;$
 $v \leftarrow v + p;$

returnează v ;

Analiza complexității algoritmului:

Vom număra numai operațiile de adunare și de înmulțire (celelalte operații care se efectuează, atribuiri și comparații, au același ordin de creștere cu cele pe care le analizăm).

Pentru fiecare $i = \overline{1, n}$ algoritmul efectuează i înmulțiri (de forma $p \cdot x$) și o adunare (de forma $v + p$), deci în total algoritmul efectuează:

- $1 + 2 + \dots + n = \frac{n(n+1)}{2}$ înmulțiri;
- n adunări.

Astfel am justificat următorul rezultat.

Propoziția 1.3.1. *Algoritmul 1.3.1 are complexitatea $\Theta(n^2)$.*

Metoda 2) Calculăm valoarea $f(x)$ utilizând *Schema lui Horner*:

	X^n	X^{n-1}	X^{n-2}	$\dots$	X	X^0
	a_n	a_{n-1}	a_{n-2}	$\dots$	a_1	a_0
x	$\underbrace{a_n}_{v_n}$	$\underbrace{x \cdot v_n + a_{n-1}}_{v_{n-1}}$	$\underbrace{x \cdot v_{n-1} + a_{n-2}}_{v_{n-2}}$	$\dots$	$\underbrace{x \cdot v_2 + a_1}_{v_1}$	$\underbrace{x \cdot v_1 + a_0}_{v_0}$

Valoarea cerută, $f(x)$, este calculată în ultima celulă a tabelului, adică $f(x) = v_0$.

Exemplul 1.3.2. De exemplu, pentru polinomul

$$f = 2X^4 - 4X^3 + 5X^2 + 7X - 8$$

și punctul $x = 3$ din exemplul anterior, Schema lui Horner este

	X^4	X^3	X^2	X	X^0
	2	-4	5	7	-8
$x = 3$	2	$3 \cdot 2 - 4$ = 2	$3 \cdot 2 + 5$ = 11	$3 \cdot 11 + 7$ = 40	$3 \cdot 40 - 8$ = 112

Deci $f(3) = 112$.

Pentru implementarea algoritmului putem utiliza o singură variabilă v pentru calculul valorilor $v_n, v_{n-1}, \dots, v_1, v_0$, deci la final avem $v = f(x)$.

Descrierea în pseudocod a algoritmului are următoarea formă.

Algoritmul 1.3.2.

VALPOL2(A, n, x, v) :

$v \leftarrow A[n];$

for $i = n - 1, 0, -1$ **do**

$v \leftarrow x \cdot v + A[i];$

returnează v ;

Analiza complexității algoritmului:

Vom număra, din nou, numai operațiile de adunare și de înmulțire (celelalte operații care se efectuează, atribuirii și comparații, au același ordin de creștere cu cele pe care le analizăm).

Pentru fiecare $i \in \{n - 1, n - 2, \dots, 1, 0\}$ algoritmul efectuează o înmulțire (de forma $x \cdot v$) și o adunare (de forma $x \cdot v + A[i]$), deci în total algoritmul efectuează:

- n înmulțiri;

- n adunări.

Astfel am justificat următorul rezultat.

Propoziția 1.3.2. Algoritmul 1.3.2 are complexitatea $\Theta(n)$, deci este un algoritm liniar.

Observația 1.3.1. Conform Propozițiilor 1.3.1 și 1.3.2 rezultă că Metoda 2 este mai eficientă decât Metoda 1.

1.4 Problema votului majoritar

Problema votului majoritar este următoarea:

Se consideră n voturi $v_1, v_2, \dots, v_n$. Un candidat este majoritar dacă a obținut mai mult de $\frac{n}{2}$ voturi.

Se cere să se determine dacă există un candidat majoritar și, în caz afirmativ, să se determine acest candidat.

Exemplul 1.4.1. De exemplu, pentru voturile

2, 3, 3, 3, 2, 1, 4, 3, 1, 1, 5, 3, 3, 4, 2, 2, 5, 2, 2, 3, 1

avem un total de $n = 21$ de voturi repartizate conform tabelului următor.

Candidat	1	2	3	4	5
Nr. voturi	4	6	7	2	2

Niciun candidat nu are mai mult de $\frac{n}{2} = 10,5$ voturi, deci nu există candidat majoritar.

Exemplul 1.4.2. Pentru voturile

2, 3, 3, 3, 2, 1, 4, 3, 1, 1, 3, 5, 3, 3, 3, 2, 3, 5, 3, 2, 3, 1, 3

avem un total de $n = 23$ de voturi repartizate conform tabelului următor.

Candidat	1	2	3	4	5
Nr. voturi	4	4	12	1	2

Candidatul cu cele mai multe voturi este candidatul 3, care are 12 voturi, deci are mai mult de $\frac{n}{2} = 11,5$ voturi, deci este majoritar.

Metoda 1) Determinăm candidatul cu cele mai multe voturi și verificăm dacă este sau nu majoritar (ca în exemplele de mai sus).

Descrierea în pseudocod a algoritmului are următoarea formă.

Algoritmul 1.4.1.

```

VOTMAJ1( $V, n, cand$ ) :           //  $V = (v_1, v_2, \dots, v_n)$ 
    //  $cand$  = candidatul cu cele mai multe voturi
     $max \leftarrow 0$ ;                //  $max$  = numărul său de voturi
    for  $i = \overline{1, n}$  do
        if  $V[i] \neq 0$  then         // candidatul  $V[i]$  nu a fost analizat
             $nr \leftarrow 1$ ;         // numărul său de voturi
            for  $j = \overline{i+1, n}$  do
                if  $V[j] = V[i]$  then
                     $nr \leftarrow nr + 1$ ;
                     $V[j] \leftarrow 0$ ; // candidatul  $V[j]$  a fost analizat
            if  $nr > max$  then         // actualizăm  $max$  și  $cand$ 
                 $max \leftarrow nr$ ;
                 $cand \leftarrow V[i]$ ;
                // verificăm dacă votul pt.  $cand$  este majoritar
                if  $max > \frac{n}{2}$  then
                    returnează  $cand$ ; //  $cand$  este majoritar
    returnează 0;                  // nu există candidat majoritar

```

Observația 1.4.1. Deoarece un candidat care nu are niciun vot printre primele $\left\lfloor \frac{n+1}{2} \right\rfloor$ voturi nu poate fi majoritar, rezultă că în algoritmul anterior putem înlocui instrucțiunea **for** $i = \overline{1, n}$ **do** cu

$$\mathbf{for} \ i = 1, \overline{\left\lfloor \frac{n+1}{2} \right\rfloor} \ \mathbf{do}$$

Analiza complexității algoritmului

Cazul cel mai defavorabil este cel în care voturile sunt distincte două câte două. Vom număra numai comparațiile de forma $V[j] = V[i]$ (celelalte operații care se efectuează au cel mult același ordin de creștere cu cele pe care le analizăm).

În cazul cel mai defavorabil algoritmul efectuează câte o astfel de comparație pentru fiecare pereche

(i, j) cu $i \in \{1, 2, \dots, n-1\}$ și $j \in \{i+1, i+2, \dots, n\}$,

deci un total de

$$(n-1) + (n-2) + \dots + 2 + 1 = \frac{n(n-1)}{2}$$

astfel de comparații.

Cu modificarea din observația anterioară, numărul acestor comparații se reduce la

$$(n-1) + (n-2) + \dots + \left(n - \left\lfloor \frac{n+1}{2} \right\rfloor\right) = \frac{1}{2} \cdot \left(2n-1 - \left\lfloor \frac{n+1}{2} \right\rfloor\right) \cdot \left\lfloor \frac{n+1}{2} \right\rfloor.$$

Astfel am justificat următorul rezultat.

Propoziția 1.4.1. *Algoritmul 1.4.1 are complexitatea $\mathcal{O}(n^2)$.*

Metoda 2) Sortăm voturile în ordine crescătoare. Apoi determinăm candidatul cu cele mai multe voturi, corespunzător celei mai lungi subsecvențe de voturi egale, și verificăm dacă este sau nu majoritar.

Descrierea în pseudocod a algoritmului are următoarea formă.

Algoritmul 1.4.2.

```

VOTMAJ2( $V, n, cand$ ):
  SORTARE( $V, n$ );           // se sortează voturile crescător,
                              // adică  $v_1 \leq v_2 \leq \dots \leq v_n$ 

   $i \leftarrow 1$ ;
   $max \leftarrow 0$ ;
  repeat
    // determinăm subsecvența voturilor egale cu  $V[i]$ 
     $j \leftarrow i + 1$ ;
    while  $j \leq n$  and  $V[j] = V[i]$  do
       $j \leftarrow j + 1$ ;
    if  $j - i > max$  then           // actualizăm  $max$  și  $cand$ 
       $max \leftarrow j - i$ ;
       $cand \leftarrow V[i]$ ;
      // verificăm dacă votul pt.  $cand$  este majoritar
      if  $max > \frac{n}{2}$  then
         $\text{returnează } cand$ ;           //  $cand$  este majoritar
       $i \leftarrow j$ ;           // trecem la următorul candidat
  while  $i \leq \left\lfloor \frac{n+1}{2} \right\rfloor$ ;
  returnează 0;           // nu există candidat majoritar

```

Analiza complexității algoritmului

Așa cum se va demonstra în Capitolul 5, orice algoritm eficient de sortare în ordine crescătoare a celor n voturi are complexitatea $\Theta(n \log_2 n)$.

Pentru restul algoritmului vom număra, din nou, numai comparațiile de forma $V[j] = V[i]$ (celelalte operații care se efectuează au cel mult același ordin de creștere cu cele pe care le analizăm). În cazul cel mai defavorabil algoritmul efectuează $n - 1$ astfel de comparații (câte una pentru fiecare $j \in \{2, 3, \dots, n\}$), deci restul algoritmului are complexitatea $\mathcal{O}(n)$.

Astfel complexitatea întregului algoritm este de ordinul $\Theta(n \log_2 n)$.

Am justificat următorul rezultat.

Propoziția 1.4.2. *Algoritmul 1.4.2 are complexitatea $\Theta(n \log_2 n)$.*

Exemplul 1.4.3. Pentru voturile din Exemplul 1.4.1, voturile sortate crescător sunt:

1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 5, 5.

Subsecvența maximă de voturi egale este cea a candidatului 3, de lungime $7 \leq \frac{n}{2} = 10,5$, deci nu există candidat majoritar.

Exemplul 1.4.4. Pentru voturile din Exemplul 1.4.2, voturile sortate crescător sunt:

1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 5, 5.

Subsecvența maximă de voturi egale este cea a candidatului 3, de lungime $12 > \frac{n}{2} = 11,5$, deci acest candidat este majoritar.

Metoda 3) Parcurgem voturile împerechind voturi diferite (adică voturi pentru candidați diferiți). La final, dacă nu rămân voturi neîmperecheate atunci nu există candidat majoritar, iar dacă rămân voturi neîmperecheate atunci ele vor fi ale unui aceluiași candidat și verificăm dacă acesta este sau nu majoritar.

Descrierea în pseudocod a algoritmului are următoarea formă.

Algoritmul 1.4.3.

```

VOTMAJ3( $V, n, cand$ ):
 $nv \leftarrow 0$ ; //  $nv$  = numărul de voturi ce rămân neîmperecheate
for  $i = \overline{1, n}$  do // analizăm votul  $V[i]$ 
|   if  $nv = 0$  then
|   |   // toate voturile anterioare au fost împerecheate
|   |    $nv \leftarrow 1$ ; // nu putem împerechea votul curent
|   |    $cand \leftarrow V[i]$ ; //  $cand$  = candidatul ce rămâne
|   |   // în urma împerecherii voturilor
|   else // există voturi neîmperecheate
|   |   if  $V[i] = cand$  then
|   |   |    $nv \leftarrow nv + 1$ ; // nu putem împerechea votul  $V[i]$ 
|   |   else
|   |   |    $nv \leftarrow nv - 1$ ; // împerechem votul  $V[i]$ 
|   if  $nv = 0$  then // nu au rămas voturi neîmperecheate
|   |   returnează 0; // nu există candidat majoritar
|   else // au rămas voturi neîmperecheate,
|   |   // verificăm dacă votul pt.  $cand$  este majoritar
|   |    $nr \leftarrow 0$ ;
|   |   for  $i = \overline{1, n}$  do
|   |   |   if  $V[i] = cand$  then
|   |   |   |    $nr \leftarrow nr + 1$ ;
|   |   |   |   if  $nr > \frac{n}{2}$  then
|   |   |   |   |   returnează  $cand$ ; //  $cand$  este majoritar
|   |   returnează 0; // nu există candidat majoritar

```

Analiza complexității algoritmului

- Prima instrucțiune este o atribuire.
- Urmează ciclul **for** ce implică $n + 1$ atribuiri și tot $n + 1$ comparații, iar în cadrul corpului său se efectuează, pentru fiecare $i \in \{1, 2, \dots, n\}$, fie câte o comparație și două atribuiri, fie câte două comparații și o atribuire; deci numărul total de atribuiri și numărul total de comparații sunt cuprinse în intervalul $[2n + 1, 3n + 1]$.
- Urmează comparația **if** $nv = 0$.

- Ramura **then** a acesteia nu efectuează instrucțiuni de atribuire sau de comparație.
- Ramura **else** efectuează doar o instrucțiune de atribuire, apoi ciclul **for** ce implică cel mult $n + 1$ atribuiri și tot cel mult $n + 1$ comparații, iar în cadrul corpului său se efectuează cel mult n atribuiri și cel mult $2n$ comparații.
- Indiferent de ramură, se apelează și o instrucțiune **return**.

Adunând toate aceste operații obținem că algoritmul efectuează:

- un număr de atribuiri cuprins în intervalul $[2n + 2, 5n + 4]$;
- un număr de comparații cuprins în intervalul $[2n + 2, 6n + 3]$;
- o instrucțiune **return**.

Astfel am justificat următorul rezultat.

Propoziția 1.4.3. *Algoritmul 1.4.3 are complexitatea $\Theta(n)$, deci este un algoritm liniar.*

Exemplul 1.4.5. Pentru voturile din Exemplul 1.4.1, aplicarea algoritmului anterior este evidențiată în tabelul următor.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
v_i	-	2	3	3	3	2	1	4	3	1	1	5	3	3	4	2
nv	0	1	0	1	2	1	0	1	0	1	2	1	0	1	0	1
$cand$	-	2	2	3	3	3	3	4	4	1	1	1	1	3	3	2

i	16	17	18	19	20	21
v_i	2	5	2	2	3	1
nv	2	1	2	3	2	1
$cand$	2	2	2	2	2	2

Avem $nv = 1$ și $cand = 2$, deci au rămas voturi neîmperecheate, ale candidatului 2. Acest candidat are 6 voturi, deci nu este majoritar. Astfel nu există candidat majoritar.

Exemplul 1.4.6. Pentru voturile din Exemplul 1.4.2, aplicarea algoritmului anterior este evidențiată în tabelul următor.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
v_i	-	2	3	3	3	2	1	4	3	1	1	3	5	3	3	3
nv	0	1	0	1	2	1	0	1	0	1	2	1	0	1	2	3
$cand$	-	2	2	3	3	3	3	4	4	1	1	1	1	3	3	3

i	16	17	18	19	20	21	22	23
v_i	2	3	5	3	2	3	1	3
nv	2	3	2	3	2	3	2	3
$cand$	3	3	3	3	3	3	3	3

Avem $nv = 3$ și $cand = 3$, deci au rămas voturi neîmperecheate, ale candidatului 3. Acest candidat are 12 voturi, deci este majoritar.

Observația 1.4.2. Conform Propozițiilor 1.4.1, 1.4.2 și 1.4.3 rezultă că Metoda 2 este mai eficientă decât Metoda 1, iar Metoda 3 este mai eficientă decât Metodele 1 și 2.

Observația 1.4.3. Evident, orice algoritm care rezolvă problema dată necesită parcurgerea (citirea și analiza) celor n voturi, deci conform Propoziției 1.4.3 rezultă că Algoritmul 1.4.3 este asimptotic-optimal.

1.5 Determinarea ultimei cifre nenule a factorialului unui număr natural

În această secțiune ne propunem să determinăm ultima cifră nenulă a lui $n!$, unde n este un număr natural arbitrar. Avem nevoie de câteva noțiuni și rezultate pregătitoare.

Definiția 1.5.1. a) Pentru orice număr natural m (scris în baza 10), notăm cu $u(m)$ **ultima cifră** a numărului m .

b) Pentru orice număr natural nenul m , notăm cu $r(m)$ **ultima cifră nenulă** a numărului m .

Observația 1.5.1. Evident, $u(m) = m \bmod 10$, pentru orice $m \in \mathbb{N}$. Pe de altă parte, pentru orice $m \in \mathbb{N}^*$ avem $r(m) = \frac{m \bmod 10^k}{10^{k-1}}$, unde k este cel mai mic număr natural nenul cu proprietatea că $m \bmod 10^k \neq 0$.

Cifra $r(m)$ poate fi calculată prin împărțiri succesive la 10, cât timp restul este egal cu zero, algoritm care are complexitatea $\mathcal{O}(\ln m)$. Într-adevăr, numărul de împărțiri succesive la 10 este egal cu 1 plus numărul de zerouri în care se termină m , deci în cazul cel mai defavorabil, și anume când $m = \overline{c_1 00 \dots 0}$, se efectuează $1 + \lfloor \lg m \rfloor$ împărțiri.

Un algoritm echivalent pentru calculul lui $r(m)$ se bazează pe următoarea proprietate.

Propoziția 1.5.1. *Pentru orice număr natural nenul m există un unic triplet (a, b, c) cu $a, b \in \mathbb{N}$ și $c \in \{1, 3, 7, 9\}$ astfel încât*

$$m = 2^a \cdot 5^b \cdot (\mathcal{M}10 + c) \quad (1.5.1)$$

(notația $\mathcal{M}10$ reprezintă un multiplu al lui 10).

Demonstrație. Fie

$$a = \max\{k \in \mathbb{N} \mid 2^k \text{ divide } m\} \text{ și } b = \max\{k \in \mathbb{N} \mid 5^k \text{ divide } m\}.$$

Atunci $m = 2^a \cdot 5^b \cdot p$, unde p este un număr natural care nu se divide nici cu 2, nici cu 5. Notând $c = u(p)$, rezultă că $m = 2^a \cdot 5^b \cdot (\mathcal{M}10 + c)$ și $c \in \{1, 3, 7, 9\}$.

Demonstrăm acum unicitatea scrierii (1.5.1). Într-adevăr, fie

$$m = 2^a \cdot 5^b \cdot (\mathcal{M}10 + c) = 2^{a'} \cdot 5^{b'} \cdot (\mathcal{M}10 + c'), \text{ cu } a, b, a', b' \in \mathbb{N} \text{ și } c, c' \in \{1, 3, 7, 9\}.$$

Cum $5^b \cdot (\mathcal{M}10 + c)$ nu se divide cu 2, rezultă că $2^{a'}$ divide 2^a , deci $a' \leq a$. Analog se obține că 2^a divide $2^{a'}$, deci $a \leq a'$ și astfel $a = a'$. Analog se arată că $b = b'$. Rezultă că $\mathcal{M}10 + c = \mathcal{M}10 + c'$, deci și $c = c'$. $\square$

Definiția 1.5.2. *Pentru orice număr natural nenul m , notăm cu $a(m)$, $b(m)$ și $c(m)$ numerele a , b , respectiv c din scrierea (1.5.1).*

Propoziția 1.5.2. *Pentru orice număr natural nenul m avem*

$$r(m) = \begin{cases} c(m), & \text{dacă } a(m) = b(m), \\ 5, & \text{dacă } a(m) < b(m), \\ u(2^{a(m)-b(m)} \cdot c(m)), & \text{dacă } a(m) > b(m). \end{cases}$$

Demonstrație. Conform definiției anterioare, $m = 2^{a(m)} \cdot 5^{b(m)} \cdot (\mathcal{M}10 + c(m))$, cu $a(m), b(m) \in \mathbb{N}$ și $c(m) \in \{1, 3, 7, 9\}$.

Cazul 1. Dacă $a(m) = b(m)$, atunci $m = 10^{a(m)} \cdot (\mathcal{M}10 + c(m))$, deci m se termină în $a(m)$ zerouri și ultima cifră nenulă a sa este $c(m)$.

Cazul 2. Dacă $a(m) < b(m)$, atunci $m = 10^{a(m)} \cdot 5^{b(m)-a(m)} \cdot (\mathcal{M}10 + c(m))$, deci m se termină în $a(m)$ zerouri și ultima sa cifră nenulă este $u(5^{b(m)-a(m)} \cdot c(m)) = 5$.

Cazul 3. Dacă $a(m) > b(m)$, atunci $m = 10^{b(m)} \cdot 2^{a(m)-b(m)} \cdot (\mathcal{M}10 + c(m))$, deci m se termină în $b(m)$ zerouri și ultima sa cifră nenulă este $u(2^{a(m)-b(m)} \cdot c(m))$. $\square$

Observația 1.5.2. Pentru orice $k \in \mathbb{N}^*$ avem $u(2^k) = u(6 \cdot 2^{k \bmod 4})$, deci formula lui $r(m)$ dată de Propoziția 1.5.2 poate fi rescrisă sub forma:

$$r(m) = \begin{cases} c(m), & \text{dacă } a(m) = b(m), \\ 5, & \text{dacă } a(m) < b(m), \\ u(6 \cdot 2^{(a(m)-b(m)) \bmod 4} \cdot c(m)), & \text{dacă } a(m) > b(m). \end{cases}$$

Exemplul 1.5.1. $15300 = 2^2 \cdot 5^2 \cdot 153$, deci $a(15300) = b(15300) = 2$, $c(15300) = 3$ și $r(15300) = 3$.

$23500 = 2^2 \cdot 5^3 \cdot 47$, deci $a(23500) = 2$, $b(23500) = 3$, $c(23500) = 7$ și $r(23500) = 5$.

$42240 = 2^8 \cdot 5 \cdot 33$, deci $a(42240) = 8$, $b(42240) = 1$, $c(42240) = 3$ și $r(42240) = u(6 \cdot 2^{(8-1) \bmod 4} \cdot 3) = u(6 \cdot 2^3 \cdot 3) = 4$.

Obținem astfel următorul algoritm pentru determinarea lui $r(m)$.

UCN(m):

$p \leftarrow m$; $a \leftarrow 0$; $b \leftarrow 0$;

while $p \bmod 2 = 0$ **do**

$a \leftarrow a + 1$; $p \leftarrow p/2$;

while $p \bmod 5 = 0$ **do**

$b \leftarrow b + 1$; $p \leftarrow p/5$;

$c \leftarrow p \bmod 10$;

if $a = b$ **then**

return c ;

else

if $a < b$ **then**

return 5;

else

for $i = 1, (a - b) \bmod 4$ **do**

$c \leftarrow 2 \cdot c$;

return $(6 \cdot c) \bmod 10$;

Propoziția 1.5.3. Algoritmul UCN are complexitatea $\Theta(a(m) + b(m) + 1)$.

Demonstrație. Conform Definiției 1.5.2 avem $m = 2^{a(m)} \cdot 5^{b(m)} \cdot (\mathcal{M}10 + c(m))$, cu $a(m), b(m) \in \mathbb{N}$ și $c(m) \in \{1, 3, 7, 9\}$. Corpul primului ciclu **while** din algoritmul UCN are două instrucțiuni și se execută de $a(m)$ ori, corpul celui de-al doilea ciclu **while** are tot două instrucțiuni și se execută de $b(m)$ ori, iar corpul ciclului **for** are o instrucțiune și se execută de cel mult 3 ori. Algoritmul mai conține 4 atribuiri și una sau două comparații, deci are complexitatea $\Theta(a(m) + b(m) + 1)$. $\square$

Corolarul 1.5.1. *Algoritmul UCN are complexitatea $\mathcal{O}(\ln m)$.*

Demonstrație. Avem $2^{a(m)} \leq m$ și $5^{b(m)} \leq m$, deci $a(m) \leq \log_2 m$ și $b(m) \leq \log_5 m$. Obținem că $a(m) + b(m) + 1 \leq (\frac{1}{\ln 2} + \frac{1}{\ln 5}) \ln m + 1$, deci conform propoziției anterioare rezultă că algoritmul UCN are complexitatea $\mathcal{O}(\ln m)$. $\square$

Observația 1.5.3. Mai mult, în cazul când m este o putere a lui 2, adică $m = 2^{a(m)}$, avem $b(m) = 0$ și astfel $a(m) + b(m) + 1 = \log_2 m + 1 = \frac{\ln m}{\ln 2} + 1$. Conform rezultatelor anterioare rezultă că algoritmul UCN are timpul de execuție în acest caz particular chiar de ordinul $\Theta(\ln m)$, care este mai precis decât $\mathcal{O}(\ln m)$. Evident, același ordin se obține și în cazul când m este o putere a lui 5 sau a lui 10, și nu numai.

Obținem astfel o primă metodă pentru calculul ultimei cifre nenule a lui $n!$, adică $r(n!)$.

Metoda 1) Calculăm $m = n!$ și determinăm $r(n!) = r(m)$, ca mai sus. Descrierea în pseudocod a algoritmului are următoarea formă.

Algoritmul 1.5.1.

UCNF1(n):

$m \leftarrow 1$;

for $i = \overline{2, n}$ **do**

$m \leftarrow m \cdot i$;

return UCN(m) ;

Analiza complexității algoritmului

Propoziția 1.5.4. *Dacă n este un număr natural nenul, atunci*

$$a(n!) = \left\lfloor \frac{n}{2} \right\rfloor + \left\lfloor \frac{n}{2^2} \right\rfloor + \left\lfloor \frac{n}{2^3} \right\rfloor + \dots + \left\lfloor \frac{n}{2^k} \right\rfloor, \text{ unde } k = \lfloor \log_2 n \rfloor$$

și

$$b(n!) = \left\lfloor \frac{n}{5} \right\rfloor + \left\lfloor \frac{n}{5^2} \right\rfloor + \left\lfloor \frac{n}{5^3} \right\rfloor + \dots + \left\lfloor \frac{n}{5^{k'}} \right\rfloor, \text{ unde } k' = \lfloor \log_5 n \rfloor.$$

Demonstrație. $a(n!)$ reprezintă puterea lui 2 din descompunerea în factori primi a produsului $1 \cdot 2 \cdot \dots \cdot n$. Printre numerele $1, 2, \dots, n$ avem exact $\left\lfloor \frac{n}{2} \right\rfloor$ numere divizibile cu 2, dintre care exact $\left\lfloor \frac{n}{2^2} \right\rfloor$ sunt divizibile și cu 2^2 , dintre care exact $\left\lfloor \frac{n}{2^3} \right\rfloor$ sunt divizibile și cu 2^3 , ș.a.m.d., deci

$$a(n!) = \left\lfloor \frac{n}{2} \right\rfloor + \left\lfloor \frac{n}{2^2} \right\rfloor + \left\lfloor \frac{n}{2^3} \right\rfloor + \dots$$

Cum $2^{k+1} > n$, deci printre numerele $1, 2, \dots, n$ nu există numere divizibile cu 2^{k+1} , rezultă că suma anterioară se poate încheia cu termenul $\left\lfloor \frac{n}{2^k} \right\rfloor$. Analog se demonstrează și formula lui $b(n!)$ din enunț. $\square$

Corolarul 1.5.2. *Avem $a(n!) = \Theta(n)$ și $b(n!) = \Theta(n)$.*

Demonstrație. Fie $n \in \mathbb{N}^*$ și $k = \lfloor \log_2 n \rfloor$. Conform propoziției anterioare,

$$a(n!) = \left\lfloor \frac{n}{2} \right\rfloor + \left\lfloor \frac{n}{2^2} \right\rfloor + \left\lfloor \frac{n}{2^3} \right\rfloor + \dots + \left\lfloor \frac{n}{2^k} \right\rfloor.$$

Avem $k \leq \log_2 n < k+1$, deci $2^k \leq n < 2^{k+1}$. Deducem că $2^{k-1} \leq \left\lfloor \frac{n}{2} \right\rfloor < 2^k$, $2^{k-2} \leq \left\lfloor \frac{n}{2^2} \right\rfloor < 2^{k-1}$, ..., $2^0 \leq \left\lfloor \frac{n}{2^k} \right\rfloor < 2^1$, deci prin adunare obținem că

$$2^k - 1 \leq a(n!) < 2(2^k - 1).$$

Cum $\log_2 n - 1 < k \leq \log_2 n$, rezultă că

$$\frac{n-2}{2} < a(n!) < 2n-2,$$

deci $a(n!) = \Theta(n)$.

Analog se demonstrează că

$$\frac{n-5}{20} < b(n!) < \frac{5n-5}{4},$$

deci și $b(n!) = \Theta(n)$. □

Corolarul 1.5.3. *Avem $a(0!) = b(0!) = a(1!) = b(1!) = 0$ și $a(n!) > b(n!)$ pentru orice $n \geq 2$.*

Demonstrație. Evident, $a(1) = b(1) = 0$. Pentru $n \geq 2$, cu notațiile din Propoziția 1.5.4, avem $k \geq k'$, $\left\lfloor \frac{n}{2} \right\rfloor > \left\lfloor \frac{n}{5} \right\rfloor$, $\left\lfloor \frac{n}{2^2} \right\rfloor \geq \left\lfloor \frac{n}{5^2} \right\rfloor$, ..., $\left\lfloor \frac{n}{2^{k'}} \right\rfloor \geq \left\lfloor \frac{n}{5^{k'}} \right\rfloor$, de unde prin adunare obținem că $a(n!) > b(n!)$. □

Corolarul 1.5.4. *Pentru orice $n \in \mathbb{N}$, numărul de zerouri în care se termină numărul $n!$ este egal cu $b(n!)$.*

Demonstrație. Conform corolarului anterior, $a(n!) \geq b(n!)$, deci conform (1.5.1) avem $n! = 10^{b(n!)} \cdot 2^{a(n!)-b(n!)} \cdot (\mathcal{M}10 + c(n!))$, de unde rezultă afirmația din enunț. □

Conform Propoziției 1.5.3 și Corolarului 1.5.2 obținem următorul rezultat.

Propoziția 1.5.5. *Algoritmul UCNF1 are complexitatea $\Theta(n)$.*

Observația 1.5.4. Algoritmul UCNF1, bazat pe calculul lui $n!$, este un algoritm liniar, dar este practic inoperabil pentru valori mari ale lui n din cauza mărimii numărului $n!$.

Metoda 2) Vom determina $r(n!)$ tot pe baza Propoziției 1.5.2, dar vom calcula numerele $a(n!)$, $b(n!)$ și $c(n!)$ evitând calculul efectiv al lui $n!$. Ne vom baza pe următorul rezultat.

Propoziția 1.5.6. *Pentru orice numere naturale nenule $m_1, m_2, \dots, m_n$ avem*

$$\begin{aligned} a(m_1 \cdot m_2 \cdot \dots \cdot m_n) &= a(m_1) + a(m_2) + \dots + a(m_n), \\ b(m_1 \cdot m_2 \cdot \dots \cdot m_n) &= b(m_1) + b(m_2) + \dots + b(m_n), \\ c(m_1 \cdot m_2 \cdot \dots \cdot m_n) &= u(c(m_1) \cdot c(m_2) \cdot \dots \cdot c(m_n)). \end{aligned}$$

Demonstrație. Avem $m_1 \cdot m_2 \cdot \dots \cdot m_n = \prod_{i=1}^n (2^{a(m_i)} \cdot 5^{b(m_i)} \cdot (\mathcal{M}10 + c(m_i))) = 2^{\sum_{i=1}^n a(m_i)} \cdot 5^{\sum_{i=1}^n b(m_i)} \cdot \left(\mathcal{M}10 + u \left(\prod_{i=1}^n c(m_i) \right) \right)$, de unde rezultă egalitățile din enunț. $\square$

Următorul rezultat este o consecință imediată a Propozițiilor 1.5.6 și 1.5.2 și a Observației 1.5.2.

Corolarul 1.5.5. *Fie $m_1, m_2, \dots, m_n$ numere naturale nenule.*

a) Dacă $a(m_1) + a(m_2) + \dots + a(m_n) = b(m_1) + b(m_2) + \dots + b(m_n)$, atunci

$$r(m_1 \cdot m_2 \cdot \dots \cdot m_n) = u(c(m_1) \cdot c(m_2) \cdot \dots \cdot c(m_n)).$$

b) Dacă $a(m_1) + a(m_2) + \dots + a(m_n) < b(m_1) + b(m_2) + \dots + b(m_n)$, atunci

$$r(m_1 \cdot m_2 \cdot \dots \cdot m_n) = 5.$$

c) Dacă $a(m_1) + a(m_2) + \dots + a(m_n) > b(m_1) + b(m_2) + \dots + b(m_n)$, atunci

$$\begin{aligned} r(m_1 \cdot m_2 \cdot \dots \cdot m_n) &= u \left(6 \cdot 2^{(a(m_1)+a(m_2)+\dots+a(m_n)-b(m_1)-b(m_2)-\dots-b(m_n)) \text{ MOD } 4} \cdot \right. \\ &\quad \left. c(m_1) \cdot c(m_2) \cdot \dots \cdot c(m_n) \right). \end{aligned}$$

Exemplul 1.5.2. $320 = 2^6 \cdot 5$ și $150 = 2 \cdot 5^2 \cdot 3$, deci $a(320) = 6$, $b(320) = 1$, $c(320) = 1$, $a(150) = 1$, $b(150) = 2$, $c(150) = 3$, $a(320) + a(150) = 7$, $b(320) + b(150) = 3$ și $r(320 \cdot 150) = u(6 \cdot 2^{(7-3) \text{ MOD } 4} \cdot 1 \cdot 3) = u(6 \cdot 2^0 \cdot 3) = 8$.

Obținem următorul algoritm pentru calculul lui $r(n!) = r(1 \cdot 2 \cdot \dots \cdot n)$.

Algoritmul 1.5.2.

```

UCNF2( $n$ ):
 $a \leftarrow 0$ ;  $b \leftarrow 0$ ;  $c \leftarrow 1$ ;
for  $k = \overline{1, n}$  do
     $p \leftarrow k$ ;
    while  $p \bmod 2 = 0$  do
         $a \leftarrow a + 1$ ;  $p \leftarrow p/2$ ;
    while  $p \bmod 5 = 0$  do
         $b \leftarrow b + 1$ ;  $p \leftarrow p/5$ ;
     $c \leftarrow (c \cdot p) \bmod 10$ ;
if  $a = b$  then
    | return  $c$ ;
else
    | if  $a < b$  then
    | | return 5;
    | else
    | | for  $i = \overline{1, (a - b) \bmod 4}$  do
    | | |  $c \leftarrow 2 \cdot c$ ;
    | | return  $(6 \cdot c) \bmod 10$ ;

```

Analiza complexității algoritmului

Propoziția 1.5.7. Algoritmul UCNF2 are complexitatea $\Theta(n)$.

Demonstrație. Conform Propoziției 1.5.6 și Corolarului 1.5.2, complexitatea algoritmului UCNF1 este $\Theta\left(\sum_{k=1}^n (1 + a(k) + b(k))\right) = \Theta(n + a(n!) + b(n!)) = \Theta(n)$. $\square$

Exemplul 1.5.3. Aplicând Metoda 2, avem $r(11!) = r(1 \cdot 2 \cdot 3 \cdot \dots \cdot 11) = r(1 \cdot 2 \cdot 3 \cdot 2^2 \cdot 5 \cdot 2 \cdot 3 \cdot 7 \cdot 2^3 \cdot 9 \cdot 2 \cdot 5 \cdot 11) = r(2^8 \cdot 5^2 \cdot 3 \cdot 3 \cdot 7 \cdot 9 \cdot 1) = u(6 \cdot 2^{(8-2) \bmod 4} \cdot 3 \cdot 3 \cdot 7 \cdot 9) = 8$.

Observația 1.5.5. Pentru valori mari ale lui n , acest procedeu de calcul este practic imposibil de aplicat fără utilizarea unui program pentru calculator.

Metoda 3) Prezentăm în continuare un algoritm subliniar pentru determinarea lui $r(n!)$. Avem nevoie de alte câteva rezultate.

Observația 1.5.6. Dacă m este un număr natural nedivizibil cu 10, atunci $u(m) \neq 0$, deci $r(m) = u(m)$.

Observația 1.5.7. Evident, $u(m_1 \cdot m_2 \cdot \dots \cdot m_n) = u(u(m_1) \cdot u(m_2) \cdot \dots \cdot u(m_n))$, pentru orice $m_1, m_2, \dots, m_n \in \mathbb{N}$ ($n \in \mathbb{N}$, $n \geq 2$).

Pe de altă parte, egalitatea

$$r(m_1 \cdot m_2 \cdot \dots \cdot m_n) = u(r(m_1) \cdot r(m_2) \cdot \dots \cdot r(m_n)) \quad (1.5.2)$$

este adevărată doar dacă $u(r(m_1) \cdot r(m_2) \cdot \dots \cdot r(m_n)) \neq 0$. Mai mult, egalitatea $r(m_1 \cdot m_2 \cdot \dots \cdot m_n) = r(r(m_1) \cdot r(m_2) \cdot \dots \cdot r(m_n))$ este adevărată dacă $u(r(m_1) \cdot r(m_2) \cdot \dots \cdot r(m_n)) \neq 0$ și nu este neapărat adevărată în caz contrar.

De exemplu, $r(320 \cdot 170) = 4 = u(r(320) \cdot r(170)) = r(r(320) \cdot r(170))$, dar $r(320 \cdot 150) = 8$ iar $u(r(320) \cdot r(150)) = 0$ și $r(r(320) \cdot r(150)) = 1$.

Observația 1.5.8. Fie $m_1, m_2, \dots, m_n$ numere naturale nenule astfel încât $r(m_i) \neq 5$ pentru orice $i = \overline{1, n}$. Atunci $u(r(m_1) \cdot r(m_2) \cdot \dots \cdot r(m_n)) \neq 0$ și conform observației anterioare are loc egalitatea (1.5.2).

Lema 1.5.1. Fie n un număr natural, $n \geq 2$. Atunci $r(n!) \in \{2, 4, 6, 8\}$.

Demonstrație. Conform Corolarului 1.5.3, $a(n!) > b(n!)$, deci conform (1.5.1) avem

$$n! = 10^{b(n!)} \cdot 2^{a(n!)-b(n!)} \cdot (\mathcal{M}10 + c(n!)),$$

de unde rezultă că

$$r(n!) = r(2^{a(n!)-b(n!)} \cdot (\mathcal{M}10 + c(n!))).$$

Dar

$$r(2^{a(n!)-b(n!)}) = u(2^{a(n!)-b(n!)}) \in \{2, 4, 6, 8\},$$

$$r(\mathcal{M}10 + c(n!)) = u(\mathcal{M}10 + c(n!)) = c(n!) \in \{1, 3, 7, 9\},$$

deci, folosind și Observația 1.5.8, $r(n!) = u(r(2^{a(n!)-b(n!)} \cdot c(n!))) \in \{2, 4, 6, 8\}$. $\square$

Lema 1.5.2. Fie m și k două numere naturale astfel încât m este divizibil cu 2^{k+1} . Atunci

$$u(m : 2^k) = u(m : 2^{k \bmod 4}) = u(m \cdot 2^{4-(k \bmod 4)}).$$

Demonstrație. Din ipoteză rezultă că $m = 2^{k+p} \cdot y$, cu $p \in \mathbb{N}^*$ și y impar. Avem

$$\begin{aligned} u(m : 2^k) &= u(2^p \cdot y) = u(u(2^p) \cdot u(y)), \\ u(m : 2^{k \bmod 4}) &= u(2^p \cdot 2^{k-(k \bmod 4)} \cdot y) = u(u(2^p) \cdot u(2^{k-(k \bmod 4)}) \cdot u(y)) \\ &= u(u(2^p) \cdot 6 \cdot u(y)) = u(u(2^p \cdot 6) \cdot u(y)) = u(u(2^p) \cdot u(y)) \end{aligned}$$

și

$$\begin{aligned} u(m \cdot 2^{4-(k \bmod 4)}) &= u(m : 2^{k \bmod 4} \cdot 2^4) = u(u(m : 2^{k \bmod 4}) \cdot u(2^4)) \\ &= u(u(u(2^p) \cdot u(y)) \cdot 6) = u(u(2^p) \cdot 6 \cdot u(y)) \\ &= u(u(2^p \cdot 6) \cdot u(y)) = u(u(2^p) \cdot u(y)), \end{aligned}$$

deci au loc egalitățile din enunț. $\square$

Propoziția 1.5.8. Pentru orice număr natural n , $n \geq 2$, avem

$$r(n!) = u\left(r\left(\left\lfloor \frac{n}{5} \right\rfloor!\right) \cdot u\left(\prod_{\substack{1 \leq i \leq n \\ i \neq 5k}} i\right) \cdot u\left(2^{4 - (\lfloor \frac{n}{5} \rfloor \bmod 4)}\right)\right).$$

Demonstrație. Avem

$$\begin{aligned} n! &= \prod_{\substack{1 \leq i \leq n \\ i \neq 5k}} i \cdot \prod_{\substack{1 \leq i \leq n \\ i = 5k}} i = \prod_{k=1}^{\lfloor n/5 \rfloor} (5k) \cdot \prod_{\substack{1 \leq i \leq n \\ i \neq 5k}} i = 5^{\lfloor n/5 \rfloor} \cdot \left\lfloor \frac{n}{5} \right\rfloor! \cdot \prod_{\substack{1 \leq i \leq n \\ i \neq 5k}} i \\ &= 10^{\lfloor n/5 \rfloor} \cdot \left\lfloor \frac{n}{5} \right\rfloor! \cdot \left(\prod_{\substack{1 \leq i \leq n \\ i \neq 5k}} i : 2^{\lfloor n/5 \rfloor}\right), \end{aligned}$$

deci utilizând Lema 1.5.1 și Observațiile 1.5.8 și 1.5.6 rezultă că

$$r(n!) = r\left(\left\lfloor \frac{n}{5} \right\rfloor! \cdot \left(\prod_{\substack{1 \leq i \leq n \\ i \neq 5k}} i : 2^{\lfloor n/5 \rfloor}\right)\right) = u\left(r\left(\left\lfloor \frac{n}{5} \right\rfloor!\right) \cdot u\left(\prod_{\substack{1 \leq i \leq n \\ i \neq 5k}} i : 2^{\lfloor n/5 \rfloor}\right)\right).$$

Cum $\prod_{\substack{1 \leq i \leq n \\ i \neq 5k}} i$ se divide cu $2^{\lfloor n/2 \rfloor}$ și $\lfloor \frac{n}{2} \rfloor > \lfloor \frac{n}{5} \rfloor$, aplicând Lema 1.5.2 obținem relația din enunț. $\square$

Definiția 1.5.3. Pentru orice număr natural n , $n \geq 2$, notăm

$$s(n) = u\left(\prod_{\substack{1 \leq i \leq n \\ i \neq 5k}} i\right).$$

Prin convenție, definim și $s(0) = s(1) = 6$.

Lema 1.5.3. Pentru orice număr natural n , $n \geq 2$, avem $s(n) = s(u(n))$.

Demonstrație. Avem

$$\begin{aligned} s(n+10) &= u\left(u\left(\prod_{\substack{1 \leq i \leq n \\ i \neq 5k}} i\right) \cdot u\left(\prod_{\substack{n+1 \leq i \leq n+10 \\ i \neq 5k}} i\right)\right) \\ &= u(s(n) \cdot u(1 \cdot 2 \cdot 3 \cdot 4 \cdot 6 \cdot 7 \cdot 8 \cdot 9)) = u(s(n) \cdot 6) = s(n), \end{aligned}$$

deoarece $s(n)$ este evident o cifră pară și nedivizibilă cu 5. Cum $s(10) = s(0) = 6$ și $s(11) = s(1) = 6$, obținem că șirul $(s(n))_{n \geq 0}$ este periodic de perioadă 10 și astfel rezultă egalitatea din enunț. $\square$

Observația 1.5.9. Conform Definiției 1.5.3 avem

$$\begin{aligned} s(0) = s(1) = s(3) = s(9) = 6, \quad s(2) = 2, \\ s(4) = s(5) = s(6) = s(8) = 4, \quad s(7) = 8. \end{aligned}$$

Definiția 1.5.4. Notăm $t(k) = u(2^{4-k})$, $k = \overline{0, 3}$.

Observația 1.5.10. Conform definiției anterioare avem

$$t(0) = 6, \quad t(1) = 8, \quad t(2) = 4, \quad t(3) = 2.$$

Teorema 1.5.1 (relația de recurență a numerelor $r(n!)$). *Avem*

$$r(n!) = \begin{cases} 1, & \text{dacă } n \in \{0, 1\}, \\ u(s(u(n)) \cdot t(\lfloor \frac{n}{5} \rfloor \text{ MOD } 4) \cdot r(\lfloor \frac{n}{5} \rfloor!)), & \text{dacă } n \geq 2. \end{cases}$$

Demonstrație. Evident, $r(0!) = r(1!) = r(1) = 1$. Pentru $n \geq 2$ egalitatea din enunț este o consecință imediată a Propoziției 1.5.8, Definițiilor 1.5.3 și 1.5.4 și Lemei 1.5.3. $\square$

Exemplul 1.5.4. Pentru a calcula $r(1918!)$ procedăm astfel:

$$\begin{aligned} \left\lfloor \frac{1918}{5} \right\rfloor &= 383, & 383 \text{ MOD } 4 &= 3, \\ \left\lfloor \frac{383}{5} \right\rfloor &= 76, & 76 \text{ MOD } 4 &= 0, \\ \left\lfloor \frac{76}{5} \right\rfloor &= 15, & 15 \text{ MOD } 4 &= 3, \\ \left\lfloor \frac{15}{5} \right\rfloor &= 3, & 3 \text{ MOD } 4 &= 3, \\ \left\lfloor \frac{3}{5} \right\rfloor &= 0, & 0 \text{ MOD } 4 &= 0. \end{aligned}$$

Astfel avem

$$\begin{aligned} r(1918!) &= u\left(s(u(1918)) \cdot t\left(\left\lfloor \frac{1918}{5} \right\rfloor \text{ MOD } 4\right) \cdot r\left(\left\lfloor \frac{1918}{5} \right\rfloor!\right)\right) \\ &= u(s(8) \cdot t(3) \cdot r(383!)) = u(4 \cdot 2 \cdot r(383!)), \\ r(383!) &= u(s(3) \cdot t(0) \cdot r(76!)) = u(6 \cdot 6 \cdot r(76!)), \\ r(76!) &= u(s(6) \cdot t(3) \cdot r(15!)) = u(4 \cdot 2 \cdot r(15!)), \\ r(15!) &= u(s(5) \cdot t(3) \cdot r(3!)) = u(4 \cdot 2 \cdot r(3!)), \\ r(3!) &= u(s(3) \cdot t(0) \cdot r(0!)) = u(6 \cdot 6 \cdot 1) = 6, \end{aligned}$$

deci revenind la relațiile anterioare obținem că

$$\begin{aligned} r(15!) &= u(4 \cdot 2 \cdot 6) = 8, \\ r(76!) &= u(4 \cdot 2 \cdot 8) = 4, \\ r(383!) &= u(6 \cdot 6 \cdot 4) = 4, \\ r(1918!) &= u(4 \cdot 2 \cdot 4) = 2. \end{aligned}$$

Conform rezultatelor de mai sus obținem următorul algoritm pentru determinarea lui $r(n!)$.

Algoritm 1.5.3.

```

 $s_0 \leftarrow 6; s_1 \leftarrow 6; s_2 \leftarrow 2; s_3 \leftarrow 6; s_4 \leftarrow 4;$ 
 $s_5 \leftarrow 4; s_6 \leftarrow 4; s_7 \leftarrow 8; s_8 \leftarrow 4; s_9 \leftarrow 6;$ 
 $t_0 \leftarrow 6; t_1 \leftarrow 8; t_2 \leftarrow 4; t_3 \leftarrow 2;$ 
UCNF3( $n$ ): // calculează  $r(n!)$ , recursiv
if  $n \leq 1$  then
  | return 1;
else
  |  $p \leftarrow \lfloor n/5 \rfloor; i \leftarrow n \bmod 10; j \leftarrow p \bmod 4;$ 
  | return  $(s_i \cdot t_j \cdot \text{UCNF3}(p)) \bmod 10;$ 

```

Analiza complexității algoritmului

Propoziția 1.5.9. *Algoritm 1.5.3 are complexitatea $\Theta(\ln n)$.*

Demonstrație. Fie $T(n)$ numărul de instrucțiuni executate de algoritm. Evident, avem $T(0) = T(1) = 1$ și

$$T(n) = T\left(\left\lfloor \frac{n}{5} \right\rfloor\right) + 5 \text{ pentru } n \geq 2.$$

Fie $n \geq 2$ și $k = \lfloor \log_5 n \rfloor$, deci $k \leq \log_5 n < k+1$ și astfel $5^k \leq n < 5^{k+1}$.

Cazul 1. Dacă $5^k < n < 5^{k+1}$, atunci $1 < \frac{n}{5^k} < 5$, $\frac{n}{5^{k+1}} < 1$, $\lfloor \frac{n}{5^{k+1}} \rfloor = 0$, deci adunând egalitățile

$$\begin{aligned} T(n) &= T\left(\left\lfloor \frac{n}{5} \right\rfloor\right) + 5, \\ T\left(\left\lfloor \frac{n}{5} \right\rfloor\right) &= T\left(\left\lfloor \frac{n}{5^2} \right\rfloor\right) + 5, \\ &\dots \\ T\left(\left\lfloor \frac{n}{5^k} \right\rfloor\right) &= T\left(\left\lfloor \frac{n}{5^{k+1}} \right\rfloor\right) + 5 \end{aligned}$$

obținem $T(n) = T(0) + 5(k+1)$, adică $T(n) = 5 \lfloor \log_5 n \rfloor + 6$.

Cazul 2. Dacă $n = 5^k$, atunci $\frac{n}{5^{k+1}} = 1$ și analog cazului 1 obținem că $T(n) = T(1) + 5k = 5 \lfloor \log_5 n \rfloor + 1$. $\square$

Observația 1.5.11. Conform Propozițiilor 1.5.5, 1.5.7 și 1.5.9 rezultă că Metoda 3 este mai eficientă decât Metodele 1 și 2.

Tema 2

Metoda Greedy

2.1 Descrierea metodei. Algoritmi generali

Metoda **Greedy** (a **optimului local**) presupune elaborarea unor strategii de rezolvare a *problemelor de optim*, în care se urmărește *maximizarea* sau *minimizarea* unei *funcții obiectiv*.

Se aplică problemelor în care se dă o mulțime finită $A = \{a_1, a_2, \dots, a_n\}$ (**mulțimea de candidați**), conținând n date de intrare, pentru care se cere să se determine o submulțime $B \subseteq A$ care să îndeplinească anumite condiții pentru a fi acceptată. Această submulțime se numește **soluție posibilă** (**soluție admisibilă**, pe scurt **soluție**).

Deoarece, în general, există mai multe *soluții posibile*, trebuie avut în vedere și un *criteriu de selecție*, conform căruia, dintre acestea, să fie aleasă una singură ca rezultat final, numită **soluție optimă**.

Soluțiile posibile au următoarele proprietăți:

- mulțimea vidă $\emptyset$ este întotdeauna soluție posibilă;
- dacă B este soluție posibilă și $C \subseteq B$, atunci și C este soluție posibilă.

În continuare sunt prezentate două scheme de lucru, care urmează aceeași idee, diferențiindu-se doar prin ordinea de efectuare a unor operații:

Algoritmul 2.1.1 (Metoda Greedy, varianta I).

- Se pleacă de la soluția vidă ($\emptyset$);
- Se alege, pe rând, într-un anumit fel, un element din A neales la pașii precedenți.

- Dacă includerea elementului ales în soluția parțială construită anterior conduce la o soluție posibilă, atunci construim noua soluție prin adăugarea elementului ales.

GREEDY1(A, n, B):
 $B \leftarrow \emptyset$;
for $i = \overline{1, n}$ **do**
 $x \leftarrow \text{ALEGE}(A, i, n)$;
 if **SOLUTIE_POSIBILA**(B, x) **then**
 $B \leftarrow B \cup \{x\}$;

Observația 2.1.1.

- Funcția **ALEGE**(A, i, n) returnează un element $x = a_j \in \{a_i, \dots, a_n\}$ și efectuează interschimbarea $a_i \leftrightarrow a_j$;
- Funcția **SOLUTIE_POSIBILA**(B, x) verifică dacă $B \cup \{x\}$ este soluție posibilă a problemei.
- Funcția **ALEGE** este cea mai dificil de realizat, deoarece trebuie să implementeze criteriul conform căruia alegerea la fiecare pas a câte unui candidat să conducă în final la obținerea soluției optime.

Algoritmul 2.1.2 (Metoda Greedy, varianta a II-a).

Metoda e asemănătoare primeia, cu excepția faptului că se stabilește de la început ordinea în care trebuie analizate elementele din A .

GREEDY2(A, n, B):
PRELUCREAZA(A, n);
 $B \leftarrow \emptyset$;
for $i = \overline{1, n}$ **do**
 if **SOLUTIE_POSIBILA**(B, a_i) **then**
 $B \leftarrow B \cup \{a_i\}$;

Observația 2.1.2. Prin apelul procedurii **PRELUCREAZA**(A, n) se efectuează o permutare a elementelor mulțimii A , stabilind ordinea de analiză a acestora. Aceasta este procedura cea mai dificil de realizat.

Observația 2.1.3.

- Metoda Greedy nu caută să determine toate soluțiile posibile și apoi să aleagă pe cea optimă conform criteriului de optimizare dat (ceea ce ar necesita în general un timp de calcul și spațiu de memorie mari), ci constă în a alege pe rând câte un element, urmând să-l "înghită" eventual în soluția optimă. De aici vine și numele metodei (*Greedy = lacom*).
- Astfel, dacă trebuie determinat maximul unei funcții de cost depinzând de $a_1, \dots, a_n$, ideea generală a metodei este de a alege la fiecare pas acel element care face să crească cât mai mult valoarea acestei funcții. Din acest motiv metoda se mai numește și **a optimului local**.
- *Optimul global* se obține prin alegeri succesive, la fiecare pas, ale *optimului local*, ceea ce permite rezolvarea problemelor fără revenire la deciziile anterioare (așa cum se întâmplă la *metoda backtracking*).
- În general metoda Greedy oferă o soluție posibilă și nu întotdeauna soluția optimă. De aceea, dacă problema cere soluția optimă, algoritmul trebuie să fie însoțit și de justificarea faptului că soluția generată este optimă. Pentru aceasta, este frecvent întâlnit următorul procedeu:
 - se demonstrează prin *inducție matematică* faptul că pentru orice pas $i \in \{0, 1, \dots, n\}$, dacă B_i este soluția posibilă construită la pasul i , atunci există o soluție optimă B^* astfel încât $B_i \subseteq B^*$;
 - se arată că pentru soluția finală, B_n , incluziunea $B_n \subseteq B^*$ devine egalitate, $B_n = B^*$, deci B_n este soluție optimă.

Exemplul 2.1.1. Se dă o mulțime $A = \{a_1, a_2, \dots, a_n\}$ cu $a_i \in \mathbb{R}$, $i = \overline{1, n}$. Se cere să se determine o submulțime $B \subseteq A$, astfel încât $\sum_{b \in B} b$ să fie maximă.

Rezolvare. Dacă $B \subseteq A$ și $b_0 \in B$, cu $b_0 \leq 0$, atunci

$$\sum_{b \in B} b \leq \sum_{b \in B \setminus \{b_0\}} b.$$

Rezultă că putem înțelege prin **soluție posibilă** o submulțime B a lui A cu toate elementele strict pozitive.

Vom aplica metoda Greedy, în **varianta I**, în care

- funcția **ALEGE** furnizează $x = a_i$;
- funcția **SOLUTIE_POSIBILA** returnează 1 (adevărat) dacă $x > 0$ și 0 (fals) în caz contrar.

□

ALEGE (A, i, n):

$x \leftarrow a_i$;

returnează x ;

SOLUTIE_POSIBILA (B, x):

if $x > 0$ **then**

returnează 1;

// adevărat

else

returnează 0;

// fals

2.2 Aplicații ale Inegalității rearanjamentelor

2.2.1 Inegalitatea rearanjamentelor

Teorema 2.2.1 (Inegalitatea rearanjamentelor). *Fie șirurile crescătoare de numere reale*

$$a_1 \leq a_2 \leq \dots \leq a_n \text{ și } b_1 \leq b_2 \leq \dots \leq b_n, \quad n \in \mathbb{N}^*.$$

Atunci, pentru orice permutare $p \in S_n$ ($S_n =$ grupul permutărilor de ordin n), avem

$$\sum_{i=1}^n a_i \cdot b_{n+1-i} \leq \sum_{i=1}^n a_i \cdot b_{p(i)} \leq \sum_{i=1}^n a_i \cdot b_i. \quad (2.2.1)$$

Demonstrație. Pentru orice permutare $p \in S_n$, notăm

$$s(p) = \sum_{i=1}^n a_i \cdot b_{p(i)}.$$

Fie

$$M = \max_{p \in S_n} s(p). \quad (2.2.2)$$

Demonstrăm că pentru orice $k \in \{0, 1, \dots, n\}$ există $p \in S_n$ astfel încât

$$s(p) = M \text{ și } p(i) = i \quad \forall 1 \leq i \leq k, \quad (2.2.3)$$

prin inducție după k .

Pentru $k = 0$ afirmația este evidentă, luând orice permutare $p \in S_n$ astfel încât $s(p) = M$.

Presupunem (2.2.3) adevărată pentru $k - 1$, adică există $p \in S_n$ astfel încât

$$s(p) = M \text{ și } p(i) = i \quad \forall 1 \leq i \leq k - 1$$

și o demonstrăm pentru k ($k \in \{1, 2, \dots, n\}$).

Avem două cazuri.

Cazul 1) $p(k) = k$. Atunci $p(i) = i \forall 1 \leq i \leq k$.

Cazul 2) $p(k) \neq k$. Cum p este o permutare și $p(i) = i \forall 1 \leq i \leq k-1$, rezultă că $p(k) > k$ și există $j > k$ a.î. $p(j) = k$. Definim permutarea $p' \in S_n$ prin

$$\begin{cases} p'(k) &= p(j), \\ p'(j) &= p(k), \\ p'(i) &= p(i), \forall i \in \{1, \dots, n\} \setminus \{k, j\}. \end{cases}$$

Avem

$$\begin{aligned} s(p') - s(p) &= \sum_{i=1}^n a_i \cdot b_{p'(i)} - \sum_{i=1}^n a_i \cdot b_{p(i)} \\ &= a_k \cdot b_{p'(k)} + a_j \cdot b_{p'(j)} - a_k \cdot b_{p(k)} - a_j \cdot b_{p(j)} \\ &= a_k \cdot b_{p(j)} + a_j \cdot b_{p(k)} - a_k \cdot b_{p(k)} - a_j \cdot b_{p(j)} \\ &= (a_j - a_k)(b_{p(k)} - b_{p(j)}) \\ &= \underbrace{(a_j - a_k)}_{\geq 0} \underbrace{(b_{p(k)} - b_k)}_{\geq 0} \geq 0 \end{aligned}$$

(deoarece $j > k$, $p(k) > k$, iar șirurile $(a_i)_{i=\overline{1,n}}$ și $(b_i)_{i=\overline{1,n}}$ sunt crescătoare). Deci $s(p') \geq s(p) = M$.

Cum, conform (2.2.2), avem $s(p') \leq M$, rezultă că

$$s(p') = s(p) = M$$

(în plus, $a_j = a_k$ sau $b_{p(k)} = b_k$).

Evident, $p'(i) = i \forall 1 \leq i \leq k$, deci relația (2.2.3) este adevărată pentru k , ceea ce încheie demonstrația prin inducție a acestei relații.

Luând $k = n$ în această relație rezultă că

$$s(e) = M, \tag{2.2.4}$$

unde $e \in S_n$ este permutarea identică, definită prin $e(i) = i \forall i \in \{1, \dots, n\}$.

Fie $p \in S_n$ o permutare arbitrară. Din (2.2.2) și (2.2.4) rezultă că $s(p) \leq s(e)$, adică

$$\sum_{i=1}^n a_i \cdot b_{p(i)} \leq \sum_{i=1}^n a_i \cdot b_i.$$

Aplicând această inegalitate pentru șirurile crescătoare

$$a_1 \leq a_2 \leq \dots \leq a_n \text{ și } -b_n \leq -b_{n-1} \leq \dots \leq -b_1$$

rezultă că

$$\sum_{i=1}^n a_i \cdot (-b_{p(i)}) \leq \sum_{i=1}^n a_i \cdot (-b_{n+1-i}),$$

adică

$$\sum_{i=1}^n a_i \cdot b_{n+1-i} \leq \sum_{i=1}^n a_i \cdot b_{p(i)}.$$

□

2.2.2 Produs scalar maxim/minim

Se consideră şirurile de numere reale

$$a_1, a_2, \dots, a_n \text{ şi } b_1, b_2, \dots, b_n, \quad n \in \mathbb{N}^*.$$

Se cere să se determine două permutări $a_{q(1)}, a_{q(2)}, \dots, a_{q(n)}$ şi $b_{p(1)}, b_{p(2)}, \dots, b_{p(n)}$ ale celor două şiruri, $q, p \in S_n$ ($S_n =$ grupul permutărilor de ordin n), astfel

încât suma $\sum_{i=1}^n a_{q(i)} \cdot b_{p(i)}$ să fie

- a) maximă;
- b) minimă.

Observația 2.2.1. Suma $\sum_{i=1}^n a_{q(i)} \cdot b_{p(i)}$ reprezintă **produsul scalar** al vectorilor $(a_{q(i)})_{i=1, \dots, n}$ şi $(b_{p(i)})_{i=1, \dots, n}$. Astfel problema anterioară cere determinarea unor permutări ale elementelor vectorilor $(a_1, a_2, \dots, a_n)$ şi $(b_1, b_2, \dots, b_n)$ astfel încât după permutare produsul lor scalar să fie maxim, respectiv minim.

Rezolvarea problemei de maxim

Algoritmul 2.2.1. Conform Teoremei 2.2.1 deducem următoarea *strategie Greedy* în varianta I pentru rezolvarea problemei:

- Pentru obținerea celor n termeni ai sumei maxime, la fiecare pas $i = \overline{1, n}$ luăm produsul dintre:
 - cel mai mic dintre termenii şirului $(a_1, a_2, \dots, a_n)$ neales la pașii anteriori;
 - cel mai mic dintre termenii şirului $(b_1, b_2, \dots, b_n)$ neales la pașii anteriori.

Descrierea în pseudocod a algoritmului are următoarea formă.

```

MAXIM1 ( $a, b, n, s$ ) :           //  $a = (a_1, \dots, a_n)$ ,  $b = (b_1, \dots, b_n)$ 
 $s \leftarrow 0$ ;                      //  $s =$  suma maximă
for  $i = \overline{1, n}$  do                // pasul  $i$ 
     $k \leftarrow i$ ;                // calculăm termenul minim  $a_k$  din  $(a_i, \dots, a_n)$ 
     $m \leftarrow a[i]$ ;
    for  $j = i + 1, n$  do
        if  $a[j] < m$  then
             $k \leftarrow j$ ;
             $m \leftarrow a[j]$ ;
     $a[i] \leftrightarrow a[k]$ ;            // interschimbăm termenii  $a_i$  și  $a_k$ 
     $k \leftarrow i$ ;                // calculăm termenul minim  $b_k$  din  $(b_i, \dots, b_n)$ 
     $m \leftarrow b[i]$ ;
    for  $j = i + 1, n$  do
        if  $b[j] < m$  then
             $k \leftarrow j$ ;
             $m \leftarrow b[j]$ ;
     $b[i] \leftrightarrow b[k]$ ;            // interschimbăm termenii  $b_i$  și  $b_k$ 
     $s \leftarrow s + a[i] \cdot b[i]$ ; // adunăm produsul termenilor minimi
                                   // la suma  $s$ 
AFISARE( $s, a, b, n$ );           // se afișează suma maximă și
                                   // permutările obținute

```

Funcția de afișare este

```

AFISARE ( $s, a, b, n$ ) :
    afișează  $s$ ;
    for  $i = \overline{1, n}$  do
        afișează  $a[i]$ ;
    for  $i = \overline{1, n}$  do
        afișează  $b[i]$ ;

```

Observația 2.2.2. Algoritmul necesită câte două comparații și câte cel mult patru atribuiri pentru fiecare pereche de indici (i, j) cu $i \in \{1, 2, \dots, n\}$ și $j \in \{i + 1, i + 2, \dots, n\}$. Numărul acestor perechi este $(n - 1) + (n - 2) + \dots + 2 + 1 = \frac{n(n - 1)}{2}$, deci algoritmul are complexitatea $\Theta(n^2)$.

Algoritmul 2.2.2. Conform Teoremei 2.2.1 obținem și următoarea *strategie Greedy în varianta a II-a* pentru rezolvarea problemei:

- ordonăm crescător elementele primului șir: $a_1 \leq a_2 \leq \dots \leq a_n$;

- ordonăm crescător elementele celui de-al doilea șir: $b_1 \leq b_2 \leq \dots \leq b_n$;
- suma maximă este $s = \sum_{i=1}^n a_i \cdot b_i$.

Descrierea în pseudocod a algoritmului are următoarea formă.

```

MAXIM2 ( $a, b, n, s$ ) :           //  $a = (a_1, \dots, a_n)$ ,  $b = (b_1, \dots, b_n)$ 
SORTARE ( $a, n$ );                // se sortează crescător vectorul  $a$ 
SORTARE ( $b, n$ );                // se sortează crescător vectorul  $b$ 
 $s \leftarrow 0$ ;                    //  $s$  = suma maximă
for  $i = \overline{1, n}$  do             // calculăm suma maximă  $s$ 
     $s \leftarrow s + a[i] \cdot b[i]$ ;
AFISARE ( $s, a, b, n$ );           // se afișează suma maximă și
                                   // permutările obținute

```

unde funcția de afișare este aceeași ca în Algoritmul 2.2.1.

Observația 2.2.3. Algoritmul anterior are complexitatea $\Theta(n \log_2 n)$, deoarece necesită sortarea celor doi vectori de dimensiune n . Rezultă că Algoritmul 2.2.2 este mai eficient decât Algoritmul 2.2.1.

Rezolvarea problemei de minim

Algoritmul 2.2.3. Conform Teoremei 2.2.1 deducem următoarea *strategie Greedy* în varianta I pentru rezolvarea problemei de minim:

- Pentru obținerea celor n termeni ai sumei minime, la fiecare pas $i = \overline{1, n}$ luăm produsul dintre:
 - cel mai mic dintre termenii șirului $(a_1, a_2, \dots, a_n)$ neales la pașii anteriori;
 - cel mai mare dintre termenii șirului $(b_1, b_2, \dots, b_n)$ neales la pașii anteriori.

Descrierea în pseudocod a algoritmului are următoarea formă.

```

MINIM1 ( $a, b, n, s$ ):           //  $a = (a_1, \dots, a_n)$ ,  $b = (b_1, \dots, b_n)$ 
 $s \leftarrow 0$ ;                     //  $s = \text{suma minimă}$ 
for  $i = \overline{1, n}$  do               // pasul  $i$ 
|    $k \leftarrow i$ ;                 // calculăm termenul minim  $a_k$  din  $(a_i, \dots, a_n)$ 
|    $m \leftarrow a[i]$ ;
|   for  $j = \overline{i+1, n}$  do
|   |   if  $a[j] < m$  then
|   |   |    $k \leftarrow j$ ;
|   |   |    $m \leftarrow a[j]$ ;
|    $a[i] \leftrightarrow a[k]$ ;         // interschimbăm termenii  $a_i$  și  $a_k$ 
|    $k \leftarrow i$ ;                 // calculăm termenul maxim  $b_k$  din  $(b_i, \dots, b_n)$ 
|    $m \leftarrow b[i]$ ;
|   for  $j = \overline{i+1, n}$  do
|   |   if  $b[j] > m$  then
|   |   |    $k \leftarrow j$ ;
|   |   |    $m \leftarrow b[j]$ ;
|    $b[i] \leftrightarrow b[k]$ ;         // interschimbăm termenii  $b_i$  și  $b_k$ 
|    $s \leftarrow s + a[i] \cdot b[i]$ ; // adunăm produsul termenilor calculați
|                                   // la suma  $s$ 
AFISARE( $s, a, b, n$ );           // se afișează suma minimă și
                                   // permutările obținute

```

Funcția de afișare este aceeași ca în Algoritmul 2.2.1.

Algoritmul 2.2.4. Conform Teoremei 2.2.1 obținem și următoarea *strategie Greedy în varianta a II-a* pentru rezolvarea problemei de minim:

- ordonăm crescător elementele primului șir: $a_1 \leq a_2 \leq \dots \leq a_n$;
- ordonăm descrescător elementele celui de-al doilea șir: $b_1 \geq b_2 \geq \dots \geq b_n$;
- suma minimă este $s = \sum_{i=1}^n a_i \cdot b_i$.

Descrierea în pseudocod a algoritmului are următoarea formă.

```

MINIM2 ( $a, b, n, s$ ) :           //  $a = (a_1, \dots, a_n)$ ,  $b = (b_1, \dots, b_n)$ 
SORTARE1 ( $a, n$ );               // se sortează crescător vectorul  $a$ 
SORTARE2 ( $b, n$ );               // se sortează descrescător vectorul  $b$ 
 $s \leftarrow 0$ ;                    //  $s$  = suma minimă
for  $i = \overline{1, n}$  do             // calculăm suma minimă  $s$ 
     $s \leftarrow s + a[i] \cdot b[i]$ ;
AFISARE ( $s, a, b, n$ );           // se afișează suma minimă și
                                   // permutările obținute

```

unde funcția de afișare este aceeași ca în Algoritmul 2.2.1.

Observația 2.2.4. Analog problemei de maxim, Algoritmul 2.2.3 are complexitatea $\Theta(n^2)$, iar Algoritmul 2.2.4 are complexitatea $\Theta(n \log_2 n)$, fiind astfel mai eficient decât Algoritmul 2.2.3.

2.2.3 Memorarea optimă a textelor pe benzi

Se dă o bandă magnetică suficient de lungă pentru a memora n texte (sau fișiere)

$$T_1, T_2, \dots, T_n$$

de lungimi date (de exemplu, în octeți)

$$L_1, L_2, \dots, \text{ respectiv } L_n.$$

La citirea unui text de pe bandă, trebuie citite și textele aflate înaintea lui.

Presupunând că frecvența de citire a celor n texte este aceeași, se cere să se determine o ordine de poziționare (memorare) optimă a acestora pe bandă, adică o poziționare astfel încât timpul mediu de citire să fie minim.

Modelarea problemei

- Evident, orice poziționare a celor n texte pe bandă este o permutare a vectorului $(T_1, T_2, \dots, T_n)$, adică are forma

$$(T_{p(1)}, T_{p(2)}, \dots, T_{p(n)}),$$

unde $p \in S_n$ este o permutare de ordin n (pentru orice $i \in \{1, \dots, n\}$, pe poziția i pe bandă se memorează textul $T_{p(i)}$).

- Evident, timpul de citire doar a unui text T_k este direct proporțional cu lungimea lui, deci putem considera că acest timp este egal cu lungimea L_k a textului.

- Pentru orice $k \in \{1, \dots, n\}$, citirea textului $T_{p(k)}$ necesită timpul

$$t_k = \sum_{i=1}^k L_{p(i)},$$

deoarece la timpul de citire efectivă a textului $T_{p(k)}$ trebuie adăugați și timpii de citire a textelor precedente $T_{p(1)}, T_{p(2)}, \dots, T_{p(k-1)}$.

- Frecvența de citire a celor n texte fiind aceeași, rezultă că *timpul mediu de citire* pentru o poziționare $p \in S_n$ este

$$\begin{aligned} t(p) &= \frac{1}{n} \sum_{k=1}^n t_k \\ &= \frac{1}{n} \sum_{k=1}^n \sum_{i=1}^k L_{p(i)} \\ &= \frac{1}{n} \sum_{i=1}^n \sum_{k=i}^n L_{p(i)} \\ &= \frac{1}{n} \sum_{i=1}^n \left(L_{p(i)} \sum_{k=i}^n 1 \right) \\ &= \frac{1}{n} \sum_{i=1}^n (n - i + 1) \cdot L_{p(i)}. \end{aligned}$$

Propoziția 2.2.1. Dacă $L_1 \leq L_2 \leq \dots \leq L_n$, atunci

$$\min_{p \in S_n} t(p) = t(e),$$

adică poziționarea corespunzătoare permutării identice este optimă.

Demonstrație. Aplicând Teorema 2.2.1 pentru șirurile crescătoare

$$-\frac{n}{n} < -\frac{n-1}{n} < \dots < -\frac{1}{n} \text{ și } L_1 \leq L_2 \leq \dots \leq L_n,$$

rezultă că pentru orice permutare $p \in S_n$ avem

$$\sum_{i=1}^n \left(-\frac{n-i+1}{n} \right) \cdot L_{p(i)} \leq \sum_{i=1}^n \left(-\frac{n-i+1}{n} \right) \cdot L_i,$$

deci

$$\frac{1}{n} \sum_{i=1}^n (n-i+1) \cdot L_{p(i)} \geq \frac{1}{n} \sum_{i=1}^n (n-i+1) \cdot L_i,$$

adică $t(p) \geq t(e)$. Rezultă că $\min_{p \in S_n} t(p) = t(e)$. □

Observația 2.2.5. Mai mult, deoarece șirul

$$-\frac{n}{n} < -\frac{n-1}{n} < \dots < -\frac{1}{n}$$

este strict crescător, conform demonstrației Teoremei 2.2.1 rezultă că orice poziționare optimă presupune memorarea textelor pe bandă în ordinea crescătoare a lungimilor lor.

Observația 2.2.6. Conform propoziției anterioare deducem următoarea *strategie Greedy* pentru rezolvarea problemei:

Varianta I: La fiecare pas $i = \overline{1, n}$ se poziționează pe bandă pe poziția curentă, i , textul de lungime minimă dintre cele nepoziționate la pașii anteriori;

Varianta II: Se sortează textele în ordinea crescătoare a lungimilor lor și se poziționează pe bandă în această ordine.

Analog problemelor de la secțiunea anterioară, Varianta I are complexitatea $\Theta(n^2)$, iar Varianta II are complexitatea $\Theta(n \log_2 n)$ (fiind astfel mai eficientă decât Varianta I).

2.3 Problema rucsacului, varianta continuă

Problema rucsacului (Knapsack) este următoarea:

Se consideră un rucsac în care se poate încărca greutatea maximă G , unde $G > 0$, și n obiecte $O_1, \dots, O_n$, $n \in \mathbb{N}^*$. Pentru fiecare obiect O_i , $i \in \{1, \dots, n\}$, se cunoaște greutatea sa, g_i , unde $g_i > 0$, și câștigul obținut la transportul său în întregime, c_i , unde $c_i > 0$.

Se cere să se determine o modalitate de încărcare a rucsacului cu obiecte astfel încât câștigul total al obiectelor încărcate să fie maxim.

În **varianta continuă (fracționară)** a problemei, pentru fiecare obiect O_i poate fi încărcată orice parte (fracțiune) $x_i \in [0, 1]$ din el, câștigul obținut fiind proporțional cu partea încărcată, adică este egal cu $x_i c_i$.

Modelarea problemei

- O *soluție (soluție posibilă)* a problemei este orice vector $x = (x_1, \dots, x_n)$ astfel încât

$$\begin{cases} x_i \in [0, 1], \forall i \in \{1, \dots, n\}, \\ \sum_{i=1}^n x_i g_i \leq G, \end{cases}$$

ultima inegalitate exprimând faptul că greutatea totală încărcată în rucsac nu trebuie să depășească greutatea maximă.

- Câștigul (*total*) corespunzător soluției $x = (x_1, \dots, x_n)$ este

$$f(x) = \sum_{i=1}^n x_i c_i.$$

- O soluție optimă a problemei este orice soluție $x^* = (x_1^*, \dots, x_n^*)$ astfel încât

$$f(x^*) = \max\{f(x) \mid x = \text{soluție a problemei}\}.$$

- Dacă suma greutatea tuturor obiectelor este mai mică sau egală cu greutatea maximă a rucsacului, adică $\sum_{i=1}^n g_i \leq G$, atunci problema este trivială, soluția $x^* = (1, 1, \dots, 1)$, corespunzătoare încărcării integrale a tuturor celor n obiecte în rucsac, fiind evident singura soluție optimă. Astfel în continuare putem presupune că

$$\sum_{i=1}^n g_i > G. \quad (2.3.1)$$

- Pentru orice soluție optimă $x^* = (x_1^*, \dots, x_n^*)$ avem

$$\sum_{i=1}^n x_i^* g_i = G \quad (2.3.2)$$

(adică rucsacul trebuie încărcat complet). Demonstrăm această afirmație prin reducere la absurd. Într-adevăr, dacă $\sum_{i=1}^n x_i^* g_i < G$, cum $\sum_{i=1}^n g_i > G$ rezultă că există un indice $k \in \{1, \dots, n\}$ a.î. $x_k^* < 1$. Considerând vectorul $x' = (x'_1, \dots, x'_n)$ definit prin

$$x'_i = \begin{cases} x_i^*, & \text{dacă } i \neq k, \\ x_i^* + u, & \text{dacă } i = k, \end{cases}$$

unde

$$u = \min \left\{ 1 - x_k^*, \frac{1}{g_k} \left(G - \sum_{i=1}^n x_i^* g_i \right) \right\},$$

avem $u > 0$, $x'_k \leq 1$, $\sum_{i=1}^n x'_i g_i = \sum_{i=1}^n x_i^* g_i + u g_k \leq \sum_{i=1}^n x_i^* g_i + G - \sum_{i=1}^n x_i^* g_i = G$

și $f(x') = \sum_{i=1}^n x'_i c_i = \sum_{i=1}^n x_i^* c_i + u c_k = f(x^*) + u c_k$, deci x' este o soluție a problemei și $f(x') > f(x^*)$, ceea ce contrazice optimalitatea soluției x^* .

Prezentăm în continuare un *algorithm Greedy* pentru rezolvarea problemei.

Algoritmul 2.3.1. Vom utiliza următoarea *strategie Greedy*:

- Ordonăm obiectele descrescător după câștigul lor unitar:

$$\frac{c_1}{g_1} \geq \frac{c_2}{g_2} \geq \dots \geq \frac{c_n}{g_n}. \quad (2.3.3)$$

- Încărcăm obiectele în rucsac, în această ordine, cât timp nu se depășește greutatea maximă G . Încărcarea obiectelor se face în întregime, cât timp este posibil; în acest fel doar ultimul obiect adăugat poate fi încărcat parțial.

Descrierea în pseudocod a algoritmului are următoarea formă.

```

RUCSAC ( $G, n, g, c, x, C$ ):      //  $g = (g_1, \dots, g_n)$ ,  $c = (c_1, \dots, c_n)$ 
                                   //  $C =$  câștigul total
SORTARE( $g, c, n$ );           // se sortează obiectele descrescător
                                   // după câștigul lor unitar

 $C \leftarrow 0$ ;
for  $i = \overline{1, n}$  do  $x[i] \leftarrow 0$ ;
 $R \leftarrow G$ ;                //  $R =$  greutatea disponibilă pentru rucsac
 $i \leftarrow 1$ ;
while  $R > 0$  do                // rucsacul nu este plin
    if  $g[i] \leq R$  then
        // obiectul curent încapă în întregime, deci
        // se adaugă în rucsac
         $x[i] \leftarrow 1$ ;
         $C \leftarrow C + c[i]$ ;
         $R \leftarrow R - g[i]$ ;      // actualizăm greutatea disponibilă
         $i \leftarrow i + 1$ ;        // trecem la obiectul următor
    else
        // obiectul curent nu încapă în întregime, deci
        // se adaugă exact acea parte din el care
        // umple rucsacul și încărcarea se încheie
         $x[i] \leftarrow \frac{R}{g[i]}$ ;
         $C \leftarrow C + x[i]c[i]$ ;
         $R \leftarrow 0$ ;

AFISARE( $C, x, n$ );           // se afișează câștigul total maxim  $C$ 
                                   // și soluția optimă  $x = (x_1, \dots, x_n)$ 

```

Teorema 2.3.1 (de corectitudine a Algoritmului 2.3.1). *În contextul Algoritmului 2.3.1, vectorul $x = (x_1, \dots, x_n)$ calculat de algoritm este o soluție optimă a problemei rucsacului.*

Demonstrație. Evident, vectorul $x = (x_1, \dots, x_n)$ calculat de algoritm verifică relațiile

$$\begin{cases} x_i \in [0, 1], \forall i \in \{1, \dots, n\}, \\ \sum_{i=1}^n x_i g_i = G, \end{cases}$$

deci este o soluție a problemei. Rămâne să demonstrăm optimalitatea acestei soluții.

Demonstrăm prin inducție după $k \in \{0, 1, \dots, n\}$ că există o soluție optimă $x^* = (x_1^*, \dots, x_n^*)$ a problemei pentru care

$$x_i = x_i^*, \forall i \text{ a.î. } 1 \leq i \leq k. \quad (2.3.4)$$

Pentru $k = 0$ afirmația este evidentă, luând x^* orice soluție optimă a problemei.

Presupunem (2.3.4) adevărată pentru $k - 1$, adică există o soluție optimă $x^* = (x_1^*, \dots, x_n^*)$ a problemei pentru care

$$x_i = x_i^*, \forall i \text{ a.î. } 1 \leq i \leq k - 1$$

și o demonstrăm pentru k ($k \in \{1, 2, \dots, n\}$).

Cum

$$\sum_{i=1}^{k-1} x_i g_i + x_k^* g_k = \sum_{i=1}^{k-1} x_i^* g_i + x_k^* g_k \leq \sum_{i=1}^n x_i^* g_i = G,$$

din descrierea algoritmului (alegerea maximală a lui x_k) rezultă că

$$x_k \geq x_k^*.$$

Avem două cazuri.

Cazul 1) $x_k = x_k^*$. Atunci $x_i = x_i^*, \forall i \in \{1, \dots, k\}$, deci (2.3.4) este adevărată pentru k .

Cazul 2) $x_k > x_k^*$. În acest caz avem $k < n$, deoarece dacă, prin reducere la absurd, am avea $k = n$, atunci ar rezulta că

$$f(x) = \sum_{i=1}^n x_i c_i = \sum_{i=1}^{n-1} x_i^* c_i + x_n c_n > \sum_{i=1}^{n-1} x_i^* c_i + x_n^* c_n = f(x^*),$$

ceea ce contrazice optimalitatea soluției x^* .

Definim vectorul $x^{**} = (x_1^{**}, \dots, x_n^{**})$ prin

$$x_i^{**} = \begin{cases} x_i, & \text{dacă } 1 \leq i \leq k, \\ \alpha^* x_i^*, & \text{dacă } k+1 \leq i \leq n, \end{cases} \quad (2.3.5)$$

unde $\alpha^* \in [0, 1)$ este o soluție a ecuației

$$h(\alpha) = 0, \text{ unde } h(\alpha) = \sum_{i=1}^k x_i g_i + \alpha \sum_{i=k+1}^n x_i^* g_i - G. \quad (2.3.6)$$

O astfel de soluție există, deoarece

$$\begin{aligned} h(0) &= \sum_{i=1}^k x_i g_i - G \leq \sum_{i=1}^n x_i g_i - G = G - G = 0, \\ h(1) &= \sum_{i=1}^k x_i g_i + \sum_{i=k+1}^n x_i^* g_i - G = \sum_{i=1}^{k-1} x_i^* g_i + x_k g_k + \sum_{i=k+1}^n x_i^* g_i - G \\ &= \sum_{i=1}^n x_i^* g_i - x_k^* g_k + x_k g_k - G = G + g_k(x_k - x_k^*) - G \\ &= g_k(x_k - x_k^*) > 0, \end{aligned}$$

iar h este o funcție continuă pe intervalul $[0, 1]$.

Conform (2.3.5) și (2.3.6) rezultă că $x_i^{**} \in [0, 1]$, $\forall i \in \{1, \dots, n\}$ și

$$\sum_{i=1}^n x_i^{**} g_i = \sum_{i=1}^k x_i g_i + \alpha^* \sum_{i=k+1}^n x_i^* g_i = h(\alpha^*) + G = 0 + G = G,$$

deci vectorul $x^{**} = (x_1^{**}, \dots, x_n^{**})$ este o soluție a problemei.

Avem

$$\begin{aligned} f(x^{**}) - f(x^*) &= \sum_{i=1}^{k-1} x_i^* c_i + x_k c_k + \alpha^* \sum_{i=k+1}^n x_i^* c_i - \sum_{i=1}^n x_i^* c_i \\ &= x_k c_k + \alpha^* \sum_{i=k+1}^n x_i^* c_i - \sum_{i=k}^n x_i^* c_i \\ &= x_k c_k + \alpha^* \sum_{i=k+1}^n x_i^* c_i - x_k^* c_k - \sum_{i=k+1}^n x_i^* c_i \\ &= c_k(x_k - x_k^*) - (1 - \alpha^*) \sum_{i=k+1}^n x_i^* c_i \\ &= \frac{c_k}{g_k} \cdot (x_k g_k - x_k^* g_k) - (1 - \alpha^*) \sum_{i=k+1}^n \frac{c_i}{g_i} \cdot x_i^* g_i. \end{aligned} \quad (2.3.7)$$

Conform (2.3.3) rezultă că

$$\frac{c_i}{g_i} \leq \frac{c_k}{g_k}, \forall i \in \{k+1, \dots, n\}. \quad (2.3.8)$$

Din (2.3.7) și (2.3.8) obținem că

$$\begin{aligned} f(x^{**}) - f(x^*) &\geq \frac{c_k}{g_k} \cdot \left[(x_k g_k - x_k^* g_k) - (1 - \alpha^*) \sum_{i=k+1}^n x_i^* g_i \right] \\ &= \frac{c_k}{g_k} \cdot \left(x_k g_k - x_k^* g_k - \sum_{i=k+1}^n x_i^* g_i + \alpha^* \sum_{i=k+1}^n x_i^* g_i \right) \\ &= \frac{c_k}{g_k} \cdot \left[x_k g_k - \left(\sum_{i=1}^n x_i^* g_i - \sum_{i=1}^{k-1} x_i^* g_i \right) + \alpha^* \sum_{i=k+1}^n x_i^* g_i \right] \\ &= \frac{c_k}{g_k} \cdot \left(x_k g_k - G + \sum_{i=1}^{k-1} x_i g_i + \alpha^* \sum_{i=k+1}^n x_i^* g_i \right) \\ &= \frac{c_k}{g_k} \cdot \left(\sum_{i=1}^k x_i g_i + \alpha^* \sum_{i=k+1}^n x_i^* g_i - G \right), \end{aligned}$$

și conform (2.3.6) rezultă că

$$f(x^{**}) - f(x^*) \geq \frac{c_k}{g_k} \cdot h(\alpha^*) = 0,$$

deci

$$f(x^{**}) \geq f(x^*).$$

Cum x^* este soluție optimă, rezultă că și x^{**} este soluție optimă (și, în plus, $f(x^{**}) = f(x^*)$). Conform (2.3.5) avem

$$x_i = x_i^{**}, \forall i \in \{1, \dots, k\},$$

deci relația (2.3.4) este adevărată pentru k , ceea ce încheie demonstrația prin inducție a acestei relații.

Luând $k = n$ în această relație rezultă că există o soluție optimă $x^* = (x_1^*, \dots, x_n^*)$ pentru care

$$x_i = x_i^*, \forall i \in \{1, \dots, n\},$$

deci $x = x^*$ și astfel x este o soluție optimă a problemei. $\square$

Exemplul 2.3.1. Considerăm un rucsac în care se poate încărca o greutate maximă $G = 40$, din $n = 10$ obiecte ce au greutatea și câștigurile date în următorul tabel:

Obiect	O_1	O_2	O_3	O_4	O_5	O_6	O_7	O_8	O_9	O_{10}
Greutate g_i	10	7	10	5	6	10	8	15	3	12
Câștig c_i	27	9	40	20	11	20	50	22	4	33

Ordinea descrescătoare a obiectelor după câștigul unitar c_i/g_i este evidențiată în următorul tabel:

Obiect	O_7	O_3	O_4	O_{10}	O_1	O_6	O_5	O_8	O_9	O_2
Câștig c_i	50	40	20	33	27	20	11	22	4	9
Greutate g_i	8	10	5	12	10	10	6	15	3	7

Aplicarea strategiei Greedy (algoritmul de mai sus) conduce la soluția optimă

$$x = (1, 1, 1, 1, 5/10, 0, 0, 0, 0, 0),$$

adică umplem rucsacul încărcând, în ordine, obiectele:

- O_7 , după care greutatea disponibilă devine $R = 40 - 8 = 32$,
- O_3 , după care $R = 32 - 10 = 22$,
- O_4 , după care $R = 22 - 5 = 17$,
- O_{10} după care $R = 17 - 12 = 5$,
- $5/10$ din O_1 , după care $R = 5 - 5 = 0$.

Câștigul (total) obținut este

$$f(x) = 50 + 40 + 20 + 33 + \frac{5}{10} \cdot 27 = 156,5.$$

Observația 2.3.1. Algoritmul 2.3.1 are complexitatea $\mathcal{O}(n \log_2 n)$, deoarece este necesară sortarea obiectelor descrescător după câștigul unitar iar blocul "while" se execută de cel mult n ori (câte o dată pentru fiecare obiect) și necesită de fiecare dată o comparație și 3 operații aritmetice.

Observația 2.3.2. În **varianta discretă a problemei rucsacului**, fiecare obiect O_i poate fi încărcat doar în întregime. În această variantă, **soluția produsă de strategia Greedy (de mai sus) nu este neapărat optimă!**

De exemplu, pentru datele din exemplul anterior, aplicarea strategiei Greedy conduce la soluția

$$x = (1, 1, 1, 1, 0, 0, 0, 0, 1, 0),$$

adică încărcăm în rucsac, în ordine, obiectele:

- O_7 , după care greutatea disponibilă devine $R = 40 - 8 = 32$,
- O_3 , după care $R = 32 - 10 = 22$,
- O_4 , după care $R = 22 - 5 = 17$,
- O_{10} după care $R = 17 - 12 = 5$,
- O_9 , după care $R = 5 - 3 = 2$ și nu mai există niciun obiect care să mai încapă în rucsac, deci încărcarea se încheie.

Câștigul (total) obținut este

$$f(x) = 50 + 40 + 20 + 33 + 4 = 147.$$

Soluția obținută nu este optimă, o soluție mai bună fiind

$$x' = (1, 1, 1, 0, 1, 0, 1, 0, 0, 0),$$

corespunzătoare încărcării obiectelor O_7 , O_3 , O_4 , O_1 și O_5 , având greutatea totală $8 + 10 + 5 + 10 + 6 = 39$ (deci rucsacul nu este plin) și câștigul (total)

$$f(x') = 50 + 40 + 20 + 27 + 11 = 148.$$

2.4 Problema planificării spectacolelor

Problema planificării spectacolelor este următoarea:

Se consideră n spectacole $S_1, \dots, S_n$, $n \in \mathbb{N}^*$. Pentru fiecare spectacol S_i , $i \in \{1, \dots, n\}$, se cunoaște intervalul orar $I_i = [a_i, b_i]$ de desfășurare, unde $a_i < b_i$.

O persoană dorește să vizioneze cât mai multe dintre aceste n spectacole. Fiecare spectacol trebuie vizionat integral, nu pot fi vizionate simultan mai multe spectacole, iar timpii necesari deplasării de la un spectacol la altul sunt nesemnificativi (egali cu zero).

Se cere să se selecteze un număr cât mai mare de spectacole ce pot fi vizionate de o singură persoană, cu respectarea cerințelor de mai sus.

Modelarea problemei

- O *soluție* (*soluție posibilă*) a problemei este orice submulțime $P \subseteq \{I_1, \dots, I_n\}$ astfel încât

$$I_i \cap I_j = \emptyset, \forall I_i, I_j \in P, i \neq j$$

(adică orice submulțime de intervale disjuncte două câte două).

- O *soluție optimă* a problemei este orice soluție $P^* \subseteq \{I_1, \dots, I_n\}$ astfel încât

$$\text{card}(P^*) = \max\{\text{card}(P) \mid P = \text{soluție a problemei}\}.$$

Prezentăm în continuare doi *algoritmi Greedy* pentru rezolvarea problemei.

Algoritmul 2.4.1. Vom utiliza următoarea *strategie Greedy*:

- Ordonăm spectacolele crescător după timpul lor de încheiere:

$$b_1 \leq b_2 \leq \dots \leq b_n. \quad (2.4.1)$$

- Parcurgem spectacolele, în această ordine, și:

- selectăm primul spectacol;
- de fiecare dată, spectacolul curent, S_i , se selectează doar dacă nu se suprapune cu niciunul dintre spectacolele selectate anterior, adică dacă timpul său de începere este mai mare decât timpul de încheiere al ultimului spectacol S_j selectat:

$$a_i > b_j.$$

Pentru memorarea soluției utilizăm un vector caracteristic $c = (c_1, \dots, c_n)$, cu semnificația

$$c_i = \begin{cases} 1, & \text{dacă intervalul } I_i \text{ a fost selectat,} \\ 0, & \text{în caz contrar.} \end{cases}$$

Descrierea în pseudocod a algoritmului are următoarea formă.

```

SPECTACOLE1 ( $a, b, n, c, m$ ): //  $a = (a_1, \dots, a_n)$ ,  $b = (b_1, \dots, b_n)$ 
    //  $c = (c_1, \dots, c_n)$ ,  $m = \text{numărul de spectacole selectate}$ 
SORTARE( $a, b, n$ ); // se sortează spectacolele crescător
    // după timpul lor de încheiere  $b_i$ 
     $m \leftarrow 0$ ; // inițializări
    for  $i = \overline{1, n}$  do  $c[i] \leftarrow 0$ ;
     $t \leftarrow a[1] - 1$ ; //  $t = \text{timpul de încheiere al ultimului}$ 
    // spectacol selectat
    for  $i = \overline{1, n}$  do // parcurgem spectacolele
        if  $a[i] > t$  then
             $c[i] \leftarrow 1$ ; // selectăm intervalul (spectacolul) curent
             $m \leftarrow m + 1$ ;
             $t \leftarrow b[i]$ ; // actualizăm  $t$ 
AFISARE( $m, c, n$ ); // se afișează numărul și
    // submulțimea intervalelor (spectacolelor) selectate

```

Funcția de afișare este

AFISARE (m, c, n) :

afișează m ;

for $i = \overline{1, n}$ **do**

if $c[i] = 1$ **then** **afișează** $[a[i], b[i]]$;

Teorema 2.4.1 (de corectitudine a Algoritmului 2.4.1). *În contextul Algoritmului 2.4.1, submulțimea intervalelor (spectacolelor) selectate de algoritm este o soluție optimă a problemei planificării spectacolelor.*

Demonstrație. Fie $P = \{I'_1, \dots, I'_m\}$ submulțimea de intervale calculată de algoritm, unde

$$I'_1 = [a'_1, b'_1], I'_2 = [a'_2, b'_2], \dots, I'_m = [a'_m, b'_m]$$

sunt intervalele selectate, în această ordine, de algoritm.

Evident, $m \geq 1$ (după sortare, primul interval este întotdeauna selectat).

Din descrierea algoritmului (alegerea intervalului curent I'_i) rezultă că

$$a'_i > b'_{i-1}, \forall i \in \{2, \dots, m\},$$

deci

$$a'_1 < b'_1 < a'_2 < b'_2 < \dots < a'_m < b'_m.$$

Rezultă că

$$I'_i \cap I'_j = \emptyset, \forall i, j \in \{1, \dots, m\}, i \neq j,$$

deci submulțimea $P = \{I'_1, \dots, I'_m\}$ a intervalelor selectate de algoritm este o soluție a problemei. Rămâne să demonstrăm optimalitatea acestei soluții.

Demonstrăm prin inducție după $k \in \{0, 1, \dots, m\}$ că există o soluție optimă $P^* = \{I_1^*, \dots, I_p^*\}$ a problemei, $p \in \mathbb{N}^*$, cu

$$I_1^* = [a_1^*, b_1^*], I_2^* = [a_2^*, b_2^*], \dots, I_p^* = [a_p^*, b_p^*], \quad b_1^* < b_2^* < \dots < b_p^*, \quad (2.4.2)$$

pentru care

$$I'_i = I_i^*, \forall i \text{ a.î. } 1 \leq i \leq k. \quad (2.4.3)$$

Pentru $k = 0$ afirmația este evidentă, luând P^* orice soluție optimă a problemei (și sortând intervalele componente I_i^* crescător după extremitățile b_i^*).

Presupunem (2.4.3) adevărată pentru $k - 1$, adică există o soluție optimă $P^* = \{I_1^*, \dots, I_p^*\}$ a problemei, ce verifică (2.4.2), pentru care

$$I'_i = I_i^*, \forall i \text{ a.î. } 1 \leq i \leq k - 1. \quad (2.4.4)$$

și o demonstrăm pentru k ($k \in \{1, 2, \dots, m\}$).

Din optimalitatea soluției $P^* = \{I_1^*, \dots, I_p^*\}$ rezultă că $p \geq m$, deci

$$p \geq m \geq k.$$

Avem două cazuri.

Cazul 1) $I'_k = I_k^*$. Atunci $I'_i = I_i^*$, $\forall i \in \{1, \dots, k\}$, deci (2.4.3) este adevărată pentru k .

Cazul 2) $I'_k \neq I_k^*$, adică $[a'_k, b'_k] \neq [a_k^*, b_k^*]$. În acest caz, pentru $k \geq 2$ avem $a_k^* > b_{k-1}^*$ (deoarece $I_k^* \cap I_{k-1}^* = \emptyset$ și $b_k^* > b_{k-1}^*$) și $b'_{k-1} = b_{k-1}^*$ (deoarece $I'_{k-1} = I_{k-1}^*$), deci

$$a_k^* > b'_{k-1}.$$

Atunci, din descrierea algoritmului, deoarece $I'_k = [a'_k, b'_k]$ este primul interval selectat după intervalul $I'_{k-1} = [a'_{k-1}, b'_{k-1}]$, rezultă că

$$b'_k \leq b_k^*. \quad (2.4.5)$$

Din descrierea algoritmului, această inegalitate este valabilă și pentru $k = 1$, deoarece $I'_1 = [a'_1, b'_1]$ este primul interval selectat.

Definim submulțimea de intervale $P^{**} = \{I_1^{**}, \dots, I_p^{**}\}$ prin

$$I_i^{**} = [a_i^{**}, b_i^{**}] = \begin{cases} I_i^*, & \text{dacă } i \neq k, \\ I'_i, & \text{dacă } i = k. \end{cases} \quad (2.4.6)$$

Deoarece $P^* = \{I_1^*, \dots, I_p^*\}$ este soluție a problemei și verifică (2.4.2), rezultă că

$$\begin{aligned} a_1^* < b_1^* < a_2^* < b_2^* < \dots < a_{k-1}^* < b_{k-1}^* < a_k^* < b_k^* < \\ < a_{k+1}^* < b_{k+1}^* < \dots < a_p^* < b_p^*. \end{aligned} \quad (2.4.7)$$

Pentru $k \geq 2$ avem $a'_k > b'_{k-1}$ (din descrierea algoritmului) și $b'_{k-1} = b_{k-1}^*$ (deoarece $I'_{k-1} = I_{k-1}^*$), deci

$$a'_k > b_{k-1}^*. \quad (2.4.8)$$

Din (2.4.7), (2.4.8) și (2.4.5) rezultă că

$$\begin{aligned} a_1^* < b_1^* < a_2^* < b_2^* < \dots < a_{k-1}^* < b_{k-1}^* < a'_k < b'_k < \\ < a_{k+1}^* < b_{k+1}^* < \dots < a_p^* < b_p^* \end{aligned}$$

(inegalitate valabilă și pentru $k = 1$), deci submulțimea $P^{**} = \{I_1^{**}, \dots, I_p^{**}\}$, definită de (2.4.6), este o soluție a problemei și

$$b_1^{**} < b_2^{**} < \dots < b_p^{**}.$$

Cum

$$\text{card}(P^{**}) = p = \text{card}(P^*)$$

și P^* este soluție optimă, rezultă că și P^{**} este soluție optimă.

Conform (2.4.6) și (2.4.4) avem

$$I'_i = I_i^{**}, \forall i \in \{1, \dots, k\},$$

deci relația (2.4.3) este adevărată pentru k , ceea ce încheie demonstrația prin inducție a acestei relații.

Luând $k = m$ în această relație rezultă că există o soluție optimă $P^* = \{I_1^*, \dots, I_p^*\}$ pentru care

$$I'_i = I_i^*, \forall i \in \{1, \dots, m\}.$$

Demonstrăm că $p = m$ prin reducere la absurd. Într-adevăr, dacă $p > m$ atunci ar exista intervalul $I_{m+1}^* = [a_{m+1}^*, b_{m+1}^*]$ astfel încât

$$a_{m+1}^* > b_m^* = b'_m$$

ceea ce ar contrazice faptul că algoritmul se încheie cu selectarea intervalului $I'_m = [a'_m, b'_m]$.

Astfel $p = m$, deci

$$P = \{I'_1, \dots, I'_m\} = \{I_1^*, \dots, I_p^*\} = P^*$$

și astfel submulțimea P este o soluție optimă a problemei.

□

Exemplul 2.4.1. Considerăm $n = 14$ spectacole ce au timpii de începere și de încheiere dați în următorul tabel (în ordinea crescătoare a timpilor de începere a_i):

Spectacol	S_1	S_2	S_3	S_4	S_5	S_6	S_7
Timp de începere a_i	8:00	8:10	8:15	8:50	9:10	9:20	9:20
Timp de încheiere b_i	9:10	9:00	9:00	10:20	10:40	10:30	11:00

Spectacol	S_8	S_9	S_{10}	S_{11}	S_{12}	S_{13}	S_{14}
Timp de începere a_i	10:45	11:00	12:00	12:10	12:30	13:00	13:40
Timp de încheiere b_i	12:00	12:30	13:30	14:00	13:50	14:30	15:00

Ordonarea spectacolele crescător după timpul lor de încheiere b_i este evidențiată în următorul tabel:

Spectacol	S_2	S_3	S_1	S_4	S_6	S_5	S_7
Timp de începere a_i	8:10	8:15	8:00	8:50	9:20	9:10	9:20
Timp de încheiere b_i	9:00	9:00	9:10	10:20	10:30	10:40	11:00

Spectacol	S_8	S_9	S_{10}	S_{12}	S_{11}	S_{13}	S_{14}
Timp de începere a_i	10:45	11:00	12:00	12:30	12:10	13:00	13:40
Timp de încheiere b_i	12:00	12:30	13:30	13:50	14:00	14:30	15:00

Aplicarea strategiei Greedy din algoritmul de mai sus conduce la soluția optimă dată de selectarea (vizionarea), în ordine, a spectacolelor:

- S_2 (primul, în ordinea impusă),
- S_6 (primul situat după S_2 și care are timpul de începere mai mare decât timpul de încheiere al lui S_2),
- S_8 (primul situat după S_6 și care are timpul de începere mai mare decât timpul de încheiere al lui S_6),
- S_{12} (primul situat după S_8 și care are timpul de începere mai mare decât timpul de încheiere al lui S_8), după care nu mai urmează niciun spectacol care să înceapă după încheierea lui S_{12} , deci selectarea se termină.

Numărul maxim de spectacole ce pot fi vizionate este deci egal cu 4.

Observația 2.4.1. Algoritmul 2.4.1 are complexitatea $\mathcal{O}(n \log_2 n)$, deoarece este necesară sortarea spectacolelor crescător după timpul lor de încheiere, iar blocul "for" prin care se parcurg spectacolele se execută de n ori (câte o dată pentru fiecare spectacol) și necesită de fiecare dată o comparație, cel mult o adunare și cel mult 3 operații de atribuire.

Algoritmul 2.4.2. O altă rezolvare a problemei spectacolelor se obține prin utilizarea următoarei *strategie Greedy*, similară cu cea de mai sus.

- Ordonăm spectacolele descrescător după timpul lor de începere:

$$a_1 \geq a_2 \geq \dots \geq a_n.$$

- Parcurgem spectacolele, în această ordine, și:
 - selectăm primul spectacol;
 - de fiecare dată, spectacolul curent, S_i , se selectează doar dacă nu se suprapune cu niciunul dintre spectacolele selectate anterior, adică dacă timpul său de încheiere este mai mic decât timpul de începere al ultimului spectacol S_j selectat:

$$b_i < a_j.$$

Pentru memorarea soluției se utilizează din nou un vector caracteristic $c = (c_1, \dots, c_n)$, cu aceeași semnificație ca în algoritmul de mai sus.

Descrierea în pseudocod a noului algoritm are următoarea formă.

```

SPECTACOLE2( $a, b, n, c, m$ ): //  $a = (a_1, \dots, a_n)$ ,  $b = (b_1, \dots, b_n)$ 
    //  $c = (c_1, \dots, c_n)$ ,  $m =$  numărul de spectacole selectate
SORTARE( $a, b, n$ ); // se sortează spectacolele crescător
    // după timpul lor de începere  $a_i$ 
     $m \leftarrow 0$ ; // inițializări
    for  $i = \overline{1, n}$  do  $c[i] \leftarrow 0$ ;
     $t \leftarrow b[n] + 1$ ; //  $t =$  timpul de începere al ultimului
    // spectacol selectat
    for  $i = \overline{n, 1, -1}$  do // parcurgem spectacolele în ordinea
    // descrescătoare a timpilor de începere
    |   if  $b[i] < t$  then
    |   |    $c[i] \leftarrow 1$ ; // selectăm intervalul (spectacolul) curent
    |   |    $m \leftarrow m + 1$ ;
    |   |    $t \leftarrow a[i]$ ; // actualizăm  $t$ 
    |
    AFISARE( $m, c, n$ ); // se afișează numărul și
    // submulțimea intervalelor (spectacolelor) selectate

```

Funcția de afișare este aceeași ca în Algoritmul 2.4.1.

Observația 2.4.2. Demonstrația corectitudinii și evaluarea complexității Algoritmului 2.4.2 sunt analoage cu cele ale Algoritmului 2.4.1.

Exemplul 2.4.2. Pentru spectacolele din Exemplul 2.4.1, ordonate crescător după timpii lor de începere a_i în primul tabel, aplicarea strategiei Greedy din Algoritmul 2.4.2 conduce la soluția optimă dată de următoarea selectare (vizionare în ordine inversă) a spectacolelor:

- S_{14} (ultimul, în ordinea descrescătoare a timpilor de începere),
- S_{10} (ultimul situat înainte de S_{14} și care are timpul de încheiere mai mic decât timpul de începere al lui S_{14}),
- S_7 (ultimul situat înainte de S_{10} și care are timpul de încheiere mai mic decât timpul de începere al lui S_{10}),
- S_3 (ultimul situat înainte de S_7 și care are timpul de încheiere mai mic decât timpul de începere al lui S_7), înainte de care nu mai avem niciun spectacol care să se încheie înainte de începerea lui S_3 , deci selectarea se termină.

Numărul maxim de spectacole ce pot fi vizionate este egal, din nou, cu 4.

2.5 Arbori parțiali de cost minim

Definiția 2.5.1. Un **graf ponderat** este o pereche (G, c) , unde $G = (V, E)$ este un graf (V fiind mulțimea nodurilor iar E mulțimea muchiilor sau arcelor) iar $c : E \rightarrow \mathbb{R}$ este o funcție numită **pondere (cost)**. Pentru orice $e \in E$, $c(e)$ se numește **ponderea (costul)** muchiei (în cazul grafurilor neorientate) sau arcului (în cazul grafurilor orientate) e .

Definiția 2.5.2. Fie (G, c) un graf ponderat, $G = (V, E)$.

- a) Dacă $H = (U, F)$ este un subgraf al lui G , atunci **costul (ponderea)** lui H este

$$c(H) = \sum_{e \in F} c(e)$$

(adică suma costurilor muchiilor sau arcelor sale).

- b) Un arbore parțial $T^* = (V, F)$ al lui G cu proprietatea că

$$c(T^*) = \min\{c(T) \mid T = \text{arbore parțial al lui } G\}$$

se numește **arbore parțial de cost minim (APM)** al grafului ponderat (G, c) .

Observația 2.5.1. Un graf ponderat are arbori parțiali de cost minim dacă și numai dacă este conex (adică între orice două noduri distincte există cel puțin un lanț).

Un arbore este un graf conex și fără cicluri.

Problema determinării arborilor parțiali de cost minim are numeroase aplicații practice. Prezentăm în continuare doi algoritmi fundamentali pentru rezolvarea acestei probleme.

Algoritmul 2.5.1 (Kruskal). Fie (G, c) un graf ponderat conex cu $G = (V, E)$, $V = \{v_1, \dots, v_n\}$. Algoritmul are $n - 1$ pași.

- La pasul i , $i = \overline{1, n-1}$, dintre muchiile neselectate la pașii anteriori se selectează o muchie $e_i \in E$ de cost minim cu proprietatea că nu formează cicluri cu muchiile $\{e_1, \dots, e_{i-1}\}$ selectate la pașii anteriori.

Algoritmul 2.5.2 (Prim). Fie (G, c) un graf ponderat conex cu $G = (V, E)$, $V = \{v_1, \dots, v_n\}$. Algoritmul are n pași.

- La pasul 0 se selectează un nod arbitrar $x_0 \in V$.

- La pasul i , $i = \overline{1, n-1}$, se selectează o muchie $e_i = [x_j, x_i] \in E$ de cost minim cu proprietatea că are ca extremități un nod $x_j \in V$ selectat la un pas anterior și celălalt nod $x_i \in V$ neselectat la pașii anteriori; se selectează și nodul x_i .

Teorema 2.5.1 (de corectitudine a algoritmilor Kruskal și Prim). În contextul Algoritmilor Kruskal sau Prim, fie $F = \{e_1, \dots, e_{n-1}\}$ mulțimea muchiilor selectate. Atunci $T = (V, F)$ este un arbore parțial de cost minim al grafului ponderat (G, c) .

Demonstrație. Fie $T_0 = (V, \emptyset)$ și $T_i = (V, \{e_1, \dots, e_i\})$, $\forall i \in \{1, 2, \dots, n-1\}$, unde $e_1, e_2, \dots, e_{n-1}$ sunt muchiile selectate, în această ordine, de Algoritmul Kruskal sau de Algoritmul Prim.

Graful G fiind conex, selectarea muchiei e_i este posibilă la fiecare pas i , iar T_i este o pădure (graf fără cicluri) parțială a lui G (afirmație evidentă pentru Algoritmul Kruskal, iar pentru Algoritmul Prim este o consecință a faptului că nodurile neselectate la pasul i sunt izolate în T_i).

Se arată prin inducție după $i \in \{0, 1, \dots, n-1\}$ că există un APM $T^* = (V, F^*)$ astfel încât

$$T_i \subseteq T^* \text{ (adică } \{e_1, \dots, e_i\} \subseteq F^* \text{)}.$$

Luând $i = n-1$ în această relație obținem că $T \subseteq T^*$, unde $T = T_{n-1} = (V, F)$, $F = \{e_1, \dots, e_{n-1}\}$ (mulțimea muchiilor selectate de algoritm) iar T^* este un APM. Dar T și T^* au fiecare câte $n-1$ muchii, deci $T = T^*$ și astfel T este un APM al grafului dat. $\square$

Exemplul 2.5.1. Fie graful ponderat (G, c) reprezentat în Figura 2.5.1, unde costul fiecărei muchii este scris lângă segmentul corespunzător acesteia.

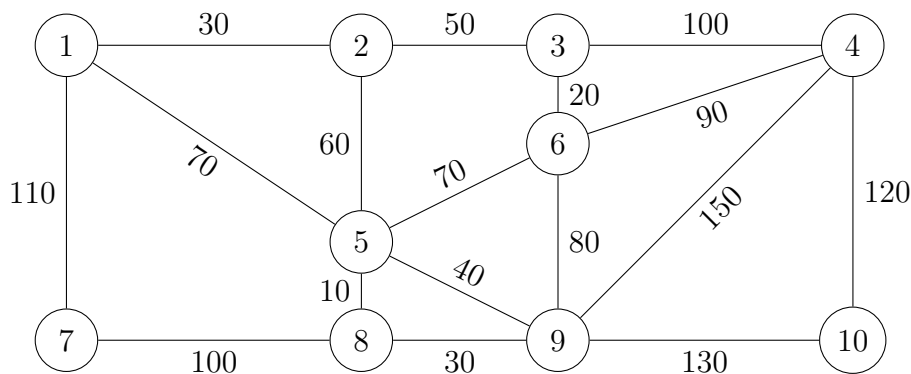


Figura 2.5.1:

Aplicarea Algoritmului Kruskal este evidențiată în următorul tabel:

Pas	Muchia selectată	Costul ei
1	[5, 8]	10
2	[3, 6]	20
3	[1, 2]	30
4	[8, 9]	30
5	[2, 3]	50
6	[2, 5]	60
7	[4, 6]	90
8	[7, 8]	100
9	[4, 10]	120

Arborele parțial de cost minim obținut este reprezentat în Figura 2.5.2. Costul acestui APM este de 510.

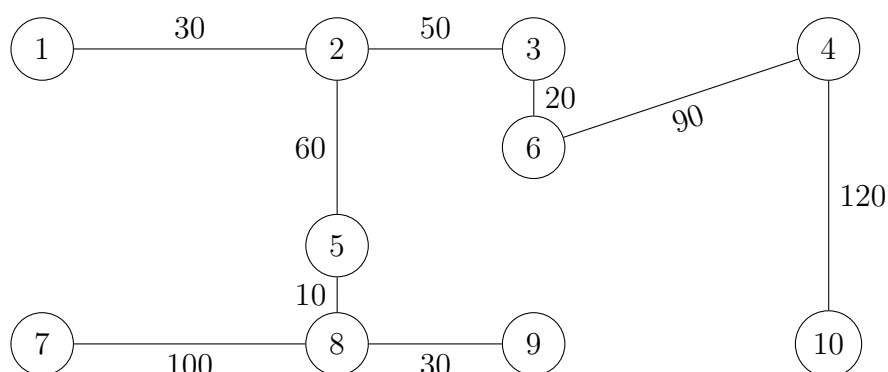


Figura 2.5.2:

Aplicarea Algoritmului Prim pentru același graf este evidențiată în următorul tabel:

Pas	Muchia selectată	Costul ei	Nodul selectat
0	-	-	1
1	[1, 2]	30	2
2	[2, 3]	50	3
3	[3, 6]	20	6
4	[2, 5]	60	5
5	[5, 8]	10	8
6	[8, 9]	30	9
7	[6, 4]	90	4
8	[8, 7]	100	7
9	[4, 10]	120	10

Arborele parțial de cost minim obținut este deci același cu cel obținut prin aplicarea Algoritmului Kruskal.

Observația 2.5.2. Algoritmii Kruskal și Prim sunt specifici **metodei de programare Greedy**. Algoritmul Kruskal selectează muchii, în ordinea crescătoare a costurilor, subgrafurile induse pe parcurs de acestea nefiind neapărat conexe. Algoritmul Prim selectează muchii și noduri, nu neapărat în ordinea crescătoare a costurilor muchiilor, iar subgrafurile induse pe parcurs de muchiile selectate sunt conexe.

În implementări optime, se poate arăta că Algoritmul Kruskal are complexitatea $\mathcal{O}(m \ln n)$ (fiind necesară sortarea muchiilor după cost), iar Algoritmul Prim are complexitatea $\mathcal{O}(n^2)$ în cazul memorării grafului prin matricea de adiacență (o astfel de implementare va fi prezentată în continuare), unde n și m reprezintă numerele de noduri, respectiv de muchii ale grafului dat. Graful fiind conex, $m \geq n - 1$.

Pentru grafuri simple, $m \leq \frac{n(n-1)}{2}$. Folosind și inegalitatea $\ln n \leq n - 1$, obținem că Algoritmul Kruskal este mai eficient pentru grafuri "sărace" în muchii, iar Algoritmul Prim este mai eficient pentru grafuri "bogate" în muchii.

Observația 2.5.3. Pentru implementarea Algoritmului Kruskal, memorăm graful ponderat conex (G, c) , unde $G = (V, E)$, $V = \{1, \dots, n\}$, $E = \{e_1, \dots, e_m\}$, într-o matrice cu 3 linii și m coloane $P = (p_{ik})_{\substack{i = \overline{1,3} \\ k = \overline{1,m}}}$ având

semnificația:

$$\text{dacă } e_k = [x_k, y_k] \in E, \text{ atunci } p_{1k} = x_k, p_{2k} = y_k \text{ și } p_{3k} = c(e_k).$$

Utilizăm un vector S cu semnificația

$$S[k] = \begin{cases} 1, & \text{dacă } e_k \text{ a fost selectată,} \\ 0, & \text{în caz contrar,} \end{cases}$$

$\forall k \in \{1, \dots, m\}$ și un vector CC cu semnificația

$CC[i] =$ numărul componentei conexe în care se află nodul i în graful indus de muchiile selectate, $\forall i \in \{1, \dots, n\}$.

Astfel o muchie $[x, y]$ nu formează cicluri cu muchiile selectate dacă și numai dacă

$$CC[x] \neq CC[y].$$

Descrierea în pseudocod a algoritmului are următoarea formă.

KRUSKAL:

```

SORTARE( $P$ );           // se sortează coloanele matricei  $P$ 
                        // crescător după costurile muchiilor
for  $i = \overline{1, m}$  do  $S[i] \leftarrow 0$ ;
for  $i = \overline{1, n}$  do  $CC[i] \leftarrow i$ ;
 $cost \leftarrow 0$ ;           // costul APM
 $poz \leftarrow 0$ ;           // căutarea următoarei muchii  $e_k$  ce va fi
                        // selectată începe de pe poziția  $poz + 1$ 
for  $l = \overline{1, n-1}$  do           // pasul  $l$ 
     $k \leftarrow poz$ ;
    repeat
         $k \leftarrow k + 1$ ;  $x \leftarrow p_{1k}$ ;  $y \leftarrow p_{2k}$ ;  $c \leftarrow p_{3k}$ ;
    while ( $CC[x] = CC[y]$ );
     $S[k] \leftarrow 1$ ;           // selectăm  $e_k = [x, y]$ 
     $cost \leftarrow cost + c$ ;  $poz \leftarrow k$ ;
     $aux \leftarrow CC[y]$ ; // actualizăm vectorul  $CC$  prin unificarea
                        // componentelor conexe ale lui  $x$  și  $y$ 
    for  $i = \overline{1, n}$  do
        if ( $CC[i] = aux$ ) then  $CC[i] \leftarrow CC[x]$ ;

```

Observația 2.5.4. Pentru implementarea Algoritmului Prim, memorăm graful ponderat conex și simplu (G, c) , unde $G = (V, E)$, $V = \{1, \dots, n\}$, $E = \{e_1, \dots, e_m\}$, cu ajutorul unei matrice $C = (c_{ij})_{i,j=\overline{1,n}}$ a **costurilor (directe)** având semnificația

$$c_{ij} = \begin{cases} c([i, j]), & \text{dacă } [i, j] \in E, \\ 0, & \text{dacă } i = j, \\ \infty, & \text{în rest,} \end{cases} \quad (2.5.1)$$

$\forall i, j \in \{1, \dots, n\}$. Pentru grafuri neorientate, matricea C este simetrică. În cazul grafurilor nesimple putem lua

$$c_{ij} = \min\{c(e) | e = [i, j] \in E\}.$$

Utilizăm un vector S cu semnificația

$$S[i] = \begin{cases} 1, & \text{dacă nodul } i \text{ a fost selectat,} \\ 0, & \text{în caz contrar} \end{cases}$$

și doi vectori t și $TATA$ având semnificația

$$\begin{aligned} t[i] &= \text{costul minim al unei muchii } [i, j] \text{ de la nodul } i \text{ la un nod selectat } j, \\ TATA[i] &= \text{nodul } j \text{ ce atinge minimul în } t[i], \quad \forall i \in \{1, \dots, n\}. \end{aligned}$$

Descrierea în pseudocod a algoritmului are următoarea formă.

PRIM:

```

 $S[1] \leftarrow 1;$                                 // selectăm nodul 1
 $cost \leftarrow 0;$                                 // costul APM
for  $i = \overline{2, n}$  do                                // inițializări
     $S[i] \leftarrow 0; t[i] \leftarrow c_{i1}; TATA[i] \leftarrow 1;$ 
for  $l = \overline{1, n-1}$  do                                // căutăm nodul  $y$  și muchia  $[x, y]$ 
    // ce vor fi selectate la pasul  $l$ 
     $min \leftarrow \infty;$ 
    for  $i = \overline{2, n}$  do
        if  $(S[i] = 0) \text{ and } (t[i] < min)$  then
             $min \leftarrow t[i]; y \leftarrow i;$ 
     $S[y] \leftarrow 1;$                                 // selectăm nodul  $y$ 
     $x \leftarrow TATA[y];$                                 // și muchia  $[x, y]$ 
     $cost \leftarrow cost + c_{xy};$ 
    for  $i = \overline{2, n}$  do                                // actualizăm vectorii  $t$  și  $TATA$ 
        if  $(S[i] = 0) \text{ and } (t[i] > c_{iy})$  then
             $t[i] \leftarrow c_{iy}; TATA[i] \leftarrow y;$ 

```

2.6 Distanțe și drumuri minime. Algoritmul lui Dijkstra

Problema determinării distanțelor și drumurilor minime între nodurile unui graf ponderat apare în numeroase aplicații practice. În continuare vom prezenta un algoritm clasic pentru rezolvarea acestei probleme.

Definiția 2.6.1. Fie (G, c) un graf ponderat, unde $G = (V, E)$, $V = \{v_1, \dots, v_n\}$ iar $c : E \rightarrow \mathbb{R}_+$.

- a) Dacă $\mu = (x_0, e_1, x_1, \dots, x_{k-1}, e_k, x_k)$ este un drum al grafului G , unde $x_0, x_1, \dots, x_k \in V$, $e_1, \dots, e_k \in E$, $k \in \mathbb{N}$, atunci **costul** (**pondera**) lui μ este

$$c(\mu) = \begin{cases} 0, & \text{dacă } k = 0, \\ \sum_{i=1}^k c(e_i), & \text{dacă } k \geq 1 \end{cases}$$

(adică suma costurilor arcelor sau muchiilor sale).

- b) Fie $x, y \in V$. Un drum $\mu^* = (x, \dots, y)$ în graful G cu proprietatea că

$$c(\mu^*) = \min\{c(\mu) \mid \mu \text{ este drum de la } x \text{ la } y \text{ în } G\}$$

se numește **drum minim** (**drum de cost minim**, **drum de pondere minimă**) de la x la y în graful ponderat (G, c) . Costul $c(\mu^*)$ al acestui drum minim se numește **distanța minimă** de la x la y în graful (G, c) .

Observația 2.6.1. Eliminând eventualele circuite $C_1, \dots, C_p$ dintr-un drum μ de la nodul x la nodul y obținem un drum elementar (adică un drum având nodurile distincte două câte două, cu excepția extremităților, care pot fi egale) μ' de la x la y cu proprietatea că $c(\mu') = c(\mu) - \sum_{k=1}^p c(C_k) \leq c(\mu)$.

Dacă drumul μ este minim atunci și drumul elementar μ' este minim și $c(e) = 0$ pentru orice muchie sau arc e al circuitelor $C_1, \dots, C_p$. Deci existența unui drum minim de la x la y implică existența unui drum minim elementar de la x la y . Astfel distanța minimă de la x la y poate fi considerată ca fiind costul minim al unui drum elementar de la x la y . Mai mult, dacă funcția cost c este strict pozitivă, atunci orice drum minim este elementar.

Observația 2.6.2. Mulțimea drumurilor elementare fiind evident finită, avem echivalența: există drumuri minime de la x la y dacă și numai dacă există drumuri de la x la y .

Observația 2.6.3. Distanța minimă de la un nod x la el însuși este egală cu zero, drumul minim elementar de la x la x fiind drumul de lungime zero, $\mu = (x)$.

Observația 2.6.4. Dacă $\mu = (x_0, x_1, \dots, x_k)$ este un drum minim de la x_0 la x_k , atunci orice subdrum $\mu' = (x_i, x_{i+1}, \dots, x_j)$ ($0 \leq i \leq j \leq k$) al său este un drum minim de la x_i la x_j (**principiul optimalității al lui Bellman**). Afirmția poate fi justificată ușor prin reducere la absurd.

Exemplul 2.6.1. Fie graful ponderat (G, c) reprezentat în Figura 2.6.1.

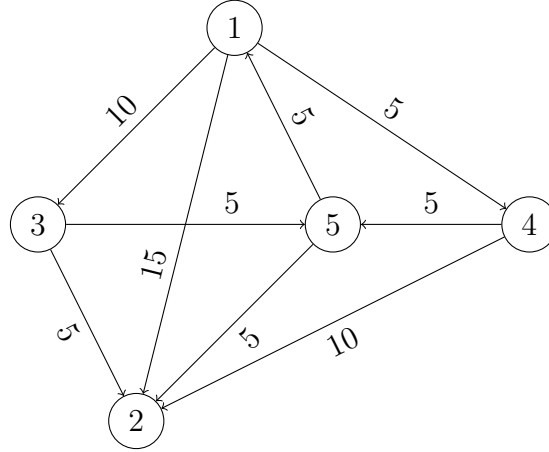


Figura 2.6.1:

Drumurile elementare de la nodul 1 la nodul 5 sunt $\mu_1 = (1, 3, 5)$, având costul $c(\mu_1) = 10 + 5 = 15$, $\mu_2 = (1, 4, 5)$, având costul $c(\mu_2) = 5 + 5 = 10$, deci μ_2 este un drum minim de la 1 la 5. Astfel distanța minimă de la nodul 1 la nodul 5 este $c(\mu_2) = 10$.

Definiția 2.6.2. Fie (G, c) un graf ponderat, unde $G = (V, E)$, $V = \{v_1, \dots, v_n\}$, $c : E \rightarrow \mathbb{R}_+$.

- a) **Matricea distanțelor (costurilor) directe** asociată grafului (G, c) este matricea $C = (c_{ij})_{i,j=\overline{1,n}}$ definită prin

$$c_{ij} = \begin{cases} 0, & \text{dacă } i = j, \\ \min\{c(e) | e = (v_i, v_j) \in E\}, & \text{dacă } i \neq j \text{ și } \exists (v_i, v_j) \in E, \\ \infty, & \text{dacă } i \neq j \text{ și } \nexists (v_i, v_j) \in E \end{cases}$$

(pentru grafuri neorientate (v_i, v_j) desemnând de fapt muchia $[v_i, v_j]$).

- b) **Matricea distanțelor (costurilor) minime** asociată grafului (G, c) este matricea $C^* = (c_{ij}^*)_{i,j=\overline{1,n}}$ definită prin

$$c_{ij}^* = \begin{cases} c(\mu^*), & \mu^* = \text{drum minim de la } v_i \text{ la } v_j, \\ & \text{dacă } \exists \mu = (v_i, \dots, v_j) \text{ drum în } G, \\ \infty, & \text{în caz contrar.} \end{cases}$$

Observația 2.6.5. Evident, pentru orice graf neorientat atât matricea distanțelor directe cât și matricea distanțelor minime sunt matrice simetrice.

Observația 2.6.6. Conform Observației 2.6.1, putem să înlocuim termenul de ”drum” cu cel de ”drum elementar” în definiția anterioară.

Conform Observației 2.6.2, punctul b) din definiția anterioară este o extindere a definiției distanței minime de la punctul b) al Definiției 2.6.1.

Conform Observației 2.6.3, $c_{ii}^* = 0 \forall i \in \{1, \dots, n\}$.

Exemplul 2.6.2. Matricele distanțelor directe, respectiv minime asociate grafului din Exemplul 2.6.1 sunt

$$C = \begin{pmatrix} 0 & 15 & 10 & 5 & \infty \\ \infty & 0 & \infty & \infty & \infty \\ \infty & 5 & 0 & \infty & 5 \\ \infty & 10 & \infty & 0 & 5 \\ 5 & 5 & \infty & \infty & 0 \end{pmatrix}, \quad C^* = \begin{pmatrix} 0 & 15 & 10 & 5 & 10 \\ \infty & 0 & \infty & \infty & \infty \\ 10 & 5 & 0 & 15 & 5 \\ 10 & 10 & 20 & 0 & 5 \\ 5 & 5 & 15 & 10 & 0 \end{pmatrix}.$$

Vom expune în continuare un algoritm pentru determinarea distanțelor minime și a drumurilor minime de la un nod fixat, numit și nod sursă, la toate nodurile grafului ponderat dat.

Algoritmul 2.6.1 (Dijkstra). Fie (G, c) un graf ponderat, $G = (V, E)$, $V = \{v_1, v_2, \dots, v_n\}$, $c : E \rightarrow \mathbb{R}_+$. Fie $C = (c_{ij})_{i,j=\overline{1,n}}$ matricea distanțelor directe asociată grafului (G, c) și fie $v_s \in V$ un nod arbitrar fixat, numit **nod sursă**. Distanțele minime de la nodul v_s la nodurile grafului sunt calculate și memorate într-un vector $t = (t_1, \dots, t_n)$ astfel:

- La pasul 1 se selectează nodul sursă v_s și se ia $t_s = 0$;
- La pasul k , $2 \leq k \leq n$, se cunosc nodurile $v_{i_1}, \dots, v_{i_{k-1}}$ selectate la pașii anteriori și distanțele corespondente $t_{i_1}, \dots, t_{i_{k-1}}$.

a) Dacă nu mai există nici-o muchie sau arc de la un nod selectat $v_j \in \{v_{i_1}, \dots, v_{i_{k-1}}\}$ la un nod neselectat $v_i \in V \setminus \{v_{i_1}, \dots, v_{i_{k-1}}\}$, atunci se ia $t_i = \infty$ pentru orice nod neselectat $v_i \in V \setminus \{v_{i_1}, \dots, v_{i_{k-1}}\}$ și algoritmul se încheie.

b) În caz contrar se selectează un nod $v_{i_k} \in V \setminus \{v_{i_1}, \dots, v_{i_{k-1}}\}$ cu proprietatea că există un nod selectat $v_{j_k} \in \{v_{i_1}, \dots, v_{i_{k-1}}\}$ astfel încât

$$t_{j_k} + c_{j_k i_k} = \min\{t_j + c_{ji} | v_j \in \{v_{i_1}, \dots, v_{i_{k-1}}\}, v_i \in V \setminus \{v_{i_1}, \dots, v_{i_{k-1}}\}\}. \quad (2.6.1)$$

Se ia

$$t_{i_k} = t_{j_k} + c_{j_k i_k} \quad (2.6.2)$$

și se trece la pasul $k + 1$.

Observația 2.6.7. Evident, algoritmul execută cel mult n pași.

Teorema 2.6.1 (de corectitudine a Algoritmului Dijkstra). *În contextul Algoritmului Dijkstra, avem*

$$t_i = c_{si}^*, \forall i \in \{1, \dots, n\}$$

(adică distanța t_i calculată de algoritm este chiar distanța minimă de la v_s la v_i).

Demonstrație. Se arată prin inducție după k că nodul v_{i_k} selectat la pasul k și distanța corespondentă t_{i_k} calculată la acel pas verifică egalitatea din enunț, adică

$$t_{i_k} = c_{si_k}^*,$$

și, în plus, $t_{i_k} < \infty$. Evident, pentru orice nod v_i rămas neselectat în urma executării ultimului pas al algoritmului avem $t_i = \infty = c_{si}^*$.

Într-adevăr, dacă ar exista un drum elementar minim

$$\mu = (v_s = y_0, y_1, \dots, y_p = v_i),$$

luând $l \in \{0, 1, \dots, p-1\}$ indicele maxim pentru care y_l este selectat (există, deoarece $y_0 = v_s$ este selectat) atunci y_{l+1} ar fi neselectat deși există o muchie sau un arc de la y_l la y_{l+1} , contradicție cu descrierea modului de încheiere a algoritmului. $\square$

Exemplul 2.6.3. Pentru graful ponderat din Exemplul 2.6.1, luând ca nod sursă nodul 1, aplicarea Algoritmului Dijkstra este evidențiată în următorul tabel:

Pas	Nodul selectat	Distanța minimă
1	1	0
2	4	5
3	3	10
4	5	10
5	2	15

De exemplu, la pasul 3 avem deja selectate nodurile $i_1 = 1$ și $i_2 = 4$, cu distanțele minime $t_1 = c_{11}^* = 0$ și $t_4 = c_{14}^* = 5$. Se selectează nodul $i_3 = 3$, cu distanța minimă $t_3 = 10$, deoarece

$$\begin{aligned} & \min\{t_1 + c_{12}, t_1 + c_{13}, t_1 + c_{15}, t_4 + c_{42}, t_4 + c_{43}, t_4 + c_{45}\} \\ &= \min\{0 + 15, 0 + 10, 0 + \infty, 5 + 10, 5 + \infty, 5 + 5\} = 10 = t_1 + c_{13}. \end{aligned}$$

Observația 2.6.8. Algoritmul Dijkstra este specific **metodei de programare Greedy**, el selectând nodurile în ordinea crescătoare a distanței față de nodul sursă.

Observația 2.6.9. Pentru implementarea Algoritmului Dijkstra, considerăm că $V = \{1, \dots, n\}$ și că nodul sursă este $s \in V$. Utilizăm un vector S având semnificația

$$S[i] = \begin{cases} 1, & \text{dacă nodul } i \text{ a fost selectat,} \\ 0, & \text{în caz contrar,} \end{cases} \quad \forall i \in \{1, \dots, n\}$$

și un vector t având semnificația

$t[i] = \text{distanța minimă de la nodul sursă } s \text{ la nodul } i, \forall i \in \{1, \dots, n\}$,
calculat conform (2.6.1) și (2.6.2).

Pentru determinarea drumurilor minime de la nodul s la nodurile grafului vom utiliza și un vector $TATA$ având semnificația

$TATA[i] = \text{nodul } j \text{ ce este predecesorul direct al nodului } i \text{ pe drumul minim de la } s \text{ la } i, \forall i \in \{1, \dots, n\}$.

Astfel în vectorul $TATA$ se memorează un arbore compus din drumuri minime de la nodul sursă la nodurile grafului, numit **arborele drumurilor minime**.

Dacă $i = i_k$ este nodul selectat la pasul k , atunci $j = j_k$ se determină conform egalității (2.6.1).

Descrierea în pseudocod a algoritmului are forma următoare.

```

DIJKSTRA( $s$ ) :
for  $i = \overline{1, n}$  do                                     // inițializări
     $S[i] \leftarrow 0$ ;  $t[i] \leftarrow \infty$ ;  $TATA[i] \leftarrow \infty$ ;
 $t[s] \leftarrow 0$ ;  $TATA[s] \leftarrow 0$ ;                 //  $s$  este nodul sursă
repeat
    // selectăm următorul nod  $x$ , în ordinea crescătoare
    // a distanțelor minime de la  $s$  la  $x$ 
     $min \leftarrow \infty$ ;
    for  $i = \overline{1, n}$  do
        if ( $S[i] = 0$ ) and ( $t[i] < min$ ) then
             $min \leftarrow t[i]$ ;
             $x \leftarrow i$ ;
    if ( $min < \infty$ ) then                               // există  $x$ , îl selectăm
         $S[x] \leftarrow 1$ ;
        for  $i = \overline{1, n}$  do                             // actualizăm vectorii  $t$  și  $TATA$ 
            if ( $S[i] = 0$ ) and ( $c_{xi} < \infty$ ) then
                if ( $t[i] > t[x] + c_{xi}$ ) then
                     $t[i] \leftarrow t[x] + c_{xi}$ ;
                     $TATA[i] \leftarrow x$ ;
    while ( $min < \infty$ );

```

Exemplul 2.6.4. Pentru graful ponderat din Exemplul 2.5.1, luând ca nod sursă nodul $s = 1$, aplicarea Algoritmului Dijkstra este evidențiată în următorul tabel:

Pas	Nodul selectat x	$TATA[x]$	Distanța minimă $t[x]$
1	1	0	0
2	2	1	30
3	5	1	70
4	3	2	80
5	8	5	80
6	6	3	100
7	7	1	110
8	9	5	110
9	4	3	180
10	10	9	240

Arborele drumurilor minime, memorat în vectorul $TATA$, este reprezentat în Figura 2.6.2.

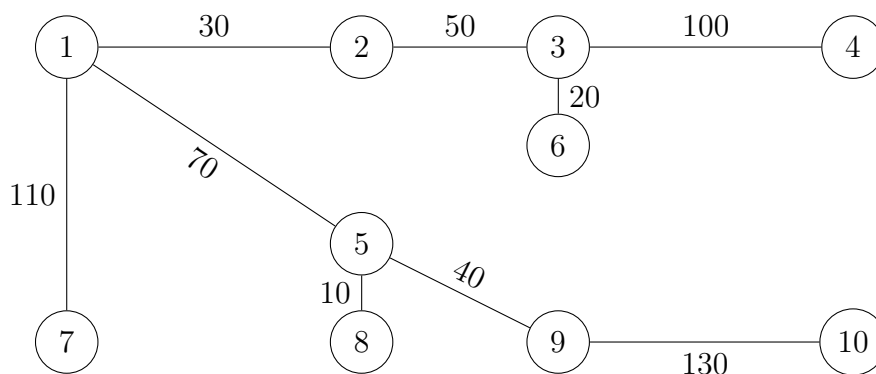


Figura 2.6.2:

Deci drumurile minime determinate de algoritm sunt:

- de la 1 la 1: [1];
- de la 1 la 2: [1, 2];
- de la 1 la 3: [1, 2, 3];
- de la 1 la 4: [1, 2, 3, 4];
- de la 1 la 5: [1, 5];
- de la 1 la 6: [1, 2, 3, 6];
- de la 1 la 7: [1, 7];
- de la 1 la 8: [1, 5, 8];
- de la 1 la 9: [1, 5, 9];
- de la 1 la 10: [1, 5, 9, 10].

Observația 2.6.10. Implementarea anterioară necesită $\mathcal{O}(n^2)$ operații (deoarece blocul ”**repeat**” se execută de cel mult n ori și necesită de fiecare dată cel mult n comparații pentru determinarea nodului selectat x și cel mult n comparații și n adunări pentru actualizarea vectorilor t și $TATA$). Aceasta este de fapt și complexitatea Algoritmului Dijkstra (în implementarea optimă) în cazul memorării grafului prin matricea distanțelor directe.

2.7 Fluxuri în rețele

Definiția 2.7.1. O rețea (rețea de transport) are forma $R = (G, s, t, c)$, unde:

- $G = (V, E)$ este un graf orientat simplu (adică fără bucle sau arce egale).

V se numește și **mulțimea nodurilor** rețelei, iar E se numește și **mulțimea arcelor** rețelei;

- $s, t \in V$ sunt două noduri a.î. $s \neq t$.

Nodul s se numește **nodul sursă (intrarea)** al rețelei, iar nodul t se numește **nodul destinație (ieșirea)** al rețelei;

- $c : E \rightarrow \mathbb{R}_+$ este o funcție numită **funcție capacitate**.

Pentru orice $e \in E$, $c(e)$ se numește **capacitatea arcului** e .

Pentru simplificarea notațiilor, extindem funcția capacitate

$$c : V \times V \rightarrow \mathbb{R}_+, \quad c(i, j) = \begin{cases} c(i, j), & \text{dacă } (i, j) \in E, \\ 0, & \text{dacă } (i, j) \notin E. \end{cases} \quad (2.7.1)$$

Exemplul 2.7.1. În Figura 2.7.1 este reprezentată o rețea R , capacitățile fiind menționate pe arce. Considerăm că această rețea are intrarea $s = 1$ și ieșirea $t = 6$.

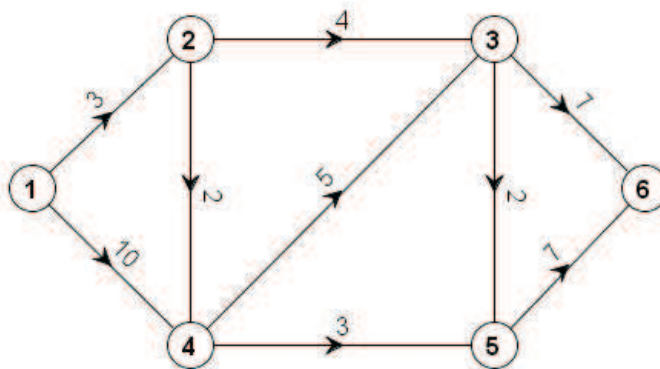


Figura 2.7.1:

Definiția 2.7.2. Fie $R = (G, s, t, c)$ o rețea, unde $G = (V, E)$. Un **flux** în rețeaua R este o funcție $f : V \times V \rightarrow \mathbb{R}$ ce verifică următoarele două proprietăți:

$$0 \leq f(i, j) \leq c(i, j), \quad \forall i, j \in V, \quad (2.7.2)$$

$$\sum_{j \in V} f(j, i) = \sum_{j \in V} f(i, j), \quad \forall i \in V \setminus \{s, t\}. \quad (2.7.3)$$

$f(i, j)$ reprezintă **fluxul transportat pe arcul** (i, j) .

Relația (2.7.2) se numește **condiția de marginire a fluxului**, iar relația (2.7.3) se numește **condiția de conservare a fluxului**.

Observația 2.7.1. Evident,

$$f(i, j) = 0 \quad \forall (i, j) \notin E,$$

deci, analog funcției capacitate c , și funcția flux f poate fi definită (restrânsă) doar pe mulțimea arcelor rețelei (adică $f : E \rightarrow \mathbb{R}$).

Observația 2.7.2. În particular, $f(i, j) = 0 \quad \forall i, j \in V$ este un flux în orice rețea, numit **fluxul nul**.

Exemplul 2.7.2. Pentru rețeaua din Figura 2.7.1, un exemplu de flux este reprezentat în Figura 2.7.2. Pe fiecare arc (i, j) sunt menționate fluxul $f(i, j)$ și capacitatea $c(i, j)$, în această ordine.

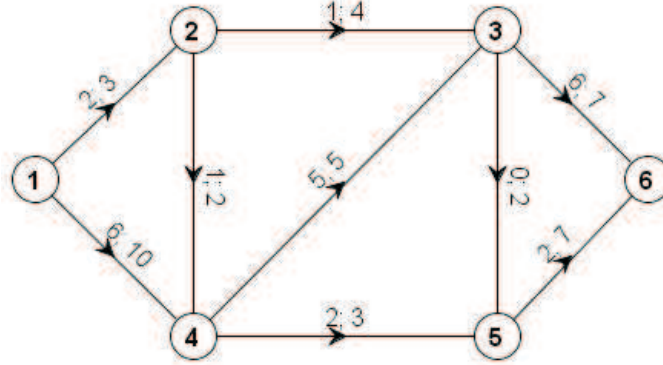


Figura 2.7.2:

Lema 2.7.1. Fie $R = (G, s, t, c)$ o rețea, unde $G = (V, E)$. Pentru orice flux f în rețeaua R are loc egalitatea

$$\sum_{i \in V} f(i, t) - \sum_{i \in V} f(t, i) = - \left(\sum_{i \in V} f(i, s) - \sum_{i \in V} f(s, i) \right).$$

Definiția 2.7.3. Fie $R = (G, s, t, c)$ o rețea, unde $G = (V, E)$. Pentru orice flux f în rețeaua R , numărul

$$v(f) = \sum_{i \in V} f(i, t) - \sum_{i \in V} f(t, i) = - \left(\sum_{i \in V} f(i, s) - \sum_{i \in V} f(s, i) \right)$$

se numește **valoarea fluxului** f (fluxul net ce ajunge la nodul destinație, fluxul net ce iese din nodul sursă).

Exemplul 2.7.3. Fluxul din Figura 2.7.2 are valoarea $v(f) = 2 + 6 = 8$.

Definiția 2.7.4. Fie $R = (G, s, t, c)$ o rețea. Un flux f^* în rețeaua R cu proprietatea că

$$v(f^*) = \max\{v(f) \mid f = \text{flux în } R\}$$

se numește **flux de valoare maximă (flux maxim)** în rețeaua R .

Problema determinării fluxurilor maxime într-o rețea are numeroase aplicații practice. Vom prezenta un algoritm pentru rezolvarea acestei probleme. Definim câteva noțiuni ajutătoare.

Definiția 2.7.5. Dacă $\mu = [x_0, x_1, \dots, x_k]$ este un lanț elementar în graful orientat $G = (V, E)$, atunci arcele sale de forma $(x_i, x_{i+1}) \in E$ ($i \in \{0, \dots, k-1\}$) se numesc **arce directe** pentru lanțul μ , iar arcele sale de forma $(x_{i+1}, x_i) \in E$ ($i \in \{0, \dots, k-1\}$) se numesc **arce inverse** pentru lanțul μ .

Exemplul 2.7.4. Pentru graful din Figura 2.7.2, lanțul $[1, 2, 3, 4, 5, 6]$ are arcele directe $(1, 2), (2, 3), (4, 5), (5, 6)$ și arcul invers $(4, 3)$.

Definiția 2.7.6. Fie f un flux în rețeaua $R = (G, s, t, c)$. Un **C-lanț** în rețeaua R relativ la fluxul f este un lanț elementar μ în graful G ce verifică următoarele două proprietăți:

$$\begin{aligned} f(i, j) &< c(i, j), \quad \forall (i, j) = \text{arc direct al lui } \mu, \\ f(i, j) &> 0, \quad \forall (i, j) = \text{arc invers al lui } \mu. \end{aligned}$$

Definiția 2.7.7. Fie f un flux în rețeaua $R = (G, s, t, c)$ și fie μ un C-lanț în rețeaua R relativ la fluxul f .

a) Pentru orice arc (i, j) al lui μ , numărul

$$r_\mu(i, j) = \begin{cases} c(i, j) - f(i, j), & \text{dacă } (i, j) \text{ este arc direct al lui } \mu, \\ f(i, j), & \text{dacă } (i, j) \text{ este arc invers al lui } \mu \end{cases}$$

se numește **capacitatea reziduală (reziduul)** a arcului (i, j) relativ la C-lanțul μ .

b) Numărul

$$r(\mu) = \min\{r_\mu(i, j) \mid (i, j) = \text{arc al lui } \mu\}$$

se numește **capacitatea reziduală (reziduul)** a C-lanțului μ .

Observația 2.7.3. Conform definițiilor anterioare, pentru orice C-lanț μ avem $r(\mu) > 0$.

Exemplul 2.7.5. Pentru rețeaua și fluxul reprezentate în Figura 2.7.2, lanțul $\mu = [1, 2, 3, 4, 5, 6]$ este un C-lanț. Reziduurile pe arcele acestui C-lanț sunt

$$r_\mu(1, 2) = 3 - 2 = 1, \quad r_\mu(2, 3) = 4 - 1 = 3, \quad r_\mu(4, 3) = 5,$$

$$r_\mu(4, 5) = 3 - 2 = 1, \quad r_\mu(5, 6) = 7 - 2 = 5,$$

deci reziduul pe acest C-lanț este $r(\mu) = 1$.

Definiția 2.7.8. Fie f un flux în rețeaua $R = (G, s, t, c)$. Un C-lanț de la s la t în rețeaua R relativ la fluxul f se numește **lanț de creștere** (în rețeaua R relativ la fluxul f).

Exemplul 2.7.6. Lanțul $\mu = [1, 2, 3, 4, 5, 6]$ din exemplul anterior este un lanț de creștere.

Lema 2.7.2. Fie $R = (G, s, t, c)$ o rețea, unde $G = (V, E)$. Fie f un flux în rețeaua R și fie μ un lanț de creștere în rețeaua R relativ la fluxul f . Atunci funcția $f' : V \times V \rightarrow \mathbb{R}$ definită prin

$$f'(i, j) = \begin{cases} f(i, j) + r(\mu), & \text{dacă } (i, j) \text{ este arc direct al lui } \mu, \\ f(i, j) - r(\mu), & \text{dacă } (i, j) \text{ este arc invers al lui } \mu, \\ f(i, j), & \text{dacă } (i, j) \text{ nu este arc al lui } \mu \end{cases}$$

este un flux în rețeaua R și

$$v(f') = v(f) + r(\mu).$$

Observația 2.7.4. În contextul lemei anterioare, conform Observației 2.7.3 avem

$$v(f') = v(f) + r(\mu) > v(f).$$

Inegalitatea justifică denumirea lui μ drept **lanț de creștere** a fluxului f , iar egalitatea justifică denumirea lui $r(\mu)$ drept **capacitatea reziduală** a lanțului μ .

Definiția 2.7.9. Fluxul f' definit în lema anterioară se numește **fluxul obținut prin mărirea fluxului f de-a lungul lanțului de creștere μ** și se notează cu

$$f' = f \oplus r(\mu).$$

Exemplul 2.7.7. Pentru rețeaua R și fluxul f reprezentate în Figura 2.7.2, lanțul $\mu = [1, 2, 3, 4, 5, 6]$ este un lanț de creștere având reziduul $r(\mu) = 1$. Fluxul $f' = f \oplus r(\mu)$ obținut prin aplicarea lemei anterioare este reprezentat în Figura 2.7.3. Valoarea acestui flux este $v(f') = v(f) + r(\mu) = 8 + 1 = 9$.

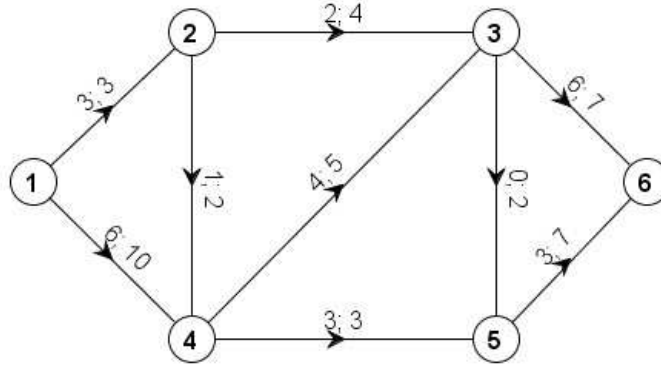


Figura 2.7.3:

Definiția 2.7.10. Fie $R = (G, s, t, c)$ o rețea, unde $G = (V, E)$. O **secțiune (tăietură)** în rețeaua R este o pereche $(S, T) \in V \times V$ ce verifică următoarele proprietăți:

$$S \cup T = V, \quad S \cap T = \emptyset, \\ s \in S, \quad t \in T.$$

Observația 2.7.5. Dacă (S, T) este o secțiune în rețeaua $R = (G, s, t, c)$, unde $G = (V, E)$, atunci $V = S \cup T$ este o partiție a mulțimii V a nodurilor rețelei (adică $V = S \cup T$, $S \neq \emptyset$, $T \neq \emptyset$, $S \cap T = \emptyset$) a.î. $s \in S$ și $t \in T$. Rezultă că numărul de secțiuni ale rețelei R este egal cu numărul de submulțimi $S \setminus \{s\} \subseteq V \setminus \{s, t\}$, deci cu

$$2^{n-2}, \text{ unde } n = \text{card}(V).$$

Definiția 2.7.11. Fie $R = (G, s, t, c)$ o rețea, unde $G = (V, E)$. Pentru orice secțiune (S, T) în rețeaua R , numărul

$$c(S, T) = \sum_{i \in S} \sum_{j \in T} c(i, j)$$

se numește **capacitatea secțiunii** (S, T) .

Observația 2.7.6. Conform relației (2.7.1), capacitatea unei secțiuni (S, T) este suma capacităților tuturor arcelor ce au extremitatea inițială în S și extremitatea finală în T .

Exemplul 2.7.8. Pentru rețeaua reprezentată în Figura 2.7.1,

$$(S, T) = (\{1, 2, 5\}, \{3, 4, 6\})$$

este o secțiune având capacitatea

$$c(S, T) = c(1, 4) + c(2, 3) + c(2, 4) + c(5, 6) = 10 + 4 + 2 + 7 = 23.$$

Lema 2.7.3. Fie $R = (G, s, t, c)$ o rețea. Pentru orice flux f și orice secțiune (S, T) în rețeaua R avem

$$v(f) = \sum_{i \in S} \sum_{j \in T} (f(i, j) - f(j, i))$$

(adică valoarea fluxului este egală cu fluxul net ce traversează secțiunea).

Lema 2.7.4. Fie $R = (G, s, t, c)$ o rețea.

a) Pentru orice flux f și orice secțiune (S, T) în rețeaua R avem

$$v(f) \leq c(S, T).$$

b) Dacă f^* este un flux și (S^*, T^*) este o secțiune în rețeaua R astfel încât

$$v(f^*) = c(S^*, T^*),$$

atunci f^* este un flux de valoare maximă și (S^*, T^*) este o secțiune de capacitate minimă în rețeaua R .

Teorema 2.7.1 (de caracterizare a unui flux de valoare maximă). Fie $R = (G, s, t, c)$ o rețea. Un flux f în rețeaua R este un flux de valoare maximă dacă și numai dacă nu există lanțuri de creștere în rețeaua R relativ la fluxul f .

Demonstrație. "⇒" Fie f un flux de valoare maximă în R . Dacă ar exista un lanț de creștere μ în R relativ la fluxul f , atunci conform Lemei 2.7.2 ar rezulta că funcția $f' = f \oplus r(\mu)$ ar fi un flux în R și $v(f') = v(f) + r(\mu) > v(f)$ (conform Observației 2.7.4) contradicție. Deci nu există lanțuri de creștere în R relativ la fluxul f .

" $\Leftarrow$ " Fie f un flux în R a.î. nu există lanțuri de creștere în R relativ la f . Fie

$$\begin{cases} S = \{i \in V \mid \text{există } C\text{-lanțuri de la } s \text{ la } i \text{ în } R \text{ relativ la } f\}, \\ T = \{i \in V \mid \text{nu există } C\text{-lanțuri de la } s \text{ la } i \text{ în } R \text{ relativ la } f\}. \end{cases} \quad (2.7.4)$$

Evident, $S \cup T = V$ și $S \cap T = \emptyset$.

De asemenea, avem $s \in S$ (deoarece $[s]$ este un C -lanț de la s la s în R relativ la f) și $t \in T$ (deoarece nu există lanțuri de creștere, adică C -lanțuri de la s la t în R relativ la f). Deci (S, T) este o secțiune în rețeaua R .

Fie $i \in S$ și $j \in T$ arbitrar fixați. Deoarece $i \in S$ rezultă că există un C -lanț $\mu = [s = x_0, x_1, \dots, x_k = i]$ în R relativ la f . Atunci $x_0, x_1, \dots, x_k \in S$ (sublanțurile C -lanțului μ sunt tot C -lanțuri), deci $j \notin V(\mu)$.

Dacă am avea $f(i, j) < c(i, j)$ sau $f(j, i) = 0$, atunci ar rezulta că $[s = x_0, x_1, \dots, x_k = i, j]$ ar fi un C -lanț în R relativ la f , ceea ce ar contrazice apartenența $j \in T$. Deci

$$f(i, j) = c(i, j) \text{ și } f(j, i) = 0,$$

pentru orice $i \in S$ și $j \in T$.

Aplicând Lema 2.7.3, avem

$$v(f) = \sum_{i \in S} \sum_{j \in T} (f(i, j) - f(j, i)) = \sum_{i \in S} \sum_{j \in T} (c(i, j) - 0) = c(S, T),$$

deci conform Lemei 2.7.4 rezultă că f este un flux de valoare maximă în rețeaua R (iar (S, T) este o secțiune de capacitate minimă în rețeaua R). $\square$

Observația 2.7.7. Dacă se cunoaște un flux f de valoare maximă într-o rețea R , atunci formulele (2.7.4) determină o secțiune (S, T) de capacitate minimă, deci orice algoritm pentru determinarea unui flux de valoare maximă bazat pe caracterizarea dată în teorema anterioară rezolvă și problema determinării unei secțiuni de capacitate minimă.

Definiția 2.7.12. Fie f un flux în rețeaua $R = (G, s, t, c)$, unde $G = (V, E)$.

Funcția $r : V \times V \rightarrow \mathbb{R}_+$ definită prin

$$r(i, j) = \begin{cases} c(i, j) - f(i, j), & \text{dacă } f(i, j) < c(i, j), \\ f(j, i), & \text{dacă } f(j, i) > 0 \text{ și } f(i, j) = c(i, j), \\ 0, & \text{în rest} \end{cases}$$

se numește **capacitatea reziduală (reziduul)** a rețelei R relativ la fluxul f , iar graful orientat $G_f = (V, E_f)$ definit prin

$$E_f = \{(i, j) \in V \times V \mid r(i, j) > 0\}$$

se numește **graful rezidual** al rețelei R relativ la fluxul f .

Observația 2.7.8. Fie f un flux în rețeaua $R = (G, s, t, c)$. Fie r și G_f capacitatea reziduală, respectiv graful rezidual ale rețelei R relativ la fluxul f . Fie $\mu = [x_0, x_1, \dots, x_k]$ un lanț elementar în graful G . Evident, avem echivalențele:

μ este un C -lanț în R relativ la $f \Leftrightarrow r(x_i, x_{i+1}) > 0, \forall i \in \{0, \dots, k-1\} \Leftrightarrow$

$\Leftrightarrow (x_0, x_1, \dots, x_k)$ este un drum elementar în graful rezidual G_f ;

μ este un lanț de creștere în R relativ la $f \Leftrightarrow$

$\Leftrightarrow (x_0, x_1, \dots, x_k)$ este un drum elementar de la s la t în graful rezidual G_f .

Exemplul 2.7.9. Pentru rețeaua R și fluxul f' reprezentate în Figura 2.7.3, graful rezidual $G_{f'}$ este reprezentat în Figura 2.7.4, capacitățile reziduale fiind menționate pe arce.

$(1, 4, 2, 3, 6)$ este un drum elementar în graful rezidual $G_{f'}$, deci $\mu' = [1, 4, 2, 3, 6]$ este un lanț de creștere în rețeaua R relativ la fluxul f' .

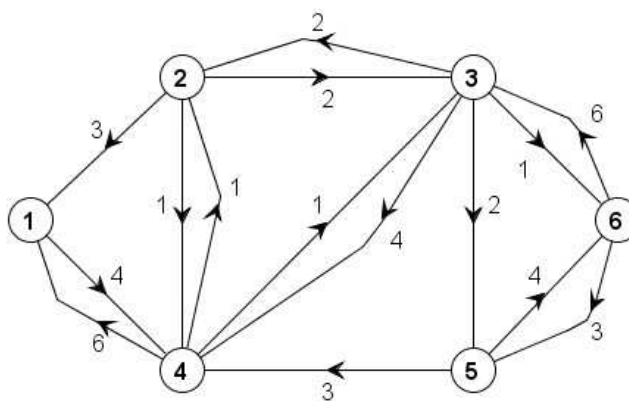


Figura 2.7.4:

Conform rezultatelor de mai sus obținem următorul algoritm pentru determinarea unui flux de valoare maximă într-o rețea.

Algoritmul 2.7.1 (Ford-Fulkerson). Fie $R = (G, s, t, c)$ o rețea, unde $G = (V, E)$.

Conform Teoremei 2.7.1, schița algoritmului, descrisă în pseudocod, are forma

```

FORD_FULKERSON :
   $f \leftarrow 0;$                                 // sau  $f \leftarrow f_0$ , unde  $f_0$  este un flux
                                              // disponibil inițial

  repeat
    if (EXISTĂ_LANȚ_DE_CREȘTERE) then
      // există lanțuri de creștere
      // relativ la fluxul curent  $f$ 
       $\mu \leftarrow \text{LANȚ\_DE\_CREȘTERE};$         // se determină
      // un astfel de lanț de creștere
       $f \leftarrow f \oplus r(\mu);$  // se mărește valoarea fluxului curent,
      // de-a lungul lanțului de creștere
  while (EXISTĂ_LANȚ_DE_CREȘTERE);
      // nu mai există lanțuri de creștere,
      // deci fluxul curent este de valoare maximă
  AFIȘARE( $f$ ); // se afișează fluxul de valoare maximă

```

În continuare detaliem implementarea acestui algoritm. Presupunem că

$$V = \{1, 2, \dots, n\}, \quad s = 1, \quad t = n,$$

cu $n \geq 2$.

Pentru depistarea și memorarea eventualelor lanțuri de creștere relativ la fluxul curent vom utiliza doi vectori SEL și $TATA$ având semnificația

$$SEL[i] = \begin{cases} 1, & \text{dacă există un } C\text{-lanț de la nodul } s = 1 \text{ la nodul } i, \\ 0, & \text{în caz contrar,} \end{cases}$$

$$TATA[i] = \text{predecesorul direct al nodului } i \text{ pe } C\text{-lanțul} \\ \text{de la nodul } s = 1 \text{ la nodul } i,$$

$$\forall i \in \{1, \dots, n\}.$$

Algoritmul descris în pseudocod are forma (detaliată)

```

FORD_FULKERSON :
CITIRE_REȚEA;                                // se citește rețeaua dată
for  $i = \overline{1, n}$  do
  for  $j = \overline{1, n}$  do  $f[i, j] \leftarrow 0;$           // fluxul inițial
 $vmax \leftarrow 0;$                                 // valoarea fluxul maxim
repeat // se caută lanțuri de creștere a fluxului curent,
  // folosind Observația 2.7.8
  CALCUL_REZIDUURI; // se determină reziduul și
  // graful rezidual ale rețelei relativ la fluxul curent
   $SEL[1] \leftarrow 1;$  // se selectează nodul sursă
  for  $i = \overline{2, n}$  do  $SEL[i] \leftarrow 0;$ 
   $TATA[1] \leftarrow 0;$  // pt. memorarea C-lanțurilor
  // ce pornesc din nodul sursă
  DF(1); // se parcurge graful rezidual, memorând
  // C-lanțurile ce pornesc din nodul sursă;
  // parcurgerea DF, poate fi înlocuită cu
  // parcurgerea BF (Algoritmul Edmonds-Karp)
  if ( $SEL[n] = 1$ ) then // s-a selectat nodul destinație,
  // deci există lanț de creștere a fluxului
   $rmin \leftarrow \infty;$  // reziduul minim, de-a lungul
  // lanțului de creștere
  DET_LANT_CR; // se determină lanțul de creștere
  // a fluxului și reziduul minim
  MĂRIRE_FLUX; // se mărește fluxul curent, de-a
  // lungul lanțului de creștere
   $vmax \leftarrow vmax + rmin;;$  // se actualizează valoarea
  // fluxului curent
while ( $SEL[n] = 1$ );
  // nu mai există lanțuri de creștere a fluxului,
  // deci fluxul curent este maxim
AFIȘARE_REZULTATE;
  // se afișează fluxul maxim și valoarea sa,
  // eventual și secțiunea de capacitate minimă,
  // calculată conform (2.7.4) astfel:
  //  $S = \{i \in \{1, \dots, n\} \mid SEL[i] = 1\},$ 
  //  $T = \{i \in \{1, \dots, n\} \mid SEL[i] = 0\}$ 

```

Funcțiile utilizate sunt descrise în continuare.

CALCUL_REZIDUURI :

```

for  $i = \overline{1, n}$  do
    for  $j = \overline{1, n}$  do
        if ( $f[i, j] < c[i, j]$ ) then
            |  $r[i, j] \leftarrow c[i, j] - f[i, j]$ ;
        else
            if ( $f[j, i] > 0$ ) then
                |  $r[i, j] \leftarrow f[j, i]$ ;
            else
                |  $r[i, j] \leftarrow 0$ ;

```

DF(i) : // recursiv

```

for  $j = \overline{1, n}$  do
    if ( $r[i, j] > 0$  and  $SEL[j] = 0$ ) then
        |  $TATA[j] \leftarrow i$ ;  $SEL[j] \leftarrow 1$ ;
        | DF( $j$ );

```

DET_LANT_CR : // se determină reziduul minim de-a
 // lungul lanțului de creștere, parcurs de la n către 1

```

 $j \leftarrow n$ ;
while ( $j \neq 1$ ) do
    |  $i \leftarrow TATA[j]$ ;
    | if ( $rmin > r[i, j]$ ) then  $rmin \leftarrow r[i, j]$ ;
    |  $j \leftarrow i$ ;

```

MĂRIRE_FLUX : // se mărește fluxul de-a lungul
 // lanțului de creștere, parcurs de la n către 1

```

 $j \leftarrow n$ ;
while ( $j \neq 1$ ) do
    |  $i \leftarrow TATA[j]$ ;
    | if ( $c[i, j] > f[i, j]$ ) then
        | |  $f[i, j] \leftarrow f[i, j] + rmin$ ;
    | else
        | |  $f[j, i] \leftarrow f[j, i] - rmin$ ;
    |  $j \leftarrow i$ ;

```

Teorema 2.7.2 (de corectitudine a Algoritmului Ford-Fulkerson).
 În contextul Algoritmului 2.7.1, fie $R = (G, s, t, c)$ rețeaua dată, unde $G = (V, E)$. Presupunem că toate capacitățile arcelor rețelei sunt numere întregi, adică

$$c(i, j) \in \mathbb{N}, \forall (i, j) \in E.$$

Fie f_0 fluxul inițial. Presupunem că toate componentele fluxului f_0 sunt numere întregi, adică

$$f_0(i, j) \in \mathbb{N}, \forall (i, j) \in E,$$

de exemplu $f_0 = \text{fluxul nul}$.

Fie $\mu_1, \mu_2, \dots, \mu_k$ lanțurile de creștere succesive obținute, $k \geq 0$, și fie $f_1 = f_0 \oplus r_0(\mu_1)$, $f_2 = f_1 \oplus r_1(\mu_2)$, $\dots$, $f_k = f_{k-1} \oplus r_{k-1}(\mu_k)$ fluxurile succesive obținute, unde μ_1 este lanț de creștere relativ la f_0 , μ_2 este lanț de creștere relativ la f_1 , $\dots$, μ_k este lanț de creștere relativ la f_{k-1} , și nu mai există lanțuri de creștere relativ la f_k . Atunci f_k este un flux de valoare maximă în rețeaua R .

Mai mult, toate componentele fluxului f_k și valoarea acestuia sunt numere întregi.

Demonstrație. Conform Lemei 2.7.2, $f_1, f_2, \dots, f_k$ sunt fluxuri și $v(f_1) = v(f_0) + r_0(\mu_1)$, $v(f_2) = v(f_1) + r_1(\mu_2) = v(f_0) + r_0(\mu_1) + r_1(\mu_2)$, $\dots$, $v(f_k) = v(f_{k-1}) + r_{k-1}(\mu_k) = v(f_0) + r_0(\mu_1) + r_1(\mu_2) + \dots + r_{k-1}(\mu_k)$.

Deoarece toate capacitățile $c(i, j)$, $i, j \in V$, și toate componentele $f_0(i, j)$, $i, j \in V$ ale fluxului inițial sunt numere întregi, conform Definiției 2.7.7 și Lemei 2.7.2 rezultă succesiv că

$$\begin{aligned} (r_0)_{\mu_1}(i, j) &\in \mathbb{N}^*, \forall (i, j) \in E(\mu_1), r_0(\mu_1) \in \mathbb{N}^*, \\ f_1(i, j) &\in \mathbb{N}, \forall i, j \in V, v(f_1) \in \mathbb{N}^*; \\ (r_1)_{\mu_2}(i, j) &\in \mathbb{N}^*, \forall (i, j) \in E(\mu_2), r_1(\mu_2) \in \mathbb{N}^*, \\ f_2(i, j) &\in \mathbb{N}, \forall i, j \in V, v(f_2) \in \mathbb{N}^*; \dots, \\ (r_{k-1})_{\mu_k}(i, j) &\in \mathbb{N}^*, \forall (i, j) \in E(\mu_k), r_{k-1}(\mu_k) \in \mathbb{N}^*, \\ f_k(i, j) &\in \mathbb{N}, \forall i, j \in V, v(f_k) \in \mathbb{N}^*. \end{aligned} \quad (2.7.5)$$

Deci

$$v(f_k) = v(f_0) + r_0(\mu_1) + r_1(\mu_2) + \dots + r_{k-1}(\mu_k) \geq v(f_0) + k.$$

Evident, $(\{s\}, V \setminus \{s\})$ este o secțiune în R având capacitatea

$$c(\{s\}, V \setminus \{s\}) = \sum_{j \in V \setminus \{s\}} c(s, j) \leq (\text{card}(V) - 1)c_{\max},$$

unde

$$c_{\max} = \max\{c(i, j) \mid (i, j) \in E\}$$

(capacitatea maximă a arcelor rețelei). Conform Lemei 2.7.4 avem

$$v(f_k) \leq c(\{s\}, V \setminus \{s\}),$$

și astfel obținem

$$k \leq (\text{card}(V) - 1)c_{\max} - v(f_0),$$

deci numărul k de pași ai algoritmului (numărul de lanțuri de creștere succesive construite) este finit, adică există $k \in \mathbb{N}$ astfel încât rețeaua R nu mai conține lanțuri de creștere relativ la fluxul f_k . Conform Teoremei 2.7.1 rezultă că f_k este un flux de valoare maximă în R . Conform (2.7.5), toate componentele fluxului f_k și valoarea acestuia sunt numere întregi. $\square$

Exemplul 2.7.10. Pentru rețeaua R din Figura 2.7.1, aplicarea Algoritmului Ford-Fulkerson este evidențiată în Figurile 2.7.5-2.7.10.

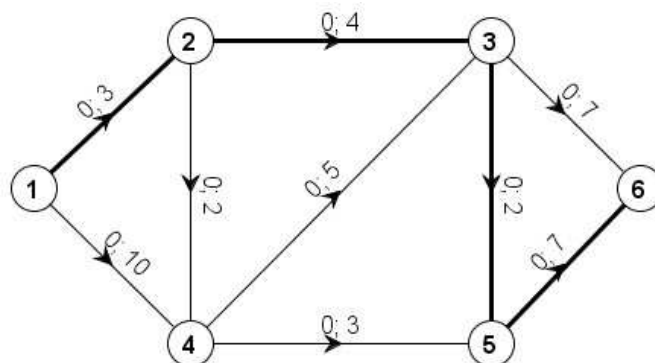


Figura 2.7.5:

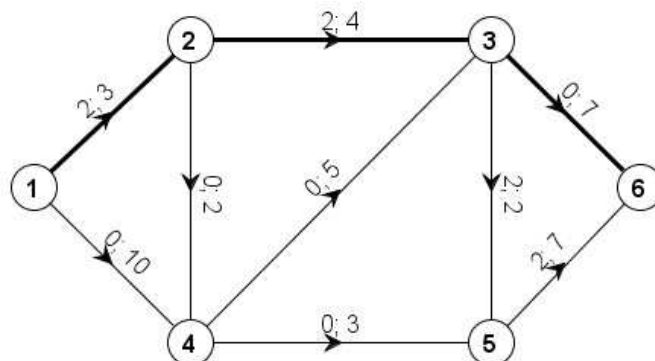


Figura 2.7.6:

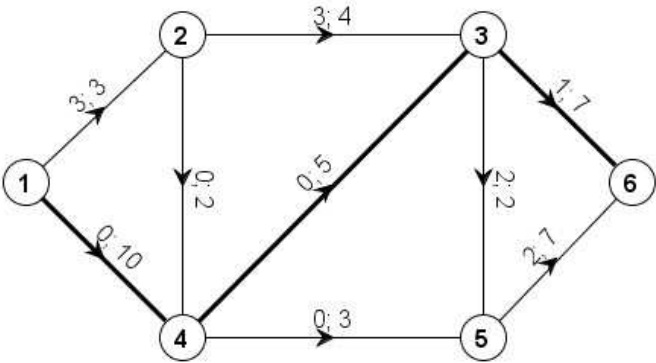


Figura 2.7.7:

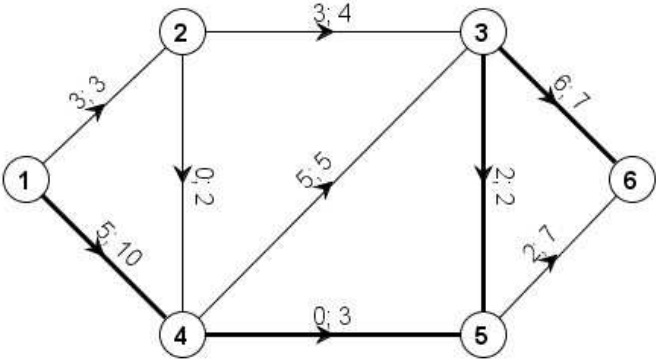


Figura 2.7.8:

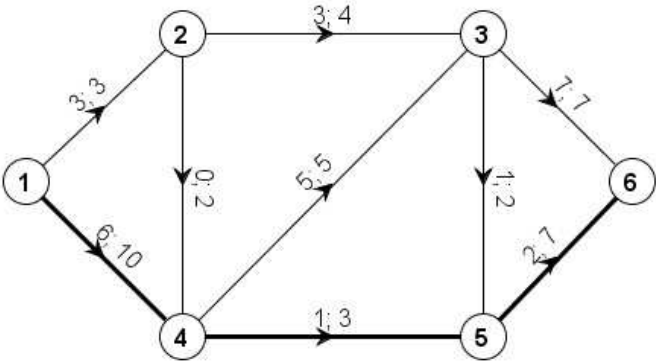


Figura 2.7.9:

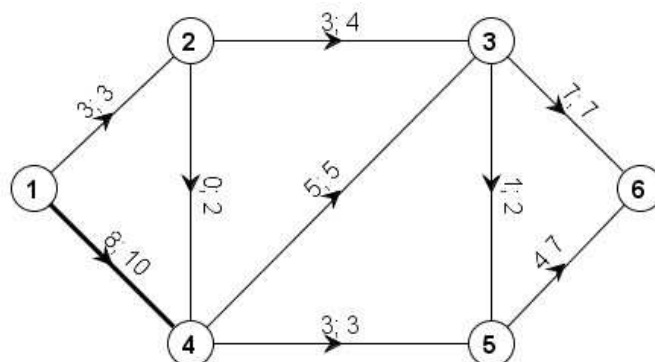


Figura 2.7.10:

Lanțurile succesive de creștere relativ la fluxul curent sunt evidențiate prin îngroșare, și anume:

$$\mu_1 = [1, 2, 3, 5, 6],$$

$$\mu_2 = [1, 2, 3, 6],$$

$$\mu_3 = [1, 4, 3, 6],$$

$$\mu_4 = [1, 4, 5, 3, 6],$$

$$\mu_5 = [1, 4, 5, 6].$$

Fluxul reprezentat în Figura 2.7.10 este de valoare maximă, deoarece nu mai există lanțuri de creștere în R relativ la acest flux. Valoarea acestui flux este 11.

Pentru acest flux maxim, avem C -lanțuri de la nodul sursă $s = 1$ doar la nodurile 1 și 4, aceste C -lanțuri sunt evidențiate prin îngroșare în Figura 2.7.10. Conform (2.7.4) rezultă că

$$(S, T) = (\{1, 4\}, \{2, 3, 5, 6\})$$

este o secțiune de capacitate minimă în R .

Observația 2.7.9. Algoritmul Ford-Fulkerson este specific **metodei de programare Greedy**. Pentru o rețea cu n noduri și m arce având toate capacitățile numere întregi și pentru un flux inițial având toate componentele numere întregi, de exemplu fluxul nul, conform demonstrației Teoremei 2.7.2 rezultă că numărul de lanțuri de creștere necesare măririi succesive a fluxului inițial, până se ajunge la un flux de valoare maximă, este de cel mult $(n - 1)c_{max}$, unde c_{max} este capacitatea maximă a arcelor rețelei. Pentru fiecare flux curent, algoritmul necesită calculul reziduurilor celor m arce, parcurgerea acestora pentru depistarea unui eventual lanț de creștere, determinarea acestui lanț (de lungime cel mult m) și mărirea fluxului curent de-a

lungul lanțului de creștere, operații având complexitatea $O(m)$. Rezultă că Algoritmul Ford-Fulkerson are complexitatea $O(mnc_{max})$.

Observația 2.7.10. Pentru o rețea având toate capacitățile numere raționale, putem aplica Algoritmul Ford-Fulkerson astfel:

- se înmulțesc toate capacitățile arcelor cu numitorul lor comun M , obținându-se o rețea având toate capacitățile numere întregi;
- se determină un flux de valoare maximă în această rețea, aplicând Algoritmul Ford-Fulkerson (pentru un flux inițial având toate componentele numere întregi, de exemplu fluxul nul);
- se împart toate componentele acestui flux prin M , obținându-se un flux de valoare maximă în rețeaua inițială.

Evident, toate componentele acestui flux maxim și valoarea lui sunt numere raționale.

Observația 2.7.11. Pentru o rețea având și capacități numere iraționale, Algoritmul Ford-Fulkerson poate să fie inoperabil, fiind posibil ca numărul de pași ai algoritmului (numărul de lanțuri de creștere succesive construite) să fie infinit, deoarece valorile reziduale ale lanțurilor de creștere succesive construite pot forma o serie convergentă.

Totuși, prin impunerea unor modalități judicioase de construire a lanțurilor de creștere succesive în cadrul Algoritmului Ford-Fulkerson, se obțin algoritmi care rezolvă problema fluxului maxim și în cazul capacităților iraționale. Un astfel de algoritm, bazat pe parcurgerea în lățime (BF) a grafului rezidual, este **Algoritmul Edmonds-Karp**.

Tema 3

Recursivitate

3.1 Introducere

Recurența este o tehnică de rezolvare a problemelor prin reducere repetată la subprobleme asemănătoare și de dimensiuni mai mici, până când dimensiunea obținută permite rezolvarea directă.

Recursivitatea este modalitatea naturală de implementare algoritmică a recurențelor și constă în definirea unor funcții având ca parametru dimensiunea problemei și care se autoapelează (*apel recursiv*) fie direct, fie indirect (prin intermediul altor funcții), până când dimensiunea permite rezolvarea directă.

Condițiile a căror verificare asigură posibilitatea rezolvării directe, deci oprirea procesului recursiv de calcul, sunt numite *condiții inițiale* pentru recurența considerată.

În mod uzual, o recurență este exprimată prin relații ce exprimă soluția problemei în funcție de soluțiile subproblemelor apelate, numite *relații de recurență*.

Implementările recursive ale recurențelor au de obicei ca dezavantaj faptul că autoapelarea funcției recursive impune nu numai memorarea valorilor deja calculate, ci și apelarea și calculul aceleiași valori de mai multe ori, ceea ce poate conduce la creșterea exponențială a timpului de execuție. În astfel de situații este de preferat ca rezolvarea problemei (determinarea soluțiilor ce verifică relațiile de recurență date) să se efectueze, atunci când este posibil, prin implementări nerecursive.

De exemplu, *metoda programării dinamice* presupune rezolvarea unei probleme prin descompunerea în subprobleme de același tip și de dimensiuni mai mici, ce pot avea (sub)subprobleme comune. Rezolvarea relațiilor de recurență aferente prin implementări recursive, în care soluția problemei

mari apelează soluțiile subproblemelor de dimensiuni mai mici (*strategia top-down*), necesită de obicei un timp de calcul exponențial, deoarece rezolvarea unei aceleiași subprobleme poate fi reluată de foarte multe ori, din cauza apelării ei repetate. Mult mai eficient este ca subproblemele să fie rezolvate în ordinea crescătoare a dimensiunii lor (*strategia bottom-up*), iar, pentru a evita repetarea procesului de calcul necesar rezolvării aceleiași subprobleme, soluțiile subproblemelor să fie memorate temporar, pentru utilizarea la posibilele reapelări.

Domeniul de aplicabilitate al tehnicilor recurente (recursive) este vast: combinatorica și teoria numerelor, metode de proiectare și de analiză a algoritmilor, algebra computațională, cercetările operaționale, inteligența artificială, etc.

Exemplul 3.1.1. Considerăm următorul algoritm recursiv, descris în pseudocod.

```

f( $A, n$ ):           // calculează  $f(A, n)$ ,  $A = (a_0, a_1, \dots, a_n)$ ,  $n \in \mathbb{N}$ 
if  $n = 0$  then           // condiție inițială
|   return  $a_0 + 2$ ;
else
|   if  $n = 1$  then           // condiție inițială
|   |   return  $a_1 - a_0 + 1$ ;
|   else
|   |    $r \leftarrow f(A, n - 2)$ ;           // apel recursiv
|   |    $a_{n-2} \leftarrow a_{n-2} + 1$ ;  $a_{n-1} \leftarrow a_{n-1} + 1$ ;
|   |    $b \leftarrow f(A, n - 2)$ ;  $c \leftarrow f(A, n - 1)$ ;           // apeluri recursive
|   |    $r \leftarrow r + b + c$ ;
|   |   for  $i = \overline{1, n - 1}$  do
|   |   |    $a_i \leftarrow a_i + a_{i+1} - a_{i-1} + 1$ ;
|   |    $b \leftarrow f(A, n - 2)$ ;  $c \leftarrow f(A, n - 1)$ ;           // apeluri recursive
|   |    $r \leftarrow r + b + c$ ;
|   |   for  $i = \overline{0, n - 1}$  do
|   |   |    $a_i \leftarrow a_i + a_n$ ;
|   |    $b \leftarrow f(A, n - 2)$ ;  $c \leftarrow f(A, n - 1)$ ;           // apeluri recursive
|   |    $r \leftarrow r + b + c$ ;
|   |   return  $r$ ;

```

Pentru analiza complexității acestui algoritm, vom evalua doar numărul de adunări și scăderi, celelalte operații nedepășind ordinul de creștere al acestora. Observăm că acest număr depinde doar dimensiunea $n + 1$ a vectorului A și nu și de valorile componentelor $a_0, a_1, \dots, a_n$.

Fie x_n numărul de adunări și scăderi efectuate de algoritm pentru un vector (arbitrar) $A = (a_0, a_1, \dots, a_n)$.

Din descrierea algoritmului avem $x_0 = 1$, $x_1 = 2$ și

$$x_n = x_{n-2} + 2 + x_{n-2} + x_{n-1} + 2 + 3(n-1) + x_{n-2} + x_{n-1} + 2 + n \\ + x_{n-2} + x_{n-1} + 2, \forall n \geq 2.$$

Astfel x_n verifică relația de recurență

$$x_n = 3x_{n-1} + 4x_{n-2} + 4n + 5, \forall n \in \mathbb{N}, n \geq 2, x_0 = 1, x_1 = 2. \quad (3.1.1)$$

Vom rezolva această recurență în una din secțiunile următoare și astfel vom putea evalua ordinul de complexitate al algoritmului de mai sus.

În continuare vom prezenta metode de rezolvare a unor relații de recurență clasice.

3.2 Recurența liniară de ordinul I cu termen liber constant

Această recurență are forma

$$x_n = ax_{n-1} + b, \forall n \in \mathbb{N}^*, \quad (3.2.1)$$

unde parametrii $a, b \in \mathbb{R}$ și primul termen $x_0 \in \mathbb{R}$ sunt numere date (cunoscute).

Exemplul 3.2.1. Considerăm relația de recurență liniară

$$x_n = 2x_{n-1} + 3, \forall n \in \mathbb{N}^*, x_0 = 1.$$

Avem

$$\begin{aligned} x_1 &= 2x_0 + 3 = 2 \cdot 1 + 3 = 5, \\ x_2 &= 2x_1 + 3 = 2 \cdot 5 + 3 = 13, \\ x_3 &= 2x_2 + 3 = 2 \cdot 13 + 3 = 29, \\ x_4 &= 2x_3 + 3 = 2 \cdot 29 + 3 = 61, \\ x_5 &= 2x_4 + 3 = 2 \cdot 61 + 3 = 125. \end{aligned}$$

Rezolvarea matematică

Avem următoarele două cazuri.

Cazul 1. Dacă $a = 1$, atunci recurența (3.2.1) devine

$$x_n = x_{n-1} + b, \forall n \in \mathbb{N}^*,$$

unde parametrul $b \in \mathbb{R}$ și primul termen $x_0 \in \mathbb{R}$ sunt numere date. Astfel șirul (x_n) este o *progresie aritmetică* de rație b , deci

$$x_n = x_0 + nb, \forall n \in \mathbb{N}$$

(relație ce poate fi demonstrată, de exemplu, prin inducție).

Cazul 2. Dacă $a \neq 1$, atunci notând

$$x_n = y_n + c, \forall n \in \mathbb{N}, \quad (3.2.2)$$

cu $c \in \mathbb{R}$, recurența (3.2.1) devine

$$y_n + c = a(y_{n-1} + c) + b, \forall n \in \mathbb{N}^*,$$

adică

$$y_n = ay_{n-1} + c(a-1) + b, \forall n \in \mathbb{N}^*.$$

Luând $c(a-1) + b = 0$, adică

$$c = -\frac{b}{a-1},$$

obținem

$$y_n = ay_{n-1}, \forall n \in \mathbb{N}^*.$$

Aceasta este o *progresie geometrică* de rație a , numită și *recurență liniară omogenă de ordinul I*. Rezultă că

$$y_n = a^n y_0, \forall n \in \mathbb{N}$$

(relație ce poate fi demonstrată, de exemplu, prin inducție), iar

$$y_0 = x_0 - c = x_0 + \frac{b}{a-1},$$

deci înlocuind în (3.2.2) obținem că soluția generală a recurenței (3.2.1) are forma

$$x_n = \left(x_0 + \frac{b}{a-1}\right) a^n - \frac{b}{a-1}, \forall n \in \mathbb{N}. \quad (3.2.3)$$

Exemplul 3.2.2. Reluăm relația de recurență liniară

$$x_n = 2x_{n-1} + 3, \forall n \in \mathbb{N}^*, x_0 = 1,$$

din Exemplul 3.2.1. Aplicând (3.2.3) obținem

$$x_n = \left(1 + \frac{3}{2-1}\right) 2^n - \frac{3}{2-1} = 4 \cdot 2^n - 3 = 2^{n+2} - 3, \forall n \in \mathbb{N}.$$

De exemplu, $x_5 = 2^7 - 3 = 125$.

3.3 Recurența liniară omogenă de ordinul al II-lea

Această recurență are forma

$$x_n = ax_{n-1} + bx_{n-2}, \quad \forall n \in \mathbb{N}, \quad n \geq 2, \quad (3.3.1)$$

unde parametrii $a, b \in \mathbb{R}$ și primii doi termeni $x_0, x_1 \in \mathbb{R}$ sunt numere date (cunoscute). Presupunem că

$$b \neq 0$$

(deoarece pentru $b = 0$ recurența devine de ordinul I).

Exemplul 3.3.1. Considerăm relația de recurență liniară

$$x_n = 2x_{n-1} + 15x_{n-2}, \quad \forall n \in \mathbb{N}, \quad n \geq 2, \quad x_0 = 1, \quad x_1 = 2.$$

Avem

$$\begin{aligned} x_2 &= 2x_1 + 15x_0 = 2 \cdot 2 + 15 \cdot 1 = 19, \\ x_3 &= 2x_2 + 15x_1 = 2 \cdot 19 + 15 \cdot 2 = 68, \\ x_4 &= 2x_3 + 15x_2 = 2 \cdot 68 + 15 \cdot 19 = 421, \\ x_5 &= 2x_4 + 15x_3 = 2 \cdot 421 + 15 \cdot 68 = 1862. \end{aligned}$$

Rezolvarea matematică

Pentru rezolvarea recurenței (3.3.1), îi asociem ecuația de gradul al doilea

$$r^2 = ar + b, \quad \text{adică} \quad r^2 - ar - b = 0, \quad (3.3.2)$$

numită *ecuația caracteristică* a acestei recurențe. Determinantul ecuației este

$$\Delta = a^2 + 4b.$$

Fie $r_1, r_2 \in \mathbb{C}$ rădăcinile acestei ecuații, adică

$$r_{1,2} = \begin{cases} \frac{a \pm \sqrt{\Delta}}{2}, & \text{dacă } \Delta \geq 0, \\ \frac{a \pm i\sqrt{-\Delta}}{2}, & \text{dacă } \Delta < 0 \end{cases}$$

(unde $i \in \mathbb{C} \setminus \mathbb{R}$, $i^2 = -1$).

Avem următoarele două cazuri.

Cazul 1. Dacă $r_1 \neq r_2$, atunci soluția generală a recurenței (3.3.1) are forma

$$x_n = c_1 r_1^n + c_2 r_2^n, \quad \forall n \in \mathbb{N},$$

unde constantele $c_1, c_2 \in \mathbb{C}$ se determină în mod unic ca soluții ale sistemului de ecuații liniare

$$\begin{cases} c_1 + c_2 = x_0, \\ c_1 r_1 + c_2 r_2 = x_1 \end{cases}$$

(această formă poate fi demonstrată, de exemplu, prin inducție). Determinantul asociat sistemului este

$$\Delta_0 = \begin{vmatrix} 1 & 1 \\ r_1 & r_2 \end{vmatrix} = r_2 - r_1 \neq 0,$$

deci sistemul are o soluție unică, dată de *formulele lui Cramer*:

$$c_1 = \frac{\Delta_1}{\Delta_0} = \frac{\begin{vmatrix} x_0 & 1 \\ x_1 & r_2 \end{vmatrix}}{\Delta_0} = \frac{x_0 r_2 - x_1}{r_2 - r_1},$$

$$c_2 = \frac{\Delta_2}{\Delta_0} = \frac{\begin{vmatrix} 1 & x_0 \\ r_1 & x_1 \end{vmatrix}}{\Delta_0} = \frac{x_1 - x_0 r_1}{r_2 - r_1}.$$

Astfel soluția generală a recurenței (3.3.1) este, în acest caz,

$$x_n = \frac{(x_0 r_2 - x_1) r_1^n + (x_1 - x_0 r_1) r_2^n}{r_2 - r_1}, \quad \forall n \in \mathbb{N}. \quad (3.3.3)$$

Exemplul 3.3.2. Reluăm relația de recurență liniară

$$x_n = 2x_{n-1} + 15x_{n-2}, \quad \forall n \in \mathbb{N}, \quad n \geq 2, \quad x_0 = 1, \quad x_1 = 2$$

din Exemplul 3.3.1. Ecuația caracteristică este

$$r^2 = 2r + 15, \text{ adică } r^2 - 2r - 15 = 0.$$

Avem $\Delta = 4 + 60 = 64$, $r_1 = \frac{2-8}{2} = -3$, $r_2 = \frac{2+8}{2} = 5$, deci $r_1 \neq r_2$.

Aplicând (3.3.3) obținem

$$x_n = \frac{3 \cdot (-3)^n + 5 \cdot 5^n}{8} = \frac{5^{n+1} + (-1)^n \cdot 3^{n+1}}{8}, \quad \forall n \in \mathbb{N}.$$

De exemplu, $x_5 = \frac{5^6 - 3^6}{8} = \frac{15625 - 729}{8} = \frac{14896}{8} = 1862$.

Cazul 2. Dacă $r_1 = r_2$, atunci soluția generală a recurenței (3.3.1) are forma

$$x_n = r_1^n(c_1 + c_2n), \quad \forall n \in \mathbb{N},$$

unde constantele $c_1, c_2 \in \mathbb{C}$ se determină în mod unic ca soluții ale sistemului de ecuații liniare

$$\begin{cases} c_1 = x_0, \\ r_1(c_1 + c_2) = x_1 \end{cases}$$

(această formă poate fi demonstrată, din nou, prin inducție). Evident, sistemul are o soluție unică, dată de:

$$\begin{aligned} c_1 &= x_0, \\ c_2 &= \frac{x_1 - x_0r_1}{r_1}. \end{aligned}$$

Astfel soluția generală a recurenței (3.3.1) este, în acest caz,

$$x_n = [(x_1 - x_0r_1)n + x_0r_1]r_1^{n-1}, \quad \forall n \in \mathbb{N}. \quad (3.3.4)$$

Exemplul 3.3.3. Considerăm relația de recurență liniară

$$x_n = 4x_{n-1} - 4x_{n-2}, \quad \forall n \in \mathbb{N}, \quad n \geq 2, \quad x_0 = 3, \quad x_1 = 9.$$

Ecuația caracteristică este

$$r^2 = 4r - 4, \quad \text{adică} \quad r^2 - 4r + 4 = 0.$$

Avem $\Delta = 16 - 16 = 0$, $r_1 = r_2 = \frac{4}{2} = 2$. Aplicând (3.3.4) obținem

$$x_n = (3n + 6)2^{n-1}, \quad \forall n \in \mathbb{N}.$$

De exemplu, $x_5 = 21 \cdot 16 = 336$.

Observația 3.3.1. Analog se definesc și rezolvă relațiile de recurență liniară omogene de ordine mai mari sau egale cu trei.

3.4 Recurența liniară de ordinul al II-lea cu termen liber constant

Această recurență are forma

$$x_n = ax_{n-1} + bx_{n-2} + c, \quad \forall n \in \mathbb{N}, \quad n \geq 2, \quad (3.4.1)$$

unde parametrii $a, b, c \in \mathbb{R}$ și primii doi termeni $x_0, x_1 \in \mathbb{R}$ sunt numere date (cunoscute). Presupunem că

$$b \neq 0$$

(deoarece pentru $b = 0$ recurența devine de ordinul I).

Exemplul 3.4.1. Considerăm relația de recurență liniară

$$x_n = 5x_{n-1} - 6x_{n-2} + 4, \forall n \in \mathbb{N}, n \geq 2, x_0 = 2, x_1 = 3.$$

Avem

$$\begin{aligned} x_2 &= 5x_1 - 6x_0 + 4 = 5 \cdot 3 - 6 \cdot 2 + 4 = 7, \\ x_3 &= 5x_2 - 6x_1 + 4 = 5 \cdot 7 - 6 \cdot 3 + 4 = 21, \\ x_4 &= 5x_3 - 6x_2 + 4 = 5 \cdot 21 - 6 \cdot 7 + 4 = 67, \\ x_5 &= 5x_4 - 6x_3 + 4 = 5 \cdot 67 - 6 \cdot 21 + 4 = 213. \end{aligned}$$

Rezolvarea matematică

Avem următoarele două cazuri.

Cazul 1. Dacă $a + b \neq 1$, atunci notând

$$x_n = y_n + d, \forall n \in \mathbb{N}, \quad (3.4.2)$$

cu $d \in \mathbb{R}$, recurența (3.4.1) devine

$$y_n + d = a(y_{n-1} + d) + b(y_{n-2} + d) + c, \forall n \in \mathbb{N}, n \geq 2,$$

adică

$$y_n = ay_{n-1} + by_{n-2} + d(a + b - 1) + c, \forall n \in \mathbb{N}, n \geq 2.$$

Luând $d(a + b - 1) + c = 0$, adică

$$d = -\frac{c}{a + b - 1},$$

obținem

$$y_n = ay_{n-1} + by_{n-2}, \forall n \in \mathbb{N}, n \geq 2, \quad (3.4.3)$$

cu

$$y_0 = x_0 - d = x_0 + \frac{c}{a + b - 1}, y_1 = x_1 - d = x_1 + \frac{c}{a + b - 1}. \quad (3.4.4)$$

Aceasta este o recurență liniară omogenă de ordinul al II-lea și poate fi rezolvată conform metodei descrise în secțiunea anterioară. Înlocuind în (3.4.2), obținem că soluția recurenței neomogene (3.4.1) în cazul $a + b \neq 1$ este

$$x_n = y_n - \frac{c}{a + b - 1}, \forall n \in \mathbb{N}. \quad (3.4.5)$$

Exemplul 3.4.2. Reluăm relația de recurență liniară cu termen liber constant

$$x_n = 5x_{n-1} - 6x_{n-2} + 4, \forall n \in \mathbb{N}, n \geq 2, x_0 = 2, x_1 = 3$$

din Exemplul 3.4.1. Avem $a + b = 5 - 6 \neq 1$. Conform relațiilor (3.4.5), (3.4.3) și (3.4.4), luând

$$y_n = x_n + \frac{c}{a+b-1} = x_n + \frac{4}{-2} = x_n - 2, \forall n \in \mathbb{N},$$

obținem că y_n verifică recurența liniară omogenă

$$y_n = 5y_{n-1} - 6y_{n-2}, \forall n \in \mathbb{N}, n \geq 2, y_0 = x_0 - 2 = 0, y_1 = x_1 - 2 = 1.$$

Ecuatia caracteristică este

$$r^2 = 5r - 6, \text{ adică } r^2 - 5r + 6 = 0.$$

Avem $\Delta = 25 - 24 = 1$, $r_1 = \frac{5-1}{2} = 2$, $r_2 = \frac{5+1}{2} = 3$, deci $r_1 \neq r_2$.

Aplicând (3.3.3) obținem

$$y_n = \frac{(y_0 r_2 - y_1) r_1^n + (y_1 - y_0 r_1) r_2^n}{r_2 - r_1} = \frac{-2^n + 3^n}{1} = 3^n - 2^n, \forall n \in \mathbb{N}.$$

Astfel

$$x_n = y_n + 2 = 3^n - 2^n + 2, \forall n \in \mathbb{N}.$$

De exemplu, $x_5 = 3^5 - 2^5 + 2 = 243 - 32 + 2 = 213$.

Cazul 2. Dacă $a + b = 1$, atunci $a = 1 - b$ și recurența (3.4.1) devine

$$x_n = (1 - b)x_{n-1} + bx_{n-2} + c, \forall n \in \mathbb{N}, n \geq 2,$$

adică

$$x_n - x_{n-1} = -b(x_{n-1} - x_{n-2}) + c, \forall n \in \mathbb{N}, n \geq 2.$$

Notând

$$y_{n-1} = x_n - x_{n-1}, \forall n \in \mathbb{N}^*, \quad (3.4.6)$$

obținem

$$y_{n-1} = -by_{n-2} + c, \forall n \in \mathbb{N}, n \geq 2,$$

adică

$$y_n = -by_{n-1} + c, \forall n \in \mathbb{N}^*, \quad (3.4.7)$$

cu

$$y_0 = x_1 - x_0. \quad (3.4.8)$$

Înlocuind în (3.4.6), obținem

$$\begin{aligned}x_1 - x_0 &= y_0, \\x_2 - x_1 &= y_1, \\x_3 - x_2 &= y_2, \\&\dots \\x_n - x_{n-1} &= y_{n-1},\end{aligned}$$

deci prin adunare rezultă că

$$x_n - x_0 = y_0 + y_1 + y_2 + \dots + y_{n-1}$$

și astfel avem

$$x_n = x_0 + \sum_{k=0}^{n-1} y_k, \quad \forall n \in \mathbb{N}. \quad (3.4.9)$$

Relațiile (3.4.7) și (3.4.8) definesc o recurență liniară de ordinul I cu termen liber constant, care se rezolvă conform rezultatelor descrise în Secțiunea 3.2. Astfel avem următoarele două subcazuri.

i) Dacă $b = -1$ și deci $a = 2$, atunci relația (3.4.7) devine

$$y_n = y_{n-1} + c, \quad \forall n \in \mathbb{N}^*,$$

adică șirul (y_n) este o progresie aritmetică de rație c . Deci

$$y_n = y_0 + nc = x_1 - x_0 + nc, \quad \forall n \in \mathbb{N}.$$

Înlocuind în (3.4.9), obținem

$$x_n = x_0 + \sum_{k=0}^{n-1} (x_1 - x_0 + ck) = x_0 + n(x_1 - x_0) + c \cdot \frac{(n-1)n}{2},$$

deci soluția recurenței (3.4.1) în cazul $a = 2$, $b = -1$ este

$$x_n = \frac{c}{2} \cdot n^2 + \left(x_1 - x_0 - \frac{c}{2}\right)n + x_0, \quad \forall n \in \mathbb{N}. \quad (3.4.10)$$

Exemplul 3.4.3. Considerăm relația de recurență liniară

$$x_n = 2x_{n-1} - x_{n-2} + 10, \quad \forall n \in \mathbb{N}, \quad n \geq 2, \quad x_0 = 3, \quad x_1 = 4.$$

Avem $a = 2$, $b = -1$, $c = 10$. Aplicând (3.4.10) obținem

$$x_n = 5n^2 - 4n + 3, \quad \forall n \in \mathbb{N}.$$

De exemplu, $x_5 = 5 \cdot 25 - 4 \cdot 5 + 3 = 108$.

ii) Dacă $b \neq -1$ și $a = 1 - b$, deci $a \neq 2$, atunci conform (3.2.3) obținem că soluția recurenței (3.4.7) este

$$\begin{aligned} y_n &= \left(y_0 + \frac{c}{-b-1} \right) (-b)^n - \frac{c}{-b-1} \\ &= \left(x_1 - x_0 - \frac{c}{b+1} \right) (-b)^n + \frac{c}{b+1}, \quad \forall n \in \mathbb{N}. \end{aligned}$$

Înlocuind în (3.4.9), obținem

$$\begin{aligned} x_n &= x_0 + \sum_{k=0}^{n-1} \left[\left(x_1 - x_0 - \frac{c}{b+1} \right) (-b)^k + \frac{c}{b+1} \right] \\ &= x_0 + \frac{c}{b+1} \cdot n + \left(x_1 - x_0 - \frac{c}{b+1} \right) \sum_{k=0}^{n-1} (-b)^k \\ &= x_0 + \frac{c}{b+1} \cdot n + \left(x_1 - x_0 - \frac{c}{b+1} \right) \cdot \frac{(-b)^n - 1}{-b-1}, \end{aligned}$$

deci soluția recurenței (3.4.1) în cazul $a + b = 1$, $b \neq -1$ este

$$x_n = \left[\frac{c}{(b+1)^2} - \frac{x_1 - x_0}{b+1} \right] [(-b)^n - 1] + \frac{c}{b+1} \cdot n + x_0, \quad \forall n \in \mathbb{N}. \quad (3.4.11)$$

Exemplul 3.4.4. Considerăm relația de recurență liniară

$$x_n = 4x_{n-1} - 3x_{n-2} + 8, \quad \forall n \in \mathbb{N}, \quad n \geq 2, \quad x_0 = 1, \quad x_1 = 5.$$

Avem $a = 4$, $b = -3$, $c = 8$, deci $a + b = 1$. Aplicând (3.4.11) obținem

$$x_n = \left(\frac{8}{4} - \frac{4}{-2} \right) (3^n - 1) + \frac{8}{-2} \cdot n + 1 = 4 \cdot 3^n - 4n - 3, \quad \forall n \in \mathbb{N}.$$

De exemplu, $x_5 = 4 \cdot 243 - 20 - 3 = 949$.

Observația 3.4.1. Analog se definesc și rezolvă relațiile de recurență liniară de ordine mai mari sau egale cu trei cu termen liber constant.

3.5 Recurența liniară de ordinul I cu termen liber liniar

Această recurență are forma

$$x_n = ax_{n-1} + bn + c, \quad \forall n \in \mathbb{N}^*, \quad (3.5.1)$$

unde parametrii $a, b, c \in \mathbb{R}$ și primul termen $x_0 \in \mathbb{R}$ sunt numere date (cunoscute). Presupunem că

$$a \neq 0 \text{ și } b \neq 0$$

(deoarece pentru $b = 0$ recurența devine cu coeficienți constanți).

Exemplul 3.5.1. Considerăm relația de recurență

$$x_n = x_{n-1} + 3n - 2, \forall n \in \mathbb{N}^*, x_0 = 1.$$

Avem

$$\begin{aligned} x_1 &= x_0 + 1 = 1 + 1 = 2, \\ x_2 &= x_1 + 4 = 2 + 4 = 6, \\ x_3 &= x_2 + 7 = 6 + 7 = 13, \\ x_4 &= x_3 + 10 = 13 + 10 = 23, \\ x_5 &= x_4 + 13 = 23 + 13 = 36. \end{aligned}$$

Rezolvarea matematică

Scriind relația (3.5.1) pentru n și pentru $n + 1$ avem

$$\begin{aligned} x_n &= ax_{n-1} + bn + c, \\ x_{n+1} &= ax_n + b(n+1) + c, \forall n \in \mathbb{N}^*, \end{aligned}$$

deci prin scădere obținem

$$x_{n+1} - x_n = a(x_n - x_{n-1}) + b, \forall n \in \mathbb{N}^*.$$

Observația 3.5.1. Aceasta este o recurență liniară de ordinul al II-lea cu termen liber constant, deci poate fi rezolvată conform metodei descrise în Secțiunea 3.4.

Descriem în continuare o abordare directă. Notând

$$y_n = x_{n+1} - x_n, \forall n \in \mathbb{N}, \quad (3.5.2)$$

obținem

$$y_n = ay_{n-1} + b, \forall n \in \mathbb{N}^*, \quad (3.5.3)$$

cu $y_0 = x_1 - x_0 = ax_0 + b + c - x_0$, adică

$$y_0 = (a - 1)x_0 + b + c. \quad (3.5.4)$$

Din (3.5.2), analog demonstrației relației (3.4.9), obținem

$$x_n = x_0 + \sum_{k=0}^{n-1} y_k, \quad \forall n \in \mathbb{N}. \quad (3.5.5)$$

Relațiile (3.5.3) și (3.5.4) definesc o recurență liniară de ordinul I cu coeficienți constanți. Pentru rezolvare, utilizăm rezultatele din Secțiunea 3.2. Astfel avem următoarele două cazuri.

Cazul 1. Dacă $a = 1$, atunci

$$y_n = y_0 + nb = b + c + nb, \quad \forall n \in \mathbb{N}.$$

Conform (3.5.5) rezultă că

$$\begin{aligned} x_n &= x_0 + \sum_{k=0}^{n-1} (b + c + kb) = x_0 + (b + c)n + b \cdot \frac{n(n-1)}{2} \\ &= \frac{b}{2} \cdot n^2 + \left(\frac{b}{2} + c\right)n + x_0, \quad \forall n \in \mathbb{N}. \end{aligned} \quad (3.5.6)$$

Exemplul 3.5.2. Reluăm relația de recurență

$$x_n = x_{n-1} + 3n - 2, \quad \forall n \in \mathbb{N}^*, \quad x_0 = 1$$

din Exemplul 3.5.1. Avem $a = 1$, $b = 3$, $c = -2$. Aplicând (3.5.6) obținem

$$x_n = \frac{3}{2} \cdot n^2 + \left(\frac{3}{2} - 2\right)n + 1 = \frac{3n^2 - n + 2}{2}, \quad \forall n \in \mathbb{N}.$$

De exemplu, $x_5 = \frac{75 - 3}{2} = 36$.

Cazul 2. Dacă $a \neq 1$, atunci

$$\begin{aligned} y_n &= \left(y_0 + \frac{b}{a-1}\right)a^n - \frac{b}{a-1} = \left[(a-1)x_0 + b + c + \frac{b}{a-1}\right]a^n - \frac{b}{a-1} \\ &= \left[(a-1)x_0 + \frac{ab}{a-1} + c\right]a^n - \frac{b}{a-1}, \quad \forall n \in \mathbb{N}. \end{aligned}$$

Conform (3.5.5) rezultă că

$$\begin{aligned} x_n &= x_0 + \sum_{k=0}^{n-1} \left\{ \left[(a-1)x_0 + \frac{ab}{a-1} + c\right]a^k - \frac{b}{a-1} \right\} \\ &= x_0 + \left[(a-1)x_0 + \frac{ab}{a-1} + c\right] \cdot \frac{a^n - 1}{a-1} - \frac{b}{a-1} \cdot n \\ &= \left(x_0 + \frac{ab}{(a-1)^2} + \frac{c}{a-1}\right)a^n - \frac{b}{a-1} \cdot n - \frac{ab}{(a-1)^2} - \frac{c}{a-1}, \quad \forall n \in \mathbb{N}. \end{aligned} \quad (3.5.7)$$

Exemplul 3.5.3. Considerăm relația de recurență

$$x_n = 3x_{n-1} + n - 1, \forall n \in \mathbb{N}^*, x_0 = 1.$$

Aplicând formula (3.5.7) (pentru $a = 3$, $b = 1$ și $c = -1$) obținem

$$x_n = \left(1 + \frac{3}{4} - \frac{1}{2}\right) 3^n - \frac{1}{2} \cdot n - \frac{3}{4} + \frac{1}{2} = \frac{5 \cdot 3^n - 2n - 1}{4}, \forall n \in \mathbb{N}.$$

De exemplu, $x_5 = \frac{5 \cdot 3^5 - 11}{4} = 301$.

3.6 Recurența liniară de ordinul I cu termen liber pătratic

Această recurență are forma

$$x_n = ax_{n-1} + bn^2 + cn + d, \forall n \in \mathbb{N}^*, \quad (3.6.1)$$

unde parametrii $a, b, c, d \in \mathbb{R}$ și primul termen $x_0 \in \mathbb{R}$ sunt numere date (cunoscute). Presupunem că

$$a \neq 0 \text{ și } b \neq 0$$

(deoarece pentru $b = 0$ recurența devine cu termen liber liniar).

Exemplul 3.6.1. Considerăm relația de recurență

$$x_n = 3x_{n-1} + 2n^2 - 3n + 2, \forall n \in \mathbb{N}^*, x_0 = 0.$$

Avem

$$\begin{aligned} x_1 &= 3x_0 + 1 = 0 + 1 = 1, \\ x_2 &= 3x_1 + 4 = 3 + 4 = 7, \\ x_3 &= 3x_2 + 11 = 21 + 11 = 32, \\ x_4 &= 3x_3 + 22 = 96 + 22 = 118, \\ x_5 &= 3x_4 + 37 = 354 + 37 = 391. \end{aligned}$$

Rezolvarea matematică

Scriind relația (3.6.1) pentru n și pentru $n + 1$ avem

$$\begin{aligned} x_n &= ax_{n-1} + bn^2 + cn + d, \\ x_{n+1} &= ax_n + b(n+1)^2 + c(n+1) + d, \forall n \in \mathbb{N}^*, \end{aligned}$$

deci prin scădere obținem

$$x_{n+1} - x_n = a(x_n - x_{n-1}) + 2bn + b + c, \forall n \in \mathbb{N}^*.$$

Notând

$$y_n = x_{n+1} - x_n, \forall n \in \mathbb{N}, \quad (3.6.2)$$

obținem

$$y_n = ay_{n-1} + 2bn + b + c, \forall n \in \mathbb{N}^*, \quad (3.6.3)$$

cu $y_0 = x_1 - x_0 = ax_0 + b + c + d - x_0$, adică

$$y_0 = (a - 1)x_0 + b + c + d. \quad (3.6.4)$$

Relațiile (3.6.3) și (3.6.4) definesc o recurență liniară de ordinul I cu termen liber liniar, care se rezolvă conform metodelor descrise în secțiunea anterioară. Din (3.6.2), analog demonstrației relației (3.4.9), obținem că soluția recurenței (3.6.1) este

$$x_n = x_0 + \sum_{k=0}^{n-1} y_k, \forall n \in \mathbb{N}. \quad (3.6.5)$$

Observația 3.6.1. Analog se rezolvă relațiile de recurență liniară de ordinul I având drept termen liber un polinom $f(n)$ de grad mai mare sau egal cu 3.

3.7 Recurența liniară de ordinul al II-lea cu termen liber liniar

Această recurență are forma

$$x_n = ax_{n-1} + bx_{n-2} + cn + d, \forall n \in \mathbb{N}, n \geq 2, \quad (3.7.1)$$

unde parametrii $a, b, c, d \in \mathbb{R}$ și primii doi termeni $x_0, x_1 \in \mathbb{R}$ sunt numere date (cunoscute). Presupunem că

$$b \neq 0 \text{ și } c \neq 0$$

(deoarece pentru $b = 0$ recurența devine de ordinul I, iar pentru $c = 0$ devine de ordinul al doilea cu coeficienți constanți).

Exemplul 3.7.1. Considerăm relația de recurență (3.1.1) din Exemplul 3.1.1

$$x_n = 3x_{n-1} + 4x_{n-2} + 4n + 5, \forall n \in \mathbb{N}, n \geq 2, x_0 = 1, x_1 = 2.$$

Avem

$$\begin{aligned}x_2 &= 3x_1 + 4x_0 + 13 = 3 \cdot 2 + 4 \cdot 1 + 13 = 23, \\x_3 &= 3x_2 + 4x_1 + 17 = 3 \cdot 23 + 4 \cdot 2 + 17 = 94, \\x_4 &= 3x_3 + 4x_2 + 21 = 3 \cdot 94 + 4 \cdot 23 + 21 = 395, \\x_5 &= 3x_4 + 4x_3 + 25 = 3 \cdot 395 + 4 \cdot 94 + 25 = 1586.\end{aligned}$$

Rezolvarea matematică

Scriind relația (3.7.1) pentru n și pentru $n + 1$ avem

$$\begin{aligned}x_n &= ax_{n-1} + bx_{n-2} + cn + d, \\x_{n+1} &= ax_n + bx_{n-1} + c(n+1) + d, \quad \forall n \in \mathbb{N}, \quad n \geq 2,\end{aligned}$$

deci prin scădere obținem

$$x_{n+1} - x_n = a(x_n - x_{n-1}) + b(x_{n-1} - x_{n-2}) + c, \quad \forall n \in \mathbb{N}^*, \quad n \geq 2.$$

Notând

$$y_n = x_{n+1} - x_n, \quad \forall n \in \mathbb{N}, \quad (3.7.2)$$

obținem

$$y_n = ay_{n-1} + by_{n-2} + c, \quad \forall n \in \mathbb{N}, \quad n \geq 2, \quad (3.7.3)$$

cu

$$y_0 = x_1 - x_0, \quad y_1 = x_2 - x_1 = (a-1)x_1 + bx_0 + 2c + d. \quad (3.7.4)$$

Din (3.7.2), analog demonstrației relației (3.4.9), obținem

$$x_n = x_0 + \sum_{k=0}^{n-1} y_k, \quad \forall n \in \mathbb{N}. \quad (3.7.5)$$

Relațiile (3.7.3) și (3.7.4) definesc o recurență liniară de ordinul al II-lea cu termen liber constant și poate fi rezolvată conform metodei descrise în Secțiunea 3.4.

Exemplul 3.7.2. Reluăm relația de recurență

$$x_n = 3x_{n-1} + 4x_{n-2} + 4n + 5, \quad \forall n \in \mathbb{N}, \quad n \geq 2, \quad x_0 = 1, \quad x_1 = 2$$

din Exemplele 3.1.1 și 3.7.1. Avem $a = 3$, $b = 4$, $c = 4$, $d = 5$. Conform relațiilor (3.7.2), (3.7.3) și (3.7.4), luând

$$y_n = x_{n+1} - x_n, \quad \forall n \in \mathbb{N},$$

obținem că y_n verifică recurența liniară de ordinul al II-lea cu termen liber constant

$$y_n = 3y_{n-1} + 4y_{n-2} + 4, \forall n \in \mathbb{N}, n \geq 2,$$

$$y_0 = x_1 - x_0 = 1, y_1 = x_2 - x_1 = 2x_1 + 4x_0 + 8 + 5 = 21.$$

Avem $a + b = 3 + 4 \neq 1$. Conform relațiilor (3.4.5), (3.4.3) și (3.4.4), luând

$$z_n = y_n + \frac{c}{a+b-1} = y_n + \frac{4}{6} = y_n + \frac{2}{3}, \forall n \in \mathbb{N},$$

obținem că z_n verifică recurența liniară omogenă

$$z_n = 3z_{n-1} + 4z_{n-2}, \forall n \in \mathbb{N}, n \geq 2, z_0 = y_0 + \frac{2}{3} = \frac{5}{3}, z_1 = y_1 + \frac{2}{3} = \frac{65}{3}.$$

Ecuția caracteristică este

$$r^2 = 3r + 4, \text{ adică } r^2 - 3r - 4 = 0.$$

Avem $\Delta = 9 + 16 = 25$, $r_1 = \frac{3-5}{2} = -1$, $r_2 = \frac{3+5}{2} = 4$, deci $r_1 \neq r_2$.

Aplicând (3.3.3) obținem

$$z_n = \frac{(z_0 r_2 - z_1) r_1^n + (z_1 - z_0 r_1) r_2^n}{r_2 - r_1} = \frac{-45 \cdot (-1)^n + 70 \cdot 4^n}{15}$$

$$= \frac{14 \cdot 4^n - 9 \cdot (-1)^n}{3}, \forall n \in \mathbb{N}.$$

Astfel

$$y_n = z_n - \frac{2}{3} = \frac{14 \cdot 4^n - 9 \cdot (-1)^n - 2}{3}, \forall n \in \mathbb{N}.$$

Conform (3.7.5) rezultă că

$$x_n = x_0 + \sum_{k=0}^{n-1} y_k = 1 + \sum_{k=0}^{n-1} \frac{14 \cdot 4^k - 9 \cdot (-1)^k - 2}{3}$$

$$= 1 + \frac{14(4^n - 1)}{3(4 - 1)} - \frac{3((-1)^n - 1)}{-1 - 1} - \frac{2n}{3}$$

$$= \frac{28 \cdot 4^n - 12n + 27 \cdot (-1)^n - 37}{18}, \forall n \in \mathbb{N}.$$

De exemplu, $x_5 = \frac{28 \cdot 1024 - 60 - 27 - 37}{18} = \frac{28548}{18} = 1586$.

Rezultă că algoritmul din Exemplul 3.1.1 are complexitatea $\Theta(4^n)$, deci necesită un timp de calcul exponențial.

Observația 3.7.1. Analog se rezolvă relațiile de recurență liniară de ordin mai mare sau egal cu 2 având drept termen liber un polinom $f(n)$.

Tema 4

Metoda Backtracking

4.1 Descrierea metodei. Algoritmi generali

Metoda Backtracking (metoda căutării cu revenire) se aplică problemelor a căror soluție se poate reprezenta sub forma unui vector

$$x = (x_1, x_2, \dots, x_n) \in S_1 \times S_2 \times \dots \times S_n,$$

unde:

- $S_1, S_2, \dots, S_n$ sunt mulțimi finite și nevide, elementele lor aflându-se într-o **relație de ordine** bine stabilită;
- între componentele $x_1, x_2, \dots, x_n$ ale vectorului x sunt precizate anumite relații, numite **condiții interne**.

Observația 4.1.1. Pentru unele probleme, numărul de componente n al soluțiilor nu este de la început cunoscut, el urmând a fi determinat pe parcurs.

De asemenea, pentru unele probleme, două soluții pot avea numere diferite de componente.

Definiția 4.1.1. Mulțimea finită $S = S_1 \times S_2 \times \dots \times S_n$ se numește **spațiul soluțiilor posibile**.

Un vector $x = (x_1, \dots, x_n) \in S$ se numește **soluție posibilă**.

Soluțiile posibile care satisfac condițiile interne se numesc **soluții rezultat**.

Observația 4.1.2. Metoda Backtracking își propune să determine:

- fie o soluție rezultat,
- fie toate soluțiile rezultat.

Obținerea tuturor soluțiilor rezultat poate constitui o etapă intermediară în rezolvarea unei alte probleme, urmând ca, în continuare, dintre acestea să fie alese soluțiile care optimizează (minimizează sau maximizează) o anumită funcție obiectiv dată.

Observația 4.1.3. O variantă de determinare a soluțiilor rezultat ar putea fi următoarea metodă, numită **metoda forței brute**:

- se generează succesiv toate *soluțiile posibile*, adică toate elementele produsului cartezian $S_1 \times S_2 \times \cdots \times S_n$;
- pentru fiecare soluție posibilă se verifică dacă sunt satisfăcute *condițiile interne*;
- se rețin cele care satisfac aceste condiții.

Această variantă are dezavantajul că timpul de execuție este foarte mare.

De exemplu, dacă $\text{card}(S_i) = 2$, $i = \overline{1, n}$, atunci spațiul soluțiilor posibile ar avea 2^n elemente, iar complexitatea ar fi de ordinul $\Omega(2^n)$ (nesatisfăcătoare!!).

Metoda Backtracking urmărește să evite generarea tuturor soluțiilor posibile, ceea ce duce la scurtarea timpului de execuție.

Definiția 4.1.2. Fie $k \in \{1, \dots, n\}$. Un set de relații definite pentru componenta $x_k \in S_k$ prin intermediul unor eventuale componente ale vectorului $(x_1, \dots, x_{k-1}) \in S_1 \times S_2 \times \cdots \times S_{k-1}$ reprezintă **condiții de continuare** pentru x_k împreună cu $(x_1, \dots, x_{k-1})$ dacă:

1. aceste relații sunt necesare pentru existența unei soluții rezultat de forma

$$(x_1, \dots, x_{k-1}, x_k, x_{k+1}, \dots, x_n),$$

adică neîndeplinirea acestor condiții implică faptul că oricum am alege $x_{k+1} \in S_{k+1}, \dots, x_n \in S_n$ vectorul $x = (x_1, \dots, x_n)$ nu poate fi o soluție rezultat (nu verifică condițiile interne);

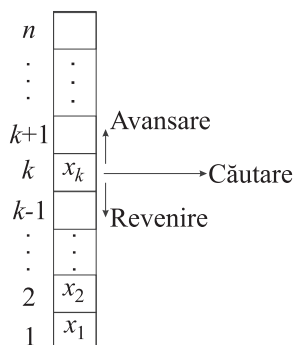
2. pentru $k = n$ condițiile de continuare coincid cu condițiile interne.

Orice vector $(x_1, \dots, x_k) \in S_1 \times S_2 \times \cdots \times S_k$, cu $1 \leq k \leq n$, care satisface condițiile de continuare, se numește **soluție parțială** (**soluție validă**).

De asemenea, vectorul vid (fără elemente) este considerat ca fiind soluție parțială.

Mecanismul metodei Backtracking

Prin metoda Backtracking soluția se construiește pas cu pas (componentă cu componentă) pe **principiul stivei**.



La fiecare nivel k , se caută un element din **mulțimea de nivel** S_k ,
care să fie atribuit componente x_k
și care împreună cu $(x_1, \dots, x_{k-1})$ să verifice condițiile de continuare

Mai precis, avem următorii pași:

1. Se începe cu soluția parțială dată de vectorul x vid.
2. Se ia primul element din mulțimea S_1 și se atribuie lui x_1 .
3. Presupunând generate elementele $(x_1, x_2, \dots, x_{k-1})$, $x_i \in S_i$, $i = \overline{1, k-1}$, se **avansează** la nivelul k și se caută primul element disponibil din S_k a cărui valoare să fie atribuită lui x_k .

Avem următoarele cazuri:

3.1. A fost găsit în S_k un element disponibil v . Atunci:

- îl atribuim lui x_k ;
- se verifică dacă acesta împreună cu elementele deja generate $x_1, \dots, x_{k-1}$ îndeplinește condițiile de continuare.

Avem următoarele subcazuri:

3.1.1. x_k verifică condițiile de continuare. Spunem că v este o **valoare validă** pentru componenta x_k .

Atunci:

- se extinde soluția parțială la $(x_1, \dots, x_{k-1}, x_k)$;
- se verifică dacă $k = n$:

3.1.1.1. Dacă **da**, atunci s-a obținut o soluție rezultat.

Acum, fie ne oprim, fie continuăm cu căutarea altei soluții rezultat, reluând algoritmul (cu pasul **3.**), considerând generate $(x_1, \dots, x_{k-1})$ și căutând în continuare pentru x_k un element netestat din S_k .

3.1.1.2. Dacă **nu** ($k < n$), se reia algoritmul cu pasul **3.**, căutând extinderea soluției parțiale $(x_1, \dots, x_{k-1}, x_k)$ cu testarea primului element din S_{k+1} (**se avansează**).

3.1.2. x_k nu verifică condițiile de continuare. Atunci, oricum am alege următoarele componente pentru x (adică pe $x_{k+1}, \dots, x_n$), nu vom obține o soluție rezultat. Prin urmare, se va relua algoritmul (cu pasul **3.**), având generate $x_1, \dots, x_{k-1}$ și căutând extinderea soluției cu următorul element netestat din S_k (continuă **căutarea**).

3.2. Nu a fost găsit în S_k un element disponibil (netestat). Atunci se consideră generate $x_1, \dots, x_{k-2}$ (**se revine**) și se reia **căutarea** cu următorul element din S_{k-1} netestat, pentru extinderea soluției parțiale $(x_1, \dots, x_{k-2})$ (pas **3.**).

Observația 4.1.4. Algoritmul se termină atunci când nu mai există nici un element din S_1 netestat.

Observația 4.1.5. După **avansare**, **căutarea** se va face începând cu testarea primului element din mulțimea de nivel S_k corespunzătoare.

Observația 4.1.6. După **revenire**, **căutarea** se va face începând cu următorul element netestat (la etapele anterioare) din mulțimea de nivel S_k corespunzătoare.

Observația 4.1.7. Condițiile de continuare sunt necesare pentru existența unei soluții rezultat, iar ideal este ca ele să fie și suficiente, ceea ce, de obicei, este imposibil!

Alegerea acestor condiții este esențială în utilizarea metodei Backtracking, deoarece cu cât condițiile de continuare sunt mai restrictive, cu atât se limitează mai mult numărul de căutări (în consecință și numărul de avansări și reveniri), deci metoda este mai eficientă.

De obicei condițiile de continuare sunt restricțiile condițiilor interne la primele k componente.

Observația 4.1.8. În general, problemele rezolvate cu Backtracking necesită un timp foarte mare de execuție. De aceea, metoda se utilizează atunci când nu avem la dispoziție un algoritm mai eficient. Problemele tipice care se pot rezolva utilizând această metodă sunt cele *nedeterminist-polinomial complete* (*NP-complete*).

În continuare considerăm **cazul particular** în care mulțimile S_k , $k = \overline{1, n}$, conțin termeni succesivi ai unor progresii aritmetice. Pentru fiecare k , $k = \overline{1, n}$, mulțimea S_k va fi dată de:

- a_k – primul termen al progresiei;
- $r_k > 0$ – rația progresiei;
- b_k – ultimul termen al progresiei,

adică

$$S_k = \{a_k, a_k + r_k, a_k + 2r_k, \dots, b_k\}, \forall k = \overline{1, n}.$$

Pentru acest caz particular, vom prezenta în continuare șase variante pentru schema Backtracking: și anume două variante iterative și patru variante recursive.

Algoritmul 4.1.1 (Schema Backtracking iterativă, varianta 1).

BACKTRACKING1:

```

k ← 1;
x[1] ← a[1] – r[1]; // pregătim introducerea valorii inițiale a1
                                     // pentru x1

while k > 0 do
    if x[k] < b[k] then
        // mai există valori netestate pentru xk
        x[k] ← x[k] + r[k]; // Căutare
        if [(x[1], ..., x[k]) verifică condițiile de continuare] then
            if k = n then
                // (x1, ..., xn) este soluție rezultat
                PRELUCREAZĂ(x[1], ..., x[n]);
                (*) // STOP, dacă se dorește
                // o singură soluție rezultat,
                // sau k ← k – 1 (revenire forțată) dacă
                // nu mai există valori valide pentru xn
            else
                k ← k + 1; // Avansare
                x[k] ← a[k] – r[k]; // pregătim introducerea
                                     // valorii inițiale ak pentru xk
        else
            // nu mai există valori netestate pentru xk
            k ← k – 1; // Revenire

```

Observația 4.1.9. Dacă după determinarea și prelucrarea soluției rezultat nu mai există valori valide pentru componenta $x[n]$, atunci în loc de (*) se poate forța revenirea ($k \leftarrow k - 1$).

Următoarea schemă este echivalentă cu cea de mai sus.

Algoritmul 4.1.2 (Schema Backtracking iterativă, varianta 2).

BACKTRACKING2:

```

 $k \leftarrow 1;$ 
 $x[1] \leftarrow a[1] - r[1];$  // pregătim introducerea valorii inițiale  $a_1$ 
                                     // pentru  $x_1$ 

while  $k > 0$  do
    if  $k \leq n$  then
        if  $x[k] < b[k]$  then
            // mai există valori netestate pentru  $x_k$ 
             $x[k] \leftarrow x[k] + r[k];$  // Căutare
            if  $[(x[1], \dots, x[k]) \text{ verifică condițiile de continuare}]$  then
                 $k \leftarrow k + 1;$  // Avansare
                 $x[k] \leftarrow a[k] - r[k];$  // pregătim introducerea
                                     // valorii inițiale  $a_k$  pentru  $x_k$ 
            else
                 $k \leftarrow k - 1;$  // Revenire
        else
            //  $(x_1, \dots, x_n)$  este soluție rezultat
            PRELUCREAZĂ $(x[1], \dots, x[n]);$ 
            (*) // STOP, dacă se dorește
                // o singură soluție rezultat
             $k \leftarrow k - 1;$  // Revenire, dacă se doresc
                // toate soluțiile rezultat

```

Observația 4.1.10. Schemele Backtracking de mai sus generează toate soluțiile problemei date. Dacă se dorește obținerea unei singure soluții, atunci în loc de (*) se poate insera o comandă de oprire (**STOP**).

Algoritmul 4.1.3 (Schema Backtracking recursivă, varianta 1).

```

BACKR1( $k$ ):                                     // se generează  $(x[k], \dots, x[n])$ 
 $x[k] \leftarrow a[k] - r[k];$  // pregătim introducerea valorii inițiale
                                     //  $a[k]$  pentru  $x[k]$ 

while  $x[k] < b[k]$  do
     $x[k] \leftarrow x[k] + r[k];$  // Căutare
    if  $[(x[1], \dots, x[k]) \text{ verifică condițiile de continuare}]$  then
        if  $k = n$  then
            //  $(x[1], \dots, x[n])$  este soluție rezultat
            PRELUCREAZĂ( $x[1], \dots, x[n]$ );
        else
            BACKR1( $k + 1$ ); // Avansare, adică
            // se generează, RECURSIV,  $(x[k + 1], \dots, x[n])$ .
            // La încheierea apelului BACKR1( $k + 1$ ) se produce
            // revenirea la funcția BACKR1( $k$ )

```

Apelare: **BACKR1**(1).

Următoarea schemă este echivalentă cu cea de mai sus.

Algoritmul 4.1.4 (Schema Backtracking recursivă, varianta 2).

```

BACKR2( $k$ ):
if  $k \leq n$  then
     $x[k] \leftarrow a[k] - r[k];$ 
    while  $x[k] < b[k]$  do
         $x[k] \leftarrow x[k] + r[k];$ 
        if  $[(x[1], \dots, x[k]) \text{ verifică condițiile de continuare}]$  then
            BACKR2( $k + 1$ );
    else
        PRELUCREAZĂ( $x[1], \dots, x[n]$ );

```

Apelare: **BACKR2**(1).

Algoritmul 4.1.5 (Schema Backtracking recursivă, varianta 3).

```

BACKR3( $k$ ):
for  $v = \overline{a[k], b[k], r[k]}$  do      // Căutarea lui  $x[k]$  în mulțimea  $S_k$ 
|    $x[k] \leftarrow v$ ;
|   if  $[(x[1], \dots, x[k]) \text{ verifică condițiile de continuare}]$  then
|       if  $k = n$  then
|           //  $(x[1], \dots, x[n])$  este soluție rezultat
|           PRELUCREAZĂ( $x[1], \dots, x[n]$ );
|       else
|           BACKR3( $k + 1$ );          // Avansare, adică
|           // se generează, RECURSIV,  $(x[k + 1], \dots, x[n])$ .
|           // La încheierea apelului BACKR3( $k + 1$ ) se produce
|           // revenirea la funcția BACKR3( $k$ )
|
|
|

```

Apelare: **BACKR3**(1).

Următoarea schemă este echivalentă cu cea de mai sus.

Algoritmul 4.1.6 (Schema Backtracking recursivă, varianta 4).

```

BACKR4( $k$ ):
if  $k \leq n$  then
|   for  $v = \overline{a[k], b[k], r[k]}$  do
|       |    $x[k] \leftarrow v$ ;
|       |   if  $[(x[1], \dots, x[k]) \text{ verifică condițiile de continuare}]$  then
|       |       |   BACKR4( $k + 1$ );
|       |
|       |
|       |
|   else
|       |   PRELUCREAZĂ( $x[1], \dots, x[n]$ );
|       |
|       |
|       |

```

Apelare: **BACKR4**(1).

Observația 4.1.11. Schemele Backtracking din Algoritmii 4.1.1, 4.1.2, 4.1.3 și 4.1.4 pot fi ușor adaptate și pentru situații în care elementele fiecărei mulțimi S_k nu sunt în progresie aritmetică.

Observația 4.1.12. În oricare din cele șase scheme de mai sus, verificarea (validarea) condițiilor de continuare

$$[(x[1], x[2], \dots, x[k]) \text{ verifică condițiile de continuare}]$$

se face adesea prin intermediul unei funcții **VALID**(x, k).

Observația 4.1.13. Dacă, în oricare din cele șase scheme de mai sus, se renunță la testul

$$[(x[1], x[2], \dots, x[k]) \text{ verifică condițiile de continuare}],$$

considerându-se astfel că orice soluție posibilă verifică condițiile interne, atunci se vor obține ca soluții rezultat ale problemei toate elementele produsului cartezian $S_1 \times S_2 \times \cdots \times S_n$.

4.2 Problema celor n dame pe tabla de șah

Problema celor n dame este următoarea:

Se consideră o tablă de șah de dimensiune $n \times n$, $n \geq 2$. Se cere să se determine toate modalitățile de aranjare a n dame astfel încât oricare două dame să nu se atace reciproc (pe linii, coloane sau diagonale).

Observația 4.2.1. Prin convenție, orice pătrățel al tablei este reprezentat prin coordonatele sale (l, c) , unde l reprezintă linia, iar c reprezintă coloana acestuia. Liniile tablei sunt numerotate cu $1, 2, \dots, n$ de sus în jos, iar coloanele sunt numerotate cu $1, 2, \dots, n$ de la stânga la dreapta.

Propoziția 4.2.1. *Pe o tablă de dimensiune $n \times n$ nu putem așeza mai mult de n dame astfel încât oricare două să nu se atace reciproc.*

Demonstrație. Oricum așezăm mai mult de n dame, cum tabla are n linii rezultă că există cel puțin două dame așezate pe aceeași linie, și acestea se atacă reciproc. $\square$

Modelarea problemei

În orice soluție rezultat, conform demonstrației propoziției anterioare obținem că pe fiecare linie a tablei se află exact o damă. Rezultă că orice soluție pentru problema celor n dame se poate reprezenta sub forma unui vector

$$x = (x_1, x_2, \dots, x_n),$$

unde x_i reprezintă coloana pe care se află dama de pe linia i , pentru $i = \overline{1, n}$.

De exemplu, pentru $n = 5$ o soluție rezultat este

$$x = (1, 3, 5, 2, 4),$$

corespunzătoare aranjării damelor din Figura 4.2.1.

	1	2	3	4	5
1					
2					
3					
4					
5					

Figura 4.2.1:

Pentru cazul general, avem

$$x = (x_1, x_2, \dots, x_n) \in S_1 \times S_2 \times \dots \times S_n,$$

unde

$$S_1 = S_2 = \dots = S_n = \{1, 2, \dots, n\}.$$

Așadar mulțimile S_k conțin termenii succesivi ai unei progresii aritmetice, în care

$$\begin{cases} a_k = 1, \\ r_k = 1, \\ b_k = n, \end{cases} \quad \forall k = \overline{1, n}.$$

Condițiile interne:

- Damele de pe liniile i și j ($i \neq j$) nu se pot afla pe aceeași coloană:

$$x_i \neq x_j, \quad \forall i \neq j, \quad i, j = \overline{1, n};$$

- Damele de pe liniile i și j ($i \neq j$) nu se pot afla pe o aceeași diagonală:

$$|i - j| \neq |x_i - x_j|, \quad \forall i \neq j, \quad i, j = \overline{1, n}$$

sau, echivalent,

$$j - i \neq |x_j - x_i|, \quad \forall i, j \text{ a.î. } 1 \leq i < j \leq n.$$

Condițiile de continuare:

Dacă avem soluția parțială $(x_1, \dots, x_{k-1})$, atunci x_k verifică condițiile de continuare (este valid) dacă

$$x_k \neq x_i \text{ și } k - i \neq |x_k - x_i|, \quad \forall i = \overline{1, k-1}.$$

Altfel spus, dacă avem soluția parțială $x = (x_1, \dots, x_{k-1})$, atunci x_k nu verifică condițiile de continuare (nu este valid) dacă

$$\exists i \in \{1, 2, \dots, k-1\} \text{ astfel încât } x_k = x_i \text{ sau } k-i = |x_k - x_i|.$$

Obținem următorul *algoritm Backtracking*, descris în pseudocod, pentru rezolvarea problemei.

*Algoritm*ul 4.2.1.

DAME(n):

$k \leftarrow 1$;

$x[1] \leftarrow 0$;

while $k > 0$ **do**

if $x[k] < n$ **then**

$x[k] \leftarrow x[k] + 1$;

if **VALID**(x, k) **then**

if $k = n$ **then**

AFISARE(x);

$k \leftarrow k - 1$;

 // forțăm revenirea, deoarece
 // nu mai există valori valide pentru $x[n]$

else

$k \leftarrow k + 1$;

$x[k] \leftarrow 0$;

else

$k \leftarrow k - 1$;

Funcția de verificare (validare) a condițiilor de continuare este

VALID(x, k):

for $i = 1, k-1$ **do**

if ($x[k] = x[i]$ or $k-i = |x[k] - x[i]|$) **then**

returnează 0;

 // fals

returnează 1;

 // adevărat

Considerații

- Pentru $n = 2$ problema nu are soluție, deoarece două dame așezate pe linii și coloane diferite se află pe aceeași diagonală, deci se atacă reciproc.
- Nici pentru $n = 3$ problema nu are soluție, deoarece orice două dame așezate pe o aceeași culoare se atacă reciproc.

- Pentru $n \geq 4$ avem următorul rezultat celebru.

Teorema 4.2.1 (E. Pauls, 1874). *Pentru orice $n \geq 4$ problema celor n dame are cel puțin o soluție.*

Demonstrație. Avem următoarele cazuri.

Cazul 1) $n = 6k$ sau $n = 6k + 4$. Atunci o soluție este dată de așezarea celor n dame în pătrățelele de coordonate (l, c) ce aparțin mulțimii

$$\left\{ (2i, i) \mid 1 \leq i \leq \frac{n}{2} \right\} \cup \left\{ (2i - 1, \frac{n}{2} + i) \mid 1 \leq i \leq \frac{n}{2} \right\}.$$

Cazul 2) $n = 6k + 1$ sau $n = 6k + 5$. Atunci o soluție este dată de așezarea celor n dame în pătrățelele de coordonate (l, c) ce aparțin mulțimii

$$\{(n, 1)\} \cup \left\{ (2i, i + 1) \mid 1 \leq i \leq \frac{n-1}{2} \right\} \cup \left\{ (2i - 1, \frac{n+1}{2} + i) \mid 1 \leq i \leq \frac{n-1}{2} \right\}.$$

Cazul 3) $n = 6k + 2$. Atunci o soluție este dată de așezarea celor n dame în pătrățelele de coordonate (l, c) ce aparțin mulțimii

$$\begin{aligned} & \left\{ (4, 1), (n, \frac{n}{2} - 1), (2, \frac{n}{2}), (n - 1, \frac{n}{2} + 1), (1, \frac{n}{2} + 2), (n - 3, n) \right\} \cup \\ & \cup \left\{ (n - 2i, i + 1) \mid 1 \leq i \leq \frac{n}{2} - 3 \right\} \cup \\ & \cup \left\{ (n - 2i - 3, \frac{n}{2} + i + 2) \mid 1 \leq i \leq \frac{n}{2} - 3 \right\}. \end{aligned}$$

Cazul 4) $n = 6k + 3$. Atunci o soluție este dată de așezarea celor n dame în pătrățelele de coordonate (l, c) ce aparțin mulțimii

$$\begin{aligned} & \left\{ (1, n), (5, 1), (n, \frac{n-1}{2} - 1), (3, \frac{n-1}{2}), (n - 1, \frac{n-1}{2} + 1) \right\} \cup \\ & \left\{ (2, \frac{n-1}{2} + 2), (n - 3, n - 1) \right\} \cup \left\{ (n - 2i, i + 1) \mid 1 \leq i \leq \frac{n-1}{2} - 3 \right\} \cup \\ & \cup \left\{ (n - 2i - 3, \frac{n-1}{2} + i + 2) \mid 1 \leq i \leq \frac{n-1}{2} - 3 \right\}, \end{aligned}$$

adică se așează o damă în colțul din dreapta sus (pătrățelul de coordonate $(1, n)$) iar celelalte $n - 1$ dame se așează conform Cazului 3 în subtabla de dimensiune $(n - 1) \times (n - 1)$ rămasă prin eliminarea primei linii și ultimei coloane. $\square$

Conform Teoremei 4.2.1, Propoziției 4.2.1 și cazurilor $n = 2$ și $n = 3$ considerate mai sus obținem următorul rezultat.

Corolarul 4.2.1. *Numărul maxim de dame ce pot fi așezate pe o tablă de șah de dimensiune $n \times n$ astfel încât oricare două dame să nu se atace reciproc este egal cu*

$$\begin{cases} n - 1, & \text{dacă } n \in \{2, 3\}, \\ n, & \text{dacă } n \geq 4. \end{cases}$$

- Problema a fost expusă inițial pentru tabla de șah obișnuită 8×8 în anul 1848 de către șahistul german Max Bezzel.
- Prima rezolvare a problemei celor 8 dame a fost publicată de către Franz Nauck în anul 1850, care a găsit toate cele 92 de soluții ale problemei. Acesta a propus și varianta generalizată la tabla de dimensiune $n \times n$.
- Problema a fost imediat studiată de mulți matematicieni, printre care și celebrul matematician Carl Friedrich Gauss.
- Emil Pauls a fost primul care a demonstrat existența soluției pentru orice $n \geq 4$, în anul 1874, prin construcția din demonstrația teoremei de mai sus.

4.3 Problema nebunilor pe tabla de șah

Problema nebunilor este următoarea:

Se consideră o tablă de șah de dimensiune $n \times n$, $n \geq 2$. Se cere să se determine toate modalitățile de aranjare a $n - 1$ nebuni pe pătrățele de aceeași culoare astfel încât oricare doi nebuni să nu se atace reciproc (pe diagonale).

Observația 4.3.1. Utilizăm în continuare convenția din Observația 4.2.1. Tot prin convenție, pătrățul situat în colțul din stânga sus al tablei, de coordonate $(1, 1)$, este colorat cu alb.

Propoziția 4.3.1. *Pe o tablă de dimensiune $n \times n$ nu putem așeza mai mult de $n - 1$ nebuni pe pătrățele de aceeași culoare astfel încât oricare doi nebuni să nu se atace reciproc.*

Demonstrație. Mulțimea $\mathcal{N}$ a pătrățelelor de culoare neagră poate fi partiționată în următoarele diagonale (cu direcția *stânga-jos* $\rightarrow$ *dreapta-sus*, ordonate crescător după distanța față de colțul din stânga-sus):

$$\mathcal{N} = D_1 \cup D_2 \cup \dots \cup D_{n-1}, \text{ unde} \\ D_k = \{(l, c) \mid l + c = 2k + 1, 1 \leq l \leq n, 1 \leq c \leq n\}, \forall k = \overline{1, n-1}. \quad (4.3.1)$$

Rezultă că oricum așezăm mai mult de $n - 1$ nebuni pe pătrățele de culoare neagră există cel puțin doi nebuni așezați pe o aceeași diagonală D_k , și aceștia se atacă reciproc.

Pe de altă parte, mulțimea $\mathcal{A}$ a pătrățelelor de culoare albă poate fi partitionată în următoarele diagonale (având, de asemenea, direcția *stânga-jos* $\rightarrow$ *dreapta-sus* și ordonate crescător după distanța față de colțul din stânga-sus):

$$\mathcal{A} = \overline{D}_1 \cup \overline{D}_2 \cup \dots \cup \overline{D}_n, \text{ unde} \\ \overline{D}_k = \{(l, c) \mid l + c = 2k, 1 \leq l \leq n, 1 \leq c \leq n\}, \forall k = \overline{1, n}. \quad (4.3.2)$$

Rezultă că oricum așezăm mai mult de $n - 1$ nebuni pe pătrățele de culoare albă, fie există cel puțin doi nebuni așezați pe o aceeași diagonală $\overline{D}_k$ și aceștia se atacă reciproc, fie există câte cel puțin un nebun pe fiecare diagonală $\overline{D}_k$ și atunci nebunii de pe diagonalele $\overline{D}_1 = \{(1, 1)\}$ și $\overline{D}_n = \{(n, n)\}$ se atacă reciproc. □

Modelarea problemei

Cazul 1) Considerăm cazul așezării celor $n - 1$ nebuni pe pătrățele de culoare neagră.

Pentru orice soluție rezultat, conform demonstrației propoziției anterioare obținem că pe fiecare din cele $n - 1$ diagonale D_k definite prin relația (4.3.1) se află exact un nebun.

Fiecare diagonală D_k , $k = \overline{1, n-1}$, poate fi rescrisă astfel

$$\begin{aligned} D_k &= \{(l, c) \mid l + c = 2k + 1, 1 \leq l \leq n, 1 \leq c \leq n\} \\ &= \{(l, 2k + 1 - l) \mid 1 \leq l \leq n, 1 \leq 2k + 1 - l \leq n\} \\ &= \{(l, 2k + 1 - l) \mid 1 \leq l \leq n, 2k + 1 - n \leq l \leq 2k\} \\ &= \{(l, 2k + 1 - l) \mid \max\{2k + 1 - n, 1\} \leq l \leq \min\{2k, n\}\}, \end{aligned}$$

deci

$$D_k = \begin{cases} \{(l, 2k + 1 - l) \mid 1 \leq l \leq 2k\}, & \text{dacă } 1 \leq k \leq \left\lfloor \frac{n}{2} \right\rfloor, \\ \{(l, 2k + 1 - l) \mid 2k + 1 - n \leq l \leq n\}, & \text{dacă } \left\lfloor \frac{n}{2} \right\rfloor < k \leq n - 1 \end{cases}$$

($\lfloor t \rfloor$ reprezintă *partea întreagă* a numărului $t \in \mathbb{R}$).

Pentru orice $k \in \{1, 2, \dots, n - 1\}$, coloana pe care se află nebunul de pe diagonala D_k este

$$c = 2k + 1 - l,$$

deci este bine determinată de linia l a acestuia.

Rezultă că orice soluție se poate reprezenta sub forma unui vector

$$x = (x_1, x_2, \dots, x_{n-1}),$$

unde x_k reprezintă linia pe care se află nebunul de pe diagonala D_k , pentru $k = \overline{1, n-1}$.

De exemplu, pentru $n = 5$ o soluție rezultat este

$$x = (1, 4, 2, 5),$$

corespunzătoare aranjării nebunilor din Figura 4.3.1.

	1	2	3	4	5
1					
2					
3					
4					
5					

Figura 4.3.1:

Pentru cazul general, avem

$$x = (x_1, x_2, \dots, x_{n-1}) \in S_1 \times S_2 \times \dots \times S_{n-1},$$

unde

$$S_k = \begin{cases} \{1, 2, \dots, 2k\}, & \text{dacă } 1 \leq k \leq \left\lceil \frac{n}{2} \right\rceil, \\ \{2k - n + 1, 2k - n + 2, \dots, n\}, & \text{dacă } \left\lceil \frac{n}{2} \right\rceil + 1 \leq k \leq n - 1. \end{cases}$$

Așadar mulțimile S_k conțin termenii succesivi ai unei progresii aritmetice, în care

$$\begin{cases} a_k = 1, \\ r_k = 1, \\ b_k = 2k, \end{cases} \quad \forall k = \overline{1, \left\lceil \frac{n}{2} \right\rceil},$$

$$\begin{cases} a_k = 2k - n + 1, \\ r_k = 1, \\ b_k = n, \end{cases} \quad \forall k = \overline{\left\lceil \frac{n}{2} \right\rceil + 1, n - 1}.$$

Condițiile interne:

Pentru orice $i \neq j$, cei doi nebuni de pe diagonalele D_i și D_j , așezați deci pe liniile x_i respectiv x_j , și pe coloanele $c_i = 2i + 1 - x_i$ respectiv $c_j = 2j + 1 - x_j$, nu se pot afla nici pe o aceeași diagonală având direcția *stânga-sus* $\rightarrow$ *dreapta-jos*:

$$x_i - x_j \neq c_i - c_j, \forall i \neq j, i, j = \overline{1, n-1},$$

sau, echivalent,

$$x_i - x_j \neq 2i + 1 - x_i - 2j - 1 + x_j, \forall i \neq j, i, j = \overline{1, n-1},$$

adică

$$x_i - x_j \neq i - j, \forall i \neq j, i, j = \overline{1, n-1}.$$

Condițiile de continuare:

Dacă avem soluția parțială $(x_1, \dots, x_{k-1})$, atunci x_k *verifică condițiile de continuare (este valid)* dacă

$$x_k - x_i \neq k - i, \forall i = \overline{1, k-1}.$$

Altfel spus, dacă avem soluția parțială $x = (x_1, \dots, x_{k-1})$, atunci x_k *nu verifică condițiile de continuare (nu este valid)* dacă

$$\exists i \in \{1, 2, \dots, k-1\} \text{ astfel încât } x_k - x_i = k - i.$$

Obținem următorul *algoritm Backtracking*, descris în pseudocod, pentru rezolvarea problemei.

Algoritmul 4.3.1.

NEBUNI1(n):

```

 $k \leftarrow 1$ ;
 $x[1] \leftarrow 0$ ;
 $ult \leftarrow 2$ ;           //  $ult$  este valoarea finală  $b_k$  pentru  $x_k$ 
while  $k > 0$  do
    if  $x[k] < ult$  then
         $x[k] \leftarrow x[k] + 1$ ;
        if VALID1( $x, k$ ) then
            if  $k = n - 1$  then
                | AFISARE( $x$ );
            else
                |  $k \leftarrow k + 1$ ;
                | // pregătim introducerea valorii inițiale  $a_k$  și
                | // actualizăm valoarea finală  $b_k$  pentru  $x_k$ 
                | if  $k \leq \frac{n}{2}$  then
                | |  $x[k] \leftarrow 0$ ;
                | |  $ult \leftarrow 2k$ ;
                | else
                | |  $x[k] \leftarrow 2k - n$ ;
                | |  $ult \leftarrow n$ ;
        else
            |  $k \leftarrow k - 1$ ;
            | // actualizăm valoarea finală  $b_k$  pentru  $x_k$ 
            | if  $k \leq \frac{n}{2}$  then
            | |  $ult \leftarrow 2k$ ;
            | else
            | |  $ult \leftarrow n$ ;

```

Funcția de verificare (validare) a condițiilor de continuare este

VALID1(x, k):

```

for  $i = 1, k - 1$  do
    if  $x[k] - x[i] = k - i$  then
        | returnează 0;           // fals
returnează 1;                   // adevărat

```

Cazul 2) Considerăm acum cazul așezării celor $n - 1$ nebuni pe pătrățele de culoare albă.

Pentru orice soluție rezultat, conform demonstrației propoziției anterioare obținem că un prim nebun se așează fie pe pătrățul $(1, 1)$ ce formează

diagonala $\overline{D}_1$, fie pe pătrățelul (n, n) ce formează diagonala $\overline{D}_n$, iar restul de $n - 2$ nebuni se așează câte unul pe fiecare din diagonalele $\overline{D}_2, \dots, \overline{D}_{n-1}$ definite prin relația (4.3.2).

Fiecare diagonală $\overline{D}_k$, $k = \overline{2, n-1}$, poate fi rescrisă astfel

$$\begin{aligned}\overline{D}_k &= \{(l, c) \mid l + c = 2k, 1 \leq l \leq n, 1 \leq c \leq n\} \\ &= \{(l, 2k - l) \mid 1 \leq l \leq n, 1 \leq 2k - l \leq n\} \\ &= \{(l, 2k - l) \mid 1 \leq l \leq n, 2k - n \leq l \leq 2k - 1\} \\ &= \{(l, 2k - l) \mid \max\{2k - n, 1\} \leq l \leq \min\{2k - 1, n\}\},\end{aligned}$$

deci

$$\overline{D}_k = \begin{cases} \{(l, 2k - l) \mid 1 \leq l \leq 2k - 1\}, & \text{dacă } 2 \leq k \leq \left\lfloor \frac{n+1}{2} \right\rfloor, \\ \{(l, 2k - l) \mid 2k - n \leq l \leq n\}, & \text{dacă } \left\lfloor \frac{n+1}{2} \right\rfloor < k \leq n - 1. \end{cases}$$

Pentru orice $k \in \{2, \dots, n - 1\}$, coloana pe care se află nebunul de pe diagonala $\overline{D}_k$ este

$$c = 2k - l,$$

deci este bine determinată de linia l a acestuia.

Rezultă că orice soluție se poate reprezenta sub forma unui vector

$$y = (y_1, y_2, \dots, y_{n-1}),$$

unde

$$y_1 = \begin{cases} 1, & \text{dacă primul nebun se află pe pătrățelul } (1, 1), \\ n, & \text{dacă primul nebun se află pe pătrățelul } (n, n), \end{cases}$$

iar pentru $k \in \{2, \dots, n - 1\}$ y_k reprezintă linia pe care se află nebunul de pe diagonala $\overline{D}_k$.

Avem

$$y = (y_1, y_2, \dots, y_{n-1}) \in S_1 \times S_2 \times \dots \times S_{n-1},$$

unde

$$S_k = \begin{cases} \{1, n\}, & \text{dacă } k = 1, \\ \{1, 2, \dots, 2k - 1\}, & \text{dacă } 2 \leq k \leq \left\lfloor \frac{n+1}{2} \right\rfloor, \\ \{2k - n, 2k - n + 1, \dots, n\}, & \text{dacă } \left\lfloor \frac{n+1}{2} \right\rfloor + 1 \leq k \leq n - 1. \end{cases}$$

Pentru $k \in \{2, \dots, n-1\}$ mulțimile S_k conțin termenii succesivi ai unei progresii aritmetice, în care

$$\begin{cases} a_k = 1, \\ r_k = 1, \\ b_k = 2k - 1, \end{cases} \quad \forall k = 2, \overline{\left[\frac{n+1}{2} \right]},$$

$$\begin{cases} a_k = 2k - n, \\ r_k = 1, \\ b_k = n, \end{cases} \quad \forall k = \overline{\left[\frac{n+1}{2} \right] + 1, n-1}.$$

Condițiile interne:

Analog cazului 1) obținem:

$$y_i - y_j \neq i - j, \quad \forall i \neq j, \quad i, j = \overline{2, n-1}.$$

În plus,

$$y_1 \in \{1, n\} \text{ și } y_i \neq i, \quad \forall i = \overline{2, n-1}$$

(nebulul de pe diagonala $\overline{D_i i = 2, n-1}$, nu se poate afla pe diagonala dintre pătrățelele $(1, 1)$ și (n, n) , deoarece s-ar ataca cu nebunul aflat pe unul din aceste două pătrățelele).

Condițiile de continuare:

Dacă avem soluția parțială $(y_1, \dots, y_{k-1})$, cu $k \geq 2$, atunci y_k verifică condițiile de continuare (este valid) dacă

$$y_k \neq k \text{ și } y_k - y_i \neq k - i, \quad \forall i = \overline{2, k-1}.$$

Altfel spus, dacă avem soluția parțială $(y_1, \dots, y_{k-1})$, cu $k \geq 2$, atunci y_k nu verifică condițiile de continuare (nu este valid) dacă

$$y_k = k \text{ sau } \exists i \in \{2, \dots, k-1\} \text{ astfel încât } y_k - y_i = k - i.$$

Obținem următorul *algoritm Backtracking*, descris în pseudocod, pentru rezolvarea problemei.

Algoritmul 4.3.2.

```

NEBUNI2( $n$ ):                                     // presupunem că  $n \geq 3$ 
 $k \leftarrow 2$ ;
 $y[2] \leftarrow 0$ ;
 $ult \leftarrow 3$ ;                                //  $ult$  este valoarea finală  $b_k$  pentru  $y_k$ 
while  $k > 1$  do
    if  $y[k] < ult$  then
         $y[k] \leftarrow y[k] + 1$ ;
        if VALID2( $y, k$ ) then
            if  $k = n - 1$  then
                 $y[1] \leftarrow 1$ ;                // primul nebun pe pătrățelul (1,1)
                AFISARE( $y$ );
                 $y[1] \leftarrow n$ ;                // primul nebun pe pătrățelul (n,n)
                AFISARE( $y$ );
            else
                 $k \leftarrow k + 1$ ;
                // pregătim introducerea valorii inițiale  $a_k$  și
                // actualizăm valoarea finală  $b_k$  pentru  $y_k$ 
                if  $k \leq \frac{n+1}{2}$  then
                     $y[k] \leftarrow 0$ ;
                     $ult \leftarrow 2k - 1$ ;
                else
                     $y[k] \leftarrow 2k - n - 1$ ;
                     $ult \leftarrow n$ ;
        else
             $k \leftarrow k - 1$ ;
            // actualizăm valoarea finală  $b_k$  pentru  $y_k$ 
            if  $k \leq \frac{n+1}{2}$  then
                 $ult \leftarrow 2k - 1$ ;
            else
                 $ult \leftarrow n$ ;

```

Funcția de verificare (validare) a condițiilor de continuare este

```

VALID2( $y, k$ ):
if  $y[k] = k$  then
    returnează 0; // fals
for  $i = \overline{2, k-1}$  do
    if  $y[k] - y[i] = k - i$  then
        returnează 0; // fals
returnează 1; // adevărat

```

Considerații

Teorema 4.3.1. *Pentru orice $n \geq 2$ problema nebunilor are cel puțin o soluție.*

Demonstrație. Avem din nou următoarele două cazuri.

Cazul 1) Cei $n - 1$ nebuni trebuie așezați pe pătrățele de culoare neagră. Atunci o soluție este dată de așezarea celor $n - 1$ nebuni în pătrățelele de coordonate (l, c) ce aparțin mulțimii

$$\left\{ (1, 2i) \mid 1 \leq i \leq \frac{n-1}{2} \right\} \cup \left\{ (n, 2i) \mid 1 \leq i \leq \frac{n-1}{2} \right\}, \text{ dacă } n \text{ este impar,}$$

respectiv mulțimii

$$\left\{ (1, 2i) \mid 1 \leq i \leq \frac{n}{2} \right\} \cup \left\{ (n, 2i-1) \mid 2 \leq i \leq \frac{n}{2} \right\}, \text{ dacă } n \text{ este par.}$$

Cazul 2) Cei $n - 1$ nebuni trebuie așezați pe pătrățele de culoare albă. Atunci o soluție este dată de așezarea celor $n - 1$ nebuni în pătrățelele de coordonate (l, c) ce aparțin mulțimii

$$\left\{ (1, 2i-1) \mid 1 \leq i \leq \frac{n+1}{2} \right\} \cup \left\{ (n, 2i-1) \mid 2 \leq i \leq \frac{n-1}{2} \right\}, \text{ dacă } n \text{ este}$$

impar, respectiv mulțimii

$$\left\{ (1, 2i-1) \mid 1 \leq i \leq \frac{n}{2} \right\} \cup \left\{ (n, 2i-2) \mid 2 \leq i \leq \frac{n}{2} \right\}, \text{ dacă } n \text{ este par.}$$

□

Conform Teoremei 4.3.1 și Propoziției 4.3.1 obținem următorul rezultat.

Corolarul 4.3.1. *Numărul maxim de nebuni ce pot fi așezați pe pătrățele de aceeași culoare ale unei table de șah de dimensiune $n \times n$, $n \geq 2$, astfel încât oricare doi nebuni să nu se atace reciproc este egal cu $n - 1$.*

Evident, orice doi nebuni așezați pe pătrățele de culori diferite nu se atacă reciproc. Astfel, conform corolarului anterior obținem următorul rezultat.

Corolarul 4.3.2. *Numărul maxim de nebuni ce pot fi așezați pe o tablă de șah de dimensiune $n \times n$, $n \geq 2$, astfel încât oricare doi nebuni să nu se atace reciproc este egal cu $2n - 2$.*

Observația 4.3.2. Pentru generarea tuturor modalităților de aranjare a $2n - 2$ nebuni pe o tablă de șah de dimensiune $n \times n$ astfel încât oricare doi nebuni să nu se atace reciproc putem proceda astfel:

- se generează toate modalitățile de așezare a $n - 1$ nebuni pe pătrățelele de culoare albă astfel încât oricare doi nebuni să nu se atace reciproc;
- pentru fiecare astfel de așezare, se generează toate modalitățile de așezare a restului de $n - 1$ nebuni pe pătrățelele de culoare neagră astfel încât oricare doi din aceștia să nu se atace reciproc.

Pentru aceasta putem utiliza funcția **NEBUNI2** (n) din Algoritmul 4.3.2 în care înlocuim funcția **AFISARE**(y) cu funcția **NEBUNI1** (n) din Algoritmul 4.3.1.

4.4 Generarea obiectelor combinatoriale

4.4.1 Preliminarii

În continuare vom prezenta formule de numărare și algoritmi de generare (enumerare) pentru cele mai cunoscute familii de obiecte combinatoriale: produs cartezian, submulțimi, aranjamente (fără repetiție sau cu repetiție), combinații (fără repetiție sau cu repetiție), permutări (fără repetiție sau cu repetiție).

Reamintim pentru început câteva noțiuni uzuale.

Definiția 4.4.1. *Fie A un alfabet (adică o mulțime finită) și $n \in \mathbb{N}^*$. O secvență de forma*

$$a = a_1 a_2 \dots a_n, \text{ cu } a_1, a_2, \dots, a_n \in A,$$

se numește cuvânt de lungime n peste alfabetul A . Lungimea cuvântului a se notează cu $|a|$.

Observația 4.4.1. Evident, cuvântul $a_1 a_2 \dots a_n$ poate fi identificat cu vectorul $(a_1, a_2, \dots, a_n)$.

Definiția 4.4.2. Fie $x \in \mathbb{R}$ și $n \in \mathbb{N}$. Notăm

$$[x]_n = \begin{cases} 1, & \text{dacă } n = 0, \\ \underbrace{x(x-1) \dots (x-n+1)}_{n \text{ factori}}, & \text{dacă } n \geq 1, \end{cases}$$

$$[x]^n = \begin{cases} 1, & \text{dacă } n = 0, \\ \underbrace{x(x+1) \dots (x+n-1)}_{n \text{ factori}}, & \text{dacă } n \geq 1. \end{cases}$$

$[x]_n$ se numește **polinomul factorial descrescător** de gradul n , iar $[x]^n$ se numește **polinomul factorial crescător** de gradul n .

4.4.2 Produs cartezian

Definiția 4.4.3. **Produsul cartezian** al mulțimilor $A_1, A_2, \dots, A_n$ ($n \in \mathbb{N}^*$) este mulțimea

$$A_1 \times A_2 \times \dots \times A_n = \{(x_1, x_2, \dots, x_n) | x_1 \in A_1, x_2 \in A_2, \dots, x_n \in A_n\}.$$

Exemplul 4.4.1. $\{a, b\} \times \{+, -\} \times \{c\} = \{(a, +, c), (a, -, c), (b, +, c), (b, -, c)\}.$

Propoziția 4.4.1 (de numărare a produsului cartezian). Fie $n \in \mathbb{N}^*$ și $A_1, A_2, \dots, A_n$ mulțimi finite. Atunci $\text{card}(A_1 \times A_2 \times \dots \times A_n) = \text{card}(A_1) \cdot \text{card}(A_2) \cdot \dots \cdot \text{card}(A_n).$

Demonstrație. Se utilizează metoda inducției matematice după n . □

Algoritmul 4.4.1 (de generare a produsului cartezian prin metoda Backtracking). Fie mulțimile standard

$$A_1 = \{1, 2, \dots, m_1\}, A_2 = \{1, 2, \dots, m_2\}, \dots, A_n = \{1, 2, \dots, m_n\}.$$

Vom utiliza metoda Backtracking. Avem

$$x = (x_1, x_2, \dots, x_n) \in S_1 \times S_2 \times \dots \times S_n,$$

unde

$$S_1 = A_1, S_2 = A_2, \dots, S_n = A_n.$$

Așadar, mulțimile S_k , conțin termeni succesivi ai unei progresii aritmetice, în care

$$\begin{cases} a_k = 1, \\ r_k = 1, \\ b_k = m_k, \end{cases} \quad \forall k = \overline{1, n}.$$

Conform Observației 4.1.13, eliminăm verificarea condițiilor de continuare.

PRODUS_CARTEZIAN(n, m) :

```

 $k \leftarrow 1$ ;
 $x[1] \leftarrow 0$ ;
while  $k > 0$  do
    if  $x[k] < m[k]$  then
         $x[k] \leftarrow x[k] + 1$ ;
        if  $k = n$  then
            | AFISARE( $x, n$ );
        else
            |  $k \leftarrow k + 1$ ;
            |  $x[k] \leftarrow 0$ ;
    else
        |  $k \leftarrow k - 1$ ;

```

Funcția de afișare este

AFISARE(x, n) :

```

for  $i = \overline{1, n}$  do
    | afișează  $x[i]$ ;

```

Observația 4.4.2. Pentru generarea produsului cartezian $A_1 \times A_2 \times \dots \times A_n$ al unor mulțimi finite arbitrare

$$\begin{aligned}
 A_1 &= \{a_{11}, a_{12}, \dots, a_{1m_1}\}, \\
 A_2 &= \{a_{21}, a_{22}, \dots, a_{2m_2}\}, \\
 &\dots, \\
 A_n &= \{a_{n1}, a_{n2}, \dots, a_{nm_n}\}
 \end{aligned}$$

se poate folosi algoritmul anterior, bazat pe generarea indicilor, înlocuind afișarea indicilor x_i cu afișarea elementelor corespunzătoare a_{ix_i} din mulțimile A_i , adică înlocuind funcția **AFISARE**(x, n) cu funcția

```

AFISARE( $x, a, n$ ) :
for  $i = \overline{1, n}$  do
    | afișează  $a[i, x[i]]$ ;

```

4.4.3 Submulțimi

Propoziția 4.4.2 (de numărare a submulțimilor). *Fie A o mulțime finită și $\mathcal{P}(A)$ mulțimea tuturor submulțimilor (părților) lui A . Atunci*

$$\text{card}(\mathcal{P}(A)) = 2^{\text{card}(A)}.$$

Demonstrație. Fie $A = \{a_1, a_2, \dots, a_n\}$, $n \in \mathbb{N}^*$. Notăm

$$\{1, 2\}^n = \underbrace{\{1, 2\} \times \dots \times \{1, 2\}}_{\text{de } n \text{ ori}}.$$

Definim funcțiile $\alpha : \mathcal{P}(A) \rightarrow \{1, 2\}^n$ și $\beta : \{1, 2\}^n \rightarrow \mathcal{P}(A)$ prin:

- $\forall B \in \mathcal{P}(A)$, $\alpha(B) = (x_1, x_2, \dots, x_n)$, unde $x_i = \begin{cases} 1, & \text{dacă } a_i \in B, \\ 2, & \text{dacă } a_i \notin B; \end{cases}$
- $\forall (x_1, x_2, \dots, x_n) \in \{1, 2\}^n$, $\beta(x_1, x_2, \dots, x_n) = \{a_i | x_i = 1, i \in \{1, \dots, n\}\}$.

Funcțiile α și β sunt inverse una celeilalte, deci sunt bijective și

$$\text{card}(\mathcal{P}(A)) = \text{card}(\{1, 2\}^n) = 2^n.$$

□

Exemplul 4.4.2. Pentru $A = \{a, b, c\}$, corespondența *submulțimi* $\leftrightarrow$ *produs cartezian* din demonstrația anterioară este redată în următorul tabel:

$(x_1, x_2, x_3) \in \{1, 2\}^3$	$B \in \mathcal{P}(A)$
(1,1,1)	$\{a, b, c\}$
(1,1,2)	$\{a, b\}$
(1,2,1)	$\{a, c\}$
(1,2,2)	$\{a\}$
(2,1,1)	$\{b, c\}$
(2,1,2)	$\{b\}$
(2,2,1)	$\{c\}$
(2,2,2)	$\emptyset$

Deci A are $2^3 = 8$ submulțimi.

Fie mulțimea $A = \{a_1, a_2, \dots, a_n\}$. Conform demonstrației anterioare, putem genera produsul cartezian $\{1, 2\}^n$ cu Algoritmul 4.4.1 înlocuind afișarea elementelor $(x_1, \dots, x_n)$ cu afișarea submulțimilor corespunzătoare

$$B = \{a_i | x_i = 1, i \in \{1, \dots, n\}\}.$$

Obținem următorul algoritm descris în pseudocod.

Algoritmul 4.4.2 (de generare a submulțimilor, folosind metoda Backtracking).

```

SUBMULTIMI( $a, n$ ) :
 $k \leftarrow 1$ ;
 $x[1] \leftarrow 0$ ;
while  $k > 0$  do
    if  $x[k] < 2$  then
         $x[k] \leftarrow x[k] + 1$ ;
        if  $k = n$  then
            AFISARE( $x, a, n$ );
        else
             $k \leftarrow k + 1$ ;
             $x[k] \leftarrow 0$ ;
    else
         $k \leftarrow k - 1$ ;

```

Funcția de afișare este

```

AFISARE( $x, a, n$ ) :
for  $i = \overline{1, n}$  do
    if  $x[i] = 1$  then afișează  $a[i]$ ;

```

4.4.4 Aranjamente cu repetiție

Propoziția 4.4.3 (de numărare a aranjamentelor cu repetiție). *Fie $m, n \in \mathbb{N}$. Atunci numărul de cuvinte de lungime n peste un alfabet cu m litere este egal cu m^n .*

Demonstrație. Fie $B = \{1, 2, \dots, m\}$ și $\mathcal{C} = \{(x_1, x_2, \dots, x_n) | x_i \in B \forall i\}$. Cum $\mathcal{C} = B^n$, conform Propoziției 4.4.1 avem $\text{card}(\mathcal{C}) = m^n$. $\square$

Definiția 4.4.4. *Cuvintele numărate în propoziția anterioară se numesc **aranjamente cu repetiție** de m luate câte n . Prin abuz de limbaj, și numărul lor, adică m^n , se numește tot **aranjamente cu repetiție de m luate câte n** .*

Exemplul 4.4.3. Pentru $m = 2$ și $n = 3$, aranjamente cu repetiție sunt, în ordine lexicografică:

$(1, 1, 1), (1, 1, 2), (1, 2, 1), (1, 2, 2), (2, 1, 1), (2, 1, 2), (2, 2, 1), (2, 2, 2)$.

Deci avem $2^3 = 8$ aranjamente cu repetiție.

Algoritmul 4.4.3 (de generare a aranjamentelor cu repetiție). Pentru mulțimea standard $B = \{1, 2, \dots, m\}$, conform demonstrației anterioare putem genera produsul cartezian B^n , cu Algoritmul 4.4.1 luând $m_1 = m_2 = \dots = m_n = m$.

Observația 4.4.3. Pentru o mulțime $A = \{a_1, a_2, \dots, a_n\}$ oarecare putem genera indicii $(x_1, \dots, x_n)$ cu algoritmul anterior și afișa elementele corespunzătoare acestor indici $(a_{x_1}, \dots, a_{x_n})$.

4.4.5 Aranjamente

Propoziția 4.4.4 (de numărare a aranjamentelor). Fie $m, n \in \mathbb{N}$. Atunci numărul de cuvinte de lungime n cu litere distincte peste un alfabet cu m litere este egal cu $[m]_n$.

Demonstrație. Fie $A = \{1, 2, \dots, m\}$ și

$$\mathcal{C}_1 = \{(x_1, x_2, \dots, x_n) | x_i \in A \forall i, x_i \neq x_j \forall i \neq j\}.$$

Avem

$$\begin{aligned} \mathcal{C}_1 = \{ & (x_1, x_2, \dots, x_n) | x_1 \in \{1, \dots, m\}, \\ & x_2 \in \{1, \dots, m\} \setminus \{x_1\}, \\ & x_3 \in \{1, \dots, m\} \setminus \{x_1, x_2\}, \\ & \dots, \\ & x_n \in \{1, \dots, m\} \setminus \{x_1, \dots, x_{n-1}\} \} \end{aligned}$$

deci $\text{card}(\mathcal{C}_1) = m(m-1)(m-2) \dots (m-n+1) = [m]_n$. □

Definiția 4.4.5. Cuvintele numărate în propoziția anterioară se numesc **aranjamente (fără repetiție)** de m luate câte n . De asemenea și numărul lor, adică $[m]_n$, se numește tot **aranjamente (fără repetiție) de m luate câte n** și se mai notează cu A_m^n .

Observația 4.4.4. Pentru $n > m$ avem

$$[m]_n = m \dots (m-m) \dots (m-n+1) = 0.$$

Exemplul 4.4.4. Pentru $m = 4$ și $n = 2$, aranjamentele sunt, în ordine lexicografică:

$$(1, 2), (1, 3), (1, 4), (2, 1), (2, 3), (2, 4), (3, 1), (3, 2), (3, 4), (4, 1), (4, 2), (4, 3).$$

Deci avem $[4]_2 = 4 \cdot 3 = 12$ aranjamente.

Algoritmul 4.4.4 (de generare a aranjamentelor prin metoda Backtracking, varianta 1). Fie mulțimea standard $A = \{1, 2, \dots, m\}$ și $n \leq m$.

Vom utiliza metoda Backtracking. Avem

$$x = (x_1, x_2, \dots, x_n) \in S_1 \times S_2 \times \dots \times S_n,$$

unde

$$S_1 = S_2 = \dots = S_n = A.$$

Așadar, mulțimile S_k , conțin termeni succesivi ai unei progresii aritmetice, în care

$$\begin{cases} a_k = 1, \\ r_k = 1, \\ b_k = m, \end{cases} \quad \forall k = \overline{1, n}.$$

Condițiile interne:

$$x_i \neq x_j, \quad \forall i \neq j, \quad i, j = \overline{1, n}.$$

Condițiile de continuare:

Dacă avem soluția parțială $(x_1, \dots, x_{k-1})$, atunci x_k verifică condițiile de continuare dacă

$$x_k \neq x_i, \quad \forall i = \overline{1, k-1}.$$

Altfel spus, dacă avem soluția parțială $x = (x_1, \dots, x_{k-1})$, atunci x_k nu verifică condițiile de continuare dacă

$$\exists i \in \{1, 2, \dots, k-1\} \text{ astfel încât } x_k = x_i.$$

ARANJAMENTE1(m, n):

$k \leftarrow 1;$

$x[1] \leftarrow 0;$

while $k > 0$ **do**

if $x[k] < m$ **then**

$x[k] \leftarrow x[k] + 1;$

if **VALID**(x, k) **then**

if $k = n$ **then**

AFISARE(x, n);

else

$k \leftarrow k + 1;$

$x[k] \leftarrow 0;$

else

$k \leftarrow k - 1;$

```

VALID( $x, k$ ):
for  $i = \overline{1, k-1}$  do
    if  $x[k] = x[i]$  then
        returnează 0;                                // fals
returnează 1;                                        // adevărat

```

Funcția de afișare este, din nou:

```

AFISARE( $x, n$ ):
for  $i = \overline{1, n}$  do
    afișează  $x[i]$ ;

```

Algoritmul 4.4.5 (de generare a aranjamentelor prin metoda Backtracking, varianta 2). Putem îmbunătăți algoritmul anterior prin utilizarea unui vector $(y_1, y_2, \dots, y_m)$ cu semnificația: dacă avem soluția parțială $(x_1, \dots, x_{k-1})$, atunci

$$y_i = \begin{cases} 1, & \text{dacă } i \in \{x_1, \dots, x_{k-1}\}, \\ 0, & \text{dacă } i \notin \{x_1, \dots, x_{k-1}\}, \end{cases} \quad \forall i = \overline{1, m}.$$

Astfel x_k verifică condițiile de continuare dacă și numai dacă

$$y_{x_k} = 0.$$

Obținem următorul algoritm:

```

ARANJAMENTE2( $m, n$ ):
for  $i = \overline{1, m}$  do  $y[i] \leftarrow 0$ ;           // inițial, soluția parțială este
                                           // vectorul vid

 $k \leftarrow 1$ ;
 $x[1] \leftarrow 0$ ;
while  $k > 0$  do
    if  $x[k] < m$  then
         $x[k] \leftarrow x[k] + 1$ ;
        if  $y[x[k]] = 0$  then                //  $x[k]$  verifică condițiile
                                           // de continuare
            if  $k = n$  then
                | AFISARE( $x, n$ );
            else
                |  $y[x[k]] \leftarrow 1$ ;      // înainte de avansare, se adaugă
                                           //  $x[k]$  la soluția parțială
                |  $k \leftarrow k + 1$ ;
                |  $x[k] \leftarrow 0$ ;
        else
            |  $k \leftarrow k - 1$ ;
            |  $y[x[k]] \leftarrow 0$ ;           // după revenire, se elimină
                                           //  $x[k]$  din soluția parțială

```

Funcția de afișare este aceeași ca în algoritmul anterior.

Observația 4.4.5. Pentru generarea aranjamentelor de m luate câte n ale unor mulțimi finite arbitrare

$$A = \{a_1, a_2, \dots, a_m\}$$

se pot folosi algoritmi anteriori, bazați pe generarea indicilor, înlocuind afișarea indicilor x_i cu afișarea elementelor corespunzătoare a_{x_i} din mulțimile A_i , adică înlocuind funcția **AFIȘARE**(x, n) cu funcția

```

AFISARE( $x, a, n$ ):
for  $i = \overline{1, n}$  do
    | afișează  $a[x[i]]$ ;

```

4.4.6 Permutări

Propoziția 4.4.5 (de numărare a permutărilor). Fie $n \in \mathbb{N}$. Atunci numărul de cuvinte ce conțin exact o dată fiecare literă a unui alfabet cu n litere este egal cu $n!$, unde

$$n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n \text{ (} n \text{ factorial)}, 0! = 1.$$

adică $n!$, se numeste tot **permutări** (fără repetitie) de n .

în ordine lexicografică:

$$(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1).$$

Deci avem $3! = 1 \cdot 2 \cdot 3 = 6$ permutări.

$(x_1, x_2, \dots, x_n)$ nu mai există alte valori valide pentru componenta x_n) obținem următorul algoritm.

 $k \leftarrow 1;$
$$x[1] \leftarrow 0;$$
| **if** $x[k] < n$ **t**
$$x[k] \leftarrow x[k] +$$

if $VALID(x, k)$

| **if** $k = n$ **then**

| AFISARE

$$k \leftarrow k - 1;$$

// 1

```
// nu mai există valori valide pentru  $x[n]$ 
```

 $|k$
$$x[k] \leftarrow 0;$$
 L^k **for** $i = \overline{1, k-1}$
$$\text{if } x[k] = x[i] \text{ then}$$

```
returnează 0;
```

Funcția de afișare este, din nou:

```
AFISARE( $x, n$ ):  
for  $i = \overline{1, n}$  do  
    afixează  $x[i]$ ;
```

Algoritmul 4.4.7 (de generare a permutărilor prin metoda Backtracking, varianta 2). Luând $m = n$ în Algoritmul 4.4.5 de generare a aranjamentelor și forțând din nou revenirea după determinarea fiecărei permutări obținem următorul algoritm.

```
PERMUTĂRI2( $n$ ):  
for  $i = \overline{1, n}$  do  $y[i] \leftarrow 0$ ;  
 $k \leftarrow 1$ ;  
 $x[1] \leftarrow 0$ ;  
while  $k > 0$  do  
    if  $x[k] < n$  then  
         $x[k] \leftarrow x[k] + 1$ ;  
        if  $y[x[k]] = 0$  then  
            if  $k = n$  then  
                AFISARE( $x, n$ );  
                 $k \leftarrow k - 1$ ;  
                 $y[x[k]] \leftarrow 0$ ;  
            else  
                 $y[x[k]] \leftarrow 1$ ;  
                 $k \leftarrow k + 1$ ;  
                 $x[k] \leftarrow 0$ ;  
        else  
             $k \leftarrow k - 1$ ;  
             $y[x[k]] \leftarrow 0$ ;
```

Funcția de afișare este aceeași ca în algoritmul anterior.

Observația 4.4.6. Pentru generarea permutărilor unei mulțimi arbitrare $A = \{a_1, a_2, \dots, a_n\}$ înlocuim afișarea indicilor $(x_1, \dots, x_n)$ cu afișarea elementelor corespunzătoare $(a_{x_1}, \dots, a_{x_n})$.

4.4.7 Combinări

Propoziția 4.4.6 (de numărare a combinațiilor). Fie $m, n \in \mathbb{N}$. Atunci numărul de cuvinte strict crescătoare de lungime n peste un alfabet (ordonat) cu m litere
 $=$ numărul de submulțimi cu n elemente ale unei mulțimi cu m elemente

$$= \frac{[m]_n}{n!} \stackrel{\text{def}}{=} \binom{m}{n}.$$

Demonstrație. Fie $A = \{1, 2, \dots, m\}$. Notăm mulțimile din enunț astfel:

$$\mathcal{C}_3 = \{(c_1, c_2, \dots, c_n) \mid c_i \in A \forall i, c_i < c_{i+1} \forall i\}, \mathcal{S}_3 = \{S \mid S \subseteq A, \text{card}(S) = n\}.$$

Între aceste mulțimi definim funcțiile $\mu : \mathcal{C}_3 \rightarrow \mathcal{S}_3$, $\nu : \mathcal{S}_3 \rightarrow \mathcal{C}_3$ prin:

- $\forall (c_1, c_2, \dots, c_n) \in \mathcal{C}_3$, $\mu(c_1, c_2, \dots, c_n) = \{c_1, c_2, \dots, c_n\}$;
- $\forall \{c_1, c_2, \dots, c_n\} \in \mathcal{S}_3$ cu $c_1 < c_2 < \dots < c_n$, $\nu(\{c_1, c_2, \dots, c_n\}) = (c_1, c_2, \dots, c_n)$.

Aceste funcții sunt inverse una celeilalte, deci sunt bijective și astfel

$$\text{card}(\mathcal{C}_3) = \text{card}(\mathcal{S}_3).$$

Permutând fiecare combinație $(c_1, c_2, \dots, c_n) \in \mathcal{C}_3$ în toate cele $n!$ moduri posibile, obținem fără repetare toate aranjamentele din mulțimea $\mathcal{C}_1 = \{(a_1, a_2, \dots, a_n) \mid a_i \in A \forall i, a_i \neq a_j \forall i \neq j\}$.

Deci $\text{card}(\mathcal{C}_3) \cdot n! = \text{card}(\mathcal{C}_1)$. Conform Propoziției 4.4.4, $\text{card}(\mathcal{C}_1) = [m]_n$, deci $\text{card}(\mathcal{C}_3) = \frac{[m]_n}{n!}$. $\square$

Definiția 4.4.7. Oricare obiecte din cele 2 tipuri numărate în propoziția anterioară se numesc **combinări (fără repetiție)** de m luate câte n . De asemenea și numărul lor, adică $\binom{m}{n}$, se numește tot **combinări (fără repetiție) de m luate câte n** și se mai notează cu C_m^n .

Observația 4.4.7. Pentru $n > m$ avem $\binom{m}{n} = 0$, deoarece $[m]_n = 0$.

Pentru $n \leq m$, deoarece $[m]_n = m(m-1) \dots (m-n+1) = \frac{m!}{(m-n)!}$ avem

$$\binom{m}{n} = \frac{m!}{n!(m-n)!}.$$

Exemplul 4.4.6. Pentru $m = 5$ și $n = 3$, corespondențele din demonstrația anterioară sunt redată în următorul tabel:

$(c_1, c_2, c_3) \in \mathcal{C}_3$	$S \in \mathcal{S}_3$
(1,2,3)	$\{1, 2, 3\}$
(1,2,4)	$\{1, 2, 4\}$
(1,2,5)	$\{1, 2, 5\}$
(1,3,4)	$\{1, 3, 4\}$
(1,3,5)	$\{1, 3, 5\}$
(1,4,5)	$\{1, 4, 5\}$
(2,3,4)	$\{2, 3, 4\}$
(2,3,5)	$\{2, 3, 5\}$
(2,4,5)	$\{2, 4, 5\}$
(3,4,5)	$\{3, 4, 5\}$

Deci avem $\binom{5}{3} = \frac{[5]_3}{3!} = \frac{5 \cdot 4 \cdot 3}{1 \cdot 2 \cdot 3} = 10$ combinații.

Algoritmul 4.4.8 (de generare a combinațiilor prin metoda Backtracking). Fie mulțimea standard $A = \{1, 2, \dots, m\}$ și $n \leq m$.

Vom utiliza metoda Backtracking. Fie soluția (combinația)

$$x = (x_1, x_2, \dots, x_n).$$

Condițiile interne:

$$1 \leq x_1 < x_2 < \dots < x_{n-1} < x_n \leq m,$$

deci

$$x_n \leq m, \quad x_{n-1} \leq m-1, \quad x_{n-2} \leq m-2, \quad \dots, \quad x_2 \leq m-n+2, \quad x_1 \leq m-n+1.$$

Astfel

$$\begin{aligned} x_1 &\in S_1 = \{1, 2, \dots, m-n+1\}, \\ x_2 &\in S_2 = \{x_1+1, x_1+2, \dots, m-n+2\}, \\ &\dots \\ x_k &\in S_k = \{x_{k-1}+1, x_{k-1}+2, \dots, m-n+k\}, \\ &\dots \\ x_n &\in S_n = \{x_{n-1}+1, x_{n-1}+2, \dots, m\}. \end{aligned}$$

Așadar, mulțimile S_k , conțin termeni succesivi ai unei progresii aritmetice, în care

$$\begin{cases} a_k = x_{k-1} + 1, \\ r_k = 1, \\ b_k = m - n + k, \end{cases} \quad \forall k = \overline{1, n},$$

considerând $x_0 = 0$.

Prin alegerea mulțimilor $S_1, S_2, \dots, S_n$ am eliminat verificarea condițiilor de continuare.

COMBINĂRI (m, n) :

$k \leftarrow 1$;

$x[1] \leftarrow 0$;

while $k > 0$ **do**

if $x[k] < m - n + k$ **then**

$x[k] \leftarrow x[k] + 1$;

if $k = n$ **then**

AFISARE (x, n) ;

else

$k \leftarrow k + 1$;

$x[k] \leftarrow x[k - 1]$;

else

$k \leftarrow k - 1$;

Funcția de afișare este, din nou:

AFISARE (x, n) :

for $i = \overline{1, n}$ **do**

afișează $x[i]$;

Observația 4.4.8. Analog Observației 4.4.6, algoritmul anterior poate fi ușor adaptat pentru generarea combinațiilor pentru mulțimi oarecare.

4.4.8 Combinări cu repetiție

Propoziția 4.4.7 (de numărare a combinațiilor cu repetiție). *Fie $m, n \in \mathbb{N}$. Atunci numărul de cuvinte crescătoare de lungime n peste un alfabet (ordonat) cu m litere este egal cu $\frac{[m]^n}{n!} \stackrel{\text{def}}{=} \binom{m+n-1}{n}$.*

Demonstrație. Fie $B = \{1, 2, \dots, m\}$, $A = \{1, 2, \dots, m + n - 1\}$,

$$\mathcal{C}_4 = \{(c_1, c_2, \dots, c_n) | c_i \in B \forall i, c_i \leq c_{i+1} \forall i\}$$

$$\mathcal{C}_3 = \{(d_1, d_2, \dots, d_n) | d_i \in A \forall i, d_i < d_{i+1} \forall i\}.$$

Definim corespondențele $\rho : \mathcal{C}_4 \rightarrow \mathcal{C}_3$, $\sigma : \mathcal{C}_3 \rightarrow \mathcal{C}_4$ astfel:

- $\rho(c_1, c_2, c_3, \dots, c_n) = (c_1, c_2 + 1, c_3 + 2, \dots, c_n + n - 1)$;
- $\sigma(d_1, d_2, d_3, \dots, d_n) = (d_1, d_2 - 1, d_3 - 2, \dots, d_n - n + 1)$.

Acestea sunt funcții inverse, deci utilizând Propoziția 4.4.6 avem

$$\text{card}(\mathcal{C}_4) = \text{card}(\mathcal{C}_3) = \binom{m+n-1}{n} = \frac{[m+n-1]_n}{n!} = \frac{[m]^n}{n!} = \left(\binom{m}{n}\right).$$

□

Definiția 4.4.8. Cuvintele numărate în propoziția anterioară se numesc **combinări cu repetiție** de m luate câte n . De asemenea și numărul lor, adică $\left(\binom{m}{n}\right)$, se numește tot **combinări cu repetiție de m luate câte n** .

Exemplul 4.4.7. Pentru $m = 3$ și $n = 4$, combinările cu repetiție sunt, în ordine lexicografică:

$$\begin{aligned} &(1, 1, 1, 1), (1, 1, 1, 2), (1, 1, 1, 3), (1, 1, 2, 2), (1, 1, 2, 3), \\ &(1, 1, 3, 3), (1, 2, 2, 2), (1, 2, 2, 3), (1, 2, 3, 3), (1, 3, 3, 3), \\ &(2, 2, 2, 2), (2, 2, 2, 3), (2, 2, 3, 3), (2, 3, 3, 3), (3, 3, 3, 3). \end{aligned}$$

Deci avem $\left(\binom{3}{4}\right) = \frac{[3]^4}{4!} = \frac{3 \cdot 4 \cdot 5 \cdot 6}{1 \cdot 2 \cdot 3 \cdot 4} = 15$ combinări cu repetiție.

Algoritmul 4.4.9 (de generare a combinărilor cu repetiție prin metoda Backtracking). Fie mulțimea standard $A = \{1, 2, \dots, m\}$ și $n \geq 1$.

Vom utiliza metoda Backtracking. Fie soluția (combinarea cu repetiție)

$$x = (x_1, x_2, \dots, x_n).$$

Condițiile interne:

$$1 \leq x_1 \leq x_2 \leq \dots \leq x_{n-1} \leq x_n \leq m.$$

Astfel

$$\begin{aligned} x_1 &\in S_1 = \{1, 2, \dots, m\}, \\ x_2 &\in S_2 = \{x_1, x_1 + 1, \dots, m\}, \\ &\dots \\ x_k &\in S_k = \{x_{k-1}, x_{k-1} + 1, \dots, m\}, \\ &\dots \\ x_n &\in S_n = \{x_{n-1}, x_{n-1} + 1, \dots, m\}. \end{aligned}$$

Așadar, mulțimile S_k , conțin termeni succesivi ai unei progresii aritmetice, în care

$$\begin{cases} a_k = x_{k-1}, \\ r_k = 1, \\ b_k = m, \end{cases} \quad \forall k = \overline{1, n},$$

considerând $x_0 = 1$.

Prin alegerea mulțimilor $S_1, S_2, \dots, S_n$ am eliminat verificarea condițiilor de continuare.

COMBINĂRI_CU_REPETIȚIE(m, n):

```

 $k \leftarrow 1$ ;
 $x[1] \leftarrow 0$ ;
while  $k > 0$  do
    if  $x[k] < m$  then
         $x[k] \leftarrow x[k] + 1$ ;
        if  $k = n$  then
            | AFISARE( $x, n$ );
        else
            |  $k \leftarrow k + 1$ ;
            |  $x[k] \leftarrow x[k - 1] - 1$ ;
    else
        |  $k \leftarrow k - 1$ ;

```

Funcția de afișare este, din nou:

AFISARE(x, n):

```

for  $i = \overline{1, n}$  do
    | afișează  $x[i]$ ;

```

Observația 4.4.9. Analog Observației 4.4.6, algoritmul anterior poate fi ușor adaptat pentru generarea combinațiilor cu repetiție pentru mulțimi oarecare.

4.4.9 Permutări cu repetiție

Propoziția 4.4.8 (de numărare a permutărilor cu repetiție). *Fie $m, t_1, t_2, \dots, t_m \in \mathbb{N}$ și $n = t_1 + t_2 + \dots + t_m$. Atunci numărul de cuvinte de lungime n ce pot fi formate peste un alfabet cu m litere astfel încât litera numărul i să apară de exact t_i ori pentru orice $i \in \{1, \dots, m\}$ este egal cu*

$$\frac{n!}{t_1!t_2!\dots t_m!} \stackrel{\text{def}}{=} \binom{n}{t_1, t_2, \dots, t_m}.$$

Demonstrație. Fie $A = \{1, 2, \dots, m\}$ și

$$\mathcal{C}_5 = \{(x_1, x_2, \dots, x_n) | x_i \in A \forall i, \text{card}(\{i | x_i = j\}) = t_j \forall j\}.$$

Numărăm cuvintele din $\mathcal{C}_5$ astfel:

- alegem cei t_1 indici (din totalul de n) ai literelor egale cu 1, rezultă $\binom{n}{t_1}$ moduri posibile;

- pentru fiecare alegere de mai sus, alegem cei t_2 indici (din restul de $n - t_1$) ai literelor egale cu 2, rezultă $\binom{n - t_1}{t_2}$ moduri posibile, deci obținem $\binom{n}{t_1} \binom{n - t_1}{t_2}$ moduri posibile de alegere a indicilor literelor 1 și 2;
- ...
- pentru fiecare alegere de mai sus, alegem cei t_m indici (din restul de $n - t_1 - \dots - t_{m-1}$) ai literelor egale cu m ; rezultă $\binom{n - t_1 - \dots - t_{m-1}}{t_m}$ moduri posibile, deci obținem

$$\binom{n}{t_1} \binom{n - t_1}{t_2} \dots \binom{n - t_1 - \dots - t_{m-1}}{t_m}$$

moduri posibile de alegere a tuturor indicilor literelor $1, 2, \dots, m$.

Astfel

$$\begin{aligned} \text{card}(\mathcal{C}_5) &= \binom{n}{t_1} \binom{n - t_1}{t_2} \dots \binom{n - t_1 - \dots - t_{m-1}}{t_m} \\ &= \frac{n!}{t_1!(n - t_1)!} \cdot \frac{(n - t_1)!}{t_2!(n - t_1 - t_2)!} \cdot \dots \cdot \frac{(n - t_1 - \dots - t_{m-1})!}{t_m!(n - t_1 - \dots - t_m)!} \\ &= \frac{n!}{t_1!t_2! \dots t_m!} \end{aligned}$$

(deoarece $(n - t_1 - \dots - t_m)! = 0! = 1$).

□

Definiția 4.4.9. Cuvintele numărate în propoziția anterioară se numesc **permutări cu repetiție (anagrame)** de n luate câte $t_1, t_2, \dots, t_m$. De asemenea și numărul lor, adică $\binom{n}{t_1, t_2, \dots, t_m}$, se numește tot **permutări cu repetiție de n luate câte $t_1, t_2, \dots, t_m$** .

Observația 4.4.10. Luând $t_1 = t_2 = \dots = t_m = 1$ obținem $n = m$ și $\binom{n}{1, 1, \dots, 1} = n!$, deci permutările (fără repetiție) sunt un caz particular al permutărilor cu repetiție. Pe de altă parte, luând $m = 2$ obținem $\binom{n}{t_1, t_2} = \frac{n!}{t_1!t_2!}$, deci și combinările (fără repetiție) sunt un caz particular al permutărilor cu repetiție.

Exemplul 4.4.8. Pentru $m = 3$ și $t_1 = 2, t_2 = t_3 = 1$, deci $n = 4$, permutările cu repetiție sunt, în ordine lexicografică:

$$(1, 1, 2, 3), (1, 1, 3, 2), (1, 2, 1, 3), (1, 2, 3, 1), (1, 3, 1, 2), (1, 3, 2, 1),$$

$$(2, 1, 1, 3), (2, 1, 3, 1), (2, 3, 1, 1), (3, 1, 1, 2), (3, 1, 2, 1), (3, 2, 1, 1).$$

Deci avem $\binom{4}{2, 1, 1} = \frac{4!}{2!1!1!} = 12$ permutări cu repetiție.

Algoritmul 4.4.10 (de generare a permutărilor cu repetiție prin metoda Backtracking, varianta 1). Fie mulțimea standard $A = \{1, 2, \dots, m\}$, $t_1, t_2, \dots, t_m \in \mathbb{N}$ și $n = t_1 + t_2 + \dots + t_m$.

Vom utiliza metoda Backtracking. Avem

$$x = (x_1, x_2, \dots, x_n) \in S_1 \times S_2 \times \dots \times S_n,$$

unde

$$S_1 = S_2 = \dots = S_n = A.$$

Așadar, mulțimile S_k , conțin termeni succesivi ai unei progresii aritmetice, în care

$$\begin{cases} a_k = 1, \\ r_k = 1, \\ b_k = m, \end{cases} \quad \forall k = \overline{1, n}.$$

Condițiile interne:

$$\text{card}(\{i | i = \overline{1, n}, x_i = j\}) = t_j \quad \forall j = \overline{1, m}.$$

Condițiile de continuare:

Dacă avem soluția parțială $(x_1, \dots, x_{k-1})$, atunci x_k verifică condițiile de continuare dacă

$$\text{card}(\{i | i = \overline{1, k-1}, x_i = x_k\}) < t_{x_k}.$$

```

PERMUTĂRI_CU_REPETIȚIE1( $n, t, m$ ) :
 $n \leftarrow 0$ ;
for  $i = \overline{1, m}$  do  $n \leftarrow n + t[i]$ ;
 $k \leftarrow 1$ ;
 $x[1] \leftarrow 0$ ;
while  $k > 0$  do
    if  $x[k] < m$  then
         $x[k] \leftarrow x[k] + 1$ ;
         $p \leftarrow t[x[k]]$ ;
        if VALID( $p, x, k$ ) then
            if  $k = n$  then
                AFISARE( $x, n$ );
                 $k \leftarrow k - 1$ ;           // forțăm revenirea, deoarece
                // nu mai există valori valide pentru  $x[n]$ 
            else
                 $k \leftarrow k + 1$ ;
                 $x[k] \leftarrow 0$ ;
        else
             $k \leftarrow k - 1$ ;

VALID( $p, x, k$ ) :
 $q \leftarrow 0$ ;           //  $q$  reprezintă  $\text{card}(\{i | i = \overline{1, k-1}, x[i] = x[k]\})$ 
if  $p = 0$  then returnează 0;
for  $i = \overline{1, k-1}$  do
    if  $x[i] = x[k]$  then
         $q \leftarrow q + 1$ ;
        if  $q \geq p$  then returnează 0;
returnează 1;

Funcția de afișare este, din nou:
AFISARE( $x, n$ ) :
for  $i = \overline{1, n}$  do
    afișează  $x[i]$ ;

```

Algoritmul 4.4.11 (de generare a permutărilor cu repetiție prin metoda Backtracking, varianta 2). Putem îmbunătăți algoritmul anterior prin utilizarea unui vector $(y_1, y_2, \dots, y_m)$ cu semnificația: dacă avem soluția parțială $(x_1, \dots, x_{k-1})$, atunci

$$y_j = \text{card}(\{i | i = \overline{1, k-1}, x_i = j\}), \quad \forall j = \overline{1, m}.$$

Astfel x_k verifică condițiile de continuare dacă și numai dacă

$$y_{x_k} < t_{x_k}.$$

Forțând din nou revenirea după determinarea fiecărei permutări cu repetiție, obținem următorul algoritm.

```

PERMUTĂRI_CU_REPETIȚIE2( $n, t, m$ ):
 $n \leftarrow 0$ ;
for  $i = \overline{1, m}$  do  $n \leftarrow n + t[i]$ ;
for  $i = \overline{1, m}$  do  $y[i] \leftarrow 0$ ;           // inițial, soluția parțială este
                                           // vectorul vid

 $k \leftarrow 1$ ;
 $x[1] \leftarrow 0$ ;
while  $k > 0$  do
    if  $x[k] < m$  then
         $x[k] \leftarrow x[k] + 1$ ;
        if  $y[x[k]] < t[x[k]]$  then           //  $x[k]$  verifică condițiile
                                           // de continuare
            if  $k = n$  then
                AFISARE( $x, n$ );
                 $k \leftarrow k - 1$ ;
                 $y[x[k]] \leftarrow y[x[k]] - 1$ ; // după forțarea revenirii,
                                           // se elimină  $x[k]$  din soluția parțială
            else
                 $y[x[k]] \leftarrow y[x[k]] + 1$ ; // înainte de avansare, se
                                           // adaugă  $x[k]$  la soluția parțială
                 $k \leftarrow k + 1$ ;
                 $x[k] \leftarrow 0$ ;
        else
             $k \leftarrow k - 1$ ;
             $y[x[k]] \leftarrow y[x[k]] - 1$ ; // după revenire, se elimină
                                           //  $x[k]$  din soluția parțială

```

Funcția de afișare este aceeași ca în algoritmul anterior.

Observația 4.4.11. Analog Observației 4.4.6, algoritmul anterior poate fi ușor adaptat pentru generarea permutărilor cu repetiție pentru mulțimi arbitrare.

4.4.10 Compuneri ale unui număr natural

Definiția 4.4.10. Fie $m, n \in \mathbb{N}$. O **compunere** a lui m este o scriere de forma

$$m = t_1 + t_2 + \cdots + t_n,$$

unde $t_1, t_2, \dots, t_n \in \mathbb{N}$ și contează ordinea dintre termenii $t_1, t_2, \dots, t_n$.

Propoziția 4.4.9 (de numărare a compunerilor unui număr natural).

Fie $m, n \in \mathbb{N}$. Atunci:

- a) numărul de compuneri ale lui m cu n termeni este egal cu $\binom{n}{m}$;
- b) numărul de compuneri ale lui m cu n termeni nenuli este egal cu $\binom{m-1}{n-1}$.

Demonstrație. a) Fie mulțimile

$$\mathcal{N} = \{(t_1, t_2, \dots, t_n) | t_i \in \mathbb{N} \forall i, t_1 + t_2 + \cdots + t_n = m\},$$

$$\mathcal{C}_4 = \{(c_1, c_2, \dots, c_m) | c_i \in \{1, 2, \dots, n\} \forall i, c_i \leq c_{i+1} \forall i\}.$$

Definim corespondențele $\alpha : \mathcal{N} \rightarrow \mathcal{C}_4$, $\beta : \mathcal{C}_4 \rightarrow \mathcal{N}$ prin:

- $\alpha(t_1, \dots, t_n) = (c_1, \dots, c_m)$, unde

$$\begin{cases} c_1 = \cdots = c_{t_1} = 1 \text{ (} t_1 \text{ litere)}, \\ c_{t_1+1} = \cdots = c_{t_1+t_2} = 2 \text{ (} t_2 \text{ litere)}, \\ \cdots \\ c_{t_1+\cdots+t_{n-1}+1} = \cdots = c_{t_1+\cdots+t_{n-1}+t_n} = n \text{ (} t_n \text{ litere)}; \end{cases}$$

- $\beta(c_1, \dots, c_m) = (t_1, \dots, t_n)$, unde $t_i =$ numărul de litere i ale cuvântului $(c_1, \dots, c_m)$, $\forall i$.

Acestea sunt funcții inverse, deci conform Propoziției 4.4.7 avem $\text{card}(\mathcal{N}) = \text{card}(\mathcal{C}_4) = \binom{n}{m}$.

- b) Fie mulțimile $\mathcal{N}_1 = \{(t_1, t_2, \dots, t_n) | t_i \in \mathbb{N}^* \forall i, t_1 + t_2 + \cdots + t_n = m\}$,

$$\mathcal{N}_2 = \{(u_1, u_2, \dots, u_n) | u_i \in \mathbb{N} \forall i, u_1 + u_2 + \cdots + u_n = m - n\}.$$

Definim corespondențele $\varphi : \mathcal{N}_1 \rightarrow \mathcal{N}_2$, $\psi : \mathcal{N}_2 \rightarrow \mathcal{N}_1$ prin:

- $\forall (t_1, \dots, t_n) \in \mathcal{N}_1$, $\varphi(t_1, \dots, t_n) = (t_1 - 1, \dots, t_n - 1)$;
- $\forall (u_1, \dots, u_n) \in \mathcal{N}_2$, $\psi(u_1, \dots, u_n) = (u_1 + 1, \dots, u_n + 1)$.

Acestea sunt funcții inverse, deci conform punctului a) avem

$$\begin{aligned} \text{card}(\mathcal{N}_1) &= \text{card}(\mathcal{N}_2) = \left(\binom{n}{m-n} \right) = \frac{[n]^{m-n}}{(m-n)!} = \frac{n(n+1)\dots(m-1)}{(m-n)!} \\ &= \frac{(m-1)!}{(n-1)!(m-n)!} = \binom{m-1}{n-1}. \end{aligned}$$

□

Exemplul 4.4.9. Pentru $m = 4$ și $n = 3$ compunerile sunt

$$\begin{aligned} 4 &= 0 + 0 + 4 = 0 + 1 + 3 = 0 + 2 + 2 = 0 + 3 + 1 = 0 + 4 + 0 \\ &= 1 + 0 + 3 = 1 + 1 + 2 = 1 + 2 + 1 = 1 + 3 + 0 = 2 + 0 + 2 \\ &= 2 + 1 + 1 = 2 + 2 + 0 = 3 + 0 + 1 = 3 + 1 + 0 = 4 + 0 + 0. \end{aligned}$$

Deci avem $\left(\binom{3}{4} \right) = \frac{3 \cdot 4 \cdot 5 \cdot 6}{1 \cdot 2 \cdot 3 \cdot 4} = 15$ compuneri, dintre care $\binom{4-1}{3-1} = \binom{3}{2} = 3$ compuneri cu termenii nenuli.

Algoritmul 4.4.12 (de generare a compunerilor unui număr natural prin metoda Backtracking). Fie $n, m \in \mathbb{N}^*$. Vom utiliza metoda Backtracking. Fie soluția (compunere a lui m cu n termeni)

$$x = (x_1, x_2, \dots, x_n).$$

Condițiile interne:

$$x_1 + x_2 + \dots + x_n = m, \quad x_1, x_2, \dots, x_n \in \mathbb{N},$$

deci

$$\begin{aligned} x_1 \leq m, \quad x_1 + x_2 \leq m, \quad x_1 + x_2 + x_3 \leq m, \quad \dots, \quad x_1 + x_2 + \dots + x_k \leq m, \quad \dots, \\ x_1 + x_2 + \dots + x_{n-1} \leq m. \end{aligned}$$

Astfel

$$\begin{aligned} x_1 &\in S_1 = \{0, 1, \dots, m\}, \\ x_2 &\in S_2 = \{0, 1, \dots, m - x_1\}, \\ x_3 &\in S_3 = \{0, 1, \dots, m - (x_1 + x_2)\}, \\ &\dots \\ x_k &\in S_k = \{0, 1, \dots, m - (x_1 + x_2 + \dots + x_{k-1})\}, \\ &\dots \\ x_{n-1} &\in S_{n-1} = \{0, 1, \dots, m - (x_1 + x_2 + \dots + x_{n-2})\}, \end{aligned}$$

iar

$$x_n = m - (x_1 + x_2 + \dots + x_{n-1}).$$

Așadar, generăm termenii $(x_1, x_2, \dots, x_{n-1})$, cu $x_k \in S_k$, $k = \overline{1, n-1}$, iar ultimul termen se calculează cu formula $x_n = m - (x_1 + x_2 + \dots + x_{n-1})$.

Mulțimile S_k , $k = \overline{1, n-1}$ conțin termeni succesivi ai unei progresii aritmetice, în care

$$\begin{cases} a_k = 0, \\ r_k = 1, \\ b_k = m - (x_1 + x_2 + \dots + x_{k-1}), \end{cases} \quad \forall k = \overline{1, n-1}.$$

Prin alegerea mulțimilor $S_1, S_2, \dots, S_{n-1}$ am eliminat verificarea condițiilor de continuare.

Vom utiliza o variabilă s cu semnificația: dacă avem soluția parțială $(x_1, x_2, \dots, x_{k-1})$, atunci

$$s = x_1 + x_2 + \dots + x_{k-1}.$$

Aplicând *Schema Backtracking iterativă, varianta 1* (Algoritmul 4.1.1), obținem următorul algoritm.

COMPUNERI1(m, n):

$k \leftarrow 1$;

$x[1] \leftarrow -1$;

$s \leftarrow 0$; // inițial, soluția parțială este vectorul vid

while $k > 0$ **do**

if $x[k] < m - s$ **then**

$x[k] \leftarrow x[k] + 1$;

if $k = n - 1$ **then**

$x[n] \leftarrow m - s - x[k]$;

AFISARE(x, n);

else

$s \leftarrow s + x[k]$; // înainte de avansare, se adaugă
 // $x[k]$ la soluția parțială

$k \leftarrow k + 1$;

$x[k] \leftarrow -1$;

else

$k \leftarrow k - 1$;

$s \leftarrow s - x[k]$; // după revenire, se elimină
 // $x[k]$ din soluția parțială

Funcția de afișare este:

AFISARE (x, n) :
for $i = \overline{1, n}$ **do**
 | **afișează** $x[i]$;

Observația 4.4.12. Algoritmul anterior funcționează pentru $n \geq 2$. Pentru $n = 1$ soluția rezultat $(x_1, x_2, \dots, x_{n-1})$ este chiar vectorul inițial, vectorul vid, și nu poate fi obținută prin utilizarea schemei menționate, deoarece în această schemă testarea dacă soluția parțială este soluție rezultat se efectuează prin condiția **if** $k = n - 1$, care este falsă, $n - 1$ fiind egal cu 0, iar k fiind egal inițial cu 1.

Acest inconvenient poate fi ușor depășit prin utilizarea algoritmului anterior doar în cazul $n \geq 2$ și considerând separat cazul $n = 1$, în care se afișează direct singura soluție, corespunzătoare compunerii $m = m$.

Inconvenientul discutat în observația anterioară nu mai apare dacă aplicăm *Schema Backtracking iterativă, varianta 2* (Algoritmul 4.1.2). În această schemă vectorul inițial, vectorul vid, este soluție rezultat pentru $n - 1 = 0$, deoarece condiția **if** $k \leq n - 1$ este falsă, k fiind egal inițial cu 1.

Astfel obținem următorul algoritm.

```

COMPUNERI2( $m, n$ ):
 $k \leftarrow 1$ ;
 $x[1] \leftarrow -1$ ;
 $s \leftarrow 0$ ;           // inițial, soluția parțială este vectorul vid
while  $k > 0$  do
    if  $k \leq n - 1$  then
        if  $x[k] < m - s$  then
             $x[k] \leftarrow x[k] + 1$ ;
             $s \leftarrow s + x[k]$ ;           // înainte de avansare, se adaugă
                                           //  $x[k]$  la soluția parțială
             $k \leftarrow k + 1$ ;
             $x[k] \leftarrow -1$ ;
        else
             $k \leftarrow k - 1$ ;
             $s \leftarrow s - x[k]$ ;           // după revenire, se elimină
                                           //  $x[k]$  din soluția parțială
    else
         $x[n] \leftarrow m - s$ ;
        AFISARE( $x, n$ );
         $k \leftarrow k - 1$ ;
         $s \leftarrow s - x[k]$ ;           // după revenire, se elimină
                                           //  $x[k]$  din soluția parțială

```

Funcția de afișare este aceeași ca în algoritmul anterior.

Algoritmul 4.4.13 (de generare a compunerilor unui număr natural cu termeni nenuli prin metoda Backtracking). Fie $n, m \in \mathbb{N}^*$. Vom utiliza metoda Backtracking. Fie soluția (compunere a lui m cu n termeni nenuli)

$$x = (x_1, x_2, \dots, x_n).$$

Condițiile interne:

$$x_1 + x_2 + \dots + x_n = m, \quad x_1, x_2, \dots, x_n \in \mathbb{N}^*,$$

deci

$$\begin{aligned}
 &x_1 + x_2 + \dots + x_{n-1} \leq m - 1, \quad x_1 + x_2 + \dots + x_{n-2} \leq m - 2, \quad \dots, \\
 &x_1 + x_2 + \dots + x_k \leq m - n + k, \quad \dots, \quad x_1 + x_2 \leq m - n + 2, \quad x_1 \leq m - n + 1.
 \end{aligned}$$

Astfel

$$\begin{aligned}
x_1 &\in S_1 = \{1, 2, \dots, m - n + 1\}, \\
x_2 &\in S_2 = \{1, 2, \dots, m - n + 2 - x_1\}, \\
x_3 &\in S_3 = \{1, 2, \dots, m - n + 3 - (x_1 + x_2)\}, \\
&\dots \\
x_k &\in S_k = \{1, 2, \dots, m - n + k - (x_1 + x_2 + \dots + x_{k-1})\}, \\
&\dots \\
x_{n-1} &\in S_{n-1} = \{1, 2, \dots, m - 1 - (x_1 + x_2 + \dots + x_{n-2})\},
\end{aligned}$$

iar

$$x_n = m - (x_1 + x_2 + \dots + x_{n-1}).$$

Așadar, din nou generăm termenii $(x_1, x_2, \dots, x_{n-1})$, cu $x_k \in S_k$, $k = \overline{1, n-1}$, iar ultimul termen se calculează cu formula $x_n = m - (x_1 + x_2 + \dots + x_{n-1})$.

Mulțimile S_k , $k = \overline{1, n-1}$ conțin termeni succesivi ai unei progresii aritmetice, în care

$$\begin{cases} a_k = 1, \\ r_k = 1, \\ b_k = m - n + k - (x_1 + x_2 + \dots + x_{k-1}), \end{cases} \quad \forall k = \overline{1, n-1}.$$

Prin alegerea mulțimilor $S_1, S_2, \dots, S_{n-1}$ am eliminat din nou verificarea condițiilor de continuare.

Vom utiliza din nou o variabilă s cu semnificația: dacă avem soluția parțială $(x_1, x_2, \dots, x_{k-1})$, atunci

$$s = x_1 + x_2 + \dots + x_{k-1}.$$

Pentru a nu discuta separat cazul $n = 1$, din nou preferăm *Schema Backtracking iterativă, varianta 2* (Algoritmul 4.1.2), și obținem următorul algoritm.

COMPUNERIL_CU_TERMENI_NENULI(m, n):

```

 $k \leftarrow 1$ ;
 $x[1] \leftarrow 0$ ;
 $s \leftarrow 0$ ;           // inițial, soluția parțială este vectorul vid
while  $k > 0$  do
    if  $k \leq n - 1$  then
        if  $x[k] < m - n + k - s$  then
             $x[k] \leftarrow x[k] + 1$ ;
             $s \leftarrow s + x[k]$ ;           // înainte de avansare, se adaugă
                                           //  $x[k]$  la soluția parțială
             $k \leftarrow k + 1$ ;
             $x[k] \leftarrow 0$ ;
        else
             $k \leftarrow k - 1$ ;
             $s \leftarrow s - x[k]$ ;           // după revenire, se elimină
                                           //  $x[k]$  din soluția parțială
    else
         $x[n] \leftarrow m - s$ ;
        AFISARE( $x, n$ );
         $k \leftarrow k - 1$ ;
         $s \leftarrow s - x[k]$ ;           // după revenire, se elimină
                                           //  $x[k]$  din soluția parțială

```

Funcția de afișare este aceeași ca în algoritmul anterior.

4.4.11 Partiții ale unui număr natural

Definiția 4.4.11. O *partiție (descompunere)* a numărului $m \in \mathbb{N}^*$ este o scriere de forma

$$m = t_1 + t_2 + \cdots + t_n,$$

unde $t_1, t_2, \dots, t_n \in \mathbb{N}^*$ ($n \in \mathbb{N}^*$) și nu contează ordinea dintre termenii $t_1, t_2, \dots, t_n$.

Observația 4.4.13. Deoarece într-o partiție ca mai sus nu contează ordinea dintre termenii, putem presupune că aceștia sunt scriși în ordine crescătoare.

Definiția 4.4.12. Fie $m, n \in \mathbb{N}^*$. Notăm cu $P(m, n)$ numărul de partiții ale lui m cu n termeni, iar cu $P(m)$ numărul tuturor partițiilor lui m .

Exemplul 4.4.10. Numărul $m = 6$ are partițiile

$$\begin{aligned} 6 &= 6 = 1 + 5 = 2 + 4 = 3 + 3 = 1 + 1 + 4 = 1 + 2 + 3 = 2 + 2 + 2 \\ &= 1 + 1 + 1 + 3 = 1 + 1 + 2 + 2 = 1 + 1 + 1 + 1 + 2 = 1 + 1 + 1 + 1 + 1 + 1, \end{aligned}$$

deci $P(6, 1) = 1$, $P(6, 2) = 3$, $P(6, 3) = 3$, $P(6, 4) = 2$, $P(6, 5) = 1$, $P(6, 6) = 1$ și $P(6) = 11$.

Observația 4.4.14. Avem $P(m) = P(m, 1) + P(m, 2) + \dots + P(m, m)$, $\forall m \in \mathbb{N}^*$.

Propoziția 4.4.10 (relația de recurență a numerelor $P(m, n)$). Pentru orice $m \in \mathbb{N}^*$ și orice $n \in \{1, \dots, m-1\}$ avem

$$P(m, n) = P(m-n, 1) + P(m-n, 2) + \dots + P(m-n, n).$$

Demonstrație. Pentru orice $m \in \mathbb{N}^*$ și orice $n \in \{1, \dots, m-1\}$ notăm

$$\mathcal{P}(m, n) = \{(t_1, t_2, \dots, t_n) | t_i \in \mathbb{N}^* \forall i, t_1 \leq t_2 \leq \dots \leq t_n, t_1 + \dots + t_n = m\}.$$

Definim corespondențele

$$\alpha : \mathcal{P}(m, n) \rightarrow \bigcup_{i=1}^n \mathcal{P}(m-n, i) \text{ și } \beta : \bigcup_{i=1}^n \mathcal{P}(m-n, i) \rightarrow \mathcal{P}(m, n)$$

prin:

- $\forall (t_1, \dots, t_n) \in \mathcal{P}(m, n)$, $\alpha(t_1, \dots, t_n) = (t_j - 1, \dots, t_n - 1)$, unde $j = \min\{i | t_i \geq 2, i \in \{1, \dots, n\}\}$ (există j , deoarece $n \leq m-1$);
- $\forall i \in \{1, \dots, n\}$, $\forall (t_1, \dots, t_i) \in \mathcal{P}(m-n, i)$,

$$\beta(t_1, \dots, t_i) = (\underbrace{1, \dots, 1}_{\text{de } n-i \text{ ori}}, t_1 + 1, \dots, t_i + 1).$$

Interpretarea acestor funcții este următoarea: aplicarea funcției α unei partiții a lui m cu n termeni constă în micșorarea cu 1 a fiecărui termen și eliminarea termenilor care astfel devin egali cu zero, obținându-se o partiție a lui $m-n$ cu cel mult n termeni. Reciproc, aplicarea funcției β unei partiții a lui $m-n$ cu cel mult n termeni constă în mărirea cu 1 a fiecărui termen și adăugarea de termeni egali cu 1 pentru a obține n termeni, astfel obținându-se o partiție a lui m cu n termeni.

Funcțiile α și β sunt bine definite și inverse una celeilalte, deci

$$\text{card}(\mathcal{P}(m, n)) = \text{card}\left(\bigcup_{i=1}^n \mathcal{P}(m-n, i)\right).$$

Cum mulțimile $\mathcal{P}(m-n, 1), \dots, \mathcal{P}(m-n, n)$ sunt evident disjuncte două câte două, rezultă că $P(m, n) = \sum_{i=1}^n P(m-n, i)$. $\square$

Observația 4.4.15. Relația de recurență din Propoziția 4.4.10, împreună cu condițiile inițiale evidente

$$P(m, 1) = P(m, m) = 1, P(m, n) = 0 \quad \forall n > m$$

permit calculul tuturor numerelor $P(m, n)$, deci și al numerelor $P(m)$, conform Corolarului 4.4.14. De exemplu, tabelul numerelor $P(m, n)$ și $P(m)$ pentru $m \leq 7$ (și $n \leq 7$) este:

$P(m, n)$	$n = 1$	$n = 2$	$n = 3$	$n = 4$	$n = 5$	$n = 6$	$n = 7$	$P(m)$
$m = 1$	1	0	0	0	0	0	0	1
$m = 2$	1	1	0	0	0	0	0	2
$m = 3$	1	1	1	0	0	0	0	3
$m = 4$	1	2	1	1	0	0	0	5
$m = 5$	1	2	2	1	1	0	0	7
$m = 6$	1	3	3	2	1	1	0	11
$m = 7$	1	3	4	3	2	1	1	15

Numererele $P(m, n)$ nenule, aflate doar pe diagonala principală și sub această diagonală (adică $1 \leq n \leq m$) formează **triunghiul numerelor** $P(m, n)$.

Algoritmul 4.4.14 (de generare a partițiilor lui m cu n termeni prin metoda Backtracking). Fie $n, m \in \mathbb{N}^*$. Vom utiliza metoda Backtracking. Fie soluția (partiție a lui m cu n termeni)

$$x = (x_1, x_2, \dots, x_n).$$

Condițiile interne:

$$x_1 + x_2 + \dots + x_n = m, \quad x_1, x_2, \dots, x_n \in \mathbb{N}^*, \quad x_1 \leq x_2 \leq \dots \leq x_n,$$

deci

$$\begin{aligned} x_1 + x_2 + \dots + x_{n-2} + 2x_{n-1} &\leq m, \quad x_1 + x_2 + \dots + x_{n-3} + 3x_{n-2} \leq m, \quad \dots, \\ x_1 + x_2 + \dots + x_{k-1} + (n-k+1)x_k &\leq m, \quad \dots, \quad x_1 + (n-1)x_2 \leq m, \quad nx_1 \leq m. \end{aligned}$$

Astfel

$$\begin{aligned}
 x_1 &\in S_1 = \left\{1, 2, \dots, \left\lfloor \frac{m}{n} \right\rfloor\right\}, \\
 x_2 &\in S_2 = \left\{x_1, x_1 + 1, \dots, \left\lfloor \frac{m - x_1}{n - 1} \right\rfloor\right\}, \\
 x_3 &\in S_3 = \left\{x_2, x_2 + 1, \dots, \left\lfloor \frac{m - (x_1 + x_2)}{n - 2} \right\rfloor\right\}, \\
 &\dots \\
 x_k &\in S_k = \left\{x_{k-1}, x_{k-1} + 1, \dots, \left\lfloor \frac{m - (x_1 + x_2 + \dots + x_{k-1})}{n - k + 1} \right\rfloor\right\}, \\
 &\dots \\
 x_{n-1} &\in S_{n-1} = \left\{x_{n-2}, x_{n-2} + 1, \dots, \left\lfloor \frac{m - (x_1 + x_2 + \dots + x_{n-2})}{2} \right\rfloor\right\},
 \end{aligned}$$

iar

$$x_n = m - (x_1 + x_2 + \dots + x_{n-1}).$$

Așadar, generăm termenii $(x_1, x_2, \dots, x_{n-1})$, cu $x_k \in S_k$, $k = \overline{1, n-1}$, iar ultimul termen se calculează cu formula $x_n = m - (x_1 + x_2 + \dots + x_{n-1})$.

Mulțimile S_k , $k = \overline{1, n-1}$ conțin termeni succesivi ai unei progresii aritmetice, în care

$$\begin{cases} a_k = x_{k-1}, \\ r_k = 1, \\ b_k = \left\lfloor \frac{m - (x_1 + x_2 + \dots + x_{k-1})}{n - k + 1} \right\rfloor, \end{cases} \quad \forall k = \overline{1, n-1},$$

considerând $x_0 = 1$.

Prin alegerea mulțimilor $S_1, S_2, \dots, S_{n-1}$ am eliminat verificarea condițiilor de continuare.

Vom utiliza o variabilă s cu semnificația: dacă avem soluția parțială $(x_1, x_2, \dots, x_{k-1})$, atunci

$$s = x_1 + x_2 + \dots + x_{k-1}.$$

Pentru a nu discuta separat cazul $n = 1$, din nou preferăm *Schema Backtracking iterativă, varianta 2* (Algoritmul 4.1.2), și obținem următorul algoritm.

```

PARTIȚII( $m, n$ ) :
 $k \leftarrow 1$ ;
 $x[1] \leftarrow 0$ ;
 $s \leftarrow 0$ ;           // inițial, soluția parțială este vectorul vid
while  $k > 0$  do
    if  $k \leq n - 1$  then
        if  $x[k] < \left\lfloor \frac{m - s}{n - k + 1} \right\rfloor$  then
             $x[k] \leftarrow x[k] + 1$ ;
             $s \leftarrow s + x[k]$ ;           // înainte de avansare, se adaugă
                                           //  $x[k]$  la soluția parțială
             $k \leftarrow k + 1$ ;
             $x[k] \leftarrow x[k - 1] - 1$ ;
        else
             $k \leftarrow k - 1$ ;
             $s \leftarrow s - x[k]$ ;           // după revenire, se elimină
                                           //  $x[k]$  din soluția parțială
    else
         $x[n] \leftarrow m - s$ ;
        AFISARE( $x, n$ );
         $k \leftarrow k - 1$ ;
         $s \leftarrow s - x[k]$ ;           // după revenire, se elimină
                                           //  $x[k]$  din soluția parțială

```

Funcția de afișare este:

```

AFISARE( $x, n$ ) :
for  $i = \overline{1, n}$  do
    afixează  $x[i]$ ;

```

Tema 5

Metoda Divide et Impera

5.1 Descrierea și analiza metodei. Algoritmi generali

Metoda **Divide et Impera** ("împarte și stăpânește") este o metodă generală de elaborare a algoritmilor care implică trei etape:

- **Divide:** Problema dată este împărțită în două sau mai multe subprobleme de același tip, dar de dimensiuni mai mici.
- **Impera:** Stăpânește subproblemele obținute. Subproblemele se rezolvă direct, dacă dimensiunea lor permite aceasta (cazuri elementare), sau, fiind de același tip, se rezolvă în mod recursiv, prin același procedeu.
- **Combină:** Se combină soluțiile tuturor subproblemelor, pentru a obține soluția problemei inițiale.

În continuare prezentăm o schemă generală de lucru.

Algoritmul 5.1.1 (Metoda Divide et Impera).

Fie $P(n)$ o problemă de dimensiune $n \in \mathbb{N}^*$. Presupunem că există $n_0 \in \mathbb{N}^*$, $a \in \mathbb{N}$, $a \geq 1$ și $b \in \mathbb{R}$, $b > 1$ astfel încât:

- $\forall n \in \mathbb{N}^*$ cu $n \leq n_0$, rezolvarea problemei $P(n)$ se poate face direct, fără descompunere.
- $\forall n \in \mathbb{N}^*$ cu $n > n_0$, $\exists m_1, \dots, m_a \in \mathbb{N}^*$ cu $m_i \leq \frac{n}{b}$, $\forall i = \overline{1, a}$, astfel încât rezolvarea problemei $P(n)$ se poate face rezolvând succesiv a subprobleme de același tip

$$P(m_1), \dots, P(m_a)$$

după care, combinând soluțiile celor a subprobleme, se obține soluția dorită a întregii probleme $P(n)$.

Algoritmul poate fi descris recursiv astfel:

```

DIVIMP( $n, S$ ) :           // Rezolvarea problemei  $P(n)$ 
if  $n \leq n_0$  then
    | PRELUCREAZA( $n, S$ );    // Rezolvarea se face direct,
    |                       // rezultând soluția  $S$  a problemei  $P(n)$ 
else
    | DESCOMPUNERE( $n, a, m$ ); // Se descompune problema,
    |                       // adică se determină  $m_1, \dots, m_a$ 
    | for  $i = \overline{1, a}$  do
    | | DIVIMP( $m_i, s_i$ );    // Se obține soluția  $s_i$  pentru
    | |                       // subproblema  $P(m_i)$ 
    | COMBINA( $s, a, S$ );    // Se combină soluțiile  $s_1, \dots, s_a$ ,
    |                       // rezultând soluția  $S$  a problemei  $P(n)$ 

```

Observația 5.1.1. Pentru orice $i = \overline{1, a}$ avem

$$m_i \leq \frac{n}{b} < n$$

(deoarece $b > 1$), adică subproblemele $P(m_1), \dots, P(m_a)$ au dimensiuni mai mici decât problema mare $P(n)$.

Definiția 5.1.1. Pentru orice număr real x , notăm cu

$$\lfloor x \rfloor = \max\{k \in \mathbb{Z} \mid k \leq x\}$$

partea întreagă inferioară (aproximarea întreagă prin lipsă sau, pe scurt, partea întreagă, notată și cu $\lfloor x \rfloor$) a lui x , iar cu

$$\lceil x \rceil = \min\{k \in \mathbb{Z} \mid k \geq x\}$$

partea întreagă superioară (aproximarea întreagă prin adaos) a lui x .

Exemplul 5.1.1. Avem $\lfloor 12,34 \rfloor = 12$, $\lceil 12,34 \rceil = 13$, $\lfloor -12,34 \rfloor = -13$, $\lceil -12,34 \rceil = -12$.

Observația 5.1.2. Pentru orice $x \in \mathbb{R}$ avem $\lceil x \rceil = -\lfloor -x \rfloor$.

Observația 5.1.3. În contextul Algoritmului 5.1.1, timpul de execuție $T(n)$ pentru rezolvarea problemei $P(n)$ verifică relația

$$T(n) = \begin{cases} t_n, & \text{dacă } n \leq n_0, \\ \sum_{i=1}^a T(m_i) + f(n), & \text{dacă } n > n_0, \end{cases} \quad (5.1.1)$$

unde:

- pentru $n \leq n_0$, t_n reprezintă timpul de execuție necesar rezolvării directe a problemei, adică timpul de execuție al procedurii **PRELUCREAZA**(n, S);
- pentru $n > n_0$, $f(n)$ reprezintă timpul total de execuție necesar descompunerii problemei în subprobleme și combinării soluțiilor acestor subprobleme, adică suma timpilor de execuție ai procedurilor **DESCOMPUNERE**(n, a, m) și **COMBINA**(s, a, S).

Adesea subproblemele $P(m_1), P(m_2), \dots, P(m_a)$, obținute la descompunerea problemei $P(n)$, au dimensiuni egale

$$m_1 = m_2 = \dots = m_a \in \left\{ \left\lfloor \frac{n}{b} \right\rfloor, \left\lceil \frac{n}{b} \right\rceil \right\}, \quad (5.1.2)$$

sau, mai general, aproape egale:

$$m_1, m_2, \dots, m_a \in \left\{ \left\lfloor \frac{n}{b} \right\rfloor, \left\lceil \frac{n}{b} \right\rceil \right\}, \quad (5.1.3)$$

În acest caz, conform (5.1.1) rezultă că timpul de execuție $T(n)$ pentru rezolvarea problemei $P(n)$ verifică relația de recurență

$$T(n) = a_1 T\left(\left\lfloor \frac{n}{b} \right\rfloor\right) + a_2 T\left(\left\lceil \frac{n}{b} \right\rceil\right) + f(n), \quad \forall n > n_0,$$

unde

$$a_1 = \text{card} \left\{ i = \overline{1, a} \mid m_i = \left\lfloor \frac{n}{b} \right\rfloor \right\}, \quad a_2 = \text{card} \left\{ i = \overline{1, a} \mid m_i = \left\lceil \frac{n}{b} \right\rceil \right\},$$

$$a_1 + a_2 = a.$$

Observația 5.1.4. Relația de recurență anterioară poate fi scrisă sub forma

$$T(n) = \underbrace{T(n/b) + \dots + T(n/b)}_{a \text{ termeni}} + f(n), \quad \forall n > n_0,$$

unde, prin convenție, fiecare termen n/b reprezintă fie $\left\lfloor \frac{n}{b} \right\rfloor$, fie $\left\lceil \frac{n}{b} \right\rceil$.

Mai mult, prin abuz de notația de înmulțire, această relație este convențional rescrisă prescurtat sub forma

$$T(n) = aT(n/b) + f(n), \quad \forall n > n_0.$$

Următorul rezultat celebru prezintă o metodă generală pentru rezolvarea acestei relații de recurență. El este deosebit de util în determinarea complexității algoritmilor specifici metodei de programare *Divide et Impera* și nu numai.

Teorema 5.1.1 (Teorema Master [10]). Fie $T : \mathbb{N} \rightarrow \mathbb{R}_+$ o funcție ce verifică relația de recurență

$$T(n) = aT(n/b) + f(n), \forall n > n_0 \quad (5.1.4)$$

din Observația 5.1.4 (cu convențiile de notație stabilite acolo), unde $n_0 \in \mathbb{N}$, $a \in \mathbb{N}$, $a \geq 1$, $b \in \mathbb{R}$, $b > 1$ și f este o funcție asimptotic pozitivă.

Cazul 1. Dacă $f(n) = \mathcal{O}(n^c)$, cu $c < \log_b a$, atunci

$$T(n) = \Theta(n^{\log_b a}).$$

Cazul 2. Dacă $f(n) = \Theta(n^c \log_2^k n)$, cu $c = \log_b a$ și $k \geq 0$, atunci

$$T(n) = \Theta(n^{\log_b a} \log_2^{k+1} n).$$

Cazul 3. Dacă $f(n) = \Omega(n^c)$, cu $c > \log_b a$, și există $k < 1$ și $n_1 \in \mathbb{N}$ astfel încât $af(n/b) \leq kf(n) \forall n \geq n_1$, atunci

$$T(n) = \Theta(f(n)).$$

În continuare prezentăm o schemă particulară de lucru.

Algoritmul 5.1.2 (Metoda Divide et Impera, variantă vectorială).

Fie $A = (a_1, a_2, \dots, a_n)$, $n \in \mathbb{N}^*$, un vector asupra componentelor căruia trebuie efectuată o prelucrare. Presupunem că există $\varepsilon \in \mathbb{N}$ astfel încât:

- $\forall p, q \in \{1, 2, \dots, n\}$ cu $q - p \leq \varepsilon$, prelucrarea secvenței $(a_p, \dots, a_q)$ se poate face direct, fără descompunere.
- $\forall p, q \in \{1, 2, \dots, n\}$ cu $q - p > \varepsilon$, $\exists m \in \{p, \dots, q - 1\}$ astfel încât prelucrarea secvenței

$$(a_p, \dots, a_q)$$

se poate face prelucrând succesiv subsecvențele

$$(a_p, \dots, a_m) \text{ și}$$

$$(a_{m+1}, \dots, a_q),$$

după care, combinând rezultatele celor două prelucrări, se obține prelucrarea dorită a întregii secvențe $(a_p, \dots, a_q)$.

Algoritmul poate fi descris recursiv astfel:

```

DIVIMP( $p, q, S$ ):           // Prelucrarea secvenței ( $a_p, \dots, a_q$ )
if  $q - p \leq \varepsilon$  then
    | PRELUCREAZA( $p, q, S$ ); // Prelucrarea se face direct,
    | // rezultând soluția  $S$  a secvenței ( $a_p, \dots, a_q$ )
else
    | DESCOMPUNERE( $p, q, m$ ); // Se descompune problema,
    | // adică se determină  $m$ 
    | DIVIMP( $p, m, S_1$ );      // Se obține soluția  $S_1$  pentru
    | // subsecvența ( $a_p, \dots, a_m$ )
    | DIVIMP( $m + 1, q, S_2$ ); // Se obține soluția  $S_2$  pentru
    | // subsecvența ( $a_{m+1}, \dots, a_q$ )
    | COMBINA( $S_1, S_2, S$ ); // Se combină soluțiile  $S_1$  și  $S_2$ ,
    | // rezultând soluția  $S$  a secvenței ( $a_p, \dots, a_q$ )

```

Procedura trebuie apelată prin:

DIVIMP(1, n , S).

În acest algoritm:

- ε reprezintă lungimea maximă a unei secvențe ($a_p, \dots, a_q$), notată prescurtat prin (p, q) , pentru care prelucrarea se poate face direct, fără a mai fi necesară împărțirea în subprobleme; procedura

PRELUCREAZA(p, q, S)

realizează prelucrarea secvențelor de acest tip, furnizând rezultatul în S .

- Procedura

COMBINA(S_1, S_2, S)

realizează combinarea rezultatelor S_1, S_2 ale prelucrării celor două subsecvențe vecine (p, m) și $(m + 1, q)$, obținându-se rezultatul S al prelucrării secvenței (p, q) .

- Valoarea m este obținută apelând procedura

DESCOMPUNERE(p, q, m).

5.2 O problemă de acoperire

Considerăm următoarea **problemă de acoperire**:

Se consideră o suprafață pătrată cu lungimea laturii $n = 2^k$, $k \geq 1$, divizată în pătrate unitare, ca o tablă de șah. Unul dintre aceste pătrate unitare este deja acoperit.

Se cere să se determine o acoperire a suprafeței rămase cu plăci de forma



constând din trei pătrate unitare.

Modelarea problemei

Memorăm suprafața pătrată într-o matrice $S = (s_{ij})_{i,j=\overline{1,n}}$, fiecare pătrat unitar fiind deci reprezentat de un element al acestei matrice.

Evident, o acoperire cu piese de forma dată utilizează un număr de

$$\frac{n^2 - 1}{3} = \frac{4^k - 1}{3}$$

piese.

Considerăm că piesele sunt numerotate cu $1, 2, \dots, \frac{n^2 - 1}{3}$ și că așezarea piesei numărul i constă în completarea elementelor corespunzătoare din matricea S cu numărul i al piesei, pentru fiecare valoare $i \in \left\{1, 2, \dots, \frac{n^2 - 1}{3}\right\}$.

Pentru rezolvarea problemei utilizăm următoarea strategie *Divide et Impera*:

- Se împarte suprafața pătrată în patru pătrate egale, formându-se patru suprafețe pătrate cu lungimea laturii 2^{k-1} .
- Una dintre cele patru suprafețe are deja un pătrat unitar acoperit.
- Acoperim cu o placă trei pătrate unitare din centrul suprafeței mari astfel încât să acoperim câte un pătrat unitar și din celelalte trei suprafețe. Un exemplu în acest sens este redat în Figura 5.2.1.

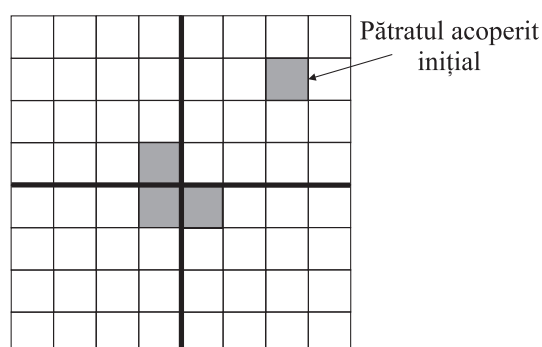


Figura 5.2.1:

- Fiecare dintre cele patru suprafețe este acum în condițiile problemei, deci pentru fiecare suprafață se repetă procedeul de descompunere descris anterior.

Obținem următorul *algoritm Divide et Impera*, descris în pseudocod, pentru rezolvarea problemei.

Algoritmul 5.2.1.

```

ACOPERIRE( $S, i, j, n, ip, jp$ ):
    //  $S$ =matricea ce va fi completată cu numerele plăcilor
    //  $(i, j)$ =colțul stânga sus
    //  $n$ =lungimea laturii
    //  $(ip, jp)$  = coordonatele pătratului unitar acoperit
if ( $n > 1$ ) then
    |  $p \leftarrow p + 1$ ; // numărul plăcii
    |  $m \leftarrow n/2$ ; // noua dimensiune a laturii
    | // Coordonatele stânga sus ale celor 4 pătrate:
    |  $i1 \leftarrow i; j1 \leftarrow j$ ;
    |  $i2 \leftarrow i; j2 \leftarrow j + m$ ;
    |  $i3 \leftarrow i + m; j3 \leftarrow j$ ;
    |  $i4 \leftarrow i + m; j4 \leftarrow j + m$ ;
    | // Coordonatele pătratelor unitare ce vor fi acoperite
    | // în cele 4 pătrate:
    |  $ip1 \leftarrow ip; jp1 \leftarrow jp$ ;
    |  $ip2 \leftarrow ip; jp2 \leftarrow jp$ ;
    |  $ip3 \leftarrow ip; jp3 \leftarrow jp$ ;
    |  $ip4 \leftarrow ip; jp4 \leftarrow jp$ ;
    | if ( $i3 \leq ip$ ) then
    | |  $ip1 \leftarrow i1 + m - 1; jp1 \leftarrow j1 + m - 1$ ;
    | |  $ip2 \leftarrow i2 + m - 1; jp2 \leftarrow j2$ ;
    | |  $S[ip1][jp1] \leftarrow p; S[ip2][jp2] \leftarrow p$ ;
    | else
    | |  $ip3 \leftarrow i3; jp3 \leftarrow j3 + m - 1$ ;
    | |  $ip4 \leftarrow i4; jp4 \leftarrow j4$ ;
    | |  $S[ip3][jp3] \leftarrow p; S[ip4][jp4] \leftarrow p$ ;
    | if ( $j2 \leq jp$ ) then
    | |  $ip1 \leftarrow i1 + m - 1; jp1 \leftarrow j1 + m - 1$ ;
    | |  $ip3 \leftarrow i3; jp3 \leftarrow j3 + m - 1$ ;
    | |  $S[ip1][jp1] \leftarrow p; S[ip3][jp3] \leftarrow p$ ;
    | else
    | |  $ip2 \leftarrow i2 + m - 1; jp2 \leftarrow j2$ ;
    | |  $ip4 \leftarrow i4; jp4 \leftarrow j4$ ;
    | |  $S[ip2][jp2] \leftarrow p; S[ip4][jp4] \leftarrow p$ ;
    | ACOPERIRE( $S, i1, j1, m, ip1, jp1$ );
    | ACOPERIRE( $S, i2, j2, m, ip2, jp2$ );
    | ACOPERIRE( $S, i3, j3, m, ip3, jp3$ );
    | ACOPERIRE( $S, i4, j4, m, ip4, jp4$ );

```

Apel: **ACOPERIRE**($S, 1, 1, n, ip, jp$).

Exemplul 5.2.1. Pentru exemplul din Figura 5.2.1, aplicarea algoritmului anterior produce acoperirea dată de matricea

$$S = \begin{pmatrix} 3 & 3 & 4 & 4 & 8 & 8 & 9 & 9 \\ 3 & 2 & 2 & 4 & 8 & 7 & 0 & 9 \\ 5 & 2 & 6 & 6 & 10 & 7 & 7 & 11 \\ 5 & 5 & 6 & 1 & 10 & 10 & 11 & 11 \\ 13 & 13 & 14 & 1 & 1 & 18 & 19 & 19 \\ 13 & 12 & 14 & 14 & 18 & 18 & 17 & 19 \\ 15 & 12 & 12 & 16 & 20 & 17 & 17 & 21 \\ 15 & 15 & 16 & 16 & 20 & 20 & 21 & 21 \end{pmatrix}.$$

Propoziția 5.2.1. Algoritmul 5.2.1 are complexitatea $\Theta(n^2)$.

Demonstrație. Fie $T(n)$ timpul de execuție al algoritmului. Vom aplica Teorema Master. Conform descrierii recursive a algoritmului, timpul de execuție $T(n)$ verifică relația de recurență

$$T(n) = 4T(n/2) + \mathcal{O}(1), \quad \forall n > 2.$$

Conform Cazului 1 al Teoremei Master, pentru

$$a = 4, \quad b = 2, \quad c = 0 < 2 = \log_b a,$$

rezultă că $T(n) = \Theta(n^2)$. □

Observația 5.2.1. Evident, orice algoritm care rezolvă problema dată necesită cel puțin $n^2 - 1$ instrucțiuni de atribuire, pentru completarea elementelor matricei S , deci conform propoziției anterioare rezultă că Algoritmul 5.2.1 este asimptotic-optimal.

5.3 Arbori binari stricți. Proprietăți

Definiția 5.3.1. Un **arbore binar** este un arbore cu rădăcină, reprezentat pe nivele, în care fiecare nod x are cel mult doi **descendenți** (**fii**, **succesori direcți** situați pe nivelul următor) și anume (cel mult) un **descendent stâng** (situat în stânga lui x) și (cel mult) un **descendent drept** (situat în dreapta lui x), contând ordinea dintre aceștia.

Nodurile cu cel puțin un descendent se numesc **interne** (**interioare**), iar nodurile fără descendenți se numesc **externe** (**exterioare**, **frunze**).

Exemplul 5.3.1. În Figura 5.3.1 este reprezentat un arbore binar având rădăcina 1. Nodurile 1, 2, 3, 4, 6 și 8 sunt interne, iar nodurile 5, 7 și 9 sunt externe.

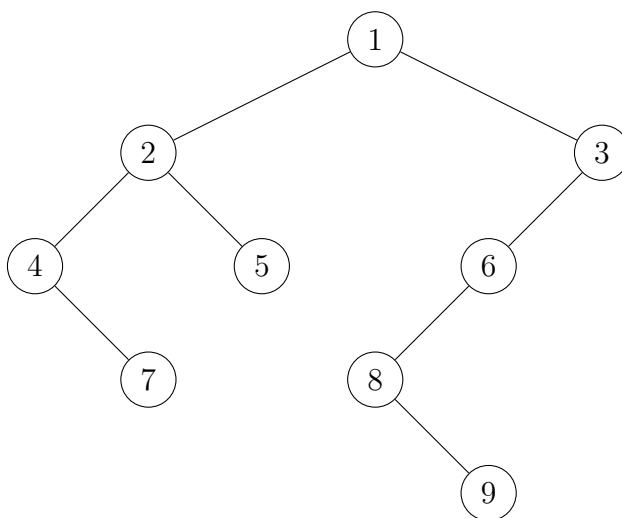


Figura 5.3.1:

Definiția 5.3.2. Fie $T = (V, F)$ un arbore binar având rădăcina r , $r \in V$.

- a) Notăm cu $I(T)$ **mulțimea nodurilor interne** și cu $E(T)$ **mulțimea nodurilor externe** ale arborelui binar T .
- b) Pentru orice nod $x \in V$, **distanța de la nodul rădăcină r la nodul x** , notată cu $D_T(x) = D(x)$, reprezintă lungimea lanțului elementar unic de la r la x .

- c) Numărul

$$h(T) = \max\{D_T(x) \mid x \in V\}$$

se numește **înălțimea arborelui binar T** .

Exemplul 5.3.2. Pentru arborele binar T din Exemplul 5.3.1 avem:

$$\begin{aligned} I(T) &= \{1, 2, 3, 4, 6, 8\}, \quad E(T) = \{5, 7, 9\}, \\ D(1) &= 0, \quad D(2) = D(3) = 1, \quad D(4) = D(5) = D(6) = 2, \\ D(7) &= D(8) = 3, \quad D(9) = 4, \end{aligned}$$

deci arborele are înălțimea $h(T) = 4$.

Observația 5.3.1. Fie $T = (V, F)$ un arbore binar. Evident, avem

$$V = I(T) \cup E(T), \quad I(T) \cap E(T) = \emptyset, \quad E(T) \neq \emptyset$$

și

$$h(T) = \max\{D_T(x) \mid x \in E(T)\}.$$

Deci

$$h(T) = k - 1,$$

unde k reprezintă numărul de nivele ale arborelui T .

Dacă nivelul pe care este reprezentată rădăcina arborelui este numerotat cu 0, atunci pentru orice nod $x \in V$ avem

$$D_T(x) = \text{numărul nivelului pe care este situat nodul } x,$$

iar înălțimea arborelui T este egală cu numărului ultimului nivel al arborelui.

Definiția 5.3.3. Un **arbore binar strict** este un arbore binar pentru care orice nod intern are exact doi descendenți (și anume un descendent stâng și un descendent drept).

Exemplul 5.3.3. Arborele binar din Exemplul 5.3.1 nu este strict, deoarece nodurile interne 4, 6 și 8 au câte un singur descendent.

Exemplul 5.3.4. În Figura 5.3.2 este reprezentat un arborele binar strict.

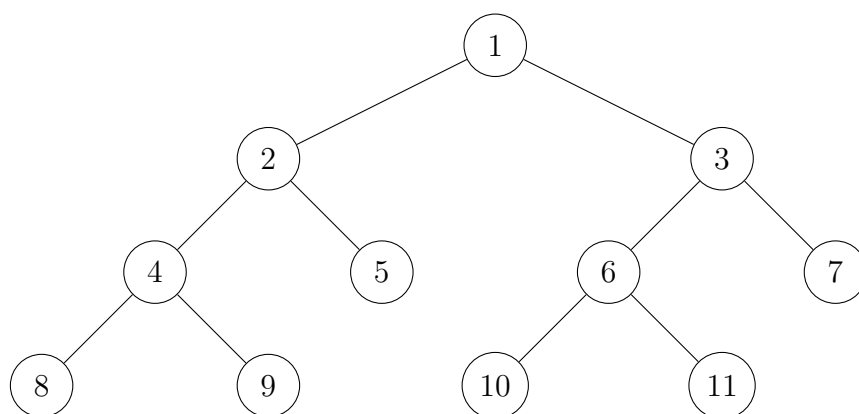


Figura 5.3.2:

Propoziția 5.3.1. Orice arbore binar strict cu n noduri interne, $n \in \mathbb{N}$, are $n + 1$ noduri externe.

Demonstrație. Demonstrăm afirmația din enunț prin inducție după n .

Pentru $n = 0$ este evident că arborele este format doar din nodul rădăcină, deci are un singur nod și acesta este extern.

Presupunem afirmația adevărată pentru $n - 1$ și o demonstrăm pentru n , $n \geq 1$. Fie $T = (V, F)$ un arbore binar strict cu n noduri interne. Fie x

un nod intern al lui T situat pe penultimul nivel, deci x are ca descendenți două noduri externe y și z , situate pe ultimul nivel. Fie

$$T' = T \setminus \{y, z\} = (V \setminus \{y, z\}, F \setminus \{[x, y], [x, z]\})$$

arborele obținut din T prin eliminarea nodurilor externe y și z . Evident, T' rămâne tot arbore binar strict și

$$I(T') = I(T) \setminus \{x\}, \quad E(T') = E(T) \setminus \{y, z\} \cup \{x\},$$

deci

$$\text{card}(I(T')) = \text{card}(I(T)) - 1 = n - 1, \quad \text{card}(E(T')) = \text{card}(E(T)) - 1.$$

Conform ipotezei de inducție, aplicată pentru T' , rezultă că

$$\text{card}(E(T')) = \text{card}(I(T')) + 1 = n,$$

deci

$$\text{card}(E(T)) = \text{card}(E(T')) + 1 = n + 1,$$

adică arborele T are $n + 1$ noduri externe. Demonstrația prin inducție este încheiată. $\square$

Exemplul 5.3.5. Arborele binar strict din Exemplul 5.3.4 are 5 noduri interne (1, 2, 3, 4 și 6) și 6 noduri externe (5, 7, 8, 9, 10 și 11).

Propoziția 5.3.2. Pentru orice arbore binar strict T cu n noduri interne avem

$$\sum_{v \in E(T)} D_T(v) = \sum_{v \in I(T)} D_T(v) + 2n.$$

Demonstrație. Demonstrăm egalitatea din enunț prin inducție după n .

Pentru $n = 0$ este evident că arborele T este format doar din nodul rădăcină r , $E(T) = \{r\}$, $I(T) = \emptyset$,

$$\sum_{v \in E(T)} D_T(v) = D_T(r) = 0, \quad \sum_{v \in I(T)} D_T(v) + 2n = 0 + 2 \cdot 0 = 0,$$

deci egalitatea din enunț este verificată.

Presupunem egalitatea adevărată pentru $n - 1$ și o demonstrăm pentru n , $n \geq 1$. Fie $T = (V, F)$ un arbore binar strict cu n noduri interne. Fie x un nod intern al lui T situat pe penultimul nivel, deci x are ca descendenți două noduri externe y și z , situate pe ultimul nivel. Evident,

$$D_T(y) = D_T(z) = 1 + D_T(x).$$

Fie

$$T' = T \setminus \{y, z\} = (V \setminus \{y, z\}, F \setminus \{[x, y], [x, z]\})$$

arborele obținut din T prin eliminarea nodurilor externe y și z . Evident, T' rămâne tot arbore binar strict și

$$I(T') = I(T) \setminus \{x\}, \quad E(T') = E(T) \setminus \{y, z\} \cup \{x\},$$

$$D_{T'}(v) = D_T(v), \quad \forall v \in I(T') \cup E(T').$$

Utilizând și ipoteza de inducție, aplicată pentru T' , avem

$$\begin{aligned} \sum_{v \in E(T)} D_T(v) &= \sum_{v \in E(T')} D_{T'}(v) - D_{T'}(x) + D_T(y) + D_T(z) \\ &= \sum_{v \in I(T')} D_{T'}(v) + 2(n-1) - D_T(x) + 2(1 + D_T(x)) \\ &= \sum_{v \in I(T) \setminus \{x\}} D_T(v) + D_T(x) + 2n \\ &= \sum_{v \in I(T)} D_T(v) + 2n. \end{aligned}$$

Demonstrația prin inducție este încheiată. $\square$

Exemplul 5.3.6. Pentru arborele binar strict T din Exemplul 5.3.4, având $n = 5$ noduri interne, avem

$$\begin{aligned} \sum_{v \in I(T)} D_T(v) &= D_T(1) + D_T(2) + D_T(3) + D_T(4) + D_T(6) \\ &= 0 + 1 + 1 + 2 + 2 = 6, \\ \sum_{v \in E(T)} D_T(v) &= D_T(5) + D_T(7) + D_T(8) + D_T(9) + D_T(10) + D_T(11) \\ &= 2 + 2 + 3 + 3 + 3 + 3 = 16, \end{aligned}$$

deci este verificată egalitatea $\sum_{v \in E(T)} D_T(v) = \sum_{v \in I(T)} D_T(v) + 2n$.

Propoziția 5.3.3. Pentru orice arbore binar strict T cu n noduri interne avem

$$h(T) \geq \lceil \log_2(n+1) \rceil.$$

Demonstrație. Considerăm că nivelul pe care este reprezentată rădăcina arborelui este numerotat cu 0. Arborele T este un arbore binar strict, cu $2n+1$

noduri (n interne și $n + 1$ externe), iar pe fiecare nivel $i \in \{0, 1, \dots, h(T)\}$ el are cel mult 2^i noduri, deci

$$\begin{aligned} 2n + 1 &\leq 1 + 2 + 2^2 + \dots + 2^{h(T)} \Leftrightarrow \\ 2n + 1 &\leq 2^{h(T)+1} - 1 \Leftrightarrow 2^{h(T)} \geq n + 1 \Leftrightarrow h(T) \geq \log_2(n + 1). \end{aligned}$$

Cum $h(T) \in \mathbb{N}$, deducem că $h(T) \geq \lceil \log_2(n + 1) \rceil$. □

Observația 5.3.2. Pentru orice $n \in \mathbb{N}^*$ are loc egalitatea

$$\lceil \log_2(n + 1) \rceil = 1 + \lfloor \log_2 n \rfloor.$$

Într-adevăr, notând $\lceil \log_2(n + 1) \rceil = k$, $k \in \mathbb{N}^*$, avem, succesiv,

$$\begin{aligned} k - 1 &< \log_2(n + 1) \leq k; \quad 2^{k-1} < n + 1 \leq 2^k; \\ 2^{k-1} &\leq n < 2^k; \quad k - 1 \leq \log_2 n < k, \end{aligned}$$

deci $\lfloor \log_2 n \rfloor = k - 1 = \lceil \log_2(n + 1) \rceil - 1$.

Propoziția 5.3.4. *Fie T un arbore binar strict cu n noduri interne. Dacă toate nodurile externe sunt situate pe ultimul și, eventual, pe penultimul nivel, atunci*

$$h(T) = \lceil \log_2(n + 1) \rceil$$

și

$$\sum_{v \in E(T)} D_T(v) = (n + 1) \lceil \log_2(n + 1) \rceil + n + 1 - 2^{\lceil \log_2(n + 1) \rceil}.$$

Demonstrație. Considerăm din nou că nivelul pe care este reprezentată rădăcina arborelui este numerotat cu 0. Rezultă că:

- arborele are $h(T) + 1$ nivele;
- pe nivelele $0, 1, \dots, h(T) - 2$ avem doar noduri interne, fiecare având câte doi descendenți;
- pe penultimul nivel, $h(T) - 1$, putem avea și noduri externe, iar nodurile interne de pe acest nivel au de asemenea câte doi descendenți;
- pe ultimul nivel, $h(T)$, avem doar noduri externe;
- pentru orice $i \in \{0, 1, \dots, h(T) - 1\}$ pe nivelul i avem exact 2^i noduri.

Notând cu a numărul de noduri interne de pe penultimul nivel și cu b numărul de noduri externe de pe același nivel, obținem că

$$a + b = 2^{h(T)-1} \quad (5.3.1)$$

(numărul total de noduri de pe penultimul nivel, nivelul $h(T) - 1$), iar

$$a \geq 1$$

(pe penultimul nivel există cel puțin un nod intern).

Pe nivelele $0, 1, \dots, h(T) - 2$ nu avem noduri externe, pe penultimul nivel, $h(T) - 1$, avem b noduri externe, iar pe ultimul nivel, $h(T)$, avem $2a$ noduri externe (și anume descendenții celor a noduri interne de pe penultimul nivel), deci numărul total de noduri externe ale arborelui este egal cu $b + 2a$. Rezultă că

$$b + 2a = n + 1, \quad (5.3.2)$$

și

$$\sum_{v \in E(T)} D_T(v) = b(h(T) - 1) + 2a \cdot h(T). \quad (5.3.3)$$

Din relațiile (5.3.1) și (5.3.2) obținem că

$$a = n + 1 - 2^{h(T)-1}. \quad (5.3.4)$$

Dar $1 \leq a \leq 2^{h(T)-1}$ (a doua inegalitate rezultă din (5.3.1)), deci

$$1 \leq n + 1 - 2^{h(T)-1} \leq 2^{h(T)-1}.$$

Rezultă că

$$2^{h(T)-1} \leq n < n + 1 \text{ și } 2^{h(T)} \geq n + 1,$$

deci

$$h(T) - 1 < \log_2(n + 1) \leq h(T)$$

și astfel $h(T) = \lceil \log_2(n + 1) \rceil$.

Conform (5.3.3), (5.3.2) și (5.3.4) obținem că

$$\begin{aligned} \sum_{v \in E(T)} D_T(v) &= (n + 1 - 2a)(h(T) - 1) + 2a \cdot h(T) \\ &= (n + 1)h(T) + 2a - n - 1 \\ &= (n + 1)h(T) + 2(n + 1 - 2^{h(T)-1}) - n - 1 \\ &= (n + 1)h(T) + n + 1 - 2^{h(T)} \\ &= (n + 1) \lceil \log_2(n + 1) \rceil + n + 1 - 2^{\lceil \log_2(n + 1) \rceil}. \end{aligned}$$

□

Exemplul 5.3.7. Arborele binar strict T din Exemplul 5.3.4, având $n = 5$ noduri interne, are toate nodurile externe situate pe ultimele două nivele. Avem $h(T) = 3$ și $\lceil \log_2(n+1) \rceil = \lceil \log_2 6 \rceil = 3$, deci este verificată egalitatea $h(T) = \lceil \log_2(n+1) \rceil$.

De asemenea, conform Exemplului 5.3.6 avem $\sum_{v \in E(T)} D_T(v) = 16$, iar $(n+1) \lceil \log_2(n+1) \rceil + n + 1 - 2^{\lceil \log_2(n+1) \rceil} = 6 \cdot 3 + 6 - 2^3 = 16$, deci este verificată și egalitatea $\sum_{v \in E(T)} D_T(v) = (n+1) \lceil \log_2(n+1) \rceil + n + 1 - 2^{\lceil \log_2(n+1) \rceil}$.

Propoziția 5.3.5. *Pentru orice arbore binar strict T cu n noduri interne avem*

$$\sum_{v \in E(T)} D_T(v) \geq (n+1) \lceil \log_2(n+1) \rceil + n + 1 - 2^{\lceil \log_2(n+1) \rceil}.$$

Mai mult, egalitatea are loc dacă și numai dacă toate nodurile externe sunt situate pe ultimul și, eventual, pe penultimul nivel.

Demonstrație. Notăm

$$\mathcal{M} = \{T \mid T = \text{arbore binar strict, } \text{card}(I(T)) = n\},$$

$$S(T) = \sum_{v \in E(T)} D_T(v), \forall T \in \mathcal{M}.$$

Fie $T^* \in \mathcal{M}$, $T = (V, F)$, a.î.

$$S(T^*) = \min\{S(T) \mid T \in \mathcal{M}\}.$$

Considerăm din nou că nivelul pe care este reprezentată rădăcina arborelui este numerotat cu 0.

Demonstrăm că arborele T^* nu are noduri externe pe nivelele $0, 1, \dots, h(T^*) - 2$ prin reducere la absurd. Să presupunem că arborele T^* ar avea un nod extern x pe un nivel i , cu $i \leq h(T^*) - 2$. Fie y un nod intern al lui T^* situat pe penultimul nivel, nivelul $h(T^*) - 1$, și fie y_1 și y_2 cei doi descendenți ai lui y , deci y_1 și y_2 sunt noduri externe situate pe ultimul nivel, nivelul $h(T^*)$. Evident,

$$D_{T^*}(x) = i, \quad D_{T^*}(y) = h(T^*) - 1, \quad D_{T^*}(y_1) = D_{T^*}(y_2) = h(T^*).$$

Fie T' arborele obținut din T^* prin mutarea nodurilor y_1 și y_2 de la descendenți ai nodului y la descendenți ai nodului x , adică

$$T' = (V, F \setminus \{[y, y_1], [y, y_2]\} \cup \{[x, y_1], [x, y_2]\}).$$

Evident, T' rămâne un arbore binar strict, $T' \in \mathcal{M}$,

$$\begin{aligned} E(T') &= E(T^*) \setminus \{x\} \cup \{y\}, \\ D_{T'}(y_1) &= D_{T'}(y_2) = i + 1, \\ D_{T'}(v) &= D_{T^*}(v), \forall v \in E(T') \setminus \{y_1, y_2\}, \end{aligned}$$

Avem

$$\begin{aligned} S(T') - S(T^*) &= \sum_{v \in E(T')} D_{T'}(v) - \sum_{v \in E(T^*)} D_{T^*}(v) \\ &= D_{T'}(y) + D_{T'}(y_1) + D_{T'}(y_2) - D_{T^*}(x) - D_{T^*}(y_1) - D_{T^*}(y_2) \\ &= h(T^*) - 1 + 2(i + 1) - i - 2h(T^*) \\ &= i + 1 - h(T^*) < 0, \end{aligned}$$

deci $S(T') < S(T^*)$, ceea ce contrazice alegerea lui T^* . Demonstrația prin reducere la absurd este încheiată.

Deoarece arborele T^* nu are noduri externe pe nivelele $0, 1, \dots, h(T^*) - 2$, rezultă că toate nodurile sale externe sunt situate pe ultimul și, eventual, pe penultimul nivel. Conform Propoziției 5.3.4 rezultă că

$$S(T^*) = (n + 1) \lceil \log_2(n + 1) \rceil + n + 1 - 2^{\lceil \log_2(n+1) \rceil},$$

deci

$$\min\{S(T) \mid T \in \mathcal{M}\} = (n + 1) \lceil \log_2(n + 1) \rceil + n + 1 - 2^{\lceil \log_2(n+1) \rceil},$$

de unde rezultă inegalitatea din enunț. Mai mult, dacă toate nodurile externe sunt situate pe ultimul și, eventual, pe penultimul nivel atunci, conform Propoziției 5.3.4, inegalitatea din enunț devine egalitate, iar în caz contrar, conform demonstrației prin reducere la absurd de mai sus, inegalitatea este strictă. $\square$

Exemplul 5.3.8. Conform Propozițiilor 5.3.3 și 5.3.5 și calculelor din Exemplul 5.3.7 rezultă că orice arborele binar strict cu 5 noduri interne (și, deci, 6 noduri externe) are înălțimea mai mare sau egală cu 3 și suma distanțelor de la rădăcină la nodurile externe mai mare sau egală cu 16.

5.4 Algoritmi de căutare

5.4.1 Problema căutării

Problema căutării este următoarea:

Se consideră un vector $A = (a_1, a_2, \dots, a_n)$, $n \geq 1$, și o valoare x . Se cere să se determine dacă x se află printre componentele vectorului A .

O căutare se poate încheia:

- fie **cu succes**, caz în care se returnează o poziție în vector pe care se află valoarea căutată;
- fie **fără succes**, caz în care se returnează -1 .

Observația 5.4.1. În practică, problema constă în căutarea unei înregistrări într-un fișier sau într-o bază de date după valoarea unui câmp, numit **cheie de căutare**.

5.4.2 Căutarea secvențială

Algoritmul 5.4.1 (Căutarea secvențială).

- se compară succesiv x cu elementele $a_1, a_2, \dots, a_n$;
- dacă $x = a_i$, atunci căutarea se termină cu succes;
- dacă $x \neq a_i$, pentru orice $i = \overline{1, n}$, atunci căutarea se termină fără succes.

CAUTSEC(A, n, x):

for $i = \overline{1, n}$ **do**

if $x = A[i]$ **then**
 returnează i ;

returnează -1 ;

Procedura trebuie apelată prin:

$$poz \leftarrow \text{CAUTSEC}(A, n, x).$$

Teorema 5.4.1 (complexitatea algoritmului de căutare secvențială).

Algoritmul CAUTSEC are complexitatea $\mathcal{O}(n)$.

Demonstrație. Considerăm căutarea secvențială a valorii x în vectorul $A = (a_1, a_2, \dots, a_n)$.

- La o căutare cu succes, $x = a_i$, numărul de comparații de chei este egal cu i , pentru orice $i \in \{1, 2, \dots, n\}$.
- La o căutare fără succes, numărul de comparații de chei este egal cu n .

Astfel:

- numărul de comparații de chei efectuate în cazul cel mai favorabil este egal cu 1;
- numărul de comparații de chei efectuate în cazul cel mai defavorabil este egal cu n .

Rezultă că algoritmul are complexitatea $\mathcal{O}(n)$. $\square$

Propoziția 5.4.1. *Algoritmul CAUTSEC este optim (în clasa algoritmilor bazați pe comparații de chei).*

Demonstrație. Evident, orice algoritm de căutare a valorii x în vectorul $A = (a_1, a_2, \dots, a_n)$, bazat pe comparații de chei, necesită cel puțin câte o comparație pentru fiecare element a_i al vectorului, $i \in \{1, 2, \dots, n\}$. Algoritmul CAUTSEC efectuează exact câte o comparație pentru fiecare element a_i , deci este optim. $\square$

5.4.3 Căutarea binară

Algoritmul 5.4.2 (Căutarea binară).

Presupunem că vectorul A are elementele în ordine crescătoare

$$a_1 \leq a_2 \leq \dots \leq a_n.$$

Vom utiliza metoda *Divide et Impera* astfel:

- se compară x cu elementul median (din "mijloc") a_m , unde

$$m = \left\lfloor \frac{1+n}{2} \right\rfloor;$$

- dacă $x = a_m$, atunci căutarea se termină cu succes;
- dacă $x < a_m$, atunci se va căuta x în subsecvența

$$(a_1, \dots, a_{m-1});$$

- dacă $x > a_m$, atunci se va căuta x în subsecvența

$$(a_{m+1}, \dots, a_n).$$

Varianta recursivă de implementare a algoritmului de căutare binară este descrisă în pseudocod astfel:

```

CAUTBIN_REC( $A, n, p, u, x$ ):
    // căutarea valorii  $x$  în subsecvența  $(a_p, \dots, a_u)$  a
    // vectorului  $(a_1, \dots, a_n)$ 

    if  $p > u$  then
        | returnează  $-1$ ;
    else
        |  $m \leftarrow \left\lfloor \frac{p+u}{2} \right\rfloor$ ;
        | if  $x = A[m]$  then
        | | returnează  $m$ ;
        | else
        | | if  $x < A[m]$  then
        | | | returnează CAUTBIN_REC( $A, n, p, m-1, x$ );
        | | else
        | | | returnează CAUTBIN_REC( $A, n, m+1, u, x$ );

```

Procedura trebuie apelată prin:

$$poz \leftarrow \text{CAUTBIN_REC}(A, n, 1, n, x).$$

Varianta iterativă de implementare a algoritmului de căutare binară este descrisă în pseudocod astfel:

```

CAUTBIN( $A, n, x$ ):
     $p \leftarrow 1$ ;  $u \leftarrow n$ ;    //  $(a_p, \dots, a_u)$  reprezintă subsecvența curentă
    // la care se restrânge căutarea

    while  $p \leq u$  do
        |  $m \leftarrow \left\lfloor \frac{p+u}{2} \right\rfloor$ ;
        | if  $x = A[m]$  then
        | | returnează  $m$ ;
        | else
        | | if  $x < A[m]$  then
        | | |  $u \leftarrow m-1$ ;
        | | else
        | | |  $p \leftarrow m+1$ ;
    returnează  $-1$ ;

```

Procedura trebuie apelată prin:

$$poz \leftarrow \text{CAUTBIN}(A, n, x).$$

Observația 5.4.2. Pentru evaluarea complexității algoritmilor care rezolvă problema considerată, vom analiza numai comparațiile în care intervin valoarea căutată x și un element al vectorului A , numite *comparații de chei*. Astfel considerăm că aceste comparații se efectuează succesiv (comparațiile simultane pot fi separate).

Prezentăm câteva noțiuni ajutătoare.

Definiția 5.4.1. *Un arbore binar de căutare (BST - Binary Search Tree) este un arbore binar cu proprietatea că orice nod intern are valoarea (eticheta) mai mare decât orice nod din subarborele său stâng și mai mică decât orice nod din subarborele său drept.*

Observația 5.4.3. Fie $\mathcal{A}$ un algoritm de căutare, bazat pe comparații de chei, ce rezolvă problema considerată. Presupunem că algoritmul nu conține instrucțiuni redundante și este aplicabil pentru orice poziționare posibilă a valorii x în raport cu componentele vectorului A , deci valoarea x este comparată exact câte o dată cu fiecare componentă a_i , $i = \overline{1, n}$.

Atunci algoritmului $\mathcal{A}$ i se poate asocia un arbore binar de căutare $\mathcal{T}(\mathcal{A})$ având n noduri, etichetate cu numerele $1, 2, \dots, n$, construit recursiv astfel: dacă prima comparație efectuată de algoritm este între valoarea căutată x și un element a_i al vectorului, atunci:

- rădăcina arborelui este etichetată cu indicele i ;
- subarborele stâng corespunde continuării algoritmului în cazul $x < a_i$, dacă acest caz este posibil după prima comparație (altfel este vid, adică nu există descendent stâng);
- subarborele drept corespunde continuării algoritmului în cazul $x > a_i$, dacă acest caz este posibil după prima comparație (altfel este vid, adică nu există descendent drept).

Reciproc, oricărui arbore binar de căutare $\mathcal{T}$ având n noduri, etichetate cu numerele $1, 2, \dots, n$, îi corespunde un algoritm de căutare $\mathcal{A}(\mathcal{T})$ a valorii x în vectorul sortat $A = (a_1, a_2, \dots, a_n)$, descris recursiv astfel:

```

 $\mathcal{A}(\mathcal{T})$ :
if [ $\mathcal{T}$  este vid] then
    | return -1;                                // căutare fără succes
else
    |  $i \leftarrow$  eticheta rădăcinii lui  $\mathcal{T}$ 
    | if  $x = a_i$  then
    | | return  $i$ ;                                // căutare cu succes
    | else
    | | if  $x < a_i$  then
    | | |  $\mathcal{T}_s \leftarrow$  subarborele stâng al lui  $\mathcal{T}$ 
    | | | return  $\mathcal{A}(\mathcal{T}_s)$ ;
    | | else
    | | |  $\mathcal{T}_d \leftarrow$  subarborele drept al lui  $\mathcal{T}$ 
    | | | return  $\mathcal{A}(\mathcal{T}_d)$ ;

```

Corespondențele $\mathcal{A} \rightarrow \mathcal{T}(\mathcal{A})$ și $\mathcal{T} \rightarrow \mathcal{A}(\mathcal{T})$ de mai sus sunt bine definite și inverse una celeilalte, deci sunt bijecții între mulțimea algoritmilor de căutare (bazați pe comparații de chei) a unei valori x într-un vector sortat cu n componente $(a_1, a_2, \dots, a_n)$ și mulțimea arborilor binari de căutare având n noduri etichetate cu numerele $1, 2, \dots, n$.

Definiția 5.4.2. *Arbore binar de căutare $\mathcal{T}(\mathcal{A})$ construit în observația anterioară se numește și **arborele de decizie al algoritmului de căutare $\mathcal{A}$** .*

Exemplul 5.4.1. Arborele binar de căutare asociat algoritmului căutării binare pentru $n = 8$ este reprezentat în Figura 5.4.1.

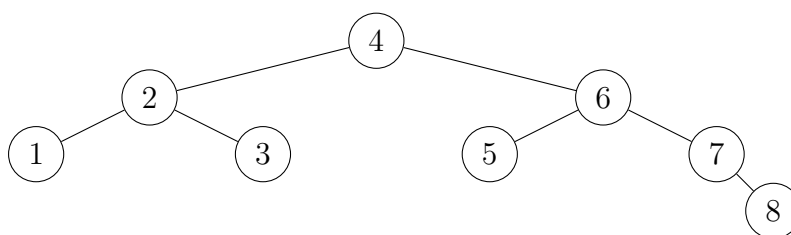


Figura 5.4.1:

Observația 5.4.4. Arborele de căutare asociat algoritmului de căutare binară este un arbore binar, nu neapărat strict.

Exemplul 5.4.2. Arborele binar de căutare asociat algoritmului căutării secvențiale pentru $n = 8$ este reprezentat în Figura 5.4.2.

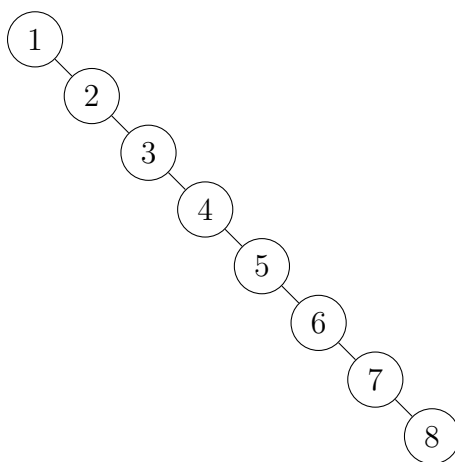


Figura 5.4.2:

Definiția 5.4.3. Fie $\mathcal{T}$ un arbore binar de căutare având n noduri, etichetate cu numerele $1, 2, \dots, n$. Extindem arborele binar $\mathcal{T}$ la un arbore binar strict $\overline{\mathcal{T}}$ astfel: pentru orice $k \in \{1, 2, \dots, n\}$, nodul lui $\mathcal{T}$ având eticheta k este reprezentat într-un cerc și devine nod intern în $\overline{\mathcal{T}}$ prin aplicarea următoarelor două reguli:

- dacă nodul nu are descendent stâng, atunci i se adaugă un descendent stâng etichetat cu $k - 1$ și reprezentat într-un pătrat;
- dacă nodul nu are descendent drept, atunci i se adaugă un descendent drept etichetat cu k și reprezentat într-un pătrat.

Arborele $\overline{\mathcal{T}}$ se numește **arbore binar extins de căutare**.

Exemplul 5.4.3. Arborele binar extins de căutare asociat algoritmului căutării binare pentru $n = 8$ este reprezentat în Figura 5.4.3.

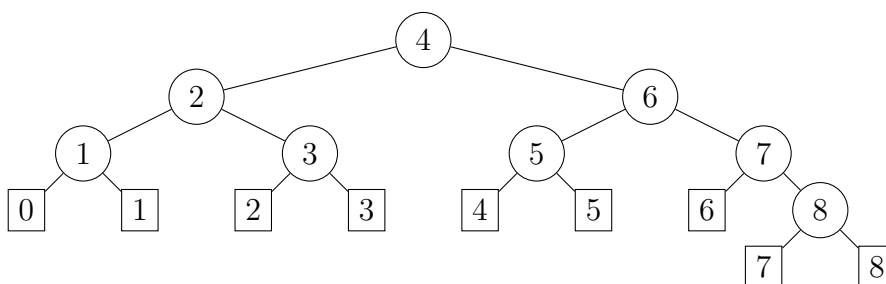


Figura 5.4.3:

Exemplul 5.4.4. Arborele binar extins de căutare asociat algoritmului căutării secvențiale pentru $n = 8$ este reprezentat în Figura 5.4.4.

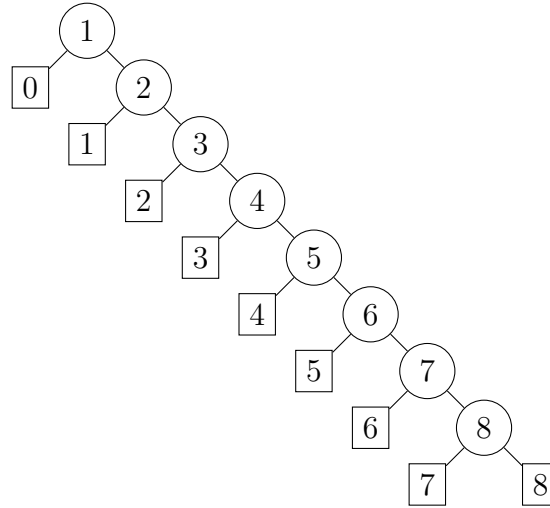


Figura 5.4.4:

Observația 5.4.5. Pentru orice algoritm de căutare $\mathcal{A}$ a valorii x în vectorul sortat $A = (a_1, a_2, \dots, a_n)$, arborele binar extins de căutare asociat $\overline{\mathcal{T}} = \overline{\mathcal{T}}(\mathcal{A})$ este un arbore binar strict ce are n noduri interne, etichetate cu numerele $1, 2, \dots, n$ reprezentate în cercuri și $n+1$ noduri externe, etichetate cu numerele $0, 1, 2, \dots, n$ reprezentate în pătrate, având semnificația următoare:

- pentru orice $i \in \{1, 2, \dots, n\}$, nodul intern etichetat cu i reprezintă comparația valorii căutate, x , cu elementul a_i din vectorul sortat;
- pentru orice $i \in \{0, 1, 2, \dots, n\}$, nodul extern etichetat cu i reprezintă încheierea fără succes a căutării, corespunzătoare situației:

$$\begin{aligned} x &< a_1, & \text{dacă } i &= 0, \\ a_i &< x < a_{i+1}, & \text{dacă } 1 \leq i \leq n-1, \\ x &> a_n, & \text{dacă } i &= n. \end{aligned}$$

Astfel, la o căutare fără succes reprezentată de nodul extern având eticheta i , dacă s-ar insera valoarea căutată x pe poziția $i+1$ a vectorului A (deplasând spre dreapta elementele $a_{i+1}, a_{i+2}, \dots, a_n$), vectorul obținut ar rămâne sortat crescător.

De asemenea, avem:

- la o căutare cu succes, $x = a_i$, numărul de comparații de chei este egal cu numărul de noduri (interne) situate pe lanțul elementar dintre nodul rădăcină și nodul intern i , adică este mai mare cu 1 decât lungimea acestui lanț;
- la o căutare fără succes numărul de comparații de chei este egal cu numărul de noduri interne situate pe lanțul elementar dintre nodul rădăcină și nodul extern corespunzător încheierii fără succes a căutării, adică cu lungimea acestui lanț.

Exemplul 5.4.5. Pentru algoritmul de căutare binară, $n = 8$ și vectorul sortat

$$A = (5, 6, 10, 12, 17, 20, 25, 32),$$

căutarea valorii $x = 17$ necesită 3 comparații de chei, puse în evidență de lanțul care unește rădăcina, etichetată cu 4, cu nodul intern etichetat cu 5 din arborele binar extins de căutare asociat, reprezentat în Figura 5.4.5, iar căutarea valorii $x = 27$ necesită 4 comparații de chei, puse în evidență de lanțul care unește rădăcina, etichetată cu 4, cu nodul extern etichetat cu 7 din arborele binar extins de căutare asociat, reprezentat în Figura 5.4.6.

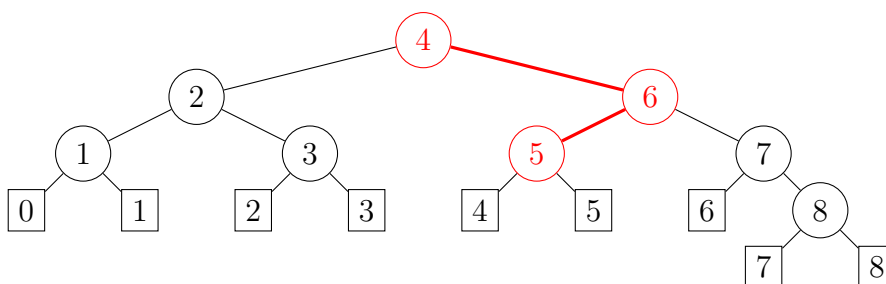


Figura 5.4.5:

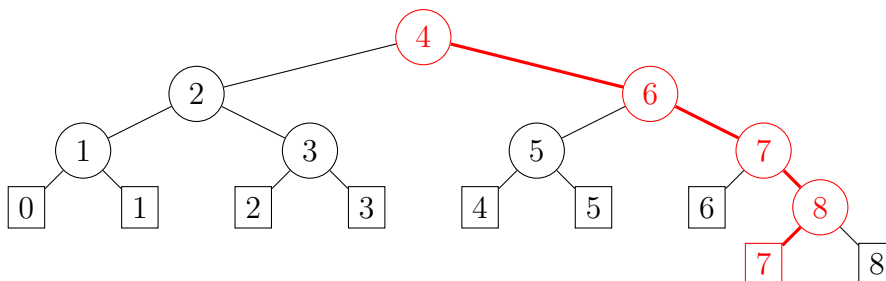


Figura 5.4.6:

Exemplul 5.4.6. Pentru algoritmul de căutare secvențială, $n = 8$ și vectorul sortat

$$A = (5, 6, 10, 12, 17, 20, 25, 32),$$

căutarea valorii $x = 17$ necesită 5 comparații de chei, puse în evidență de lanțul care unește rădăcina, etichetată cu 1, cu nodul intern etichetat cu 5 din arborele binar extins de căutare asociat, reprezentat în Figura 5.4.7, iar căutarea valorii $x = 27$ necesită 8 comparații de chei, puse în evidență de lanțul care unește rădăcina, etichetată cu 1, cu nodul extern etichetat cu 7 din arborele binar extins de căutare asociat, reprezentat în Figura 5.4.8.

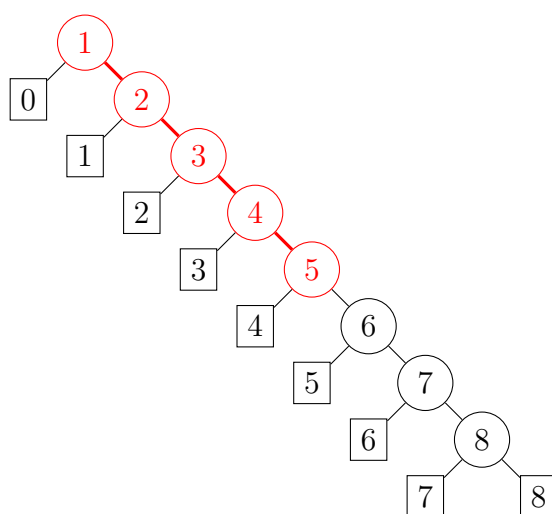


Figura 5.4.7:

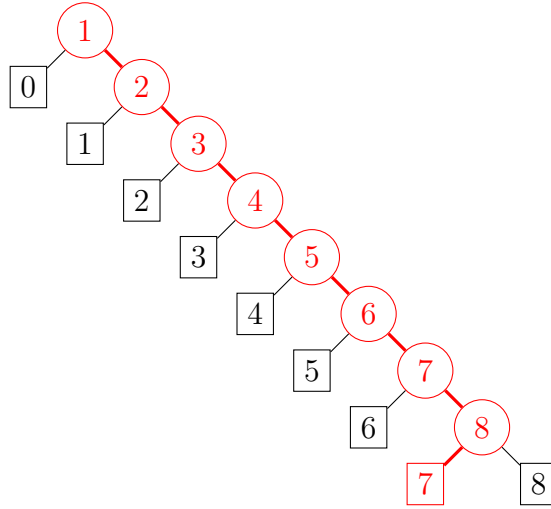


Figura 5.4.8:

Teorema 5.4.2 (complexitatea algoritmilor de căutare). Pentru orice algoritm de căutare $\mathcal{A}$, bazat pe comparații de chei, a valorii x în vectorul sortat $A = (a_1, a_2, \dots, a_n)$ avem:

- 1) numărul de comparații de chei efectuate în cazul cel mai defavorabil este mai mare sau egal cu $\lceil \log_2(n+1) \rceil$;
- 2) numărul mediu de comparații de chei efectuate este mai mare sau egal cu

$$\frac{2n+2}{2n+1} \cdot \lceil \log_2(n+1) \rceil + \frac{n+2 - 2 \cdot 2^{\lceil \log_2(n+1) \rceil}}{2n+1}.$$

Demonstrație. 1) Analizăm cazul cel mai defavorabil. Conform Observației 5.4.5, numărul de comparații de chei efectuate în cazul cel mai defavorabil este egal cu înălțimea h a arborelui binar extins de căutare asociat $\overline{\mathcal{T}} = \overline{\mathcal{T}}(\mathcal{A})$. Arborele $\overline{\mathcal{T}}$ este un arbore binar strict, cu n noduri interne (și $n+1$ noduri externe), deci conform Propoziției 5.3.3 rezultă că $h \geq \lceil \log_2(n+1) \rceil$.

2) Analizăm acum cazul mediu (cazul timpului mediu de execuție). Conform Observației 5.4.5, numărul mediu de comparații de chei are valoarea

$$N_{med}(n) = \frac{\sum_{k \in I} (1 + D(k)) + \sum_{k \in E} D(k)}{2n+1} = \frac{n + \sum_{k \in I} D(k) + \sum_{k \in E} D(k)}{2n+1},$$

unde I este mulțimea nodurilor interne, E este mulțimea nodurilor externe, iar, pentru fiecare nod k , $D(k)$ reprezintă distanța de la rădăcina arborelui

$\overline{\mathcal{T}} = \overline{\mathcal{T}}(\mathcal{A})$ la nodul k . Conform Propozițiilor 5.3.2 și 5.3.5 rezultă că

$$N_{med}(n) = \frac{2 \sum_{k \in E} D(k) - n}{2n + 1} \geq \frac{2((n + 1) \lceil \log_2(n + 1) \rceil + n + 1 - 2^{\lceil \log_2(n + 1) \rceil}) - n}{2n + 1},$$

adica inegalitatea din enunț. $\square$

Propoziția 5.4.2. *Pentru algoritmul de căutare binară într-un vector sortat cu n componente, o căutare cu succes necesită cel mult h comparații de chei, iar o căutare fără succes necesită $h - 1$ sau h comparații de chei, unde $h = \lceil \log_2(n + 1) \rceil = \lfloor \log_2 n \rfloor + 1$.*

Demonstrație. Fie $\mathcal{T}(n)$ arbore binar de căutare asociat algoritmului de căutare binară într-un vector sortat cu n componente. Conform descrierii algoritmului de căutare binară și a construirii arborelui binar de căutare asociat, rezultă că subarboarele stâng al lui $\mathcal{T}(n)$ corespunde căutării binare în subvectorul $(a_1, \dots, a_{m-1})$, unde $m = \lfloor \frac{n+1}{2} \rfloor$, deci este chiar $\mathcal{T}(\lfloor \frac{n-1}{2} \rfloor)$, iar subarboarele drept corespunde căutării binare în subvectorul $(a_{m+1}, \dots, a_n)$ de lungime $n - m = n - \lfloor \frac{n+1}{2} \rfloor = \lceil \frac{n-1}{2} \rceil$, deci are aceeași structură cu $\mathcal{T}(\lceil \frac{n-1}{2} \rceil)$ (doi arbori binari au aceeași structură dacă sunt identici, cu excepția etichetelor, adică prin suprapunerea rădăcinilor ei se suprapun perfect). Aplicând metoda inducției matematice, rezultă că arborele binar $\mathcal{T}(n+1)$ are aceeași structură cu arborele binar $\mathcal{T}(n)$ plus un nod extern, situat pe ultimul nivel (la pasul inductiv se analizează separat cazurile n par, respectiv impar și se aplică ipoteza de inducție pentru cei doi subarbori). Cum $\lceil \frac{n-1}{2} \rceil = \lfloor \frac{n-1}{2} \rfloor$ pentru n impar, respectiv $\lceil \frac{n-1}{2} \rceil = \lfloor \frac{n-1}{2} \rfloor + 1$ pentru n par, rezultă că subarboarele drept al lui $\mathcal{T}(n)$, având aceeași structură cu $\mathcal{T}(\lceil \frac{n-1}{2} \rceil)$, are aceeași structură cu subarboarele stâng $\mathcal{T}(\lfloor \frac{n-1}{2} \rfloor)$ plus un eventual nod extern (când n este par). Aplicând din nou metoda inducției matematice, rezultă că în arborele binar $\mathcal{T}(n)$ toate nodurile care nu au doi descendenți (fii) sunt situate pe ultimul și, eventual, pe penultimul nivel (la pasul inductiv se folosește relația de structură menționată dintre cei doi subarbori și se aplică ipoteza de inducție pentru subarboarele drept).

Rezultă că arborele binar extins de căutare asociat $\overline{\mathcal{T}}(n)$ are nodurile externe situate pe cel mult două nivele. Notând cu h înălțimea arborelui extins (adică lungimea maximă de la rădăcină la un nod extern) și considerând că rădăcina arborelui este reprezentată pe nivelul 0, rezultă că: arborele extins are $h + 1$ nivele, n noduri interne și $n + 1$ noduri externe; pe nivelele $0, 1, \dots, h - 2$ avem doar noduri interne, fiecare având câte doi descendenți; pe penultimul nivel, $h - 1$, putem avea și noduri externe, iar nodurile interne de pe acest nivel au de asemenea câte doi descendenți; pe ultimul nivel, h , avem doar noduri externe.

Conform Propoziției 5.3.4 și Observației 5.3.2 rezultă că $h = \lceil \log_2(n + 1) \rceil = \lfloor \log_2 n \rfloor + 1$. Conform Observației 5.4.5 rezultă că o căutare cu succes necesită cel mult h comparații de chei, iar o căutare fără succes necesită $h - 1$ sau h comparații de chei. $\square$

Observația 5.4.6. Conform propoziției anterioare, algoritmul de căutare binară are timpul de execuție în cazul cel mai defavorabil de ordinul $\Theta(\log_2 n)$ (celelalte operații nu depășesc ordinul de creștere al comparațiilor de chei), deci timpul (mediu) de execuție are ordinul $\mathcal{O}(\log_2 n)$. Teorema următoare îmbunătățește acest rezultat.

Teorema 5.4.3 (complexitatea algoritmului de căutare binară). *Fie $T(n)$ timpul mediu de execuție al algoritmului de căutare binară într-un vector sortat cu n componente. Avem $T(n) = \Theta(\log_2 n)$.*

Demonstrație. Considerăm că timpul mediu de execuție $T(n)$ al algoritmului de căutare binară este dat de numărul de comparații de chei efectuate (celelalte operații nu depășesc ordinul de creștere al acestora). Vom demonstra că $T(n) = \Theta(\log_2 n)$ prin două metode.

Metoda 1) Vom aplica Teorema Master. Conform descrierii recursive a algoritmului de căutare binară, timpul de execuție $T(n)$ verifică relația de recurență

$$T(n) = T(n/2) + \mathcal{O}(1), \quad \forall n > 0,$$

cu convențiile de notație din Teorema Master. Conform Cazului 2 al acelei teoreme, pentru $a = 1$, $b = 2$, $c = 0 = \log_b a$ și $k = 0$, rezultă că $T(n) = \Theta(\log_2 n)$.

Observația 5.4.7. Ca o consecință imediată a Metodei 1, Propoziției 5.4.2 și Teoremei 5.4.2 rezultă că algoritmul de căutare binară este asimptotic optim (în clasa algoritmilor bazați pe comparații de chei, atât în raport cu timpul de execuție în cazul cel mai defavorabil, cât și în raport cu timpul mediu de execuție). Pentru a justifica faptul că algoritmul este chiar optim, nu este suficientă evaluarea complexității cu Teorema Master. Aici intervine metoda următoare, prin care se numără comparațiile efectuate de algoritm.

Metoda 2) Vom calcula efectiv timpul mediu de execuție $T(n)$. Conform demonstrației Propoziției 5.4.2, arborele binar extins de căutare asociat algoritmului căutării binare este un arbore binar strict cu n noduri interne și $n + 1$ noduri externe, iar toate nodurile externe sunt situate pe ultimul și, eventual, pe penultimul nivel. Notăm cu I mulțimea nodurilor interne, cu E mulțimea nodurilor externe, iar pentru fiecare nod k notăm cu $D(k)$ distanța de la rădăcina arborelui la nodul k . Conform Observației 5.4.5, timpul mediu

de execuție are valoarea

$$T(n) = \frac{\sum_{k \in I} (1 + D(k)) + \sum_{k \in E} D(k)}{2n + 1} = \frac{n + \sum_{k \in I} D(k) + \sum_{k \in E} D(k)}{2n + 1}.$$

Conform Propozițiilor 5.3.2 și 5.3.4 rezultă că

$$T(n) = \frac{2 \sum_{k \in E} D(k) - n}{2n + 1} = \frac{2n + 2}{2n + 1} \cdot \lceil \log_2(n + 1) \rceil + \frac{n + 2 - 2 \cdot 2^{\lceil \log_2(n + 1) \rceil}}{2n + 1}. \quad (5.4.1)$$

Dar, conform Observației 5.3.2, $\lceil \log_2(n + 1) \rceil = 1 + \lfloor \log_2 n \rfloor$, deci

$$T(n) = \frac{2n + 2}{2n + 1} \cdot \lfloor \log_2 n \rfloor + \frac{3n + 4 - 4 \cdot 2^{\lfloor \log_2 n \rfloor}}{2n + 1}.$$

Evident, $\log_2 n - 1 < \lfloor \log_2 n \rfloor \leq \log_2 n$, deci $\frac{n}{2} < 2^{\lfloor \log_2 n \rfloor} \leq n$. Rezultă că

$$\begin{aligned} \log_2 n - \frac{3n - 2 - \log_2 n}{2n + 1} &< T(n) < \log_2 n + \frac{n + 4 + \log_2 n}{2n + 1}, \text{ deci} \\ 1 - \frac{3n - 2 - \log_2 n}{(2n + 1) \log_2 n} &< \frac{T(n)}{\log_2 n} < 1 + \frac{n + 4 + \log_2 n}{(2n + 1) \log_2 n}, \quad \forall n \geq 2. \end{aligned}$$

Aplicând *Criteriul cleștelui* rezultă că

$$\lim_{n \rightarrow \infty} \frac{T(n)}{\log_2 n} = 1, \quad (5.4.2)$$

deci conform Propoziției 1.1.1 obținem că $T(n) = \Theta(\log_2 n)$. $\square$

Mai mult, conform relației (5.4.2) am obținut următorul rezultat.

Corolarul 5.4.1. *Numărul mediu de comparații de chei efectuate de algoritmul de căutare binară într-un vector sortat cu n componente este asimptotic echivalent cu $\log_2 n$.*

Teorema 5.4.4 (optimalitatea algoritmului de căutare binară). *Algoritmul de căutare binară este optim (în clasa algoritmilor bazați pe comparații de chei), atât în raport cu timpul de execuție în cazul cel mai defavorabil, cât și în raport cu timpul mediu de execuție.*

Demonstrație. Optimalitatea în raport cu timpul de execuție în cazul cel mai defavorabil este o consecință directă a Propoziției 5.4.2 și a primei părți din Teorema 5.4.2. Optimalitatea în raport cu timpul mediu de execuție este o consecință directă a relației (5.4.1) din demonstrația Teoremei 5.4.3 și a părții secunde din Teorema 5.4.2. $\square$

5.5 Algoritmi de sortare

5.5.1 Problema sortării

Problema sortării este următoarea:

Se consideră un vector $A = (a_1, a_2, \dots, a_n)$, $n \geq 1$. Se cere să se ordoneze crescător (sau descrescător) elementele acestui vector.

Observația 5.5.1. În practică, problema apare, în general, în ordonarea crescătoare (sau descrescătoare) a unei mulțimi finite de articole (înregistrări) după un câmp (sau mai multe) numit **cheie de sortare**.

5.5.2 Complexitatea algoritmilor de sortare

Observația 5.5.2. Fie $\mathcal{A}$ un algoritm de sortare, bazat pe comparații de chei, a unui vector $A = (a_1, a_2, \dots, a_n)$, $n \geq 1$.

Presupunem că algoritmul nu conține instrucțiuni redundante și este aplicabil pentru orice poziționare posibilă a componentelor vectorului A , adică pentru orice permutare a acestora.

Atunci algoritmului $\mathcal{A}$ i se poate asocia un arbore binar $\mathcal{T}(\mathcal{A})$ în care fiecare nod are o etichetă de forma $i?j$, construit recursiv astfel: dacă prima comparație efectuată de algoritm este între elementele a_i și a_j ale vectorului, atunci

- rădăcina arborelui este etichetată cu $i?j$;
- subarborele stâng corespunde continuării algoritmului în cazul $a_i \leq a_j$, dacă acest caz este posibil după prima comparație (altfel este vid, adică nu există descendent stâng);
- subarborele drept corespunde continuării algoritmului în cazul $a_i > a_j$, dacă acest caz este posibil după prima comparație (altfel este vid, adică nu există descendent drept).

Definiția 5.5.1. *Arbore binar $\mathcal{T}(\mathcal{A})$ construit în observația anterioară se numește **arborele de sortare** sau **arborele de decizie al algoritmului de sortare $\mathcal{A}$** .*

Definiția 5.5.2. *Fie $\mathcal{A}$ un algoritm de sortare, bazat pe comparații de chei, a unui vector $A = (a_1, a_2, \dots, a_n)$, $n \geq 1$, și fie $\mathcal{T}(\mathcal{A})$ arborele de sortare corespunzător. Extindem arborele binar $\mathcal{T}(\mathcal{A})$ la un arbore binar strict $\overline{\mathcal{T}}(\mathcal{A})$ astfel: orice nod al lui $\mathcal{T}(\mathcal{A})$, având eticheta $i?j$, este reprezentat într-un cerc și devine nod intern în $\overline{\mathcal{T}}(\mathcal{A})$ prin aplicarea următoarelor două reguli:*

- dacă nodul nu are descendent stâng i se adaugă un descendent stâng, reprezentat într-un dreptunghi, corespunzător cazului $a_i \leq a_j$, și etichetat cu permutarea σ a mulțimii de indici $\{1, 2, \dots, n\}$ pentru care lanțul de la rădăcină la nodul adăugat corespunde ordinii

$$a_{\sigma(1)} \leq a_{\sigma(2)} \leq \dots \leq a_{\sigma(n)};$$

- dacă nodul nu are descendent drept i se adaugă un descendent drept, reprezentat într-un dreptunghi, corespunzător cazului $a_i > a_j$, și etichetat cu permutarea σ a mulțimii de indici $\{1, 2, \dots, n\}$ pentru care lanțul de la rădăcină la nodul adăugat corespunde ordinii

$$a_{\sigma(1)} \leq a_{\sigma(2)} \leq \dots \leq a_{\sigma(n)}.$$

Arborele $\overline{\mathcal{T}}(\mathcal{A})$ se numește **arbore extins de sortare**.

Observația 5.5.3. Deoarece algoritmul $\mathcal{A}$ este aplicabil pentru orice permutare a vectorului $A = (a_1, a_2, \dots, a_n)$ și nu conține instrucțiuni redundante, rezultă că în arborele extins de sortare $\overline{\mathcal{T}}(\mathcal{A})$ fiecare permutarea σ a mulțimii de indici $\{1, 2, \dots, n\}$ apare exact o dată ca etichetă a unui nod extern, deci acest arbore are $n!$ noduri externe și, conform Propoziției 5.3.1, $n! - 1$ noduri interne.

Propoziția 5.5.1. Pentru orice algoritm de sortare $\mathcal{A}$, bazat pe comparații de chei, a unui vector cu n componente, avem:

1) numărul de comparații de chei efectuate în cazul cel mai defavorabil, notat cu $N_{def}(n)$, verifică inegalitatea

$$N_{def}(n) \geq \lceil \log_2 n! \rceil;$$

2) numărul mediu de comparații de chei efectuate, notat cu $N_{med}(n)$, verifică inegalitățile

$$N_{med}(n) \geq \lceil \log_2 n! \rceil + 1 - \frac{2^{\lceil \log_2 n! \rceil}}{n!} \geq \log_2 n! - 1 + \frac{1}{n!}.$$

Demonstrație. Cazul 1) Analizăm cazul cel mai defavorabil.

Evident, numărul de comparații de chei efectuate în cazul cel mai defavorabil este egal cu înălțimea h a arborelui extins de sortare asociat $\overline{\mathcal{T}} = \overline{\mathcal{T}}(\mathcal{A})$.

Arborele $\overline{\mathcal{T}}$ este un arbore binar strict, cu $n! - 1$ noduri interne (și $n!$ noduri externe), deci conform Propoziției 5.3.3 rezultă că

$$h \geq \lceil \log_2 n! \rceil.$$

Cazul 2) Analizăm acum cazul mediu (cazul timpului mediu de execuție).

Pentru fiecare din cele $n!$ permutări σ ale mulțimii de indici $\{1, 2, \dots, n\}$, numărul de comparații de chei efectuate de algoritmul $\mathcal{A}$ în cazul ordinii $a_{\sigma(1)} \leq a_{\sigma(2)} \leq \dots \leq a_{\sigma(n)}$ este egal cu distanța de la rădăcina arborelui extins de sortare asociat la nodul extern etichetat cu σ . Deci numărul mediu de comparații de chei are valoarea

$$N_{med}(n) = \frac{\sum_{k \in E} D(k)}{n!},$$

unde E este mulțimea nodurilor externe, iar, pentru fiecare nod k , $D(k)$ reprezintă distanța de la rădăcina arborelui la nodul k .

Conform Propoziției 5.3.5 rezultă că

$$\begin{aligned} N_{med}(n) &\geq \frac{n! \lceil \log_2 n! \rceil + n! - 2^{\lceil \log_2 n! \rceil}}{n!} \\ &= \lceil \log_2 n! \rceil + 1 - \frac{2^{\lceil \log_2 n! \rceil}}{n!}. \end{aligned}$$

Cum $\log_2 n! \leq \lceil \log_2 n! \rceil < 1 + \log_2 n!$, rezultă că $2^{\lceil \log_2 n! \rceil} < 2 \cdot n!$, adică $2^{\lceil \log_2 n! \rceil} \leq 2 \cdot n! - 1$, deci $\lceil \log_2 n! \rceil + 1 - \frac{2^{\lceil \log_2 n! \rceil}}{n!} \geq \log_2 n! - 1 + \frac{1}{n!}$. $\square$

Corolarul 5.5.1. Fie $\mathcal{A}$ un algoritm de sortare, bazat pe comparații de chei, a unui vector cu n componente și fie $\overline{\mathcal{T}} = \overline{\mathcal{T}}(\mathcal{A})$ arborele extins de sortare asociat.

1) Algoritmul $\mathcal{A}$ este optim în raport cu timpul de execuție în cazul cel mai defavorabil dacă arborele $\overline{\mathcal{T}}$ are toate nodurile externe situate pe ultimul și, eventual, pe penultimul nivel. Mai mult, în acest caz numărul de comparații de chei efectuate în cazul cel mai defavorabil, notat cu $N_{def}(n)$, verifică egalitatea

$$N_{def}(n) = \lceil \log_2 n! \rceil;$$

2) Algoritmul $\mathcal{A}$ este optim în raport cu timpul mediu de execuție dacă și numai dacă arborele $\overline{\mathcal{T}}$ are toate nodurile externe situate pe ultimul și, eventual, pe penultimul nivel. Mai mult, în acest caz numărul mediu de comparații de chei efectuate, notat cu $N_{med}(n)$, verifică relațiile

$$N_{med}(n) = \lceil \log_2 n! \rceil + 1 - \frac{2^{\lceil \log_2 n! \rceil}}{n!} \geq \log_2 n! - 1 + \frac{1}{n!}.$$

Demonstrație. Optimalitatea în raport cu timpul de execuție în cazul cel mai defavorabil este o consecință directă a Cazului 1 din propoziția anterioară și a Propoziției 5.3.4.

Optimalitatea în raport cu timpul mediu de execuție este o consecință directă a Cazului 2 din propoziția anterioară și a Propoziției 5.3.5. $\square$

Exemplul 5.5.1. Un algoritm de sortare, bazat pe comparații de chei, a unui vector cu $n = 3$ componente este optim în raport cu timpul de execuție în cazul cel mai defavorabil dacă și numai dacă în acest caz numărul de comparații de chei efectuate este

$$\lceil \log_2 3! \rceil = 3,$$

adică dacă și numai dacă arborele extins de sortare asociat are 4 nivele.

Pe de altă parte, un algoritm de sortare, bazat pe comparații de chei, a unui vector cu $n = 3$ componente este optim în raport cu timpul mediu de execuție dacă și numai dacă arborele extins de sortare asociat are 4 nivele și toate nodurile externe situate pe ultimele două nivele. Mai mult, în acest caz numărul mediu de comparații de chei efectuate este

$$\lceil \log_2 3! \rceil + 1 - \frac{2^{\lceil \log_2 3! \rceil}}{3!} = \frac{16}{6} = 2, (6).$$

Analog, un algoritm de sortare, bazat pe comparații de chei, a unui vector cu $n = 4$ componente este optim în raport cu timpul de execuție în cazul cel mai defavorabil dacă și numai dacă în acest caz numărul de comparații de chei efectuate este

$$\lceil \log_2 4! \rceil = 5,$$

adică dacă și numai dacă arborele extins de sortare asociat are 6 nivele.

Pe de altă parte, un algoritm de sortare, bazat pe comparații de chei, a unui vector cu $n = 4$ componente este optim în raport cu timpul mediu de execuție dacă și numai dacă arborele extins de sortare asociat are 6 nivele și toate nodurile externe situate pe ultimele două nivele. Mai mult, în acest caz numărul mediu de comparații de chei efectuate este

$$\lceil \log_2 4! \rceil + 1 - \frac{2^{\lceil \log_2 4! \rceil}}{4!} = \frac{112}{24} = 4, (6).$$

Lema 5.5.1. Pentru orice $n \in \mathbb{N}^*$ au loc inegalitățile:

$$\frac{1}{n+1} < \ln(n+1) - \ln n < \frac{1}{n}; \quad (5.5.1)$$

$$n \log_2 n - (n-1) \log_2 e \leq \log_2 n! \leq n \log_2 n. \quad (5.5.2)$$

Demonstrație. Aplicând Teorema lui Lagrange pentru funcția

$$f : [n, n+1] \rightarrow \mathbb{R}, \quad f(x) = \ln x$$

(f este derivabilă), rezultă că există $c \in (n, n+1)$ a.î. $f(n+1) - f(n) = f'(c)$, adică $\ln(n+1) - \ln n = \frac{1}{c}$. Dar $\frac{1}{n+1} < \frac{1}{c} < \frac{1}{n}$, deci obținem relația (5.5.1).

Din această relație rezultă că $(n+1)\ln(n+1) - n\ln n < 1 + \ln(n+1)$, deci

$$\begin{aligned} 2\ln 2 - \ln 1 &< 1 + \ln 2, \\ 3\ln 3 - 2\ln 2 &< 1 + \ln 3, \\ &\dots \\ n\ln n - (n-1)\ln(n-1) &< 1 + \ln n. \end{aligned}$$

Prin adunare obținem că

$$n\ln n < n - 1 + \ln n!, \quad \forall n \geq 2,$$

de unde, utilizând formula $\log_2 x = \log_2 e \cdot \ln x$, $\forall x > 0$, rezultă prima inegalitate din (5.5.2), valabilă și pentru $n = 1$. A doua inegalitate din (5.5.2) este o consecință a inegalității evidente $n! \leq n^n$. $\square$

Lema 5.5.2. *Avem*

$$\log_2 n! \sim n \log_2 n$$

(adică funcțiile $\log_2 n!$ și $n \log_2 n$ sunt asimptotic echivalente).

Demonstrație. Din (5.5.2) rezultă că

$$1 - \frac{(n-1)\log_2 e}{n \log_2 n} \leq \frac{\log_2 n!}{n \log_2 n} \leq 1, \quad \forall n \in \mathbb{N}^*,$$

deci, aplicând *Criteriul cleștelui* obținem

$$\lim_{n \rightarrow \infty} \frac{\log_2 n!}{n \log_2 n} = 1,$$

adică $\log_2 n! \sim n \log_2 n$. $\square$

Următorul rezultat este o consecință imediată a Corolarului 5.5.1, Lemei 5.5.2 și Propoziției 1.1.1.

Teorema 5.5.1 (complexitatea algoritmilor de sortare). *Numărul de comparații de chei efectuate de un algoritm optim de sortare a unui vector cu n componente este asimptotic echivalent cu*

$$n \log_2 n,$$

atât în cazul cel mai defavorabil, cât și în cazul mediu (cazul timpului mediu de execuție), deci în ambele cazuri un astfel de algoritm are complexitatea

$$\Theta(n \log_2 n).$$

Observația 5.5.4. Dacă un algoritm de sortare conține instrucțiuni redundante, atunci în arborele extins de sortare asociat orice nod extern care nu corespunde unei ordini $a_{\sigma(1)} \leq a_{\sigma(2)} \leq \dots \leq a_{\sigma(n)}$ (calea de la rădăcină la el fiind imposibilă la executarea algoritmului) este etichetat cu $\emptyset$.

În acest caz arborele extins de sortare va conține mai mult de $n!$ noduri externe (iar arborele de sortare va conține mai mult de $n! - 1$ noduri), deci un astfel de algoritm nu poate fi optim în raport cu timpul mediu de execuție.

Observația 5.5.5. În anumite cazuri particulare pot exista algoritmi de sortare a unui vector cu n componente, nebazăți pe comparații de chei, a căror complexitate să fie mai bună decât $\Theta(n \log_2 n)$.

De exemplu, dacă vectorul $A = (a_1, a_2, \dots, a_n)$, $n \geq 1$, are toate componentele numere naturale nenule mai mici sau egale cu m , $m \in \mathbb{N}^*$, adică

$$a_1, a_2, \dots, a_n \in \{1, 2, \dots, m\},$$

un algoritm eficient de sortare a vectorului A este **sortarea prin numărarea frecvențelor componentelor**:

- pentru fiecare valoare k , $k = \overline{1, m}$, numărăm câte elemente ale vectorului A sunt egale cu k , adică determinăm numărul

$$f_k = \text{card} \{i \mid i = \overline{1, n}, a_i = k\},$$

numit *frecvența absolută* de apariție a valorii k în vectorul A ;

- vectorul sortat crescător este

$$\underbrace{(1, 1, \dots, 1)}_{\text{de } f_1 \text{ ori}}, \underbrace{(2, 2, \dots, 2)}_{\text{de } f_2 \text{ ori}}, \dots, \underbrace{(m, m, \dots, m)}_{\text{de } f_m \text{ ori}}.$$

Evident, $f_1 + f_2 + \dots + f_m = n$.

Descrierea în pseudocod a algoritmului are următoarea formă.

SORTNUMFREC $V(A, n, B)$: // vectorul $B = (b_1, b_2, \dots, b_n)$ va
 // conține elementele vectorului $A = (a_1, a_2, \dots, a_n)$
 // sortate crescător

for $k = \overline{1, m}$ **do**

$F[k] \leftarrow 0$; // $F = (f_1, f_2, \dots, f_m)$ este vectorul frecvențelor

for $i = \overline{1, n}$ **do**

$F[A[i]] \leftarrow F[A[i]] + 1$;

$i \leftarrow 0$;

for $k = \overline{1, m}$ **do**

for $j = \overline{1, F[k]}$ **do**

$i \leftarrow i + 1$;

$B[i] \leftarrow k$;

Algoritmul SORTNUMFRECVC efectuează m atribuiri de forma $F[k] \leftarrow 0$, n atribuiri de forma $F[A[i]] \leftarrow F[A[i]] + 1$ și tot n atribuiri de forma $B[i] \leftarrow k$ (iar celelalte operații nu depășesc ordinul de creștere al acestora), deci are complexitatea

$$\Theta(n + m).$$

Astfel, dacă $m = n$ sau, mai general, $p \cdot n \leq m \leq q \cdot n$ cu p și q numere reale pozitive fixate (independente de n), atunci algoritmul SORTNUMFRECVC are complexitatea $\Theta(n)$, adică este un algoritm de sortare liniar!

5.5.3 Sortarea prin inserție directă

Algoritmul 5.5.1 (Sortarea prin inserție directă). Pentru a sorta crescător vectorul $A = (a_1, a_2, \dots, a_n)$, $n \geq 1$, procedăm astfel:

- pentru fiecare indice $i = \overline{2, n}$, cunoscând că subvectorul $(a_1, a_2, \dots, a_{i-1})$ este deja sortat crescător, inserăm elementul a_i în acest subvector conform ordinii crescătoare;
- pentru aceasta comparăm succesiv elementul a_i cu elementele $a_{i-1}, a_{i-2}, \dots$ și cât timp aceste elemente sunt mai mari decât a_i le deplasăm câte o poziție spre dreapta; elementul a_i va fi inserat pe ultima poziție eliberată.

Descrierea în pseudocod a algoritmului are următoarea formă.

SORTINSDIR(A, n) :

for $i = \overline{2, n}$ **do**

```

     $x \leftarrow A[i];$ 
     $j \leftarrow i - 1;$ 
    while  $x < A[j]$  and  $j \geq 1$  do
         $A[j + 1] \leftarrow A[j];$ 
         $j \leftarrow j - 1;$ 
     $A[j + 1] \leftarrow x;$ 
```

Exemplul 5.5.2. În Figura 5.5.1 este reprezentat arborele extins de sortare asociat algoritmului de sortare prin inserție directă pentru $n = 3$.

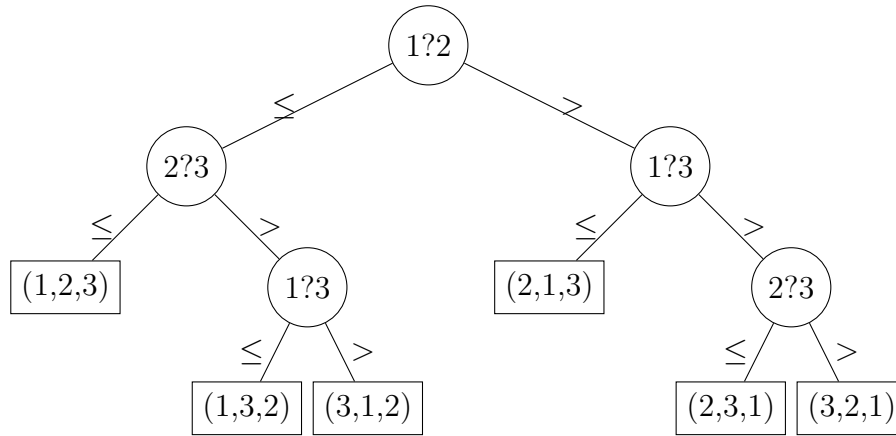


Figura 5.5.1:

Observăm că algoritmul nu conține instrucțiuni redundante.

Propoziția 5.5.2 (complexitatea algoritmului de sortare prin inserție directă). Algoritmul de sortare prin inserție directă a unui vector cu n componente are complexitatea de ordinele $\mathcal{O}(n^2)$ și $\Omega(n)$.

Demonstrație. Vom evalua atât numărul $N_c(n)$ de comparații de chei efectuate de algoritm, cât și numărul de atribuiri în care intervin elemente ale vectorului, numite și **deplasări**, notat cu $N_d(n)$. Celelalte operații nu depășesc ordinele de creștere ale acestora.

Pentru fiecare $i \in \{2, 3, \dots, n\}$, algoritmul efectuează cel puțin o comparație de chei de forma $x < A[j]$, unde $x = A[i]$, și anume pentru $j = i - 1$, și cel mult $i - 1$ astfel de comparații, și anume pentru $j \in \{i - 1, i - 2, \dots, 1\}$.

De asemenea, pentru fiecare $i \in \{2, 3, \dots, n\}$, algoritmul efectuează obligatoriu deplasările $x \leftarrow A[i]$ și $A[j + 1] \leftarrow x$, plus un număr de maxim încă $i - 1$ deplasări de forma $A[j + 1] \leftarrow A[j]$, pentru $j \in \{i - 1, i - 2, \dots, 1\}$.

Astfel, în cazul cel mai defavorabil, când elementele vectorului A sunt inițial sortate descrescător, pentru fiecare $i \in \{2, 3, \dots, n\}$ algoritmul efectuează câte $i - 1$ comparații de chei (de forma $x < A[j]$, pentru $j \in \{i - 1, i - 2, \dots, 1\}$, unde $x = A[i]$) și câte $i + 1$ deplasări (prima de forma $x \leftarrow A[i]$, următoarele $i - 1$ de forma $A[j + 1] \leftarrow A[j]$, pentru $j \in \{i - 1, i - 2, \dots, 1\}$, și ultima de forma $A[j + 1] \leftarrow x$, unde $x = A[i]$), deci

$$N_c(n) = 1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2}$$

și

$$N_d(n) = 3 + 4 + \dots + (n + 1) = \frac{(n + 4)(n - 1)}{2}.$$

Prin urmare algoritmul are complexitatea $\mathcal{O}(n^2)$.

Pe de altă parte, în cazul cel mai favorabil, când elementele vectorului A sunt inițial sortate crescător, pentru fiecare $i \in \{2, 3, \dots, n\}$ algoritmul efectuează câte o comparație de chei (de forma $x < A[j]$, pentru $j = i - 1$, unde $x = A[i]$) și câte două deplasări (prima de forma $x \leftarrow A[i]$, a doua de forma $A[j + 1] \leftarrow x$, unde $j = i - 1$), deci

$$N_c(n) = n - 1 \text{ și } N_d(n) = 2(n - 1).$$

Prin urmare algoritmul are și complexitatea $\Omega(n)$. □

5.5.4 Sortarea prin inserție binară

Algoritmul 5.5.2 (Sortarea prin inserție binară). Pentru a sorta crescător vectorul $A = (a_1, a_2, \dots, a_n)$, $n \geq 1$, procedăm astfel:

- ca și în cazul inserției directe, pentru fiecare indice $i = \overline{2, n}$, cunoscând că subvectorul $(a_1, a_2, \dots, a_{i-1})$ este deja sortat crescător, inserăm elementul a_i în acest subvector conform ordinii crescătoare;
- pentru aceasta, acum procedăm ca la căutarea binară (Observația 5.4.5):

- comparăm a_i cu elementul median, a_m , al subvectorului sortat $(a_1, a_2, \dots, a_{i-1})$, unde

$$m = \left\lfloor \frac{1 + (i - 1)}{2} \right\rfloor;$$

- dacă $a_i < a_m$, atunci continuăm procedeul de inserare a lui a_i în subvectorul

$$(a_1, \dots, a_{m-1});$$

- dacă $a_i \geq a_m$, atunci continuăm procedeul de inserare a lui a_i în subvectorul

$$(a_{m+1}, \dots, a_{i-1});$$

- după determinarea poziției p din subvectorul $(a_1, a_2, \dots, a_{i-1})$ pe care trebuie inserat elementul a_i , conform ordinii crescătoare (adică $a_{p-1} \leq a_i < a_p$ cu $p \in \{2, 3, \dots, i-1\}$, sau $a_{p-1} \leq a_i$ cu $p = i$, sau $a_i < a_p$ cu $p = 1$), elementele $a_{i-1}, a_{i-2}, \dots, a_p$ (adică toate elementele mai mari decât a_i ale subvectorului) se deplasează câte o poziție spre dreapta și elementul a_i va fi inserat pe ultima poziție eliberată, p .

SORTINSBIN(A, n) :

$$A[p] \leftarrow x;$$

```

graph TD
    N1((1?2)) -- "≤" --> N2((1?3))
    N1 -- ">" --> N3((2?3))
    N2 -- "≤" --> N4((2?3))
    N2 -- ">" --> R1["(3,1,2)"]
    N3 -- "≤" --> N5((1?3))
    N3 -- ">" --> R2["(3,2,1)"]
    N4 -- "≤" --> R3["(1,2,3)"]
    N4 -- ">" --> R4["(1,3,2)"]
    N5 -- "≤" --> R5["(2,1,3)"]
    N5 -- ">" --> R6["(2,3,1)"]
    style N1 fill:#fff,stroke:#000,stroke-width:1px
    style N2 fill:#fff,stroke:#000,stroke-width:1px
    style N3 fill:#fff,stroke:#000,stroke-width:1px
    style N4 fill:#fff,stroke:#000,stroke-width:1px
    style N5 fill:#fff,stroke:#000,stroke-width:1px
    style R1 fill:#fff,stroke:#000,stroke-width:1px
    style R2 fill:#fff,stroke:#000,stroke-width:1px
    style R3 fill:#fff,stroke:#000,stroke-width:1px
    style R4 fill:#fff,stroke:#000,stroke-width:1px
    style R5 fill:#fff,stroke:#000,stroke-width:1px
    style R6 fill:#fff,stroke:#000,stroke-width:1px
  
```

Observăm că algoritmul nu conține instrucțiuni redundante.

Lema 5.5.3. Pentru orice $n \in \mathbb{N}^*$ avem

$$\sum_{i=1}^n \lceil \log_2 i \rceil = n \lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1.$$

Demonstrație. Fie $n \geq 2$ și $p = \lceil \log_2 n \rceil$, deci $p \in \mathbb{N}^*$ și $p - 1 < \log_2 n \leq p$, deci $2^{p-1} < n \leq 2^p$.

Pentru orice $i \in \{2, 3, \dots, n\}$ avem echivalențele

$$\lceil \log_2 i \rceil = j \Leftrightarrow j - 1 < \log_2 i \leq j \Leftrightarrow 2^{j-1} < i \leq 2^j \Leftrightarrow 2^{j-1} + 1 \leq i \leq 2^j.$$

Astfel avem

$$\begin{aligned} \sum_{i=1}^n \lceil \log_2 i \rceil &= \sum_{i=2}^n \lceil \log_2 i \rceil = \sum_{j=1}^{p-1} \sum_{i=2^{j-1}+1}^{2^j} \lceil \log_2 i \rceil + \sum_{i=2^{p-1}+1}^n \lceil \log_2 i \rceil \\ &= \sum_{j=1}^{p-1} \sum_{i=2^{j-1}+1}^{2^j} j + \sum_{i=2^{p-1}+1}^n p = \sum_{j=1}^{p-1} j(2^j - 2^{j-1}) + p(n - 2^{p-1}) \\ &= \sum_{j=1}^{p-1} j \cdot 2^{j-1} + p(n - 2^{p-1}) = p \cdot 2^{p-1} - 2^p + 1 + p(n - 2^{p-1}) \\ &= np - 2^p + 1, \end{aligned}$$

deci

$$\sum_{i=1}^n \lceil \log_2 i \rceil = n \lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1.$$

Egalitatea este evident valabilă și pentru $n = 1$. □

Propoziția 5.5.3 (complexitatea algoritmului de sortare prin inserție binară). a) Numărul de comparații de chei efectuate de algoritmul de sortare prin inserție binară a unui vector cu n componente este de ordinul $\Theta(n \log_2 n)$.

b) Algoritmul de sortare prin inserție binară a unui vector cu n componente are complexitatea de ordinele $\mathcal{O}(n^2)$ și $\Omega(n \log_2 n)$.

Demonstrație. a) Evaluăm numai numărul $N_c(n)$ de comparații de chei efectuate de algoritm. Pentru fiecare $i \in \{2, 3, \dots, n\}$, conform descrierii algoritmului și Propoziției 5.4.2 numărul de comparații de chei efectuate pentru inserarea elementului a_i în subvectorul $(a_1, a_2, \dots, a_{i-1})$ este $h_{i-1} - 1$ sau h_{i-1} , unde

$$h_{i-1} = \lceil \log_2(i - 1 + 1) \rceil = \lceil \log_2 i \rceil.$$

Rezultă că

$$\sum_{i=2}^n (h_{i-1} - 1) \leq N_c(n) \leq \sum_{i=2}^n h_{i-1},$$

adică

$$\sum_{i=2}^n \lceil \log_2 i \rceil - (n-1) \leq N_c(n) \leq \sum_{i=2}^n \lceil \log_2 i \rceil,$$

deci conform Lemei 5.5.3 obținem că

$$n \lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} - n + 2 \leq N_c(n) \leq n \lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1.$$

Dar

$$\log_2 n \leq \lceil \log_2 n \rceil < \log_2 n + 1,$$

deci

$$n \leq 2^{\lceil \log_2 n \rceil} < 2n.$$

Astfel rezultă că numărul de *comparații de chei* verifică inegalitățile

$$n \log_2 n - 3n + 2 < N_c(n) < n \log_2 n + 1, \quad \forall n \in \mathbb{N}^*, \quad (5.5.3)$$

deci $N_c(n) = \Theta(n \log_2 n)$.

b) Evaluăm acum și numărul de *deplasări* $N_d(n)$ (atribuiri în care intervin elemente ale vectorului) efectuate de algoritm. Celelalte operații nu depășesc ordinul de creștere al acestora.

Pentru fiecare $i \in \{2, 3, \dots, n\}$, după determinarea prin căutare binară a poziției p din subvectorul $(a_1, a_2, \dots, a_{i-1})$ sortat crescător pe care trebuie inserat elementul a_i , algoritmul SORTINSBIN efectuează exact aceleași deplasări ale elementelor $a_{i-1}, a_{i-2}, \dots, a_p$ câte o poziție spre dreapta și a elementului a_i pe poziția p ca și algoritmul SORTINSDIR. Conform demonstrației Propoziției 5.5.2 rezultă că

$$2(n-1) \leq N_d(n) \leq \frac{(n+4)(n-1)}{2}$$

(cazurile cel mai favorabil și cel mai defavorabil fiind când elementele vectorului A sunt inițial sortate crescător, respectiv descrescător). Prin urmare avem

$$N_d(n) = \mathcal{O}(n^2) \text{ și } N_d(n) = \Omega(n),$$

deci, folosind și punctul a), rezultă că algoritmul SORTINSBIN are complexitatea de ordinele $\mathcal{O}(n^2)$ și $\Omega(n \log_2 n)$. $\square$

Observația 5.5.6. Conform Propoziției 5.5.3 și Teoremei 5.5.1 rezultă că algoritmul de sortare prin inserție binară este asimptotic-optimal în raport cu numărul de comparații de chei (în clasa algoritmilor bazați pe comparații de chei).

Mai mult, conform relațiilor (5.5.3) și Teoremei 5.5.1 rezultă că, exact ca la algoritmi optimi de sortare bazați pe comparații de chei, numărul de comparații de chei efectuate de algoritmul de sortare prin inserție binară a unui vector cu n componente este asimptotic echivalent cu $n \log_2 n$.

5.5.5 Sortarea prin interclasare (mergesort)

Algoritmul de sortare prin interclasare (mergesort), inventat de către faimosul savant John von Neumann în anul 1945, este unul dintre cei mai performanți algoritmi de sortare.

Algoritmul 5.5.3 (Sortarea prin interclasare (mergesort)). Pentru a sorta crescător un vector dat $A = (a_1, a_2, \dots, a_n)$, $n \geq 1$, utilizăm următorul procedeu specific metodei de programare *Divide et Impera*:

- Dacă $n = 1$, atunci vectorul este deja sortat (având un singur element);
- Dacă $n \geq 2$, atunci:
 - Se împarte vectorul A în doi subvectori

$$(a_1, \dots, a_m) \text{ și } (a_{m+1}, \dots, a_n), \text{ unde } m = \left\lfloor \frac{n+1}{2} \right\rfloor = \left\lceil \frac{n}{2} \right\rceil$$

(deci a_m este elementul median și cei doi subvectori au dimensiuni cât mai apropiate posibil);

- Se sortează crescător cei doi subvectori, prin același procedeu;
- Se **interclasează** cei doi subvectori sortați, adică se reconstruiește vectorul A astfel încât să conțină toate componentele celor doi subvectori sortați (deci toate cele n componente ale vectorului inițial) și care să fie, de asemenea, sortat crescător.

Descrierea în pseudocod a algoritmului are următoarea formă.

```
SORTINT( $A, p, u$ ):           // sortarea secvenței ( $a_p, \dots, a_u$ )  
if  $p < u$  then  
     $m \leftarrow \left\lfloor \frac{p+u}{2} \right\rfloor$ ;  
    SORTINT( $A, p, m$ );        // sortarea subsecvenței ( $a_p, \dots, a_m$ )  
    SORTINT( $A, m+1, u$ );      // sortarea subsecvenței  
                               // ( $a_{m+1}, \dots, a_u$ )  
    INTERCLAS( $A, p, u, m$ );  // interclasarea celor două  
                               // subsecvențe sortate
```

unde procedura de interclasare este:

```

INTERCLAS( $A, p, u, m$ );    // interclasarea subsecvențelor
    // ( $a_p, \dots, a_m$ ) și ( $a_{m+1}, \dots, a_u$ ), sortate crescător
 $i \leftarrow p$ ;                // poziția curentă din subsecvența ( $a_p, \dots, a_m$ )
 $j \leftarrow m + 1$ ;          // poziția curentă din subsecvența ( $a_{m+1}, \dots, a_u$ )
 $k \leftarrow 0$ ;              // dimensiunea curentă a unui vector  $B$ , sortat
    // crescător, ce va fi construit prin interclasarea celor
    // două subsecvențe, comparând componentele curente
    //  $a_i$  și  $a_j$  din cele două subsecvențe
    // și copiind-o pe cea mai mică în  $B$ 
while  $i \leq m$  and  $j \leq u$  do
    // mai există componente necopiate în
    // ambele subsecvențe
    if  $A[i] \leq A[j]$  then
        // mai mică este componenta curentă
        // din prima subsecvență,
         $k \leftarrow k + 1$ ;  $B[k] \leftarrow A[i]$ ;    // deci o copiem în  $B$ 
         $i \leftarrow i + 1$ ;    // și avansăm în prima subsecvență
    else
        // mai mică este componenta curentă
        // din a doua subsecvență,
         $k \leftarrow k + 1$ ;  $B[k] \leftarrow A[j]$ ;    // deci o copiem în  $B$ 
         $j \leftarrow j + 1$ ;    // și avansăm în a doua subsecvență
    // una dintre subsecvențe nu mai are elemente necopiate;
    // copiem în  $B$  și componentele rămase necopiate
    // din cealaltă subsecvență
for  $q = \overline{i, m}$  do    // componente rămase necopiate în prima
    // subsecvență
     $k \leftarrow k + 1$ ;  $B[k] \leftarrow A[q]$ ;
for  $q = \overline{j, u}$  do    // componente rămase necopiate în a doua
    // subsecvență
     $k \leftarrow k + 1$ ;  $B[k] \leftarrow A[q]$ ;
    // La final aducem componentele (sortate crescător) din
    //  $B = (b_1, \dots, b_k)$  înapoi în subsecvența ( $a_p, \dots, a_u$ ):
for  $i = \overline{1, k}$  do
     $A[p + i - 1] \leftarrow B[i]$ ;

Apel:
SORTINT( $A, 1, n$ )    // pentru sortarea întregului vector
    //  $A = (a_1, \dots, a_n)$ .

```

Exemplul 5.5.4. În Figura 5.5.3 este reprezentat arborele extins de sortare asociat algoritmului de sortare prin interclasare pentru $n = 3$. Observăm că algoritmul nu conține comparații de chei redundante.

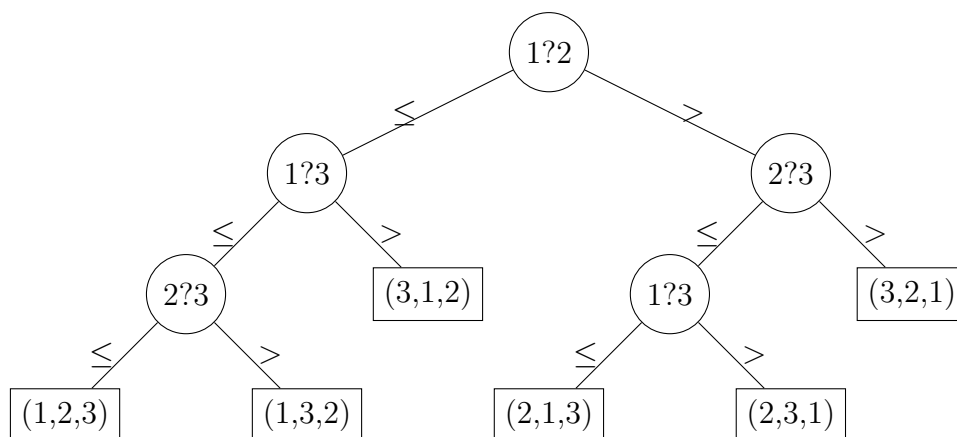


Figura 5.5.3:

Observația 5.5.7. Procedura de interclasare poate fi ușor îmbunătățită, eliminând copierile inutile în vectorul intermediar B și înapoi în vectorul A ale componentelor rămase necopiate (la ieșirea din ciclul **while**) în una dintre cele două subsecvențe interclasate. În acest sens, putem renunța la ciclul **for** $q = \overline{j, u}$, deoarece elementele rămase necopiate în a doua subsecvență sunt deja pe pozițiile bune, deci nu are rost să le mai copiem în B . De asemenea, elementele rămase necopiate în prima subsecvență pot fi copiate direct pe ultimele poziții ale secvenței $(a_p, \dots, a_u)$ (în ordine inversă, pentru a evita pierderea valorilor unor componente prin suprapunere), fără a le mai copia în B ; pentru aceasta, putem înlocui ciclul **for** $q = \overline{i, m}$ cu:

```

for  $q = \overline{m, i, -1}$  do
     $A[u + q - m] \leftarrow A[q];$ 

```

Observația 5.5.8. Implementarea aleasă pentru algoritmul de sortare prin interclasare este una recursivă, în care soluția problemei apelează soluțiile subproblemelor de dimensiuni mai mici, adică folosește **strategia Divide et Impera în varianta top-down**. În cazul acestei variante este important că subproblemele sunt disjuncte și astfel calculul necesar rezolvării unei subprobleme nu este reluat prin apelarea repetată a acesteia (ceea ce ar conduce la un timp de calcul exponențial, așa cum se întâmplă la utilizarea acestei variante în rezolvarea unor probleme prin *metoda programării dinamice*). Menționăm că algoritmul poate fi descris și implementat și iterativ, utilizând

varianta *bottom-up* a strategiei Divide et Impera, în care subproblemele se rezolvă în ordinea crescătoare a dimensiunii lor. Această variantă este mult mai eficientă în cazul *programării dinamice*, când subproblemele se suprapun, dar în cazul sortării prin interclasare are aceeași complexitate ca și varianta noastră, de tip *top-down*. Mai multe detalii privind implementarea și complexitatea acestor două forme, precum și a altor variante ale algoritmului de sortare prin interclasare, pot fi găsite în [38] și [39].

Propoziția 5.5.4 (complexitatea algoritmului de sortare prin interclasare). *Algoritmul de sortare prin interclasare a unui vector cu n componente are complexitatea $\Theta(n \log_2 n)$.*

Demonstrație. Fie $T(n)$ timpul de execuție al algoritmului **SORTINT**($A, 1, n$). Vom demonstra că $T(n) = \Theta(n \log_2 n)$ prin două metode.

Metoda 1) Vom estima efectiv timpul de execuție $T(n)$. Vom număra numai operațiile de **comparație de chei** (comparații în care intervin elemente ale vectorului) și de **deplasări** (atribuiri în care intervin elemente ale vectorului). Celelalte operații care se efectuează au același ordin de creștere cu cele pe care le analizăm. Avem

$$T(n) = c_1 \cdot N_c(n) + c_2 \cdot N_d(n), \quad (5.5.4)$$

unde $N_c(n)$ și $N_d(n)$ reprezintă numărul de *comparații de chei*, respectiv numărul de *deplasări* efectuate de algoritm, iar $c_1, c_2 > 0$ sunt constante ce reprezintă costurile în timp ale executării unei *comparații de chei* și, respectiv, ale unei *deplasări*.

Conform descrierii recursive a algoritmului, rezultă că numerele $N_c(n)$ și $N_d(n)$ verifică, respectiv, relațiile de recurență

$$N_c(n) = \begin{cases} 0, & \text{dacă } n = 1, \\ N_c\left(\left\lceil \frac{n}{2} \right\rceil\right) + N_c\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + \overline{N}_c\left(\left\lceil \frac{n}{2} \right\rceil, \left\lfloor \frac{n}{2} \right\rfloor\right), & \text{dacă } n > 1, \end{cases} \quad (5.5.5)$$

$$N_d(n) = \begin{cases} 0, & \text{dacă } n = 1, \\ N_d\left(\left\lceil \frac{n}{2} \right\rceil\right) + N_d\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + \overline{N}_d\left(\left\lceil \frac{n}{2} \right\rceil, \left\lfloor \frac{n}{2} \right\rfloor\right), & \text{dacă } n > 1, \end{cases} \quad (5.5.6)$$

unde $\overline{N}_c\left(\left\lceil \frac{n}{2} \right\rceil, \left\lfloor \frac{n}{2} \right\rfloor\right)$ și $\overline{N}_d\left(\left\lceil \frac{n}{2} \right\rceil, \left\lfloor \frac{n}{2} \right\rfloor\right)$ reprezintă numărul de *comparații de chei*, respectiv numărul de *deplasări* efectuate la interclasarea celor doi subvectori sortați, $(a_1, \dots, a_m)$ și $(a_{m+1}, \dots, a_n)$, cu $m = \left\lfloor \frac{n+1}{2} \right\rfloor = \left\lceil \frac{n}{2} \right\rceil$, de dimensiuni $m = \left\lceil \frac{n}{2} \right\rceil$, respectiv $n - m = n - \left\lceil \frac{n}{2} \right\rceil = \left\lfloor \frac{n}{2} \right\rfloor$.

Conform descrierii procedurii de interclasare, la interclasarea celor doi subvectorilor avem:

- Numărul de *deplasări* $\overline{N}_d \left(\left\lceil \frac{n}{2} \right\rceil, \left\lfloor \frac{n}{2} \right\rfloor \right)$ este egal cu dublul numărul de componente ale lui A (fiecare componentă - fie că se află în primul subvector, fie că se află în al doilea subvector - este mutată o dată în vectorul intermediar B și o dată adusă înapoi în A), deci

$$\overline{N}_d \left(\left\lceil \frac{n}{2} \right\rceil, \left\lfloor \frac{n}{2} \right\rfloor \right) = 2 \left(\left\lceil \frac{n}{2} \right\rceil + \left\lfloor \frac{n}{2} \right\rfloor \right) = 2n. \quad (5.5.7)$$

- Numărul de *comparații de chei* $\overline{N}_c \left(\left\lceil \frac{n}{2} \right\rceil, \left\lfloor \frac{n}{2} \right\rfloor \right)$ este egal cu numărul de execuții ale ciclului **while**. Cu fiecare astfel de comparație are loc și o copiere în vectorul B a unei componente din A , adică o *deplasare*. La ieșirea din ciclul **while**, componentele unuia dintre cei doi subvectori ai lui A au fost toate copiate în B , iar în celălalt subvector au mai rămas r componente necopiate în B , care vor fi și ele copiate în B , dar după ieșirea din acest ciclu. Rezultă că

$$\overline{N}_c \left(\left\lceil \frac{n}{2} \right\rceil, \left\lfloor \frac{n}{2} \right\rfloor \right) = n - r.$$

Evident,

$$1 \leq r \leq \max \left\{ \left\lceil \frac{n}{2} \right\rceil, \left\lfloor \frac{n}{2} \right\rfloor \right\} = \left\lceil \frac{n}{2} \right\rceil,$$

deci

$$\overline{N}_c \left(\left\lceil \frac{n}{2} \right\rceil, \left\lfloor \frac{n}{2} \right\rfloor \right) \in \left\{ \left\lfloor \frac{n}{2} \right\rfloor, \left\lfloor \frac{n}{2} \right\rfloor + 1, \dots, n - 1 \right\}. \quad (5.5.8)$$

Notând cu $N_c^+(n)$ numărul de *comparații de chei în cazul cel mai defavorabil*, din (5.5.5) și (5.5.8) rezultă relația de recurență

$$N_c^+(n) = \begin{cases} 0, & \text{dacă } n = 1, \\ N_c^+ \left(\left\lceil \frac{n}{2} \right\rceil \right) + N_c^+ \left(\left\lfloor \frac{n}{2} \right\rfloor \right) + n - 1, & \text{dacă } n > 1. \end{cases} \quad (5.5.9)$$

Rezolvăm această relație de recurență.

Pentru $n = \text{număr par}$, adică $n = 2m$, $m \geq 1$, avem

$$\begin{aligned} N_c^+(n+1) - N_c^+(n) &= N_c^+(2m+1) - N_c^+(2m) \\ &= (N_c^+(m+1) + N_c^+(m) + 2m) - (N_c^+(m) + N_c^+(m) + 2m - 1) \\ &= N_c^+(m+1) - N_c^+(m) + 1, \end{aligned}$$

iar pentru $n = \text{număr impar}$, adică $n = 2m - 1$, $m \geq 2$, avem

$$\begin{aligned} N_c^+(n+1) - N_c^+(n) &= N_c^+(2m) - N_c^+(2m-1) \\ &= (N_c^+(m) + N_c^+(m) + 2m - 1) - (N_c^+(m) + N_c^+(m-1) + 2m - 2) \\ &= N_c^+(m) - N_c^+(m-1) + 1, \end{aligned}$$

deci pentru orice $n > 1$ avem

$$N_c^+(n+1) - N_c^+(n) = N_c^+ \left(\left\lfloor \frac{n}{2} \right\rfloor + 1 \right) - N_c^+ \left(\left\lfloor \frac{n}{2} \right\rfloor \right) + 1.$$

Notând

$$f(n) = N_c^+(n+1) - N_c^+(n), \quad \forall n \in \mathbb{N}^*, \quad (5.5.10)$$

obținem că

$$f(n) = \begin{cases} 1, & \text{dacă } n = 1, \\ f \left(\left\lfloor \frac{n}{2} \right\rfloor \right) + 1, & \text{dacă } n > 1. \end{cases} \quad (5.5.11)$$

Demonstrăm că

$$f(n) = \lceil \log_2(n+1) \rceil, \quad \forall n \in \mathbb{N}^*, \quad (5.5.12)$$

prin inducție.

Pentru $n = 1$ avem $f(1) = 1 = \lceil \log_2 2 \rceil$.

Presupunem egalitatea adevărată pentru orice număr din mulțimea $\{1, \dots, n-1\}$ și o demonstrăm pentru n ($n \geq 2$). Într-adevăr, folosind relația de recurență (5.5.11) și ipoteza de inducție pentru $\left\lfloor \frac{n}{2} \right\rfloor$ avem

$$f(n) = f \left(\left\lfloor \frac{n}{2} \right\rfloor \right) + 1 = \lceil \log_2 \left(\left\lfloor \frac{n}{2} \right\rfloor + 1 \right) \rceil + 1.$$

Dar, notând $\lceil \log_2(n+1) \rceil = k$, avem $k \in \mathbb{N}^*$, $k \geq 2$ și, succesiv:

$$\begin{aligned} k-1 &< \log_2(n+1) \leq k; \quad 2^{k-1} < n+1 \leq 2^k; \\ 2^{k-1} &\leq n < 2^k; \quad 2^{k-2} \leq \frac{n}{2} < 2^{k-1}; \\ 2^{k-2} &\leq \left\lfloor \frac{n}{2} \right\rfloor < 2^{k-1}; \quad 2^{k-2} < \left\lfloor \frac{n}{2} \right\rfloor + 1 \leq 2^{k-1}; \\ k-2 &< \log_2 \left(\left\lfloor \frac{n}{2} \right\rfloor + 1 \right) \leq k-1; \\ \lceil \log_2 \left(\left\lfloor \frac{n}{2} \right\rfloor + 1 \right) \rceil &= k-1, \end{aligned}$$

deci

$$f(n) = k-1+1 = k = \lceil \log_2(n+1) \rceil,$$

ceea ce încheie demonstrația prin inducție a relației (5.5.12).

Din (5.5.10) și (5.5.12) rezultă că

$$N_c^+(n+1) - N_c^+(n) = \lceil \log_2(n+1) \rceil, \forall n \in \mathbb{N}^*.$$

Aplicând succesiv această relație avem

$$\begin{aligned} N_c^+(n) &= N_c^+(n-1) + \lceil \log_2 n \rceil \\ &= N_c^+(n-2) + \lceil \log_2(n-1) \rceil + \lceil \log_2 n \rceil \\ &\dots \\ &= N_c^+(1) + \lceil \log_2 2 \rceil + \lceil \log_2 3 \rceil + \dots + \lceil \log_2 n \rceil. \end{aligned}$$

Cum $N_c^+(1) = 0$, obținem că

$$N_c^+(n) = \lceil \log_2 1 \rceil + \lceil \log_2 2 \rceil + \dots + \lceil \log_2 n \rceil, \forall n \in \mathbb{N}^*. \quad (5.5.13)$$

Conform Lemei 5.5.3, rezultă că

$$N_c^+(n) = \sum_{i=1}^n \lceil \log_2 i \rceil = n \lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1, \forall n \in \mathbb{N}^*.$$

Dar

$$\log_2 n \leq \lceil \log_2 n \rceil < \log_2 n + 1,$$

deci

$$n \leq 2^{\lceil \log_2 n \rceil} < 2n.$$

Astfel rezultă că în *cazul cel mai defavorabil* numărul de *comparații de chei* verifică inegalitățile

$$n \log_2 n - 2n + 1 < N_c^+(n) < n \log_2 n + 1, \forall n \in \mathbb{N}^*. \quad (5.5.14)$$

Notând cu $N_c^-(n)$ numărul de *comparații de chei în cazul cel mai favorabil*, din (5.5.5) și (5.5.8) rezultă relația de recurență

$$N_c^-(n) = \begin{cases} 0, & \text{dacă } n = 1, \\ N_c^-\left(\left\lceil \frac{n}{2} \right\rceil\right) + N_c^-\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + \left\lfloor \frac{n}{2} \right\rfloor, & \text{dacă } n > 1. \end{cases} \quad (5.5.15)$$

Demonstrăm că șirul $(N_c^-(n))_{n \geq 1}$ este crescător, adică

$$N_c^-(n+1) \geq N_c^-(n), \forall n \in \mathbb{N}^*,$$

prin inducție.

Pentru $n = 1$ avem $N_c^-(2) = 1 > 0 = N_c^-(1)$.

Presupunem inegalitatea adevărată pentru orice număr din mulțimea $\{1, \dots, n-1\}$ și o demonstrăm pentru n ($n \geq 2$). Într-adevăr, folosind relația de recurență (5.5.15), ipoteza de inducție pentru $\left\lceil \frac{n}{2} \right\rceil$ când n este par, respectiv pentru $\left\lfloor \frac{n}{2} \right\rfloor$ când n este impar, precum și inegalitatea evidentă $\left\lfloor \frac{n+1}{2} \right\rfloor \geq \left\lfloor \frac{n}{2} \right\rfloor$, avem

$$\begin{aligned} N_c^-(n+1) &= N_c^- \left(\left\lceil \frac{n+1}{2} \right\rceil \right) + N_c^- \left(\left\lfloor \frac{n+1}{2} \right\rfloor \right) + \left\lfloor \frac{n+1}{2} \right\rfloor \\ &\geq N_c^- \left(\left\lceil \frac{n}{2} \right\rceil \right) + N_c^- \left(\left\lfloor \frac{n}{2} \right\rfloor \right) + \left\lfloor \frac{n}{2} \right\rfloor \\ &= N_c^-(n). \end{aligned}$$

Demonstrația prin inducție este încheiată.

Fie $n \in \mathbb{N}^*$ și

$$t = \lfloor \log_2 n \rfloor, \quad (5.5.16)$$

deci $t \in \mathbb{N}$ și $t \leq \log_2 n < t+1$, deci $2^t \leq n < 2^{t+1}$. Deoarece șirul $(N_c^-(n))_{n \geq 1}$ este crescător, rezultă că

$$N_c^-(n) \geq N_c^-(2^t). \quad (5.5.17)$$

Dar, conform relației de recurență (5.5.15) avem

$$N_c^-(2^t) = 2N_c^-(2^{t-1}) + 2^{t-1}, \quad \forall t \in \mathbb{N}^*.$$

Rescriem această egalitate sub forma

$$N_c^-(2^t) - t \cdot 2^{t-1} = 2(N_c^-(2^{t-1}) - (t-1)2^{t-2}), \quad \forall t \in \mathbb{N}^*,$$

Aplicând succesiv această relație avem

$$\begin{aligned} N_c^-(2^t) - t \cdot 2^{t-1} &= 2(N_c^-(2^{t-1}) - (t-1)2^{t-2}) \\ &= 2^2(N_c^-(2^{t-2}) - (t-2)2^{t-3}) \\ &\dots \\ &= 2^t(N_c^-(2^{t-t}) - (t-t)2^{t-t-1}) \end{aligned}$$

Cum $N_c^-(1) = 0$ obținem că $N_c^-(2^t) - t \cdot 2^{t-1} = 0$, deci

$$N_c^-(2^t) = t \cdot 2^{t-1}, \quad \forall t \in \mathbb{N}. \quad (5.5.18)$$

Din (5.5.17), (5.5.18) și (5.5.16) rezultă că în *cazul cel mai favorabil* numărul de *comparații de chei* verifică inegalitățile

$$N_c^-(n) \geq \lfloor \log_2 n \rfloor \cdot 2^{\lfloor \log_2 n \rfloor - 1} > \frac{1}{4} \cdot n \log_2 n - \frac{1}{4} \cdot n, \quad \forall n \in \mathbb{N}^*. \quad (5.5.19)$$

Din (5.5.14) și (5.5.19) rezultă că numărul de *comparații de chei* verifică inegalitățile

$$\frac{1}{4} \cdot n \log_2 n - \frac{1}{4} \cdot n < N_c(n) < n \log_2 n + 1, \quad \forall n \in \mathbb{N}^*. \quad (5.5.20)$$

Din relațiile (5.5.6) și (5.5.7) rezultă că numărul de *deplasări* efectuate de algoritm verifică relația de recurență

$$N_d(n) = \begin{cases} 0, & \text{dacă } n = 1, \\ N_d\left(\left\lceil \frac{n}{2} \right\rceil\right) + N_d\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 2n, & \text{dacă } n > 1. \end{cases}$$

Această relație se rezolvă analog relației (5.5.9), obținându-se

$$N_d(n) = 2(n \lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + n), \quad \forall n \in \mathbb{N}^*,$$

deci

$$2n \log_2 n - 2n < N_d(n) < 2n \log_2 n + 2n, \quad \forall n \in \mathbb{N}^*. \quad (5.5.21)$$

Din relațiile (5.5.4), (5.5.20) și (5.5.21) rezultă că $T(n) = \Theta(n \log_2 n)$.

Metoda 2) Vom aplica Teorema Master. Conform descrierii recursive a algoritmului și utilizând complexitatea procedurii de interclasare (formulele pentru $\overline{N}_c\left(\left\lceil \frac{n}{2} \right\rceil, \left\lfloor \frac{n}{2} \right\rfloor\right)$ și $\overline{N}_d\left(\left\lceil \frac{n}{2} \right\rceil, \left\lfloor \frac{n}{2} \right\rfloor\right)$), timpul de execuție $T(n)$ verifică relația de recurență

$$T(n) = 2T(n/2) + \Theta(n), \quad \forall n > 1,$$

cu convențiile de notație din Teorema Master. Conform Cazului 2 al acelei teoreme (pentru $a = 2$, $b = 2$, $c = 1 = \log_b a$ și $k = 0$) rezultă că $T(n) = \Theta(n \log_2 n)$. $\square$

Următorul rezultat este o consecință directă a propoziției anterioare și a Teoremei 5.5.1.

Corolarul 5.5.2 (optimalitatea algoritmului de sortare prin interclasare). *Algoritmul de sortare prin interclasare este asimptotic-optimal (în clasa algoritmilor bazați pe comparații de chei), atât în raport cu timpul de execuție în cazul cel mai defavorabil, cât și în raport cu timpul mediu de execuție.*

Observația 5.5.9. Mai mult, conform relațiilor (5.5.14) și (5.5.20) și Teoremei 5.5.1 rezultă că, exact ca la algoritmi optimi de sortare bazați pe comparații de chei, numărul de comparații de chei efectuate de algoritmul de sortare prin interclasare a unui vector cu n componente este chiar asimptotic echivalent cu $n \log_2 n$, atât în cazul cel mai defavorabil, cât și în cazul mediu (cazul timpului mediu de execuție).

Totuși, conform Corolarului 5.5.1, **algoritmul de sortare prin interclasare nu este optim, nici în raport cu timpul mediu de execuție**, deoarece arborele extins de sortare asociat nu are întotdeauna toate nodurile externe situate doar pe ultimul și, eventual, pe penultimul nivel, **nici în raport cu timpul de execuție în cazul cel mai defavorabil**, deoarece înălțimea acestui arbore nu este întotdeauna egală cu $\lceil \log_2 n! \rceil$. De exemplu, pentru $n = 5$ există noduri externe nesituate pe același nivel sau pe două nivele consecutive, iar înălțimea arborele extins de sortare asociat este mai mare decât $\lceil \log_2 5! \rceil = \lceil \log_2 120 \rceil = 7$, așa cum reiese din Figura 5.5.4, în care este reprezentată doar o parte a arborelui extins de sortare.

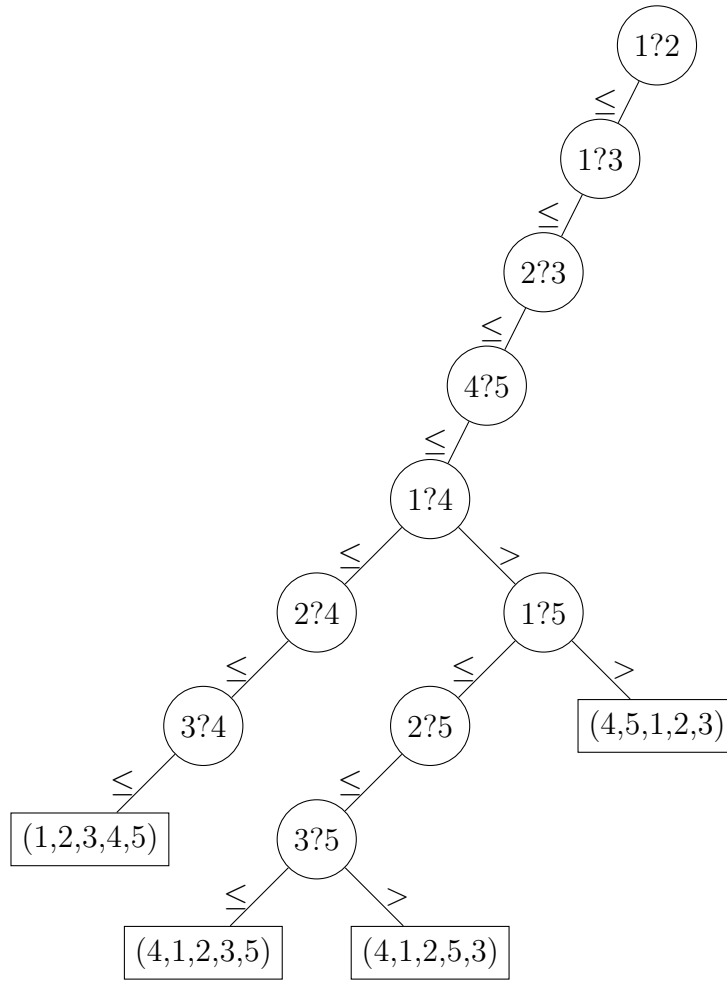


Figura 5.5.4:

Observația 5.5.10. O altă justificare a neoptimalității (pure a) algoritmului de sortare prin interclasare este următoarea: în cazul cel mai defavorabil la sortarea unui vector cu n componente, numărul de *comparații de chei* efectuate de algoritmul mergesort este, conform (5.5.13),

$$N_c^+(n) = \lceil \log_2 1 \rceil + \lceil \log_2 2 \rceil + \dots + \lceil \log_2 n \rceil,$$

iar numărul de *comparații de chei* efectuate de orice algoritm optim este, conform Corolarului 5.5.1,

$$N_{def}(n) = \lceil \log_2 n! \rceil = \lceil \log_2 1 + \log_2 2 + \dots + \log_2 n \rceil.$$

Evident, pentru orice numere reale x și y avem

$$\lceil x \rceil + \lceil y \rceil \geq \lceil x + y \rceil \quad (5.5.22)$$

(deoarece $\lceil x \rceil \geq x$, $\lceil y \rceil \geq y$, $\lceil x \rceil + \lceil y \rceil \geq x + y$ și $\lceil x \rceil + \lceil y \rceil \in \mathbb{Z}$, deci $\lceil x \rceil + \lceil y \rceil = \lceil \lceil x \rceil + \lceil y \rceil \rceil \geq \lceil x + y \rceil$). Folosind (5.5.22) rezultă că

$$N_c^+(n) = \lceil \log_2 1 \rceil + \lceil \log_2 2 \rceil + \dots + \lceil \log_2 n \rceil \geq \lceil \log_2 1 + \log_2 2 + \dots + \log_2 n \rceil = N_{def}(n),$$

ceea ce era de așteptat, conform optimalității invocate! Mai mult, avem $N_c^+(5) = \lceil \log_2 1 \rceil + \lceil \log_2 2 \rceil + \lceil \log_2 3 \rceil + \lceil \log_2 4 \rceil + \lceil \log_2 5 \rceil = 0 + 1 + 2 + 2 + 3 = 8$, iar $N_{def}(5) = \lceil \log_2 5! \rceil = \lceil \log_2 120 \rceil = 7$, deci

$$N_c^+(5) > N_{def}(5).$$

Folosind această inegalitate strictă și inegalitatea (5.5.22), rezultă că pentru $n \geq 5$ avem

$$\begin{aligned} N_c^+(n) &= (\lceil \log_2 1 \rceil + \dots + \lceil \log_2 5 \rceil) + (\lceil \log_2 6 \rceil + \dots + \lceil \log_2 n \rceil) \\ &> \lceil \log_2 1 + \dots + \log_2 5 \rceil + \lceil \log_2 6 + \dots + \log_2 n \rceil \\ &\geq \lceil \log_2 1 + \dots + \log_2 5 + \log_2 6 + \dots + \log_2 n \rceil = N_{def}(n), \end{aligned}$$

deci

$$N_c^+(n) > N_{def}(n), \quad \forall n \geq 5.$$

Prin urmare algoritmul mergesort nu este optim în raport cu timpul de execuție în cazul cel mai defavorabil. Cum optimalitatea în cazul timpului mediu de execuție implică optimalitatea în cazul cel mai defavorabil (afirmație evidentă din noțiunile de *caz mediu*, *caz defavorabil* și *algoritm optim*, dar care rezultă imediat și conform Corolarului 5.5.1), rezultă că algoritmul mergesort nu este optim nici în raport cu timpul mediu de execuție.

Observația 5.5.11. Complexitatea algoritmului mergesort este una dintre cele mai bune din punct de vedere al numărului de comparații de chei, în clasa algoritmilor de sortare bazați pe astfel de comparații. Un dezavantaj este însă faptul că se utilizează un vector suplimentar de lucru, care poate avea dimensiunea foarte mare (fiind egală cu cea a vectorului ce trebuie sortat).

5.5.6 Sortarea rapidă (quicksort)

Algoritmul de sortare rapidă (quicksort), inventat de către celebrul informatician britanic Sir Charles Antony Richard Hoare în anul 1959, este considerat - în urma experienței practice - ca fiind cel mai performant algoritm de sortare.

Algoritmul 5.5.4 (Sortarea rapidă (quicksort)). Pentru a sorta crescător un vector dat $A = (a_1, a_2, \dots, a_n)$, $n \geq 1$, utilizăm următorul procedeu specific metodei de programare *Divide et Impera*:

- Se alege un element, de exemplu $x = a_1$, care se duce, prin inter-schimbări succesive, pe o poziție k , astfel încât după această etapă să avem $a_k = x$ și

$$a_i \leq a_k \leq a_j, \forall i, j \text{ cu } 1 \leq i < k < j \leq n.$$

Acest element, devenit acum elementul a_k , se numește **pivot** și are proprietatea că a ajuns pe poziția sa finală în vectorul sortat, deoarece subvectorul $(a_1, \dots, a_{k-1})$ al elementelor din stânga sa conține doar elemente mai mici sau egale cu el, iar subvectorul $(a_{k+1}, \dots, a_n)$ al elementelor din dreapta sa conține doar elemente mai mari sau egale cu el. Acești doi subvectori se numesc **partițiile** în care pivotul a_k împarte vectorul.

Pentru realizarea acestei etape putem proceda astfel:

- se compară succesiv elementul ales, $x = a_1$, cu elementele din dreapta (marginea opusă) $a_n, a_{n-1}, \dots$, iar când se ajunge la un element a_u cu $a_1 > a_u$ atunci se interschimbă $x = a_1$ cu a_u ;
 - se compară apoi succesiv $x = a_u$ cu elementele neanalizate din stânga (marginea opusă) $a_2, a_3, \dots$, iar când se ajunge la un element a_p cu $a_p > a_u$ atunci se interschimbă $x = a_u$ cu a_p ;
 - se compară apoi din nou $x = a_p$ cu elementele neanalizate din dreapta (marginea opusă) $a_{u-1}, a_{u-2}, \dots$, iar când se ajunge la un nou element a_u cu $a_p > a_u$ atunci se interschimbă $x = a_p$ cu noul a_u ;
 - se compară apoi din nou $x = a_u$ cu elementele neanalizate din stânga (marginea opusă) $a_{p+1}, a_{p+2}, \dots$, iar când se ajunge la un nou element a_p cu $a_p > a_u$ atunci din nou se interschimbă $x = a_u$ cu noul a_p ;
 - se continuă acest procedeu cât timp subsecvența $(a_p, \dots, a_u)$ formată din x , care este a_p sau a_u , și elemente rămase neanalizate verifică inegalitatea $p < u$, adică are cel puțin două elemente;
 - la final se ajunge ca $p = u$ și astfel $x = a_p = a_u$, deci $k = p = u$ este poziția pivotului x și procedeul de partiționare (determinarea poziției pivotului și astfel a celor două partiții în care acesta împarte vectorul) este încheiat.
- Se sortează crescător cele două partiții $(a_1, \dots, a_{k-1})$ și $(a_{k+1}, \dots, a_n)$, prin același procedeu, dacă au cel puțin două elemente. Partițiile vide și cele care au un singur element sunt, bineînțeles, deja sortate.

Observația 5.5.12. Spre deosebire de *algoritmul mergesort*, unde împărțirea vectorului în doi subvectori (*etapa „Divide”* din strategia Divide et Impera) se realiza instantaneu prin intermediul elementului median, la sortarea rapidă această etapă necesită rearanjarea elementelor pentru a obține pivotul a_k și cele două partiții. Pe de altă parte, după sortarea independentă a celor două partiții vectorul A va fi acum sortat, spre deosebire de *algoritmul mergesort*, unde după sortarea celor doi subvectori urma interclasarea lor (*etapa „Combină”* din strategia Divide et Impera, care astfel nu mai este necesară la *algoritmul quicksort*).

Descrierea în pseudocod a algoritmului are următoarea formă.

```

QUICKSORT( $A, p, u$ ):           // sortarea secvenței ( $a_p, \dots, a_u$ )
if  $p < u$  then
     $k \leftarrow$  PIVOT( $A, p, u$ );           // se determină, rearanjând
                                           // elementele, poziția  $k$  a pivotului
    QUICKSORT( $A, p, k - 1$ );           // sortarea partiției
                                           // ( $a_p, \dots, a_{k-1}$ )
    QUICKSORT( $A, k + 1, u$ );           // sortarea partiției
                                           // ( $a_{k+1}, \dots, a_u$ )

```

unde procedura de partiționare este:

```

PIVOT( $A, p, u$ ):
 $d \leftarrow 0$ ;           //  $d$  = pasul de deplasare spre dreapta a lui  $p$ 
                        //  $1 - d$  = pasul de deplasare spre stânga a lui  $u$ 
                        //  $d = 0$  corespunde deplasării lui  $u$  (marginea din
                        // dreapta), iar  $d = 1$  corespunde
                        // deplasării lui  $p$  (marginea din stânga)

while  $p < u$  do
    if  $A[p] > A[u]$  then
         $A[p] \leftrightarrow A[u]$ ;           // interschimbare
         $d \leftarrow 1 - d$ ;           // se comută marginea unde se face
                                           // deplasarea
     $p \leftarrow p + d$ ;  $u \leftarrow u - 1 + d$ ;           // deplasarea marginii
                                           // corespunzătoare

    // s-a ajuns la  $p = u$  = poziția pivotului

return  $p$ ;

```

Apel:

```

QUICKSORT( $A, 1, n$ )           // pentru sortarea întregului vector
//  $A = (a_1, \dots, a_n)$ .

```

Observația 5.5.13. În procedura de partiționare **PIVOT**(A, p, u), elementul ales pentru a deveni pivot este cel situat inițial la marginea din stânga, a_p . Dacă inițializarea $d \leftarrow 0$ se înlocuiește cu $d \leftarrow 1$, atunci elementul ce va deveni pivot este cel situat inițial la marginea din dreapta, a_u .

Exemplul 5.5.5. În Figura 5.5.5 este reprezentat arborele extins de sortare asociat algoritmului de sortare rapidă pentru $n = 3$. Observăm că algoritmul nu conține comparații de chei redundante.

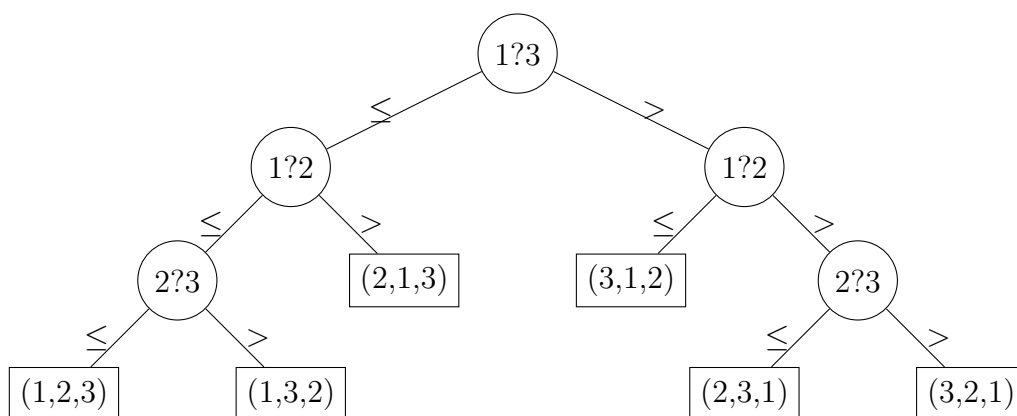


Figura 5.5.5:

Propoziția 5.5.5 (complexitatea algoritmului de sortare rapidă). Algoritmul de sortare rapidă a unui vector cu n componente are complexitatea

$$\begin{cases} \Theta(n^2), & \text{pentru timpul de execuție în cazul cel mai defavorabil,} \\ \Theta(n \log_2 n), & \text{pentru timpul de execuție în cazul cel mai favorabil,} \\ \Theta(n \log_2 n), & \text{pentru timpul mediu de execuție.} \end{cases}$$

Demonstrație. Vom evalua doar numărul de **comparație de chei** (comparații în care intervin elemente ale vectorului), celelalte operații nedepășind ordinul de creștere al acestora.

Fie $N(n)$ numărul de comparații de chei efectuate de algoritm pentru un vector cu n componente.

În procedura **PIVOT**(A, p, u) se efectuează $u - p$ comparații de chei.

Conform descrierii recursive a algoritmului, rezultă că numerele $N(n)$ verifică relația de recurență

$$N(n) = \begin{cases} 0, & \text{dacă } n \in \{0, 1\}, \\ N(k-1) + N(n-k) + n-1, & \text{dacă } n \geq 2, \end{cases} \quad (5.5.23)$$

unde k este indicele pivotului, $k \in \{1, 2, \dots, n\}$.

Analizăm întâi cazurile extreme, în raport cu diferența absolută a dimensiunilor celor două partiții rezultate după fixarea pivotului. Această diferență este

$$|(n - k) - (k - 1)| = |n + 1 - 2k|.$$

Cazul 1) Notăm cu $N^+(n)$ numărul de comparații de chei efectuate de algoritm în cazul în care la fiecare partiționare cele două partiții obținute sunt cât mai „neechilibrate”, adică diferența absolută a dimensiunilor lor este maximă. Evident, $|n + 1 - 2k|$ este maxim pentru $k \in \{1, n\}$, deci

$$N^+(n) = \begin{cases} 0, & \text{dacă } n \in \{0, 1\}, \\ N^+(n-1) + n - 1, & \text{dacă } n \geq 2 \end{cases}$$

(deoarece $N^+(0) = 0$). Această situație apare, de exemplu, atunci când vectorul A este de la început sortat crescător (deoarece procedura **PIVOT** va fi apelată succesiv pentru perechile de indici $(1, n), (2, n), \dots, (n-1, n)$). Avem

$$\begin{aligned} N^+(n) &= N^+(n-1) + n - 1 \\ &= N^+(n-2) + (n-2) + (n-1) \\ &\dots \\ &= N^+(1) + 1 + 2 + \dots + (n-2) + (n-1), \end{aligned}$$

deci

$$N^+(n) = \frac{n(n-1)}{2}, \quad \forall n \in \mathbb{N}. \quad (5.5.24)$$

Demonstrăm că

$$N(n) \leq N^+(n), \quad \forall n \in \mathbb{N}$$

prin inducție.

Pentru $n \in \{0, 1\}$ avem $N(n) = N^+(n) = 0$.

Presupunem inegalitatea adevărată pentru orice $i \in \{0, 1, \dots, n-1\}$ și o demonstrăm pentru n ($n \geq 2$). Într-adevăr, folosind relația de recurență (5.5.23), ipoteza de inducție pentru $k-1$ și $n-k$ și relația (5.5.24) avem

$$\begin{aligned} N(n) &= N(k-1) + N(n-k) + n - 1 \\ &\leq N^+(k-1) + N^+(n-k) + n - 1 \\ &= \frac{(k-1)(k-2)}{2} + \frac{(n-k)(n-k-1)}{2} + n - 1 \\ &= \frac{n(n-1)}{2} - (k-1)(n-k) \\ &\leq \frac{n(n-1)}{2}. \end{aligned}$$

Mai mult, egalitatea are loc dacă și numai dacă $k \in \{1, n\}$ (la fiecare partiționare!).

Astfel Cazul 1) este cel mai defavorabil, având numărul de comparații $N^+(n) = \frac{n(n-1)}{2}$ și, prin urmare, complexitatea $\Theta(n^2)$.

Cazul 2) Notăm cu $N^-(n)$ numărul de comparații de chei efectuate de algoritm în cazul în care la fiecare partiționare cele două partiții obținute sunt cât mai „echilibrate”, adică diferența absolută a dimensiunilor lor este minimă. Evident, $|n+1-2k|$ este minim pentru $k \in \left\{ \left\lfloor \frac{n+1}{2} \right\rfloor, \left\lceil \frac{n+1}{2} \right\rceil \right\}$, deci

$$N^-(n) = \begin{cases} 0, & \text{dacă } n \in \{0, 1\}, \\ N^-\left(\left\lceil \frac{n-1}{2} \right\rceil\right) + N^-\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right) + n - 1, & \text{dacă } n \geq 2, \end{cases} \quad (5.5.25)$$

Rezolvăm relația de recurență (5.5.25). Pentru $n =$ număr par, adică $n = 2m$, $m \geq 1$, avem

$$\begin{aligned} N^-(n+1) - N^-(n) &= N^-(2m+1) - N^-(2m) \\ &= (N^-(m) + N^-(m) + 2m) - (N^-(m) + N^-(m-1) + 2m - 1) \\ &= N^-(m) - N^-(m-1) + 1, \end{aligned}$$

iar pentru $n =$ număr impar, adică $n = 2m+1$, $m \geq 1$, avem

$$\begin{aligned} N^-(n+1) - N^-(n) &= N^-(2m+2) - N^-(2m+1) \\ &= (N^-(m+1) + N^-(m) + 2m+1) - (N^-(m) + N^-(m) + 2m) \\ &= N^-(m+1) - N^-(m) + 1, \end{aligned}$$

deci pentru orice $n \geq 2$ avem

$$N^-(n+1) - N^-(n) = N^-\left(\left\lfloor \frac{n-1}{2} \right\rfloor + 1\right) - N^-\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right) + 1,$$

egalitatea fiind evident valabilă și pentru $n = 1$. Notând

$$v(n) = N^-(n+1) - N^-(n), \quad \forall n \in \mathbb{N}, \quad (5.5.26)$$

obținem că

$$v(n) = \begin{cases} 0, & \text{dacă } n = 0, \\ v\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right) + 1, & \text{dacă } n \geq 1. \end{cases} \quad (5.5.27)$$

Demonstrăm că

$$v(n) = \lceil \log_2(n+2) \rceil - 1, \quad \forall n \in \mathbb{N}, \quad (5.5.28)$$

prin inducție.

Pentru $n = 0$ avem $v(0) = 0 = \lceil \log_2 2 \rceil - 1$.

Presupunem egalitatea adevărată pentru orice $i \in \{0, 1, \dots, n-1\}$ și o demonstrăm pentru n ($n \geq 1$). Într-adevăr, folosind relația de recurență (5.5.27) și ipoteza de inducție pentru $\left\lfloor \frac{n-1}{2} \right\rfloor$ avem

$$v(n) = v\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right) + 1 = \left\lceil \log_2 \left(\left\lfloor \frac{n-1}{2} \right\rfloor + 2\right) \right\rceil = \left\lceil \log_2 \left(\left\lfloor \frac{n+1}{2} \right\rfloor + 1\right) \right\rceil.$$

Dar, notând $\lceil \log_2(n+2) \rceil = r$, avem $r \in \mathbb{N}^*$, $r \geq 2$ și, succesiv:

$$\begin{aligned} r-1 &< \log_2(n+2) \leq r; \quad 2^{r-1} < n+2 \leq 2^r; \\ 2^{r-1} &\leq n+1 < 2^r; \quad 2^{r-2} \leq \frac{n+1}{2} < 2^{r-1}; \\ 2^{r-2} &\leq \left\lfloor \frac{n+1}{2} \right\rfloor < 2^{r-1}; \quad 2^{r-2} < \left\lfloor \frac{n+1}{2} \right\rfloor + 1 \leq 2^{r-1}; \\ r-2 &< \log_2 \left(\left\lfloor \frac{n+1}{2} \right\rfloor + 1\right) \leq r-1; \\ \left\lceil \log_2 \left(\left\lfloor \frac{n+1}{2} \right\rfloor + 1\right) \right\rceil &= r-1, \end{aligned}$$

deci

$$v(n) = r-1 = \lceil \log_2(n+2) \rceil - 1,$$

ceea ce încheie demonstrația prin inducție a relației (5.5.28).

Din (5.5.26) și (5.5.28) rezultă că

$$N^-(n+1) - N^-(n) = \lceil \log_2(n+2) \rceil - 1, \quad \forall n \in \mathbb{N}.$$

Aplicând succesiv această relație avem

$$\begin{aligned} N^-(n) &= N^-(n-1) + \lceil \log_2(n+1) \rceil - 1 \\ &= N^-(n-2) + \lceil \log_2 n \rceil + \lceil \log_2(n+1) \rceil - 2 \\ &\dots \\ &= N^-(0) + \lceil \log_2 2 \rceil + \lceil \log_2 3 \rceil + \dots + \lceil \log_2(n+1) \rceil - n. \end{aligned}$$

Cum $N^-(0) = 0$ obținem că

$$N^-(n) = \lceil \log_2 1 \rceil + \lceil \log_2 2 \rceil + \dots + \lceil \log_2(n+1) \rceil - n, \quad \forall n \in \mathbb{N}. \quad (5.5.29)$$

Conform Lemei 5.5.3, rezultă că

$$N^-(n) = (n+1)\lceil \log_2(n+1) \rceil - 2^{\lceil \log_2(n+1) \rceil} - (n-1), \quad \forall n \in \mathbb{N}. \quad (5.5.30)$$

Demonstrăm că

$$N(n) \geq N^-(n), \quad \forall n \in \mathbb{N} \quad (5.5.31)$$

prin inducție.

Pentru $n \in \{0, 1\}$ avem $N(n) = N^-(n) = 0$.

Presupunem inegalitatea adevărată pentru orice $i \in \{0, 1, \dots, n-1\}$ și o demonstrăm pentru n ($n \geq 2$). Într-adevăr, folosind relațiile de recurență (5.5.23) și (5.5.25) și ipoteza de inducție pentru $k-1$ și $n-k$ avem

$$\begin{aligned} N(n) - N^-(n) &= N(k-1) + N(n-k) - N^-\left(\left\lceil \frac{n-1}{2} \right\rceil\right) - N^-\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right) \\ &\geq \varphi(k), \end{aligned} \quad (5.5.32)$$

unde

$$\varphi(k) = N^-(k-1) + N^-(n-k) - N^-\left(\left\lceil \frac{n-1}{2} \right\rceil\right) - N^-\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right).$$

Deoarece $\varphi(k) = \varphi(n+1-k)$, rezultă că putem presupune că avem $k \leq n+1-k$, adică

$$k \leq \left\lfloor \frac{n-1}{2} \right\rfloor + 1$$

(în caz contrar, notând $k' = n+1-k$ obținem $\varphi(k) = \varphi(k')$ și $k' \in \left\{1, 2, \dots, \left\lfloor \frac{n-1}{2} \right\rfloor + 1\right\}$).

Pentru $k = \left\lfloor \frac{n-1}{2} \right\rfloor + 1$ avem $\varphi(k) = 0$, deci $N(n) - N^-(n) \geq 0$.

Pentru $k \in \left\{1, 2, \dots, \left\lfloor \frac{n-1}{2} \right\rfloor\right\}$, conform (5.5.29) avem

$$\begin{aligned}
\varphi(k) &= N^-(k-1) + N^-(n-k) - N^-\left(\left\lceil \frac{n-1}{2} \right\rceil\right) - N^-\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right) \\
&= \sum_{i=1}^k \lceil \log_2 i \rceil - k + 1 + \sum_{i=1}^{n-k+1} \lceil \log_2 i \rceil - n + k \\
&\quad - \sum_{i=1}^{\lceil \frac{n-1}{2} \rceil + 1} \lceil \log_2 i \rceil + \left\lfloor \frac{n-1}{2} \right\rfloor - \sum_{i=1}^{\lfloor \frac{n-1}{2} \rfloor + 1} \lceil \log_2 i \rceil + \left\lfloor \frac{n-1}{2} \right\rfloor \\
&= \left(\sum_{i=1}^{n-k+1} \lceil \log_2 i \rceil - \sum_{i=1}^{\lceil \frac{n-1}{2} \rceil + 1} \lceil \log_2 i \rceil \right) - \left(\sum_{i=1}^{\lfloor \frac{n-1}{2} \rfloor + 1} \lceil \log_2 i \rceil - \sum_{i=1}^k \lceil \log_2 i \rceil \right) \\
&= \sum_{i=\lceil \frac{n-1}{2} \rceil + 2}^{n-k+1} \lceil \log_2 i \rceil - \sum_{i=k+1}^{\lfloor \frac{n-1}{2} \rfloor + 1} \lceil \log_2 i \rceil \\
&= \sum_{i=1}^{\lfloor \frac{n-1}{2} \rfloor + 1 - k} \left\lceil \log_2 \left(i + \left\lfloor \frac{n-1}{2} \right\rfloor + 1 \right) \right\rceil - \sum_{i=1}^{\lfloor \frac{n-1}{2} \rfloor + 1 - k} \lceil \log_2 (i+k) \rceil \\
&= \sum_{i=1}^{\lfloor \frac{n-1}{2} \rfloor + 1 - k} \left(\left\lceil \log_2 \left(i + \left\lfloor \frac{n-1}{2} \right\rfloor + 1 \right) \right\rceil - \lceil \log_2 (i+k) \rceil \right) \\
&\geq 0.
\end{aligned}$$

Conform (5.5.32) rezultă din nou că $N(n) - N^-(n) \geq 0$ și astfel demonstrația prin inducție a inegalității (5.5.31) este încheiată.

Astfel Cazul 2) este cel mai favorabil, având numărul de comparații dat de relația (5.5.30) și, prin urmare, complexitatea $\Theta(n \log_2 n)$.

Semnalăm că egalitatea poate avea loc în (5.5.31) și pentru valori $k \notin \left\{ \left\lfloor \frac{n+1}{2} \right\rfloor, \left\lceil \frac{n+1}{2} \right\rceil \right\}$, de exemplu pentru $n = 10$ și $k = 4$ ($N^-(3) + N^-(6) = 2 + 8 = 10$, $N^-(4) + N^-(5) = 4 + 6 = 10$).

Cazul 3) Analizăm acum cazul mediu (cazul timpului mediu de execuție). Notăm cu $N_k(n)$ numărul de comparații de chei efectuate de algoritm pentru un vector cu n componente în cazul în care indicele pivotului este k . Evident k poate lua oricare dintre valorile $1, 2, \dots, n$, iar toate aceste situații sunt echiprobabile, deci numărul mediu de comparații de chei are valoarea

$$N_{med}(n) = \frac{\sum_{k=1}^n N_k(n)}{n}.$$

Conform descrierii algoritmului (a se vedea relația (5.5.23)) avem

$$N_k(n) = \begin{cases} 0, & \text{dacă } n \in \{0, 1\}, \\ N_{med}(k-1) + N_{med}(n-k) + n-1, & \text{dacă } n \geq 2, \end{cases}$$

pentru orice $k \in \{1, 2, \dots, n\}$, deci

$$N_{med}(n) = \begin{cases} 0, & \text{dacă } n \in \{0, 1\}, \\ \frac{\sum_{k=1}^n (N_{med}(k-1) + N_{med}(n-k) + n-1)}{n}, & \text{dacă } n \geq 2. \end{cases}$$

Pentru orice $n \geq 2$ avem

$$\begin{aligned} N_{med}(n) &= \frac{\sum_{k=1}^n N_{med}(k-1) + \sum_{k=1}^n N_{med}(n-k) + n(n-1)}{n} \\ &= \frac{2 \sum_{k=1}^n N_{med}(k-1) + n(n-1)}{n}, \end{aligned}$$

deci numărul mediu de comparații de chei verifică relația de recurență

$$N_{med}(n) = \begin{cases} 0, & \text{dacă } n \in \{0, 1\}, \\ \frac{2 \sum_{k=1}^n N_{med}(k-1) + n(n-1)}{n}, & \text{dacă } n \geq 2. \end{cases}$$

Formula dată de a doua ramură este valabilă și pentru $n = 1$. Pentru $n \geq 2$ avem

$$\begin{aligned} nN_{med}(n) &= 2 \sum_{k=1}^n N_{med}(k-1) + n^2 - n, \\ (n-1)N_{med}(n-1) &= 2 \sum_{k=1}^{n-1} N_{med}(k-1) + (n-1)^2 - (n-1), \end{aligned}$$

deci prin scădere obținem

$$nN_{med}(n) - (n-1)N_{med}(n-1) = 2N_{med}(n-1) + 2n - 2,$$

adică

$$nN_{med}(n) = (n+1)N_{med}(n-1) + 2n - 2.$$

Această egalitate poate fi rescrisă sub forma

$$n(N_{med}(n) - 2) = (n + 1)(N_{med}(n - 1) - 2) + 2n.$$

Împărțind prin $n(n + 1)$ obținem

$$\frac{N_{med}(n) - 2}{n + 1} = \frac{N_{med}(n - 1) - 2}{n} + \frac{2}{n + 1}.$$

Aplicând succesiv această relație avem

$$\begin{aligned} \frac{N_{med}(n) - 2}{n + 1} &= \frac{N_{med}(n - 1) - 2}{n} + \frac{2}{n + 1} \\ &= \frac{N_{med}(n - 2) - 2}{n - 1} + \frac{2}{n} + \frac{2}{n + 1} \\ &\dots \\ &= \frac{N_{med}(1) - 2}{2} + \frac{2}{3} + \dots + \frac{2}{n} + \frac{2}{n + 1}. \end{aligned}$$

Cum $N_{med}(1) = 0$, obținem că

$$N_{med}(n) = 2(n + 1) \left(1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \right) - 4n, \quad \forall n \geq 1.$$

Aplicând *Teorema Stolz-Cesàro* pentru șirurile $a_n = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$ și $b_n = \ln n$ (strict crescător și având limita ∞) avem

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}}{\ln n} &= \lim_{n \rightarrow \infty} \frac{a_n}{b_n} = \lim_{n \rightarrow \infty} \frac{a_{n+1} - a_n}{b_{n+1} - b_n} \\ &= \lim_{n \rightarrow \infty} \frac{\frac{1}{n+1}}{\ln(n+1) - \ln n} = \lim_{n \rightarrow \infty} \frac{n}{n+1} \cdot \frac{1}{\ln \left(1 + \frac{1}{n} \right)^n} = \frac{1}{\ln e} = 1. \end{aligned}$$

Deducem că

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{N_{med}(n)}{n \log_2 n} &= \lim_{n \rightarrow \infty} \frac{\ln 2 \cdot N_{med}(n)}{n \ln n} \\ &= \ln 2 \cdot \lim_{n \rightarrow \infty} \left(\frac{2(n+1)}{n} \cdot \frac{1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}}{\ln n} - \frac{4}{\ln n} \right) \\ &= 2 \ln 2, \end{aligned} \tag{5.5.33}$$

deci $N_{med}(n) = \Theta(n \log_2 n)$, □

Următorul rezultat este o consecință directă a propoziției anterioare și a Teoremei 5.5.1.

Corolarul 5.5.3 (optimalitatea algoritmului de sortare rapidă). *Algoritmul de sortare rapidă este asimptotic-optimal (în clasa algoritmilor bazați pe comparații de chei), în raport cu timpul mediu de execuție.*

Observația 5.5.14. În cazul cel mai favorabil, conform relației (5.5.30) rezultă că numărul de comparații de chei efectuate de algoritmul de sortare rapidă este asimptotic echivalent cu $n \log_2 n$.

În cazul timpului mediu de execuție, conform relației (5.5.33) rezultă că numărul de comparații de chei efectuate de algoritmul de sortare rapidă este asimptotic echivalent cu

$$2 \ln 2 \cdot n \log_2 n.$$

Cum conform Teoremei 5.5.1, numărul de comparații de chei efectuate de un algoritm optim de sortare a unui vector cu n componente este asimptotic echivalent cu $n \log_2 n$ (atât în cazul cel mai defavorabil, cât și în cazul timpului mediu de execuție), rezultă că algoritmul de sortare rapidă nu este optim în raport cu timpul mediu de execuție.

Pe de altă parte, cum $2 \ln 2 \simeq 1,39$, rezultă că în cazul mediu numărul de comparații efectuate este mai mare doar cu aproximativ 39% decât în cazul cel mai favorabil (sau decât un algoritm optim).

Observația 5.5.15. În încercarea de a evita cazurile defavorabile ale algoritmului de sortare rapidă, s-au propus variante ale algoritmului în care se urmărește o echilibrare a celor două partiții. De exemplu, o variantă constă în alegerea drept pivot a elementului din mijloc din ordonarea crescătoare a trei elemente: elementele de la cele două margini (a_p și a_u) și elementul median (a_m , $m = \left\lfloor \frac{p+u}{2} \right\rfloor$), variantă numită **metoda medianei din trei** (**median-of-three quicksort**). Mai multe detalii privind implementarea și complexitatea acestei variante, precum și a altor variante ale algoritmului de sortare rapidă, pot fi găsite în [38] și [39].

Tema 6

Metoda programării dinamice

6.1 Descrierea metodei. Algoritmi generali

Ca și metoda Divide et Impera, **metoda programării dinamice** implică rezolvarea unei probleme prin descompunerea în subprobleme de același tip de dimensiuni mai mici, dar de această dată subproblemele nu sunt disjuncte (ca la metoda Divide et Impera), ci se pot suprapune, în sensul că două subprobleme pot avea (sub)subprobleme comune.

Soluția problemei se construiește combinând soluțiile subproblemelor, numite și *subsoluții* (soluții parțiale).

Metoda se adresează problemelor de optimizare care îndeplinesc **principiul optimalității al lui Bellman**: *orice subsoluție a unei soluții optime este la rândul ei o soluție optimă pentru subproblema corespondentă.*

De regulă, subproblemele se rezolvă în ordinea crescătoare a dimensiunii lor (**strategia bottom-up**), iar, pentru a evita repetarea procesului de calcul necesar rezolvării aceleiași subprobleme, soluțiile subproblemelor se memorează temporar (de obicei în structuri de date de tip tablouri) pentru reutilizări la posibile viitoare apeluri ale acestor subprobleme.

Soluțiile optime ale problemelor se exprimă în funcție de soluțiile optime ale subproblemelor lor, de obicei prin intermediul unor **relații de recurență**, și astfel determinarea soluțiilor optime revine la scrierea și rezolvarea acestor relații de recurență.

Relațiile de recurență pot fi rezolvate și prin implementări recursive, în care soluția optimă a problemei apelează soluțiile optime ale subproblemelor (de dimensiuni mai mici, **strategia top-down**), dar în acest caz calculul necesar rezolvării unei aceleiași subprobleme poate fi reluat de foarte multe ori prin apelarea repetată a acesteia, ceea ce conduce frecvent la un timp de calcul exponențial. Totuși, acest dezavantaj major poate fi evitat dacă după

fiecare apel recursiv se memorează, într-un tablou de date, valoarea calculată a soluției subproblemei, iar apelul este recursiv doar dacă subproblema nu este deja rezolvată, ceea ce poate fi verificat în tabloul de date utilizat pentru memorarea soluțiilor subproblemelor, în rest soluția subproblemei se preia din acest tablou de date. Astfel sunt rezolvate doar subproblemele ale căror soluții sunt necesare în construirea soluției problemei date, iar fiecare astfel de subproblemă este rezolvată o singură dată. Aceasta este o tehnică de optimizare a programelor recursive, numită **memoizare**.

Deci este de preferat ca relațiile de recurență să fie rezolvate fie prin implementări nerecursive, utilizând strategia *bottom-up*, fie prin implementări recursive, utilizând tehnica *memoizării*, în ambele cazuri cu memorarea temporară a soluțiilor optime ale subproblemelor.

Prezentăm o schemă generală de lucru:

- Problema dată se descompune în subprobleme de același tip, dar de dimensiuni mai mici.
- Se alege o structură de date în care se memorează soluțiile optime ale problemei și subproblemelor.
- Se determină relația de recurență ce caracterizează soluția optimă.
- Se rezolvă relația de recurență prin strategia *bottom-up* (sau prin tehnica *memoizării*).

Observația 6.1.1. Presupunem că soluțiile optime ale subproblemelor sunt memorate într-un tablou

$$S = (S[i_1, \dots, i_r])_{(i_1, \dots, i_r) \in I_1 \times \dots \times I_r},$$

în vederea aplicării strategiei *bottom-up*.

a) Dacă relația de recurență ce caracterizează soluția optimă exprimă elementul $S[i_1, \dots, i_r]$ în funcție de elemente următoare

$$S[j_1, \dots, j_r], (j_1, \dots, j_r) \succ (i_1, \dots, i_r)$$

(conform ordinii lexicografice), atunci calculul acestor elemente se face în ordinea inversă, descrescătoare, a indicilor $(i_1, \dots, i_r)$. Această abordare este denumită și programare dinamică în **varianta înapoi**.

b) Dacă relația de recurență ce caracterizează soluția optimă exprimă elementul $S[i_1, \dots, i_r]$ în funcție de elemente anterioare

$$S[j_1, \dots, j_r], (j_1, \dots, j_r) \prec (i_1, \dots, i_r)$$

(conform ordinii lexicografice), atunci calculul acestor elemente se face în ordinea directă, crescătoare, a indicilor $(i_1, \dots, i_r)$. Această abordare este denumită și programare dinamică în **varianta înainte**.

6.2 Șirul numerelor lui Fibonacci

Șirul numerelor lui Fibonacci $(F_n)_{n \in \mathbb{N}}$ este definit prin:

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}, \forall n \geq 2.$$

Se cere să se determine termenul F_n , unde $n \in \mathbb{N}$.

Exemplul 6.2.1. Primii termeni ai șirului sunt

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots$$

De exemplu, $F_9 = 34$.

Vom prezenta patru algoritmi de tip *programare dinamică* pentru calculul lui F_n .

Algoritmul 6.2.1. Varianta *top-down*, implementată recursiv:

```

F(n):                                     // calculul valorii F_n
if n ≤ 1 then
  | returnează n;
else
  | returnează F(n - 1) + F(n - 2);

```

Observația 6.2.1. Pentru evaluarea complexității algoritmului anterior, notăm cu x_n numărul de adunări efectuate de funcția $F(n)$, adică necesare calculului termenului F_n (celelalte operații au același ordin de creștere cu x_n). Conform descrierii recursive a algoritmului, rezultă că numerele x_n verifică relația de recurență

$$x_n = \begin{cases} 0, & \text{dacă } n = 0 \text{ sau } n = 1, \\ x_{n-1} + x_{n-2} + 1, & \text{dacă } n \geq 2. \end{cases}$$

De exemplu, $x_2 = 1$, $x_3 = 2$, $x_4 = 4$, $x_5 = 7$, $x_6 = 12$, $x_7 = 20$, $x_8 = 33$, $x_9 = 54$.

Rezolvând relația de recurență obținută, găsim că

$$x_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1 + \sqrt{5}}{2} \right)^{n+1} - \left(\frac{1 - \sqrt{5}}{2} \right)^{n+1} \right] - 1, \forall n \in \mathbb{N},$$

egalitate ce poate fi demonstrată și prin inducție. Rezultă că algoritmul are complexitatea $\Theta(\varphi^n)$, unde $\varphi = \frac{1 + \sqrt{5}}{2}$ (numit *numărul de aur*), deci necesită un timp de calcul exponențial.

Algoritmul 6.2.2. Varianta *top-down*, folosind tehnica *memoizării*, prin utilizarea unui vector $t = (t[0], t[1], \dots, t[n])$ cu semnificația

$$t[i] = \begin{cases} 0, & \text{dacă } F_i \text{ nu a fost calculat,} \\ F_i, & \text{dacă } F_i \text{ a fost calculat.} \end{cases}$$

```

F( $n$ ):
  if  $n \leq 1$  then
    | returnează  $n$ ;
  else
    | if  $t[n] \neq 0$  then           //  $t[n] = F_n$  este deja calculat
    |   | returnează  $t[n]$ ;       // îl utilizăm, fără apel recursiv
    | else
    |   |           // calculăm  $t[n] = F_n$  prin apel recursiv
    |   |  $t[n] \leftarrow F(n-1) + F(n-2)$ ;
    |   | returnează  $t[n]$ ;

```

Algoritmul 6.2.3. Varianta *bottom-up*, folosind un vector $F = (F_0, F_1, \dots, F_n)$ pentru memorarea numerelor lui Fibonacci:

```

Fib( $F, n$ ):
   $F[0] \leftarrow 0$ ;
   $F[1] \leftarrow 1$ ;
  for  $i = \overline{2, n}$  do
    |  $F[i] \leftarrow F[i-1] + F[i-2]$ ;
  afișează  $F[n]$ ;

```

Algoritmul 6.2.4. Varianta *bottom-up*, folosind doar trei variabile a , b și c pentru memorarea numerelor F_{i-2} , F_{i-1} , respectiv F_i , pentru valorile succesive $i \in \{2, 3, \dots, n\}$.

```

F( $n$ ):
  if  $n \leq 1$  then
    | returnează  $n$ ;
  else
    |  $a \leftarrow 0$ ;
    |  $b \leftarrow 1$ ;
    | for  $i = \overline{2, n}$  do
    |   |  $c \leftarrow b + a$ ;
    |   |  $a \leftarrow b$ ;
    |   |  $b \leftarrow c$ ;
    | returnează  $c$ ;

```

Observația 6.2.2. Fiecare dintre ultimii trei algoritmi efectuează câte $n - 1$ adunări pentru calculul termenului F_n , $n \geq 1$, deci acești algoritmi au

complexitatea $\Theta(n)$ (algoritmi liniari).

Ultimul algoritm prezintă avantajul că este iterativ, față de algoritmul al doilea, care folosește $2n - 2$ apeluri recursive, $n \geq 1$, și avantajul că utilizează doar trei variabile suplimentare pentru memorarea numerelor $F_0, F_1, \dots, F_n$, față de ceilalți doi algoritmi, care folosesc câte un vector suplimentar cu $n + 1$ elemente.

6.3 Problema subșirului crescător de lungime maximă

Problema subșirului crescător de lungime maximă este următoarea:

Se consideră un șir de numere reale

$$a_1, a_2, \dots, a_n, \quad n \geq 1.$$

Se cere să se determine un subșir crescător de lungime maximă, adică un subșir

$$a_{i_1} \leq a_{i_2} \leq \dots \leq a_{i_k}$$

($1 \leq i_1 < i_2 < \dots < i_k \leq n$) cu $k = \text{maxim}$.

Exemplul 6.3.1. De exemplu, pentru șirul

$$10, 16, 2, 8, 4, 10, 3, 4, 9, 8$$

lungimea maximă a unui subșir crescător este egală cu 4 (nu există subșiruri crescătoare de lungime 5), iar un subșir crescător de lungime maximă este, de exemplu,

$$2, 3, 4, 8.$$

Modelarea problemei

Metoda 1) Utilizăm *programarea dinamică, varianta înapoi*. Considerăm subproblemele constând în determinarea a câte unui subșir crescător de lungime maximă care începe cu elementul a_i , pentru orice $i \in \{1, 2, \dots, n\}$.

Pentru memorarea soluțiilor subproblemelor utilizăm doi vectori $L = (L[1], \dots, L[n])$ și $P = (P[1], \dots, P[n - 1])$ având semnificația

$L[i]$ = lungimea maximă a unui subșir crescător care începe cu elementul a_i ,

$P[i]$ = indicele primului element care este succesor al lui a_i într-un subșir crescător de lungime maximă care începe cu a_i , dacă există un astfel de indice,

$P[i] = -1$, dacă nu există un astfel de indice,

pentru orice $i \in \{1, 2, \dots, n\}$.

Evident,

$$L[n] = 1$$

$$L[i] = \max_{j=i+1, n} \{1 + L[j] \mid a_i \leq a_j\}, \text{ dacă există un astfel de indice } j,$$

$$L[i] = 1, \text{ dacă nu există un astfel de indice } j,$$

$$P[i] = \min_{j=i+1, n} \{j \mid L[i] = 1 + L[j], a_i \leq a_j\}, \text{ dacă există un astfel de indice } j,$$

$$P[i] = -1, \text{ dacă nu există un astfel de indice } j,$$

pentru orice $i \in \{1, 2, \dots, n-1\}$.

Relația de recurență de mai sus exprimă elementul $L[i]$ în funcție de elementele următoare $L[i+1], \dots, L[n]$.

Rezolvarea relației de recurență a elementelor $L[i]$ și calculul elementelor $P[i]$ conform strategiei *bottom-up* revine la calculul acestor elemente în ordinea inversă a indicilor, adică

$$i = n, n-1, \dots, 2, 1.$$

O soluție a problemei este subșirul

$$a_{t_1} \leq a_{t_2} \leq \dots \leq a_{t_k}$$

de lungime

$$k = \max_{i=1, n} L[i],$$

construit astfel:

$$t_1 = \min_{i=1, n} \{i \mid L[i] = k\},$$

$$t_j = P[t_{j-1}], \forall j \in \{2, \dots, k\}.$$

Obținem următorul *algoritm de programare dinamică în varianta înapoi*, descris în pseudocod, pentru rezolvarea problemei.

Algoritmul 6.3.1.

SUBSIRMAX1(A, n) :

$L[n] \leftarrow 1$;

for $i = \overline{n-1, 1, -1}$ **do**

$L[i] \leftarrow 1$;

$P[i] \leftarrow -1$;

for $j = \overline{i+1, n}$ **do**

if ($a_j \geq a_i$) *and* ($1 + L[j] > L[i]$) **then**

$L[i] \leftarrow 1 + L[j]$;

$P[i] \leftarrow j$;

$k \leftarrow L[1]$;

$t_1 \leftarrow 1$;

for $i = \overline{2, n}$ **do**

if ($L[i] > k$) **then**

$k \leftarrow L[i]$;

$t_1 \leftarrow i$;

for $j = \overline{2, k}$ **do**

$t_j \leftarrow P[t_{j-1}]$;

for $j = \overline{1, k}$ **do**

afișează a_{t_j} ;

Exemplul 6.3.2. La aplicarea algoritmului anterior pentru șirul din Exemplul 6.3.1 avem $n = 10$,

$$L[10] = 1;$$

$$L[9] = 1, P[9] = -1;$$

$$L[8] = 2, P[8] = 9;$$

$$L[7] = 3, P[7] = 8;$$

$$L[6] = 1, P[6] = -1;$$

$$L[5] = 3, P[5] = 8;$$

$$L[4] = 2, P[4] = 6;$$

$$L[3] = 4, P[3] = 5;$$

$$L[2] = 1, P[2] = -1;$$

$$L[1] = 2, P[1] = 2,$$

deci lungimea maximă a unui subșir crescător este

$$k = 4$$

iar un subșir crescător de lungime maximă este dat de indicii

$$t_1 = 3, t_2 = P[t_1] = 5, t_3 = P[t_2] = 8, t_4 = P[t_3] = 9,$$

adică subșirul

$$2, 4, 4, 9.$$

Observația 6.3.1. Instrucțiunea

$$\mathbf{if}(a_j \geq a_i) \text{ and } (1 + L[j] > L[i])$$

se execută câte o dată pentru fiecare pereche de indici (i, j) cu $i \in \{n-1, n-2, \dots, 1\}$ și $j \in \{i+1, i+2, \dots, n\}$, adică de $1 + 2 + \dots + n - 1 = \frac{n(n-1)}{2}$ ori, deci algoritmul are complexitatea $\Theta(n^2)$.

Observația 6.3.2. Algoritmul anterior poate fi adaptat pentru determinarea **tuturor** subșirurilor crescătoare de lungime maximă. Pentru aceasta, păstrând semnificația și construcția vectorului L , reconsiderăm semnificația și construcția vectorului P astfel:

$P[i]$ = mulțimea indicilor tuturor elementelor care sunt succesori direcți ai lui a_i într-un subșir crescător de lungime maximă care începe cu a_i

(posibil vidă), pentru orice $i \in \{1, 2, \dots, n-1\}$. Evident,

$$P[i] = \{j = \overline{i+1, n} \mid L[i] = 1 + L[j], a_i \leq a_j\}, \forall i \in \{1, 2, \dots, n-1\}.$$

Orice soluție a problemei este un subșir de forma

$$a_{t_1} \leq a_{t_2} \leq \dots \leq a_{t_k}$$

de lungime

$$k = \max_{i=\overline{1, n}} L[i],$$

construit astfel:

$$\begin{aligned} t_1 &\in \{i = \overline{1, n} \mid L[i] = k\}, \\ t_j &\in P[t_{j-1}], \forall j \in \{2, \dots, k\}. \end{aligned}$$

Pentru generarea tuturor acestor vectori $(t_1, t_2, \dots, t_k)$ se poate utiliza *metoda Backtracking*.

De exemplu, pentru șirul din Exemplul 6.3.1 avem

$$\begin{aligned}
 L[10] &= 1; \\
 L[9] &= 1, P[9] = \emptyset; \\
 L[8] &= 2, P[8] = \{9, 10\}; \\
 L[7] &= 3, P[7] = \{8\}; \\
 L[6] &= 1, P[6] = \emptyset; \\
 L[5] &= 3, P[5] = \{8\}; \\
 L[4] &= 2, P[4] = \{6, 9, 10\}; \\
 L[3] &= 4, P[3] = \{5, 7\}; \\
 L[2] &= 1, P[2] = \emptyset; \\
 L[1] &= 2, P[1] = \{2, 6\},
 \end{aligned}$$

deci lungimea maximă a unui subșir crescător este $k = 4$ iar orice subșir crescător de lungime maximă este dat de indicii

$$t_1 \in \{3\}, t_2 \in P[t_1], t_3 \in P[t_2], t_4 \in P[t_3].$$

Obținem că subșirurile crescătoare de lungime maximă sunt

$$\begin{aligned}
 (a_3 = 2, a_5 = 4, a_8 = 4, a_9 = 9), (a_3 = 2, a_5 = 4, a_8 = 4, a_{10} = 8), \\
 (a_3 = 2, a_7 = 3, a_8 = 4, a_9 = 9), (a_3 = 2, a_7 = 3, a_8 = 4, a_{10} = 8),
 \end{aligned}$$

deci avem 4 soluții (subșiruri crescătoare de lungime maximă).

Metoda 2) Utilizăm *programarea dinamică, varianta înainte*. Considerăm subproblemele constând în determinarea a câte unui subșir crescător de lungime maximă care se încheie cu elementul a_i , pentru orice $i \in \{1, 2, \dots, n\}$.

Pentru memorarea soluțiilor subproblemelor utilizăm din nou doi vectori $L = (L[1], \dots, L[n])$ și $P = (P[2], \dots, P[n])$ având acum semnificația

$L[i]$ = lungimea maximă a unui subșir crescător care se încheie cu elementul a_i ,

$P[i]$ = indicele primului element care este predecesor al lui a_i într-un subșir crescător de lungime maximă care se încheie cu a_i , dacă există un astfel de indice,

$P[i] = -1$, dacă nu există un astfel de indice,

pentru orice $i \in \{1, 2, \dots, n\}$.

Evident,

$$L[1] = 1,$$

$$L[i] = \max_{j=1, i-1} \{1 + L[j] \mid a_j \leq a_i\}, \text{ dacă există un astfel de indice } j,$$

$$L[i] = 1, \text{ dacă nu există un astfel de indice } j,$$

$$P[i] = \min_{j=1, i-1} \{j \mid L[i] = 1 + L[j], a_j \leq a_i\}, \text{ dacă există un astfel de indice } j,$$

$$P[i] = -1, \text{ dacă nu există un astfel de indice } j,$$

pentru orice $i \in \{2, 3, \dots, n\}$.

Relația de recurență de mai sus exprimă elementul $L[i]$ în funcție de elementele anterioare $L[1], \dots, L[i-1]$.

Rezolvarea relației de recurență a elementelor $L[i]$ și calculul elementelor $P[i]$ conform strategiei *bottom-up* revine la calculul acestor elemente în ordinea directă a indicilor, adică

$$i = 1, 2, \dots, n-1, n.$$

O soluție a problemei este subșirul

$$a_{t_1} \leq a_{t_2} \leq \dots \leq a_{t_k}$$

de lungime

$$k = \max_{i=1, n} L[i],$$

construit astfel:

$$t_k = \min_{i=1, n} \{i \mid L[i] = k\},$$

$$t_j = P[t_{j+1}], \forall j \in \{k-1, \dots, 1\}.$$

Obținem următorul *algorithm de programare dinamică în varianta înainte*, descris în pseudocod, pentru rezolvarea problemei.

Algoritmul 6.3.2.

SUBSIRMAX2(A, n) :

```

 $L[1] \leftarrow 1;$ 
for  $i = \overline{2, n}$  do
     $L[i] \leftarrow 1;$ 
     $P[i] \leftarrow -1;$ 
    for  $j = \overline{1, i-1}$  do
        if  $(a_j \leq a_i)$  and  $(1 + L[j] > L[i])$  then
             $L[i] \leftarrow 1 + L[j];$ 
             $P[i] \leftarrow j;$ 
 $k \leftarrow L[1];$ 
 $m \leftarrow 1;$ 
for  $i = \overline{2, n}$  do
    if  $(L[i] > k)$  then
         $k \leftarrow L[i];$ 
         $m \leftarrow i;$ 
 $t_k \leftarrow m;$ 
for  $j = \overline{k-1, 1, -1}$  do
     $t_j \leftarrow P[t_{j+1}];$ 
for  $j = \overline{1, k}$  do
    afișează  $a_{t_j};$ 

```

$// k = \max\{i = \overline{1, n} \mid L[i]\}$
 $// m = \min\{i = \overline{1, n} \mid L[i] = k\}$

Exemplul 6.3.3. La aplicarea algoritmului anterior pentru șirul din Exemplul 6.3.1 avem $n = 10$,

$L[1] = 1;$
 $L[2] = 2, P[2] = 1;$
 $L[3] = 1, P[3] = -1;$
 $L[4] = 2, P[4] = 3;$
 $L[5] = 2, P[5] = 3;$
 $L[6] = 3, P[6] = 4;$
 $L[7] = 2, P[7] = 3;$
 $L[8] = 3, P[8] = 5;$
 $L[9] = 4, P[9] = 8;$
 $L[10] = 4, P[10] = 8,$

deci lungimea maximă a unui subșir crescător este

$$k = 4$$

iar un subșir crescător de lungime maximă este $a_{t_1} \leq a_{t_2} \leq \dots \leq a_{t_k}$ dat de indicii

$$t_4 = 9, t_3 = P[t_4] = 8, t_2 = P[t_3] = 5, t_1 = P[t_2] = 3,$$

adică subșirul

$$2, 4, 4, 9.$$

Observația 6.3.3. Analog algoritmului de la Metoda 1, și algoritmul anterior are complexitatea $\Theta(n^2)$.

Observația 6.3.4. Printr-un procedeu similar celui descris în Observația 6.3.2, și algoritmul anterior poate fi adaptat pentru determinarea tuturor subșirurilor crescătoare de lungime maximă.

6.4 Problema subșirului comun de lungime maximă

Problema subșirului comun de lungime maximă este următoarea:

Se consideră două șiruri de numere reale (sau de caractere)

$$a = (a_1, a_2, \dots, a_m) \text{ și } b = (b_1, b_2, \dots, b_n) \text{ } m, n \geq 1.$$

Se cere să se determine un subșir comun de lungime maximă, adică un subșir comun

$$(a_{i_1}, a_{i_2}, \dots, a_{i_k}) = (b_{j_1}, b_{j_2}, \dots, b_{j_k})$$

($1 \leq i_1 < i_2 < \dots < i_k \leq m$, $1 \leq j_1 < j_2 < \dots < j_k \leq n$) cu $k = \text{maxim}$.

Exemplul 6.4.1. De exemplu, pentru șirurile

$$a = (5, 0, 2, 1, 3, 4, 0, 2) \text{ și } b = (2, 3, 0, 0, 1, 2, 5, 3, 0)$$

lungimea maximă a unui subșir comun este egală cu 4 (nu există subșiruri comune de lungime 5), iar un subșir comun de lungime maximă este, de exemplu,

$$(a_2, a_3, a_5, a_7) = (0, 2, 3, 0) = (b_3, b_6, b_8, b_9).$$

Modelarea problemei

Metoda 1) Utilizăm *programarea dinamică, varianta înapoi*. Considerăm subproblemele constând în determinarea a câte unui subșir comun de lungime maximă pentru șirurile parțiale

$$(a_i, a_{i+1}, \dots, a_m) \text{ și } (b_j, b_{j+1}, \dots, b_n),$$

pentru orice $i \in \{1, 2, \dots, m\}$ și $j \in \{1, 2, \dots, n\}$.

Pentru memorarea soluțiilor subproblemelor utilizăm două matrice
 $L = (L[i, j])_{\substack{i = \overline{1, m+1} \\ j = \overline{1, n+1}}}$ și $P = (P[i, j])_{\substack{i = \overline{1, m+1} \\ j = \overline{1, n+1}}}$ având semnificația

$L[i, j]$ = lungimea maximă a unui subșir comun pentru șirurile parțiale
 $(a_i, a_{i+1}, \dots, a_m)$ și $(b_j, b_{j+1}, \dots, b_n)$,

$P[i, j] = (P1[i, j], P2[i, j])$, unde

$P1[i, j]$ = indicele primului element din $(a_i, a_{i+1}, \dots, a_m)$ cu care începe un
subșir comun de lungime maximă pentru cele două șiruri parțiale, iar

$P2[i, j]$ = indicele unui element din $(b_j, b_{j+1}, \dots, b_n)$ cu care începe acest
subșir comun de lungime maximă, dacă există un subșir comun (nevid),

$P1[i, j] = P2[i, j] = -1$, dacă nu există un subșir comun (nevid),

pentru orice $i \in \{1, 2, \dots, m+1\}$ și $j \in \{1, 2, \dots, n+1\}$.

Avem

$$L[i, j] = \begin{cases} 0, & \text{dacă } i = m+1 \text{ sau } j = n+1, \\ 1 + L[i+1, j+1], & \text{dacă } i \leq m, j \leq n \text{ și } a_i = b_j, \\ \max \{L[i, j+1], L[i+1, j]\}, & \text{dacă } i \leq m, j \leq n \text{ și } a_i \neq b_j, \end{cases}$$

$$P[i, j] = \begin{cases} (-1, -1), & \text{dacă } i = m+1 \text{ sau } j = n+1, \\ (i, j), & \text{dacă } i \leq m, j \leq n \text{ și } a_i = b_j, \\ P[i, j+1], & \text{dacă } i \leq m, j \leq n, a_i \neq b_j \text{ și } L[i, j+1] \geq L[i+1, j], \\ P[i+1, j], & \text{dacă } i \leq m, j \leq n, a_i \neq b_j \text{ și } L[i, j+1] < L[i+1, j]. \end{cases}$$

Relația de recurență de mai sus exprimă elementul $L[i, j]$ în funcție de elementele următoare de pe linia i și de elementele de pe linia următoare, $i+1$.

Rezolvarea relației de recurență a elementelor $L[i, j]$ și calculul elementelor $P[i, j]$ conform strategiei *bottom-up* revine la calculul acestor elemente în ordinea inversă a liniilor, adică $i = m+1, m, \dots, 2, 1$, și, pentru fiecare linie i , în ordinea inversă a coloanelor, adică $j = n+1, n, \dots, 2, 1$.

Dacă $L[1, 1] = 0$, atunci problema nu are soluție, adică șirurile a și b nu au subșiruri comune (nevide).

Dacă $L[1, 1] > 0$, atunci o soluție a problemei este subșirul comun

$$(a_{s_1}, a_{s_2}, \dots, a_{s_k}) = (b_{t_1}, b_{t_2}, \dots, b_{t_k})$$

de lungime

$$k = L[1, 1]$$

construit astfel:

$$(s_1, t_1) = P[1, 1], (s_j, t_j) = P[s_{j-1} + 1, t_{j-1} + 1], \forall j \in \{2, \dots, k\}.$$

Obținem următorul *algoritm de programare dinamică în varianta înapoi*, descris în pseudocod, pentru rezolvarea problemei.

Algoritmul 6.4.1.

```

SUBSIRCOMUNMAX1( $A, m, B, n$ ) :
  for  $j = \overline{1, n+1}$  do
     $L[m+1, j] \leftarrow 0$ ;
     $P1[m+1, j] \leftarrow -1$ ;  $P2[m+1, j] \leftarrow -1$ ;
  for  $i = \overline{m, 1, -1}$  do
     $L[i, n+1] \leftarrow 0$ ;
     $P1[i, n+1] \leftarrow -1$ ;  $P2[i, n+1] \leftarrow -1$ ;
    for  $j = \overline{n, 1, -1}$  do
      if  $a[i] = b[j]$  then
         $L[i, j] \leftarrow 1 + L[i+1, j+1]$ ;
         $P1[i, j] \leftarrow i$ ;  $P2[i, j] \leftarrow j$ ;
      else
        if  $L[i, j+1] \geq L[i+1, j]$  then
           $L[i, j] \leftarrow L[i, j+1]$ ;
           $P1[i, j] \leftarrow P1[i, j+1]$ ;  $P2[i, j] \leftarrow P2[i, j+1]$ ;
        else
           $L[i, j] \leftarrow L[i+1, j]$ ;
           $P1[i, j] \leftarrow P1[i+1, j]$ ;  $P2[i, j] \leftarrow P2[i+1, j]$ ;
     $k \leftarrow L[1, 1]$ ;
    if  $k = 0$  then
      afișează "nu exista subsir comun";
    else
       $s[1] \leftarrow P1[1, 1]$ ;  $t[1] \leftarrow P2[1, 1]$ ;
      for  $j = \overline{2, k}$  do
         $s[j] \leftarrow P1[s[j-1] + 1, t[j-1] + 1]$ ;
         $t[j] \leftarrow P2[s[j-1] + 1, t[j-1] + 1]$ ;
      for  $j = \overline{1, k}$  do
        afișează ( $a[s[j]], b[t[j]]$ );

```

Exemplul 6.4.2. Aplicarea algoritmului anterior pentru șirurile din Exemplul 6.4.1 este ilustrată în tabelul următor, în care pe prima coloană avem șirul a , pe prima linie avem șirul b , iar elementele calculate în interiorul tabelului au forma $L[i, j], (P1[i, j], P2[i, j])$. Reamintim că aceste elemente sunt calculate, conform relațiilor de recurență de mai sus, în ordinea inversă a liniilor

și a coloanelor.

	2	3	0	0	1	2	5	3	0	
5	4,(2,4)	4,(2,4)	4,(2,4)	4,(2,4)	3,(1,7)	3,(1,7)	3,(1,7)	2,(5,8)	1,(2,9)	0,(-1,-1)
0	4,(2,3)	4,(2,3)	4,(2,3)	4,(2,4)	3,(3,6)	3,(3,6)	2,(5,8)	2,(5,8)	1,(2,9)	0,(-1,-1)
2	4,(3,1)	3,(3,6)	3,(3,6)	3,(3,6)	3,(3,6)	3,(3,6)	2,(5,8)	2,(5,8)	1,(7,9)	0,(-1,-1)
1	3,(4,5)	3,(4,5)	3,(4,5)	3,(4,5)	3,(4,5)	2,(5,8)	2,(5,8)	2,(5,8)	1,(7,9)	0,(-1,-1)
3	3,(5,2)	3,(5,2)	2,(5,8)	2,(5,8)	2,(5,8)	2,(5,8)	2,(5,8)	2,(5,8)	1,(7,9)	0,(-1,-1)
4	2,(7,4)	2,(7,4)	2,(7,4)	2,(7,4)	1,(7,9)	1,(7,9)	1,(7,9)	1,(7,9)	1,(7,9)	0,(-1,-1)
0	2,(7,3)	2,(7,3)	2,(7,3)	2,(7,4)	1,(7,9)	1,(7,9)	1,(7,9)	1,(7,9)	1,(7,9)	0,(-1,-1)
2	1,(8,1)	1,(8,6)	1,(8,6)	1,(8,6)	1,(8,6)	1,(8,6)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)
	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)

Lungimea maximă a unui subșir comun are valoarea

$$k = L[1, 1] = 4.$$

Avem

$$\begin{aligned}(s_1, t_1) &= (P1[1, 1], P2[1, 1]) = (2, 4), \\(s_2, t_2) &= (P1[s_1 + 1, t_1 + 1], P2[s_1 + 1, t_1 + 1]) = (P1[3, 5], P2[3, 5]) = (3, 6), \\(s_3, t_3) &= (P1[s_2 + 1, t_2 + 1], P2[s_2 + 1, t_2 + 1]) = (P1[4, 7], P2[4, 7]) = (5, 8), \\(s_4, t_4) &= (P1[s_3 + 1, t_3 + 1], P2[s_3 + 1, t_3 + 1]) = (P1[6, 9], P2[6, 9]) = (7, 9),\end{aligned}$$

deci subșirul comun de lungime maximă produs de algoritm are elementele

$$a_2 = b_4 = 0, a_3 = b_6 = 2, a_5 = b_8 = 3, a_7 = b_9 = 0.$$

Observația 6.4.1. Pentru calculul fiecărui element al matricelor L , $P1$ și $P2$ sunt necesare câte o atribuire și cel mult două comparații, deci algoritmul are complexitatea $\Theta(mn)$.

Observația 6.4.2. Algoritmul anterior poate fi adaptat pentru determinarea **tuturor** subșirurilor comune de lungime maximă. Pentru aceasta, păstrând semnificația și construcția matricei L , reconsiderăm semnificația și construcția matricei P astfel:

$P[i, j]$ = mulțimea tuturor perechilor de indici (p_1, p_2) cu proprietatea că

p_1 = indicele unui element din $(a_i, a_{i+1}, \dots, a_m)$ cu care începe un subșir comun de lungime maximă pentru cele două șiruri parțiale, iar

p_2 = indicele unui element din $(b_j, b_{j+1}, \dots, b_n)$ cu care începe acest subșir comun de lungime maximă

(mulțime posibil vidă), pentru orice $i \in \{1, 2, \dots, m+1\}$, $j \in \{1, 2, \dots, n+1\}$.

Pentru $i = m + 1$ sau $j = n + 1$ avem $P[i, j] = \emptyset$.

Pentru $i \in \{1, 2, \dots, m\}$ și $j \in \{1, 2, \dots, n\}$, dacă $a_i = b_j$ avem

$$P[i, j] = \begin{cases} \{(i, j)\} \cup P[i, j+1] \cup P[i+1, j], & \text{dacă } L[i, j] = L[i, j+1] = L[i+1, j], \\ \{(i, j)\} \cup P[i, j+1], & \text{dacă } L[i, j] = L[i, j+1] > L[i+1, j], \\ \{(i, j)\} \cup P[i+1, j], & \text{dacă } L[i, j] = L[i+1, j] > L[i, j+1], \\ \{(i, j)\}, & \text{în rest} \end{cases}$$

(adică dacă $L[i, j] > L[i, j+1]$ și $L[i, j] > L[i+1, j]$), iar dacă $a_i \neq b_j$ avem

$$P[i, j] = \begin{cases} P[i, j+1] \cup P[i+1, j], & \text{dacă } L[i, j+1] = L[i+1, j], \\ P[i, j+1], & \text{dacă } L[i, j+1] > L[i+1, j], \\ P[i+1, j], & \text{dacă } L[i, j+1] < L[i+1, j]. \end{cases}$$

Dacă $L[1, 1] > 0$, atunci orice soluție a problemei este un subșir comun de forma

$$(a_{s_1}, a_{s_2}, \dots, a_{s_k}) = (b_{t_1}, b_{t_2}, \dots, b_{t_k})$$

având lungimea

$$k = L[1, 1]$$

și construit astfel:

$$(s_1, t_1) \in P[1, 1], (s_j, t_j) \in P[s_{j-1} + 1, t_{j-1} + 1], \forall j \in \{2, \dots, k\}.$$

Pentru generarea tuturor acestor șiruri de perechi de indici $(s_1, t_1), (s_2, t_2), \dots, (s_k, t_k)$ se poate utiliza *metoda Backtracking*.

Metoda 2) Utilizăm *programarea dinamică, varianta înainte*. Considerăm subproblemele constând în determinarea a câte unui subșir comun de lungime maximă pentru șirurile parțiale

$$(a_1, a_2, \dots, a_i) \text{ și } (b_1, b_2, \dots, b_j),$$

pentru orice $i \in \{1, 2, \dots, m\}$ și $j \in \{1, 2, \dots, n\}$.

Pentru memorarea soluțiilor subproblemelor utilizăm din nou două matrice $L = (L[i, j])_{\substack{i = \overline{0, m} \\ j = \overline{0, n}}}$ și $P = (P[i, j])_{\substack{i = \overline{0, m} \\ j = \overline{0, n}}}$ având acum semnificația

$L[i, j]$ = lungimea maximă a unui subșir comun pentru șirurile parțiale

$$(a_1, a_2, \dots, a_i) \text{ și } (b_1, b_2, \dots, b_j),$$

$P[i, j] = (P1[i, j], P2[i, j])$, unde

$P1[i, j]$ = indicele ultimului element din $(a_1, a_2, \dots, a_i)$ cu care se încheie un

subșir comun de lungime maximă pentru cele două șiruri parțiale, iar

$P2[i, j]$ = indicele unui element din $(b_1, b_2, \dots, b_j)$ cu care se încheie acest

subșir comun de lungime maximă, dacă există un subșir comun (nevid),

$P1[i, j] = P2[i, j] = -1$, dacă nu există un subșir comun (nevid),

pentru orice $i \in \{0, 1, \dots, m\}$ și $j \in \{0, 1, \dots, n\}$.

Avem

$$L[i, j] = \begin{cases} 0, & \text{dacă } i = 0 \text{ sau } j = 0, \\ 1 + L[i - 1, j - 1], & \text{dacă } i \geq 1, j \geq 1 \text{ și } a_i = b_j, \\ \max \{L[i, j - 1], L[i - 1, j]\}, & \text{dacă } i \geq 1, j \geq 1 \text{ și } a_i \neq b_j, \end{cases}$$

$$P[i, j] = \begin{cases} (-1, -1), & \text{dacă } i = 0 \text{ sau } j = 0, \\ (i, j), & \text{dacă } i \geq 1, j \geq 1 \text{ și } a_i = b_j, \\ P[i, j - 1], & \text{dacă } i \geq 1, j \geq 1, a_i \neq b_j \text{ și } L[i, j - 1] \geq L[i - 1, j], \\ P[i - 1, j], & \text{dacă } i \geq 1, j \geq 1, a_i \neq b_j \text{ și } L[i, j - 1] < L[i - 1, j]. \end{cases}$$

Relația de recurență de mai sus exprimă elementul $L[i, j]$ în funcție de elementele anterioare de pe linia i și de elementele de pe linia anterioară, $i - 1$.

Rezolvarea relației de recurență a elementelor $L[i, j]$ și calculul elementelor $P[i, j]$ conform strategiei *bottom-up* revine la calculul acestor elemente în ordinea directă a liniilor, adică $i = 0, 1, \dots, m - 1, m$, și, pentru fiecare linie i , în ordinea directă a coloanelor, adică $j = 0, 1, \dots, n - 1, n$.

Dacă $L[m, n] = 0$, atunci problema nu are soluție, adică șirurile a și b nu au subșiruri comune (nevide).

Dacă $L[m, n] > 0$, atunci o soluție a problemei este subșirul comun

$$(a_{s_1}, a_{s_2}, \dots, a_{s_k}) = (b_{t_1}, b_{t_2}, \dots, b_{t_k})$$

de lungime

$$k = L[m, n]$$

construit astfel:

$$(s_k, t_k) = P[m, n], (s_j, t_j) = P[s_{j+1} - 1, t_{j+1} - 1], \forall j \in \{1, \dots, k - 1\}.$$

Obținem următorul *algoritm de programare dinamică în varianta înainte*, descris în pseudocod, pentru rezolvarea problemei.

Algoritmul 6.4.2.

```

SUBSIRCOMUNMAX2( $A, m, B, n$ ) :
  for  $j = \overline{0, n}$  do
     $L[0, j] \leftarrow 0$ ;
     $P1[0, j] \leftarrow -1$ ;  $P2[0, j] \leftarrow -1$ ;
  for  $i = \overline{1, m}$  do
     $L[i, 0] \leftarrow 0$ ;
     $P1[i, 0] \leftarrow -1$ ;  $P2[i, 0] \leftarrow -1$ ;
    for  $j = \overline{1, n}$  do
      if  $a[i] = b[j]$  then
         $L[i, j] \leftarrow 1 + L[i - 1, j - 1]$ ;
         $P1[i, j] \leftarrow i$ ;  $P2[i, j] \leftarrow j$ ;
      else
        if  $L[i, j - 1] \geq L[i - 1, j]$  then
           $L[i, j] \leftarrow L[i, j - 1]$ ;
           $P1[i, j] \leftarrow P1[i, j - 1]$ ;  $P2[i, j] \leftarrow P2[i, j - 1]$ ;
        else
           $L[i, j] \leftarrow L[i - 1, j]$ ;
           $P1[i, j] \leftarrow P1[i - 1, j]$ ;  $P2[i, j] \leftarrow P2[i - 1, j]$ ;
     $k \leftarrow L[m, n]$ ;
    if  $k = 0$  then
      afișează "nu exista subsir comun";
    else
       $s[k] \leftarrow P1[m, n]$ ;  $t[k] \leftarrow P2[m, n]$ ;
      for  $j = \overline{k - 1, 1 - 1}$  do
         $s[j] \leftarrow P1[s[j + 1] - 1, t[j + 1] - 1]$ ;
         $t[j] \leftarrow P2[s[j + 1] - 1, t[j + 1] - 1]$ ;
      for  $j = \overline{1, k}$  do
        afișează ( $a[s[j]]$ ,  $b[t[j]]$ );

```

Exemplul 6.4.3. Aplicarea algoritmului anterior pentru șirurile din Exemplul 6.4.1 este ilustrată în tabelul următor, în care pe prima coloană avem șirul a , pe prima linie avem șirul b , iar elementele calculate în interiorul tabelului au forma $L[i, j]$, $(P1[i, j], P2[i, j])$. Reamintim că aceste elemente sunt calculate, conform relațiilor de recurență de mai sus, în ordinea directă a liniilor și a coloanelor.

		2	3	0	0	1	2	5	3	0
	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)
5	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	1,(1,7)	1,(1,7)	1,(1,7)
0	0,(-1,-1)	0,(-1,-1)	0,(-1,-1)	1,(2,3)	1,(2,4)	1,(2,4)	1,(2,4)	1,(2,4)	1,(2,4)	2,(2,9)
2	0,(-1,-1)	1,(3,1)	1,(3,1)	1,(3,1)	1,(3,1)	1,(3,1)	2,(3,6)	2,(3,6)	2,(3,6)	2,(3,6)
1	0,(-1,-1)	1,(3,1)	1,(3,1)	1,(3,1)	1,(3,1)	2,(4,5)	2,(4,5)	2,(4,5)	2,(4,5)	2,(4,5)
3	0,(-1,-1)	1,(3,1)	2,(5,2)	2,(5,2)	2,(5,2)	2,(5,2)	2,(5,2)	2,(5,2)	3,(5,8)	3,(5,8)
4	0,(-1,-1)	1,(3,1)	2,(5,2)	2,(5,2)	2,(5,2)	2,(5,2)	2,(5,2)	2,(5,2)	3,(5,8)	3,(5,8)
0	0,(-1,-1)	1,(3,1)	2,(5,2)	3,(7,3)	3,(7,4)	3,(7,4)	3,(7,4)	3,(7,4)	3,(7,4)	4,(7,9)
2	0,(-1,-1)	1,(8,1)	2,(5,2)	3,(7,3)	3,(7,3)	3,(7,3)	4,(8,6)	4,(8,6)	4,(8,6)	4,(8,6)

Lungimea maximă a unui subșir comun are valoarea

$$k = L[8, 9] = 4.$$

Avem

$$\begin{aligned}(s_4, t_4) &= (P1[8, 9], P2[8, 9]) = (8, 6), \\(s_3, t_3) &= (P1[s_4 - 1, t_4 - 1], P2[s_4 - 1, t_4 - 1]) = (P1[7, 5], P2[7, 5]) = (7, 4), \\(s_2, t_2) &= (P1[s_3 - 1, t_3 - 1], P2[s_3 - 1, t_3 - 1]) = (P1[6, 3], P2[6, 3]) = (5, 2), \\(s_1, t_1) &= (P1[s_2 - 1, t_2 - 1], P2[s_2 - 1, t_2 - 1]) = (P1[4, 1], P2[4, 1]) = (3, 1),\end{aligned}$$

deci subșirul comun de lungime maximă produs de algoritmul are elementele

$$a_3 = b_1 = 2, a_5 = b_2 = 3, a_7 = b_4 = 0, a_8 = b_6 = 2.$$

Observația 6.4.3. Analog algoritmului de la Metoda 1, și algoritmul anterior are complexitatea $\Theta(mn)$.

Observația 6.4.4. Printr-un procedeu similar celui descris în Observația 6.4.2, și algoritmul anterior poate fi adaptat pentru determinarea tuturor subșirurilor comune de lungime maximă.

Observația 6.4.5. Algoritmii pentru rezolvarea problemei subșirului comun de lungime maximă pot fi utilizați și pentru rezolvarea problemei subșirului crescător de lungime maximă conform următoarei proprietăți: un subșir al unui șir $a = (a_1, a_2, \dots, a_n)$ este un subșir crescător (de lungime maximă) dacă și numai dacă el este un subșir comun (de lungime maximă) pentru șirurile a și b , unde șirul b se obține prin ordonarea crescătoare a elementelor șirului a .

De exemplu, pentru șirul

$$a = (10, 16, 2, 8, 4, 10, 3, 4, 9, 8)$$

din Exemplul 6.3.1, avem, sortându-l crescător,

$$b = (2, 3, 4, 4, 8, 8, 9, 10, 10, 16).$$

Un subșir comun de lungime maximă pentru a și b este

$$2, 4, 4, 9,$$

deci acesta este un subșir crescător de lungime maximă al șirului a .

Cum obținerea șirului sortat b are complexitatea $\Theta(n \log_2 n)$ iar algoritmi de mai sus pentru determinarea unui subșir comun de lungime maximă pentru șirurile a și b au complexitatea $\Theta(n^2)$, rezultă că algoritmi derivați pentru determinarea unui subșir crescător de lungime maximă pentru șirul a are complexitatea $\Theta(n^2)$, exact ca și algoritmi prezentați în secțiunea anterioară.

6.5 Problema drumurilor de sumă maximă/minimă într-o matrice

Problema drumului de sumă maximă/minimă într-o matrice este următoarea:

Se consideră o matrice de numere reale

$$A = (a_{ij})_{\substack{i = \overline{1, m} \\ j = \overline{1, n}}}, \quad m, n \in \mathbb{N}^*.$$

Se cere ca dintre toate drumurile care încep cu un element de pe prima linie, coboară de fiecare dată pe verticală sau pe diagonală la un element de pe linia următoare și se încheie cu un element de pe ultima linie, să se determine un drum pentru care suma elementelor parcurse este:

- a) maximă;
- b) minimă.

Exemplul 6.5.1. De exemplu, pentru matricea

$$A = \begin{pmatrix} 40 & 30 & \boxed{10} & \textcircled{35} & 20 & 25 \\ 45 & \boxed{30} & \textcircled{40} & 30 & 40 & 45 \\ \boxed{20} & 25 & \textcircled{50} & 50 & 20 & 10 \\ 35 & \boxed{15} & 10 & \textcircled{30} & 15 & 20 \\ \boxed{40} & 50 & 60 & 55 & \textcircled{65} & 80 \\ 40 & \boxed{20} & 15 & \textcircled{20} & 10 & 10 \end{pmatrix}$$

un drum de sumă maximă este format din elementele încadrate în cercuri, având suma 240.

Pentru aceeași matrice, un drum de sumă minimă este format din elementele încadrate în pătrate, având suma 135.

Modelarea problemei

a) Considerăm problema drumului de sumă maximă.

Metoda 1) Utilizăm *programarea dinamică, varianta înapoi*. Considerăm subproblemele constând în determinarea a câte unui drum de sumă maximă care începe cu elementul a_{ij} (și se încheie cu un element de pe ultima linie), pentru orice $i \in \{1, 2, \dots, m\}$ și orice $j \in \{1, 2, \dots, n\}$.

Pentru memorarea soluțiilor subproblemelor utilizăm două matrice $S = (S[i, j])_{\substack{i = \overline{1, m} \\ j = \overline{1, n}}}$ și $P = (P[i, j])_{\substack{i = \overline{1, m-1} \\ j = \overline{1, n}}}$ având semnificația

$$S[i, j] = \text{suma maximă a unui drum de la } a_{ij} \text{ la ultima linie,}$$

$$P[i, j] = \begin{cases} -1, & \text{dacă există un drum de sumă maximă de la } a_{ij} \text{ la ultima} \\ & \text{linie având al doilea element pe } a_{i+1, j-1} \text{ (adică prima} \\ & \text{coborâre are forma } \swarrow), \\ 0, & \text{dacă există un drum de sumă maximă de la } a_{ij} \text{ la ultima} \\ & \text{linie având al doilea element pe } a_{i+1, j} \text{ (adică prima} \\ & \text{coborâre are forma } \downarrow) \text{ și nu există un drum de sumă} \\ & \text{maximă de la } a_{ij} \text{ la ultima linie având al doilea element} \\ & \text{pe } a_{i+1, j-1}, \\ 1, & \text{dacă există un drum de sumă maximă de la } a_{ij} \text{ la ultima} \\ & \text{linie având al doilea element pe } a_{i+1, j+1} \text{ (adică prima} \\ & \text{coborâre are forma } \searrow) \text{ și nu există un drum de sumă} \\ & \text{maximă de la } a_{ij} \text{ la ultima linie având al doilea element} \\ & \text{pe } a_{i+1, j-1} \text{ sau pe } a_{i+1, j}. \end{cases}$$

Evident,

$$S[m, j] = a_{mj}, \quad \forall j \in \{1, \dots, n\}$$

și pentru orice $i \in \{1, \dots, m-1\}$ avem

$$S[i, 1] = a_{i1} + \max\{S[i+1, 1], S[i+1, 2]\},$$

$$P[i, 1] = \begin{cases} 0, & \text{dacă } S[i+1, 1] \geq S[i+1, 2], \\ 1, & \text{dacă } S[i+1, 1] < S[i+1, 2], \end{cases}$$

$$\begin{aligned}
S[i, j] &= a_{ij} + \max\{S[i+1, j-1], S[i+1, j], S[i+1, j+1]\}, \\
&\quad \forall j \in \{2, \dots, n-1\} \\
P[i, j] &= \begin{cases} -1, & \text{dacă } S[i+1, j-1] \geq S[i+1, j] \text{ și} \\
& S[i+1, j-1] \geq S[i+1, j+1], \\
0, & \text{dacă } S[i+1, j] > S[i+1, j-1] \text{ și} \\
& S[i+1, j] \geq S[i+1, j+1], \\
1, & \text{dacă } S[i+1, j+1] > S[i+1, j-1] \text{ și} \\
& S[i+1, j+1] > S[i+1, j], \\
\end{cases} \\
&\quad \forall j \in \{2, \dots, n-1\}
\end{aligned}$$

$$\begin{aligned}
S[i, n] &= a_{in} + \max\{S[i+1, n-1], S[i+1, n]\}, \\
P[i, n] &= \begin{cases} -1, & \text{dacă } S[i+1, n-1] \geq S[i+1, n], \\
0, & \text{dacă } S[i+1, n-1] < S[i+1, n]. \end{cases}
\end{aligned}$$

Relațiile de recurență de mai sus exprimă elementul $S[i, j]$ în funcție de elementele de pe linia următoare $S[i+1, j-1]$, $S[i+1, j]$, $S[i+1, j+1]$.

Rezolvarea relației de recurență a elementelor $S[i, j]$ și calculul elementelor $P[i, j]$ conform strategiei *bottom-up* revine la calculul acestor elemente în ordinea inversă a liniilor, adică

$$i = m, m-1, \dots, 2, 1.$$

O soluție a problemei este drumul

$$a_{1t_1}, a_{2t_2}, \dots, a_{mt_m}$$

de sumă

$$M = \max_{j=1, n} S[1, j],$$

construit astfel:

$$t_1 = \min_{j=1, n} \{j \mid S[1, j] = M\},$$

$$t_i = t_{i-1} + P[i-1, t_{i-1}], \quad \forall i \in \{2, \dots, m\}.$$

Obținem următorul *algoritm de programare dinamică în varianta înapoi*, descris în pseudocod, pentru rezolvarea problemei.

Algoritmul 6.5.1.

```

DRUMMAX1( $A, m, n$ ) :
  for  $j = \overline{1, n}$  do  $S[m, j] \leftarrow a_{mj}$ ;
  for  $i = \overline{m-1, 1, -1}$  do
     $P[i, 1] \leftarrow 0$ ;
    if  $S[i+1, 2] > S[i+1, 1]$  then  $P[i, 1] \leftarrow 1$ ;
     $S[i, 1] \leftarrow a_{i1} + S[i+1, 1 + P[i, 1]]$ ;
    for  $j = \overline{2, n-1}$  do
       $P[i, j] \leftarrow -1$ ;
      if  $S[i+1, j] > S[i+1, j-1]$  then  $P[i, j] \leftarrow 0$ ;
      if  $S[i+1, j+1] > S[i+1, j + P[i, j]]$  then  $P[i, j] \leftarrow 1$ ;
       $S[i, j] \leftarrow a_{ij} + S[i+1, j + P[i, j]]$ ;
     $P[i, n] \leftarrow -1$ ;
    if  $S[i+1, n] > S[i+1, n-1]$  then  $P[i, n] \leftarrow 0$ ;
     $S[i, n] \leftarrow a_{in} + S[i+1, n + P[i, n]]$ ;
   $M \leftarrow S[1, 1]$ ;
   $t_1 \leftarrow 1$ ;
  for  $j = \overline{2, n}$  do
    if  $S[1, j] > M$  then
       $M \leftarrow S[1, j]$ ;
       $t_1 \leftarrow j$ ;
  for  $i = \overline{2, m}$  do                                     // (*)
     $t_i \leftarrow t_{i-1} + P[i-1, t_{i-1}]$ ;
  afișează  $M$ ;
  for  $i = \overline{1, m}$  do
    afișează  $a_{it_i}$ ;

```

Exemplul 6.5.2. La aplicarea algoritmului anterior pentru matricea A din Exemplul 6.5.1 avem

$$S = \begin{pmatrix} 235 & 235 & 215 & 240 & 225 & 230 \\ 195 & 195 & 205 & 195 & 205 & 180 \\ 145 & 150 & 165 & 165 & 135 & 120 \\ 125 & 105 & 100 & 115 & 105 & 110 \\ 80 & 90 & 80 & 75 & 85 & 90 \\ 40 & 20 & 15 & 20 & 10 & 10 \end{pmatrix},$$

$$P = \begin{pmatrix} 0 & 1 & 0 & -1 & 0 & -1 \\ 1 & 1 & 0 & -1 & -1 & -1 \\ 0 & -1 & 1 & 0 & -1 & 0 \\ 1 & 0 & -1 & 1 & 1 & 0 \\ 0 & -1 & -1 & 0 & -1 & -1 \end{pmatrix}.$$

Suma maximă are valoarea

$$M = \max_{j=\overline{1,6}} S[1, j] = 240.$$

Avem

$$\begin{aligned} t_1 &= \min_{j=\overline{1,6}} \{j \mid S[1, j] = M\} = 4, \\ t_2 &= t_1 + P[1, t_1] = 4 + P[1, 4] = 4 - 1 = 3, \\ t_3 &= t_2 + P[2, t_2] = 3 + P[2, 3] = 3 + 0 = 3, \\ t_4 &= t_3 + P[3, t_3] = 3 + P[3, 3] = 3 + 1 = 4, \\ t_5 &= t_4 + P[4, t_4] = 4 + P[4, 4] = 4 + 1 = 5, \\ t_6 &= t_5 + P[5, t_5] = 5 + P[5, 5] = 5 - 1 = 4, \end{aligned}$$

deci drumul de sumă maximă produs de algoritm are elementele

$$\begin{aligned} a_{1t_1} &= a_{14} = 35, a_{2t_2} = a_{23} = 40, a_{3t_3} = a_{33} = 50, \\ a_{4t_4} &= a_{44} = 30, a_{5t_5} = a_{55} = 65, a_{6t_6} = a_{64} = 20. \end{aligned}$$

Observația 6.5.1. Pentru calculul fiecărui element al matricelor S și P sunt necesare câte o atribuire și cel mult două comparații, deci algoritmul are complexitatea $\Theta(mn)$.

Observația 6.5.2. Problema poate fi rezolvată și fără utilizarea matricei P . În acest caz, pentru determinarea unui drum de sumă maximă înlocuim instrucțiunea (*) cu următoarea secvență de instrucțiuni:

```

for  $i = \overline{1, m-1}$  do
    if ( $t_i > 1$ ) and ( $S[i+1, t_i-1] = S[i, t_i] - a_{it_i}$ ) then
        |  $t_{i+1} \leftarrow t_i - 1;$                                 // coborâre de forma  $\swarrow$ 
    else
        | if  $S[i+1, t_i] = S[i, t_i] - a_{it_i}$  then
            | |  $t_{i+1} \leftarrow t_i;$                                 // coborâre de forma  $\downarrow$ 
        | else
            | |  $t_{i+1} \leftarrow t_i + 1;$                             // coborâre de forma  $\searrow$ 

```

Observația 6.5.3. Algoritmul 6.5.1 poate fi adaptat pentru determinarea **tuturor** drumurilor de sumă maximă. Pentru aceasta, păstrând semnificația și construcția matricei S , reconsiderăm semnificația și construcția matricei P astfel:

$$P[i, j] = \left\{ k \in \{-1, 0, 1\} \mid \text{există un drum de sumă maximă de la } a_{ij} \right. \\ \left. \text{la ultima linie având al doilea element pe } a_{i+1, j+k} \right\},$$

pentru orice $i \in \{1, 2, \dots, m-1\}$ și orice $j \in \{1, 2, \dots, n\}$.

Evident, pentru orice $i \in \{1, 2, \dots, m-1\}$ avem

$$\begin{aligned} P[i, 1] &= \left\{ k \in \{0, 1\} \mid a_{i+1,1+k} = \max\{a_{i+1,1}, a_{i+1,2}\} \right\}, \\ P[i, j] &= \left\{ k \in \{-1, 0, 1\} \mid a_{i+1,j+k} = \max\{a_{i+1,j-1}, a_{i+1,j}, a_{i+1,j+1}\} \right\}, \\ &\quad \forall j \in \{2, \dots, n-1\}, \\ P[i, n] &= \left\{ k \in \{-1, 0\} \mid a_{i+1,n+k} = \max\{a_{i+1,n-1}, a_{i+1,n}\} \right\}. \end{aligned}$$

Orice soluție a problemei este un drum de forma

$$a_{1t_1}, a_{2t_2}, \dots, a_{mt_m}$$

de sumă

$$M = \max_{j=\overline{1,n}} S[1, j],$$

construit astfel:

$$\begin{aligned} t_1 &\in \{j = \overline{1, n} \mid S[1, j] = M\}, \\ t_i &\in \{t_{i-1} + k \mid k \in P[i-1, t_{i-1}]\}, \quad \forall i \in \{2, \dots, m\}. \end{aligned}$$

Pentru generarea tuturor acestor vectori $(t_1, t_2, \dots, t_m)$ se poate utiliza *metoda Backtracking*.

De exemplu, pentru matricea A din Exemplul 6.5.1 avem

$$\begin{aligned} S &= \begin{pmatrix} 235 & 235 & 215 & 240 & 225 & 230 \\ 195 & 195 & 205 & 195 & 205 & 180 \\ 145 & 150 & 165 & 165 & 135 & 120 \\ 125 & 105 & 100 & 115 & 105 & 110 \\ 80 & 90 & 80 & 75 & 85 & 90 \\ 40 & 20 & 15 & 20 & 10 & 10 \end{pmatrix}, \\ P &= \begin{pmatrix} \{0, 1\} & \{1\} & \{0\} & \{-1, 1\} & \{0\} & \{-1\} \\ \{1\} & \{1\} & \{0, 1\} & \{-1, 0\} & \{-1\} & \{-1\} \\ \{0\} & \{-1\} & \{1\} & \{0\} & \{-1\} & \{0\} \\ \{1\} & \{0\} & \{-1\} & \{1\} & \{1\} & \{0\} \\ \{0\} & \{-1\} & \{-1, 1\} & \{0\} & \{-1\} & \{-1, 0\} \end{pmatrix}. \end{aligned}$$

Suma maximă are valoarea $M = \max_{j=\overline{1,6}} S[1, j] = 240$. Avem

$$\begin{aligned} t_1 &\in \{j = \overline{1,6} \mid S[1, j] = M\} = \{4\}, \\ t_2 &\in \{4 + k \mid k \in P[1, 4]\} = \{3, 5\}, \\ \text{pentru } t_2 = 3 : t_3 &\in \{3 + k \mid k \in P[2, 3]\} = \{3, 4\}, \\ \text{pentru } t_2 = 5 : t_3 &\in \{5 + k \mid k \in P[2, 5]\} = \{4\}, \\ \text{pentru } t_3 = 3 : t_4 &\in \{3 + k \mid k \in P[3, 3]\} = \{4\}, \\ \text{pentru } t_3 = 4 : t_4 &\in \{4 + k \mid k \in P[3, 4]\} = \{4\}, \\ t_5 &\in \{4 + k \mid k \in P[4, 4]\} = \{5\}, \\ t_6 &\in \{5 + k \mid k \in P[5, 5]\} = \{4\}, \end{aligned}$$

deci drumurile de sumă maximă sunt

$$\begin{aligned} (a_{14} = 35, a_{23} = 40, a_{33} = 50, a_{44} = 30, a_{55} = 65, a_{64} = 20), \\ (a_{14} = 35, a_{23} = 40, a_{34} = 50, a_{44} = 30, a_{55} = 65, a_{64} = 20), \\ (a_{14} = 35, a_{25} = 40, a_{34} = 50, a_{44} = 30, a_{55} = 65, a_{64} = 20), \end{aligned}$$

deci avem 3 soluții (drumuri de sumă maximă).

Metoda 2) Utilizăm *programarea dinamică, varianta înainte*. Considerăm subproblemele constând în determinarea a câte unui drum de sumă maximă care se încheie cu elementul a_{ij} (și se începe cu un element de pe prima linie), pentru orice $i \in \{1, 2, \dots, m\}$ și orice $j \in \{1, 2, \dots, n\}$.

Pentru memorarea soluțiilor subproblemelor utilizăm din nou două matrice $S = (S[i, j])_{\substack{i=\overline{1,m} \\ j=\overline{1,n}}}$ și $P = (P[i, j])_{\substack{i=\overline{1,m-1} \\ j=\overline{1,n}}}$ având acum semnificația

$$\begin{aligned} S[i, j] &= \text{suma maximă a unui drum de la prima linie la } a_{ij}, \\ P[i-1, j] &= \begin{cases} 1, & \text{dacă există un drum de sumă maximă de la prima} \\ & \text{linie la } a_{ij} \text{ având penultimul element pe } a_{i-1, j-1} \\ & \text{(adică ultima coborâre are forma } \searrow), \\ 0, & \text{dacă există un drum de sumă maximă de la prima} \\ & \text{linie la } a_{ij} \text{ având penultimul element pe } a_{i-1, j} \\ & \text{(adică ultima coborâre are forma } \downarrow) \text{ și nu există un} \\ & \text{drum de sumă maximă de la prima linie la } a_{ij} \text{ având} \\ & \text{penultimul element pe } a_{i-1, j-1}, \\ -1, & \text{dacă există un drum de sumă maximă de la prima} \\ & \text{linie la } a_{ij} \text{ având penultimul element pe } a_{i-1, j+1} \\ & \text{(adică ultima coborâre are forma } \swarrow) \text{ și nu există un} \\ & \text{drum de sumă maximă de la prima linie la } a_{ij} \text{ având} \\ & \text{penultimul element pe } a_{i-1, j-1} \text{ sau pe } a_{i-1, j}. \end{cases} \end{aligned}$$

Evident,

$$S[1, j] = a_{1j}, \quad \forall j \in \{1, \dots, n\}$$

și pentru orice $i \in \{2, \dots, m\}$ avem

$$S[i, 1] = a_{i1} + \max\{S[i-1, 1], S[i-1, 2]\},$$

$$P[i-1, 1] = \begin{cases} 0, & \text{dacă } S[i-1, 1] \geq S[i-1, 2], \\ -1, & \text{dacă } S[i-1, 1] < S[i-1, 2], \end{cases}$$

$$S[i, j] = a_{ij} + \max\{S[i-1, j-1], S[i-1, j], S[i-1, j+1]\},$$

$$\forall j \in \{2, \dots, n-1\}$$

$$P[i-1, j] = \begin{cases} 1, & \text{dacă } S[i-1, j-1] \geq S[i-1, j] \text{ și} \\ & S[i-1, j-1] \geq S[i-1, j+1], \\ 0, & \text{dacă } S[i-1, j] > S[i-1, j-1] \text{ și} \\ & S[i-1, j] \geq S[i-1, j+1], \\ -1, & \text{dacă } S[i-1, j+1] > S[i-1, j-1] \text{ și} \\ & S[i-1, j+1] > S[i-1, j], \end{cases}$$

$$\forall j \in \{2, \dots, n-1\}$$

$$S[i, n] = a_{in} + \max\{S[i-1, n-1], S[i-1, n]\},$$

$$P[i-1, n] = \begin{cases} 1, & \text{dacă } S[i-1, n-1] \geq S[i-1, n], \\ 0, & \text{dacă } S[i-1, n-1] < S[i-1, n]. \end{cases}$$

Relațiile de recurență de mai sus exprimă elementul $S[i, j]$ în funcție de elementele de pe linia anterioară $S[i-1, j-1], S[i-1, j], S[i-1, j+1]$.

Rezolvarea relației de recurență a elementelor $S[i, j]$ și calculul elementelor $P[i, j]$ conform strategiei *bottom-up* revine la calculul acestor elemente în ordinea directă a liniilor, adică

$$i = 1, 2, \dots, m-1, m.$$

O soluție a problemei este drumul

$$a_{1t_1}, a_{2t_2}, \dots, a_{mt_m}$$

de sumă

$$M = \max_{j=1, \dots, n} S[m, j],$$

construit astfel:

$$t_m = \min_{j=1, n} \{j \mid S[m, j] = M\},$$

$$t_i = t_{i+1} - P[i, t_{i+1}], \forall i \in \{1, \dots, m-1\}.$$

Obținem următorul *algoritm de programare dinamică în varianta înainte*, descris în pseudocod, pentru rezolvarea problemei.

Algoritmul 6.5.2.

```

DRUMMAX2( $A, m, n$ ) :
  for  $j = \overline{1, n}$  do  $S[1, j] \leftarrow a_{1j}$ ;
  for  $i = \overline{1, m-1}$  do
     $P[i, 1] \leftarrow 0$ ;
    if  $S[i, 2] > S[i, 1]$  then  $P[i, 1] \leftarrow -1$ ;
     $S[i+1, 1] \leftarrow a_{i+1,1} + S[i, 1 - P[i, 1]]$ ;
    for  $j = \overline{2, n-1}$  do
       $P[i, j] \leftarrow 1$ ;
      if  $S[i, j] > S[i, j-1]$  then  $P[i, j] \leftarrow 0$ ;
      if  $S[i, j+1] > S[i, j - P[i, j]]$  then  $P[i, j] \leftarrow -1$ ;
       $S[i+1, j] \leftarrow a_{i+1,j} + S[i, j - P[i, j]]$ ;
     $P[i, n] \leftarrow 1$ ;
    if  $S[i, n] > S[i, n-1]$  then  $P[i, n] \leftarrow 0$ ;
     $S[i+1, n] \leftarrow a_{i+1,n} + S[i, n - P[i, n]]$ ;
   $M \leftarrow S[m, 1]$ ;
   $t_m \leftarrow 1$ ;
  for  $j = \overline{2, n}$  do
    if  $S[m, j] > M$  then
       $M \leftarrow S[m, j]$ ;
       $t_m \leftarrow j$ ;
  for  $i = \overline{m-1, 1, -1}$  do                                // (*)
     $t_i \leftarrow t_{i+1} - P[i, t_{i+1}]$ ;
  afișează  $M$ ;
  for  $i = \overline{1, m}$  do
    afișează  $a_{it_i}$ ;

```

Exemplul 6.5.3. La aplicarea algoritmului anterior pentru matricea A din

Exemplul 6.5.1 avem

$$S = \begin{pmatrix} 40 & 30 & 10 & 35 & 20 & 25 \\ 85 & 70 & 75 & 65 & 75 & 70 \\ 105 & 110 & 125 & 125 & 95 & 85 \\ 145 & 140 & 135 & 155 & 140 & 115 \\ 185 & 195 & 215 & 210 & 220 & 220 \\ 235 & 235 & 230 & 240 & 230 & 230 \end{pmatrix},$$

$$P = \begin{pmatrix} 0 & 1 & -1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ -1 & -1 & 0 & 1 & 1 & 1 \\ 0 & 1 & -1 & 0 & 1 & 1 \\ -1 & -1 & 0 & -1 & 0 & 1 \end{pmatrix}.$$

Suma maximă are valoarea

$$M = \max_{j=\overline{1,6}} S[6, j] = 240.$$

Avem

$$\begin{aligned} t_6 &= \min_{j=\overline{1,6}} \{j \mid S[6, j] = M\} = 4, \\ t_5 &= t_6 - P[5, t_6] = 4 - P[5, 4] = 4 + 1 = 5, \\ t_4 &= t_5 - P[4, t_5] = 5 - P[4, 5] = 5 - 1 = 4, \\ t_3 &= t_4 - P[3, t_4] = 4 - P[3, 4] = 4 - 1 = 3, \\ t_2 &= t_3 - P[2, t_3] = 3 - P[2, 3] = 3 - 0 = 3, \\ t_1 &= t_2 - P[1, t_2] = 3 - P[1, 3] = 3 + 1 = 4, \end{aligned}$$

deci drumul de sumă maximă produs de algoritm are elementele

$$\begin{aligned} a_{1t_1} &= a_{14} = 35, a_{2t_2} = a_{23} = 40, a_{3t_3} = a_{33} = 50, \\ a_{4t_4} &= a_{44} = 30, a_{5t_5} = a_{55} = 65, a_{6t_6} = a_{64} = 20. \end{aligned}$$

Observația 6.5.4. Analog algoritmului de la Metoda 1, și algoritmul anterior are complexitatea $\Theta(mn)$.

Observația 6.5.5. Problema poate fi rezolvată, din nou, și fără utilizarea matricei P . În acest caz, pentru determinarea unui drum de sumă maximă înlocuim instrucțiunea (*) cu următoarea secvență de instrucțiuni:

```

for  $i = \overline{m, 2, -1}$  do
  if  $(t_i > 1)$  and  $(S[i - 1, t_i - 1] = S[i, t_i] - a_{it_i})$  then
     $t_{i-1} \leftarrow t_i - 1;$  // coborâre de forma  $\searrow$ 
  else
    if  $S[i - 1, t_i] = S[i, t_i] - a_{it_i}$  then
       $t_{i-1} \leftarrow t_i;$  // coborâre de forma  $\downarrow$ 
    else
       $t_{i-1} \leftarrow t_i + 1;$  // coborâre de forma  $\swarrow$ 

```

Observația 6.5.6. Analog Algoritmului 6.5.1, și Algoritmul 6.5.2 poate fi adaptat pentru determinarea **tuturor** drumurilor de sumă maximă.

b) Considerăm problema drumului de sumă minimă.

Metoda 1) Utilizăm *programarea dinamică, varianta înapoi*. Considerăm subproblemele constând în determinarea a câte unui drum de sumă minimă care începe cu elementul a_{ij} (și se încheie cu un element de pe ultima linie), pentru orice $i \in \{1, 2, \dots, m\}$ și orice $j \in \{1, 2, \dots, n\}$.

Pentru memorarea soluțiilor subproblemelor utilizăm două matrice $S = (S[i, j])_{\substack{i = \overline{1, m} \\ j = \overline{1, n}}}$ și $P = (P[i, j])_{\substack{i = \overline{1, m-1} \\ j = \overline{1, n}}}$ având semnificația

$$\begin{aligned}
 &S[i, j] = \text{suma minimă a unui drum de la } a_{ij} \text{ la ultima linie,} \\
 &P[i, j] = \begin{cases} -1, & \text{dacă există un drum de sumă minimă de la } a_{ij} \text{ la ultima} \\ & \text{linie având al doilea element pe } a_{i+1, j-1} \text{ (adică prima} \\ & \text{coborâre are forma } \swarrow), \\ 0, & \text{dacă există un drum de sumă minimă de la } a_{ij} \text{ la ultima} \\ & \text{linie având al doilea element pe } a_{i+1, j} \text{ (adică prima} \\ & \text{coborâre are forma } \downarrow) \text{ și nu există un drum de sumă} \\ & \text{minimă de la } a_{ij} \text{ la ultima linie având al doilea element} \\ & \text{pe } a_{i+1, j-1}, \\ 1, & \text{dacă există un drum de sumă minimă de la } a_{ij} \text{ la ultima} \\ & \text{linie având al doilea element pe } a_{i+1, j+1} \text{ (adică prima} \\ & \text{coborâre are forma } \searrow) \text{ și nu există un drum de sumă} \\ & \text{minimă de la } a_{ij} \text{ la ultima linie având al doilea element} \\ & \text{pe } a_{i+1, j-1} \text{ sau pe } a_{i+1, j}. \end{cases}
 \end{aligned}$$

Evident,

$$S[m, j] = a_{mj}, \quad \forall j \in \{1, \dots, n\}$$

și pentru orice $i \in \{1, \dots, m-1\}$ avem

$$S[i, 1] = a_{i1} + \min\{S[i+1, 1], S[i+1, 2]\},$$

$$P[i, 1] = \begin{cases} 0, & \text{dacă } S[i+1, 1] \leq S[i+1, 2], \\ 1, & \text{dacă } S[i+1, 1] > S[i+1, 2], \end{cases}$$

$$S[i, j] = a_{ij} + \min\{S[i+1, j-1], S[i+1, j], S[i+1, j+1]\},$$

$$\forall j \in \{2, \dots, n-1\}$$

$$P[i, j] = \begin{cases} -1, & \text{dacă } S[i+1, j-1] \leq S[i+1, j] \text{ și} \\ & S[i+1, j-1] \leq S[i+1, j+1], \\ 0, & \text{dacă } S[i+1, j] < S[i+1, j-1] \text{ și} \\ & S[i+1, j] \leq S[i+1, j+1], \\ 1, & \text{dacă } S[i+1, j+1] < S[i+1, j-1] \text{ și} \\ & S[i+1, j+1] < S[i+1, j], \end{cases}$$

$$\forall j \in \{2, \dots, n-1\}$$

$$S[i, n] = a_{in} + \min\{S[i+1, n-1], S[i+1, n]\},$$

$$P[i, n] = \begin{cases} -1, & \text{dacă } S[i+1, n-1] \leq S[i+1, n], \\ 0, & \text{dacă } S[i+1, n-1] > S[i+1, n]. \end{cases}$$

Relațiile de recurență de mai sus exprimă elementul $S[i, j]$ în funcție de elementele de pe linia următoare $S[i+1, j-1], S[i+1, j], S[i+1, j+1]$.

Rezolvarea relației de recurență a elementelor $S[i, j]$ și calculul elementelor $P[i, j]$ conform strategiei *bottom-up* revine la calculul acestor elemente în ordinea inversă a liniilor, adică

$$i = m, m-1, \dots, 2, 1.$$

O soluție a problemei este drumul

$$a_{1t_1}, a_{2t_2}, \dots, a_{mt_m}$$

de sumă

$$M = \min_{j=1, n} S[1, j],$$

construit astfel:

$$t_1 = \min_{j=1, n} \{j \mid S[1, j] = M\},$$

$$t_i = t_{i-1} + P[i-1, t_{i-1}], \forall i \in \{2, \dots, m\}.$$

Obținem următorul *algoritm de programare dinamică în varianta înapoi*, descris în pseudocod, pentru rezolvarea problemei.

Algoritmul 6.5.3.

```

DRUMMIN1( $A, m, n$ ):
  for  $j = \overline{1, n}$  do  $S[m, j] \leftarrow a_{mj}$ ;
  for  $i = m-1, 1, -1$  do
     $P[i, 1] \leftarrow 0$ ;
    if  $S[i+1, 2] < S[i+1, 1]$  then  $P[i, 1] \leftarrow 1$ ;
     $S[i, 1] \leftarrow a_{i1} + S[i+1, 1 + P[i, 1]]$ ;
    for  $j = \overline{2, n-1}$  do
       $P[i, j] \leftarrow -1$ ;
      if  $S[i+1, j] < S[i+1, j-1]$  then  $P[i, j] \leftarrow 0$ ;
      if  $S[i+1, j+1] < S[i+1, j + P[i, j]]$  then  $P[i, j] \leftarrow 1$ ;
       $S[i, j] \leftarrow a_{ij} + S[i+1, j + P[i, j]]$ ;
     $P[i, n] \leftarrow -1$ ;
    if  $S[i+1, n] < S[i+1, n-1]$  then  $P[i, n] \leftarrow 0$ ;
     $S[i, n] \leftarrow a_{in} + S[i+1, n + P[i, n]]$ ;
   $M \leftarrow S[1, 1]$ ;
   $t_1 \leftarrow 1$ ;
  for  $j = \overline{2, n}$  do
    if  $S[1, j] < M$  then
       $M \leftarrow S[1, j]$ ;
       $t_1 \leftarrow j$ ;
  for  $i = \overline{2, m}$  do                                     // (*)
     $t_i \leftarrow t_{i-1} + P[i-1, t_{i-1}]$ ;
  afișează  $M$ ;
  for  $i = \overline{1, m}$  do
    afișează  $a_{it_i}$ ;

```

Exemplul 6.5.4. La aplicarea algoritmului anterior pentru matricea A din Exemplul 6.5.1 avem

$$S = \begin{pmatrix} 165 & 155 & 135 & 165 & 150 & 155 \\ 140 & 125 & 140 & 130 & 130 & 135 \\ 95 & 100 & 125 & 125 & 100 & 90 \\ 95 & 75 & 75 & 95 & 80 & 95 \\ 60 & 65 & 75 & 65 & 75 & 90 \\ 40 & 20 & 15 & 20 & 10 & 10 \end{pmatrix},$$

$$P = \begin{pmatrix} 1 & 0 & -1 & 0 & -1 & -1 \\ 0 & -1 & -1 & 1 & 1 & 0 \\ 1 & 0 & -1 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & -1 \\ 1 & 1 & 0 & 1 & 0 & -1 \end{pmatrix}.$$

Suma minimă are valoarea

$$M = \min_{j=\overline{1,6}} S[1, j] = 135.$$

Avem

$$\begin{aligned} t_1 &= \min_{j=\overline{1,6}} \{j \mid S[1, j] = M\} = 3, \\ t_2 &= t_1 + P[1, t_1] = 3 + P[1, 3] = 3 - 1 = 2, \\ t_3 &= t_2 + P[2, t_2] = 2 + P[2, 2] = 2 - 1 = 1, \\ t_4 &= t_3 + P[3, t_3] = 1 + P[3, 1] = 1 + 1 = 2, \\ t_5 &= t_4 + P[4, t_4] = 2 + P[4, 2] = 2 - 1 = 1, \\ t_6 &= t_5 + P[5, t_5] = 1 + P[5, 1] = 1 + 1 = 2, \end{aligned}$$

deci drumul de sumă minimă produs de algoritm are elementele

$$\begin{aligned} a_{1t_1} &= a_{13} = 10, a_{2t_2} = a_{22} = 30, a_{3t_3} = a_{31} = 20, \\ a_{4t_4} &= a_{42} = 15, a_{5t_5} = a_{51} = 40, a_{6t_6} = a_{62} = 20. \end{aligned}$$

Observația 6.5.7. Problema poate fi rezolvată și fără utilizarea matricei P . În acest caz, pentru determinarea unui drum de sumă minimă înlocuim instrucțiunea (*) cu secvența de instrucțiuni din Observația 6.5.2.

Metoda 2) Utilizăm *programarea dinamică, varianta înainte*. Considerăm subproblemele constând în determinarea a câte unui drum de sumă minimă care se încheie cu elementul a_{ij} (și se începe cu un element de pe prima linie), pentru orice $i \in \{1, 2, \dots, m\}$ și orice $j \in \{1, 2, \dots, n\}$.

Pentru memorarea soluțiilor subproblemelor utilizăm din nou două ma-

trice $S = (S[i, j])_{\substack{i = \overline{1, m} \\ j = \overline{1, n}}}$ și $P = (P[i, j])_{\substack{i = \overline{1, m-1} \\ j = \overline{1, n}}}$ având acum semnificația

$$S[i, j] = \text{suma minimă a unui drum de la prima linie la } a_{ij},$$

$$P[i-1, j] = \begin{cases} 1, & \text{dacă există un drum de sumă minimă de la prima} \\ & \text{linie la } a_{ij} \text{ având penultimul element pe } a_{i-1, j-1} \\ & \text{(adică ultima coborâre are forma } \searrow \text{)}, \\ 0, & \text{dacă există un drum de sumă minimă de la prima} \\ & \text{linie la } a_{ij} \text{ având penultimul element pe } a_{i-1, j} \\ & \text{(adică ultima coborâre are forma } \downarrow \text{) și nu există un} \\ & \text{drum de sumă minimă de la prima linie la } a_{ij} \text{ având} \\ & \text{penultimul element pe } a_{i-1, j-1}, \\ -1, & \text{dacă există un drum de sumă minimă de la prima} \\ & \text{linie la } a_{ij} \text{ având penultimul element pe } a_{i-1, j+1} \\ & \text{(adică ultima coborâre are forma } \swarrow \text{) și nu există un} \\ & \text{drum de sumă minimă de la prima linie la } a_{ij} \text{ având} \\ & \text{penultimul element pe } a_{i-1, j-1} \text{ sau pe } a_{i-1, j}. \end{cases}$$

Evident,

$$S[1, j] = a_{1j}, \quad \forall j \in \{1, \dots, n\}$$

și pentru orice $i \in \{2, \dots, m\}$ avem

$$S[i, 1] = a_{i1} + \min\{S[i-1, 1], S[i-1, 2]\},$$

$$P[i-1, 1] = \begin{cases} 0, & \text{dacă } S[i-1, 1] \leq S[i-1, 2], \\ -1, & \text{dacă } S[i-1, 1] > S[i-1, 2], \end{cases}$$

$$S[i, j] = a_{ij} + \min\{S[i-1, j-1], S[i-1, j], S[i-1, j+1]\},$$

$$\forall j \in \{2, \dots, n-1\}$$

$$P[i-1, j] = \begin{cases} 1, & \text{dacă } S[i-1, j-1] \leq S[i-1, j] \text{ și} \\ & S[i-1, j-1] \leq S[i-1, j+1], \\ 0, & \text{dacă } S[i-1, j] < S[i-1, j-1] \text{ și} \\ & S[i-1, j] \leq S[i-1, j+1], \\ -1, & \text{dacă } S[i-1, j+1] < S[i-1, j-1] \text{ și} \\ & S[i-1, j+1] < S[i-1, j], \\ \end{cases}$$

$$\forall j \in \{2, \dots, n-1\}$$

$$S[i, n] = a_{in} + \min\{S[i-1, n-1], S[i-1, n]\},$$

$$P[i-1, n] = \begin{cases} 1, & \text{dacă } S[i-1, n-1] \leq S[i-1, n], \\ 0, & \text{dacă } S[i-1, n-1] > S[i-1, n]. \end{cases}$$

Relațiile de recurență de mai sus exprimă elementul $S[i, j]$ în funcție de elementele de pe linia anterioară $S[i-1, j-1], S[i-1, j], S[i-1, j+1]$.

Rezolvarea relației de recurență a elementelor $S[i, j]$ și calculul elementelor $P[i, j]$ conform strategiei *bottom-up* revine la calculul acestor elemente în ordinea directă a liniilor, adică

$$i = 1, 2, \dots, m-1, m.$$

O soluție a problemei este drumul

$$a_{1t_1}, a_{2t_2}, \dots, a_{mt_m}$$

de sumă

$$M = \min_{j=1, n} S[m, j],$$

construit astfel:

$$t_m = \min_{j=1, n} \{j \mid S[m, j] = M\},$$

$$t_i = t_{i+1} - P[i, t_{i+1}], \quad \forall i \in \{1, \dots, m-1\}.$$

Obținem următorul *algorithm de programare dinamică în varianta înainte*, descris în pseudocod, pentru rezolvarea problemei.

Algoritmul 6.5.4.

```

DRUMMIN2( $A, m, n$ ) :
  for  $j = \overline{1, n}$  do  $S[1, j] \leftarrow a_{1j}$ ;
  for  $i = \overline{1, m-1}$  do
     $P[i, 1] \leftarrow 0$ ;
    if  $S[i, 2] < S[i, 1]$  then  $P[i, 1] \leftarrow -1$ ;
     $S[i+1, 1] \leftarrow a_{i+1,1} + S[i, 1 - P[i, 1]]$ ;
    for  $j = \overline{2, n-1}$  do
       $P[i, j] \leftarrow 1$ ;
      if  $S[i, j] < S[i, j-1]$  then  $P[i, j] \leftarrow 0$ ;
      if  $S[i, j+1] < S[i, j - P[i, j]]$  then  $P[i, j] \leftarrow -1$ ;
       $S[i+1, j] \leftarrow a_{i+1,j} + S[i, j - P[i, j]]$ ;
     $P[i, n] \leftarrow 1$ ;
    if  $S[i, n] < S[i, n-1]$  then  $P[i, n] \leftarrow 0$ ;
     $S[i+1, n] \leftarrow a_{i+1,n} + S[i, n - P[i, n]]$ ;
   $M \leftarrow S[m, 1]$ ;
   $t_m \leftarrow 1$ ;
  for  $j = \overline{2, n}$  do
    if  $S[m, j] < M$  then
       $M \leftarrow S[m, j]$ ;
       $t_m \leftarrow j$ ;
  for  $i = \overline{m-1, 1, -1}$  do                                     // (*)
     $t_i \leftarrow t_{i+1} - P[i, t_{i+1}]$ ;
  afișează  $M$ ;
  for  $i = \overline{1, m}$  do
    afișează  $a_{it_i}$ ;

```

Exemplul 6.5.5. La aplicarea algoritmului anterior pentru matricea A din Exemplul 6.5.1 avem

$$S = \begin{pmatrix} 40 & 30 & 10 & 35 & 20 & 25 \\ 75 & 40 & 50 & 40 & 60 & 65 \\ 60 & 65 & 90 & 90 & 60 & 70 \\ 95 & 75 & 75 & 90 & 75 & 80 \\ 115 & 125 & 135 & 130 & 140 & 155 \\ 155 & 135 & 140 & 150 & 140 & 150 \end{pmatrix},$$

$$P = \begin{pmatrix} -1 & -1 & 0 & 1 & 0 & 1 \\ -1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & -1 & 0 & 1 \\ -1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 \end{pmatrix}.$$

Suma minimă are valoarea

$$M = \min_{j=\overline{1,6}} S[6, j] = 135.$$

Avem

$$\begin{aligned} t_6 &= \min_{j=\overline{1,6}} \{j \mid S[6, j] = M\} = 2, \\ t_5 &= t_6 - P[5, t_6] = 2 - P[5, 2] = 2 - 1 = 1, \\ t_4 &= t_5 - P[4, t_5] = 1 - P[4, 1] = 1 + 1 = 2, \\ t_3 &= t_4 - P[3, t_4] = 2 - P[3, 2] = 2 - 1 = 1, \\ t_2 &= t_3 - P[2, t_3] = 1 - P[2, 1] = 1 + 1 = 2, \\ t_1 &= t_2 - P[1, t_2] = 2 - P[1, 2] = 2 + 1 = 3, \end{aligned}$$

deci drumul de sumă minimă produs de algoritm are elementele

$$\begin{aligned} a_{1t_1} &= a_{13} = 10, a_{2t_2} = a_{22} = 30, a_{3t_3} = a_{31} = 20, \\ a_{4t_4} &= a_{42} = 15, a_{5t_5} = a_{51} = 40, a_{6t_6} = a_{62} = 20. \end{aligned}$$

Observația 6.5.8. Problema poate fi rezolvată, din nou, și fără utilizarea matricei P . În acest caz, pentru determinarea unui drum de sumă minimă înlocuim instrucțiunea (*) cu secvența de instrucțiuni din Observația 6.5.5.

Observația 6.5.9. Analog Algoritmului 6.5.1, și Algoritmii 6.5.3 și 6.5.4 au complexitatea $\Theta(mn)$.

De asemenea, ei pot fi adaptați pentru determinarea **tuturor** drumurilor de sumă minimă.

6.6 Matricea drumurilor. Algoritmul Roy-Warshall

Definiția 6.6.1. Fie $G = (V, E)$ un graf (neorientat sau orientat), unde $V = \{v_1, \dots, v_n\}$. **Matricea drumurilor** asociată grafului G este matricea $D = (d_{ij})_{i,j=\overline{1,n}}$ definită prin

$$d_{ij} = \begin{cases} 1, & \text{dacă } \exists \mu = (v_i, \dots, v_j) \text{ drum cu } l(\mu) > 0, \\ 0, & \text{în caz contrar,} \end{cases}$$

unde $l(\mu)$ reprezintă lungimea drumului μ .

Exemplul 6.6.1. Matricea drumurilor asociată grafului neorientat din Figura 6.6.1

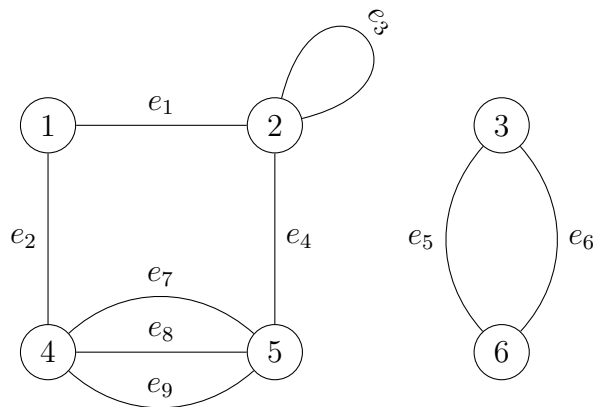


Figura 6.6.1:

este

$$D = \begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}.$$

Matricea drumurilor asociată grafului orientat din Figura 6.6.2 este

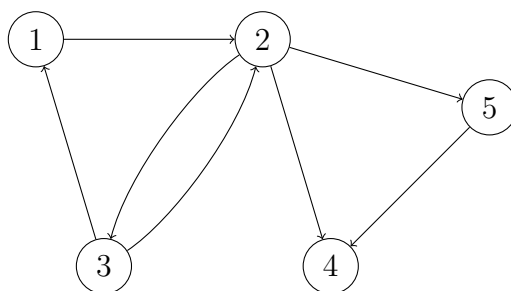


Figura 6.6.2:

$$D = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

Observația 6.6.1. Evident, orice graf neorientat are matricea drumurilor simetrică.

Observația 6.6.2. Pentru $i \neq j$ putem înlocui termenul de "drum" cu "drum elementar" în definiția matricei drumurilor.

Următorul algoritm determină matricea drumurilor unui graf pornind de la matricea de adiacență.

Algoritmul 6.6.1 (Roy-Warshall). Fie $G = (V, E)$ un graf simplu cu $V = \{v_1, \dots, v_n\}$ și fie $A = (a_{ij})_{i,j=\overline{1,n}}$ matricea sa de adiacență (având toate elementele 0 sau 1). Se calculează matricele

$$D^{(k)} = (d_{ij}^{(k)})_{i,j=\overline{1,n}}, \quad k \in \{0, 1, \dots, n\},$$

definite prin

$$D^{(0)} = A, \tag{6.6.1}$$

$$d_{ij}^{(k)} = d_{ij}^{(k-1)} \vee d_{ik}^{(k-1)} d_{kj}^{(k-1)}, \quad \forall k, i, j \in \{1, \dots, n\}, \tag{6.6.2}$$

unde $0 \vee 0 = 0$, $0 \vee 1 = 1 \vee 0 = 1 \vee 1 = 1$ (operația de disjuncție, "sau").

Teorema 6.6.1 (de corectitudine a Algoritmului Roy-Warshall). În contextul Algoritmului Roy-Warshall, ultima matrice calculată este chiar matricea drumurilor asociată grafului G , adică

$$D^{(n)} = D.$$

Demonstrație. Vom demonstra prin inducție după $k \in \{0, 1, \dots, n\}$ că pentru orice $i, j \in \{1, \dots, n\}$ avem

$$d_{ij}^{(k)} = \begin{cases} 1, & \text{dacă } \exists \mu = (v_i, \dots, v_j) \text{ drum cu } l(\mu) > 0 \text{ și } I(\mu) \subseteq \{v_1, \dots, v_k\}, \\ 0, & \text{în caz contrar,} \end{cases} \tag{6.6.3}$$

unde $l(\mu)$ reprezintă lungimea drumului μ , $I(\mu)$ reprezintă mulțimea nodurilor intermediare ale drumului μ , iar $\{v_1, \dots, v_k\}$ reprezintă mulțimea $\{v_i \mid 1 \leq i \leq k\}$, deci pentru $k = 0$ această mulțime este $\emptyset$.

Pentru $k = 0$ avem

$$d_{ij}^{(0)} = a_{ij} \text{ (conform (6.6.1)) și } a_{ij} = \begin{cases} 1, & \text{dacă } (v_i, v_j) \in E, \\ 0, & \text{în caz contrar} \end{cases}$$

(din definiția matricei de adiacență), iar $(v_i, v_j) \in E$ dacă și numai dacă $\exists \mu = (v_i, \dots, v_j)$ drum cu $l(\mu) > 0$ și $I(\mu) = \emptyset$, deci obținem egalitatea (6.6.3).

Presupunem acum egalitatea (6.6.3) adevărată pentru $k - 1$ ($1 \leq k \leq n$) și o demonstrăm pentru k . Folosind (6.6.2), definiția operațiilor $\vee$ și $\cdot$ și ipoteza de inducție avem echivalențele:

$$\begin{aligned} d_{ij}^{(k)} = 1 &\Leftrightarrow d_{ij}^{(k-1)} = 1 \text{ sau } d_{ik}^{(k-1)} = d_{kj}^{(k-1)} = 1 \Leftrightarrow \exists \mu = (v_i, \dots, v_j) \text{ drum cu} \\ &l(\mu) > 0, I(\mu) \subseteq \{v_1, \dots, v_{k-1}\} \text{ sau } \exists \mu_1 = (v_i, \dots, v_k), \mu_2 = (v_k, \dots, v_j) \\ &\text{drumuri cu } l(\mu_1), l(\mu_2) > 0, I(\mu_1), I(\mu_2) \subseteq \{v_1, \dots, v_{k-1}\} \\ &\Leftrightarrow \exists \mu = (v_i, \dots, v_j) \text{ cu } l(\mu) > 0, I(\mu) \subseteq \{v_1, \dots, v_k\}, v_k \notin I(\mu) \text{ sau} \\ &\exists \mu' = (v_i, \dots, v_k, \dots, v_j) \text{ cu } l(\mu') > 0, I(\mu') \subseteq \{v_1, \dots, v_k\}, v_k \in I(\mu') \end{aligned}$$

(μ' se obține parcurgând întâi μ_1 apoi μ_2 și, reciproc, μ_1, μ_2 sunt porțiunile din μ' dintre v_i și prima apariție a lui v_k în $I(\mu')$, respectiv dintre ultima apariție a lui v_k în $I(\mu')$ și v_j)

$$\Leftrightarrow \exists \mu = (v_i, \dots, v_j) \text{ drum cu } l(\mu) > 0 \text{ și } I(\mu) \subseteq \{v_1, \dots, v_k\}.$$

Demonstrația prin inducție a egalității (6.6.3) este astfel încheiată.

Pentru $k = n$ condiția $I(\mu) \subseteq \{v_1, \dots, v_n\}$ poate fi eliminată, fiind întotdeauna adevărată, deci din (6.6.3) și Definiția 6.6.1 obținem că

$$d_{ij}^{(n)} = d_{ij}, \quad \forall i, j \in \{1, \dots, n\},$$

adică egalitatea din enunț. □

Observația 6.6.3. Pentru n fixat, Algoritmul Roy-Warshall necesită $2n^3$ operații (câte o operație $\cdot$ și $\vee$ pentru fiecare $(k, i, j) \in \{1, \dots, n\} \times \{1, \dots, n\} \times \{1, \dots, n\}$), deci acest algoritm are complexitatea $\Theta(n^3)$.

Observația 6.6.4. Algoritmul Roy-Warshall rămâne evident valabil și pentru grafuri nesimple, cu modificarea

$$d_{ij}^{(0)} = \begin{cases} 1, & \text{dacă } a_{ij} > 0, \\ 0, & \text{dacă } a_{ij} = 0, \end{cases} \quad \forall i, j \in \{1, \dots, n\}.$$

Exemplul 6.6.2. Pentru graful din Figura 6.6.2, prin aplicarea Algoritmului Roy-Warshall obținem matricele:

$$D^{(0)} = A = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix} \text{ (matricea de adiacență);}$$

$$D^{(1)} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}; \quad D^{(2)} = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

(deoarece, de exemplu, $d_{13}^{(2)} = d_{13}^{(1)} \vee d_{12}^{(1)} d_{23}^{(1)} = 0 \vee 1 \cdot 1 = 0 \vee 1 = 1$);

$$D^{(3)} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}; \quad D^{(4)} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix};$$

$$D^{(5)} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix} = D \text{ (matricea drumurilor).}$$

Observația 6.6.5. Conform ecuațiilor (6.6.3), Algoritmul Roy-Warshall este un algoritm specific **metodei programării dinamice**.

Conform ecuațiilor (6.6.2) și definiției operației $\vee$ avem $d_{ik}^{(k)} = d_{ik}^{(k-1)}$ și $d_{kj}^{(k)} = d_{kj}^{(k-1)}$, $\forall k, i, j \in \{1, \dots, n\}$, deci pentru implementarea algoritmului putem utiliza o singură matrice D . Astfel obținem următoarea descriere în pseudocod a algoritmului.

```

ROY_WARSHALL :
for  $i = \overline{1, n}$  do                                     // inițializări
┌   for  $j = \overline{1, n}$  do
└   ┌  $d_{ij} \leftarrow a_{ij}$ ;
    for  $k = \overline{1, n}$  do
└   ┌   for  $i = \overline{1, n}$  do
└   └   ┌   for  $j = \overline{1, n}$  do
└   └   └    $d_{ij} \leftarrow d_{ij} \vee d_{ik}d_{kj}$ ;

```

Pentru grafuri oarecare (simple sau nesimple) inițializarea are forma

```

for  $i = \overline{1, n}$  do                                     // inițializări
┌   for  $j = \overline{1, n}$  do
└   ┌   if  $(a_{ij} > 0)$  then
└   └   ┌  $d_{ij} \leftarrow 1$ ;
└   └   └ else
└   └   └    $d_{ij} \leftarrow 0$ ;

```

Observația 6.6.6. Dacă $G = (V, E)$ este un graf neorientat și $V = \{1, \dots, n\}$, atunci pentru orice nod $x \in V$ componenta conexă a nodului x este (indusă de) mulțimea

$$\{x\} \cup \{y | y \in V \setminus \{x\}, d_{xy} = 1\}.$$

Astfel Algoritmul Roy-Warshall poate fi utilizat pentru determinarea componentelor conexe și pentru verificarea conexității grafului.

Algoritmul 6.6.2 (determinarea componentelor tare-conexe). Dacă $G = (V, E)$ este un graf orientat și $V = \{1, \dots, n\}$, atunci pentru orice nod $x \in V$ componenta tare-conexă a nodului x este (indusă de) mulțimea

$$\{x\} \cup \{y | y \in V \setminus \{x\}, d_{xy} = d_{yx} = 1\},$$

deci Algoritmul Roy-Warshall poate fi utilizat pentru determinarea componentelor tare-conexe și pentru verificarea tare-conexității grafului.

Descrierea în pseudocod a algoritmului are forma

COMPONENTE_TARE_CONEXE :

```

nrc ← 0; // numărul de componente tare-conexe
for i =  $\overline{1, n}$  do
  CC[i] ← 0; // vectorul componentelor tare-conexe
ROY_WARSHALL;
for i =  $\overline{1, n}$  do
  if (CC[i] = 0) then
    nrc ← nrc + 1;
    CC[i] ← nrc;
    for j =  $\overline{i+1, n}$  do
      if (CC[j] = 0) and ( $d_{ij} = 1$ ) and ( $d_{ji} = 1$ ) then
        CC[j] ← nrc;

```

unde $CC[i]$ reprezintă numărul componentei tare-conexe în care se află nodul i , $\forall i \in \{1, \dots, n\}$.

6.7 Distanțe și drumuri minime. Algoritmul Roy-Floyd

Vom expune un algoritm pentru determinarea distanțelor minime și a drumurilor minime între orice două noduri ale grafului ponderat dat. Acest algoritm este asemănător cu Algoritmul Roy-Warshall pentru determinarea matricei drumurilor.

Algoritmul 6.7.1 (Roy-Floyd). Fie (G, c) un graf ponderat, $G = (V, E)$, $V = \{v_1, v_2, \dots, v_n\}$, $c : E \rightarrow \mathbb{R}_+$. Fie $C = (c_{ij})_{i,j=\overline{1,n}}$ matricea distanțelor directe asociată grafului (G, c) . Se calculează matricele

$$C^{(k)} = (c_{ij}^{(k)})_{i,j=\overline{1,n}}, \quad k \in \{0, 1, \dots, n\}$$

definite prin

$$C^{(0)} = C, \tag{6.7.1}$$

$$c_{ij}^{(k)} = \min\{c_{ij}^{(k-1)}, c_{ik}^{(k-1)} + c_{kj}^{(k-1)}\}, \quad \forall k, i, j \in \{1, \dots, n\}. \tag{6.7.2}$$

Teorema 6.7.1 (de corectitudine a Algoritmului Roy-Floyd). În contextul Algoritmului Roy-Floyd, ultima matrice calculată este chiar matricea distanțelor minime asociată grafului (G, c) , adică

$$C^{(n)} = C^*.$$

Demonstrație. Vom demonstra prin inducție după $k \in \{0, 1, \dots, n\}$ că pentru orice $i, j \in \{1, \dots, n\}$ avem

$$c_{ij}^{(k)} = \begin{cases} \min\{c(\mu) \mid \mu = (v_i, \dots, v_j), I(\mu) \subseteq \{v_1, \dots, v_k\}\}, & \text{dacă} \\ \quad \exists \mu = (v_i, \dots, v_j) \text{ drum cu } I(\mu) \subseteq \{v_1, \dots, v_k\}, \\ \infty, & \text{în caz contrar,} \end{cases} \quad (6.7.3)$$

unde $I(\mu)$ reprezintă mulțimea nodurilor intermediare ale drumului μ și $\{v_1, \dots, v_k\}$ reprezintă mulțimea $\{v_i \mid 1 \leq i \leq k\}$, deci pentru $k = 0$ această mulțime este $\emptyset$.

Pentru $k = 0$ avem $c_{ij}^{(0)} = c_{ij}$ (conform (6.7.1)) și egalitatea (6.7.3) este evidentă din definiția matricei C a costurilor directe și faptul că $I(\mu) = \emptyset$ înseamnă că drumul μ este de fapt un arc sau o muchie pentru $i \neq j$, respectiv că μ este o buclă sau drumul (v_i) de lungime 0 pentru $i = j$.

Presupunem acum egalitatea (6.7.3) adevărată pentru $k - 1$ ($1 \leq k \leq n$) și o demonstrăm pentru k . Fie $d \in [0, \infty)$. Folosind (6.7.2), ipoteza de inducție și principiul optimalității al lui Bellman avem echivalențele:

$$\begin{aligned} c_{ij}^{(k)} = d &\Leftrightarrow c_{ij}^{(k-1)} = d \leq c_{ik}^{(k-1)} + c_{kj}^{(k-1)} \text{ sau } c_{ik}^{(k-1)} + c_{kj}^{(k-1)} = d < c_{ij}^{(k-1)} \\ &\Leftrightarrow \exists \mu = (v_i, \dots, v_j) \text{ drum minim cu proprietatea că } I(\mu) \subseteq \{v_1, \dots, v_{k-1}\} \\ &\text{(adică de cost minim dintre toate drumurile de la } v_i \text{ la } v_j \text{ ce satisfac această} \\ &\text{proprietate), } c(\mu) = d \leq c(\mu_1) + c(\mu_2) \forall \mu_1 = (v_i, \dots, v_k), \mu_2 = (v_k, \dots, v_j) \\ &\text{drumuri cu } I(\mu_1), I(\mu_2) \subseteq \{v_1, \dots, v_{k-1}\} \end{aligned}$$

sau

$$\begin{aligned} &\exists \mu'_1 = (v_i, \dots, v_k), \mu'_2 = (v_k, \dots, v_j) \text{ drumuri minime cu proprietatea că} \\ &I(\mu'_1), I(\mu'_2) \subseteq \{v_1, \dots, v_{k-1}\}, c(\mu'_1) + c(\mu'_2) = d < c(\mu''), \forall \mu'' = (v_i, \dots, v_j) \\ &\text{drum cu } I(\mu'') \subseteq \{v_1, \dots, v_{k-1}\} \end{aligned}$$

$$\begin{aligned} &\Leftrightarrow \exists \mu = (v_i, \dots, v_j) \text{ drum minim cu proprietatea că} \\ &I(\mu) \subseteq \{v_1, \dots, v_k\}, v_k \notin I(\mu), c(\mu) = d \end{aligned}$$

sau

$$\begin{aligned} &\exists \mu' = (v_i, \dots, v_k, \dots, v_j) \text{ drum minim cu proprietatea că} \\ &I(\mu') \subseteq \{v_1, \dots, v_k\}, v_k \in I(\mu'), c(\mu') = d \\ &(\mu' \text{ se obține parcurgând întâi } \mu'_1, \text{ apoi } \mu'_2 \text{ și, reciproc,} \\ &\mu'_1 \text{ și } \mu'_2 \text{ sunt porțiunile din } \mu' \text{ dintre } v_i \text{ și prima apariție a lui } v_k \text{ în } I(\mu'), \\ &\text{respectiv dintre ultima apariție a lui } v_k \text{ în } I(\mu') \text{ și } v_j; \text{ între aceste apariții} \\ &\text{eventualele arce sau muchii ale lui } \mu' \text{ au costul 0 conform Observației 2.6.1)} \\ &\Leftrightarrow \exists \mu = (v_i, \dots, v_j) \text{ drum minim cu proprietatea că} \\ &I(\mu) \subseteq \{v_1, \dots, v_k\}, c(\mu) = d. \end{aligned}$$

Demonstrația prin inducție a egalității (6.7.3) este astfel încheiată.

Pentru $k = n$ condiția $I(\mu) \subseteq \{v_1, \dots, v_n\}$ poate fi eliminată, fiind întotdeauna adevărată, deci din (6.7.3) și Definiția 2.6.2 obținem $c_{ij}^{(n)} = c_{ij}^*$, $\forall i, j \in \{1, \dots, n\}$, adică egalitatea din enunț. $\square$

Observația 6.7.1. Algoritmul Roy-Floyd are complexitatea $\Theta(n^3)$ (deoarece necesită câte o adunare și o comparație pentru fiecare triplet (k, i, j) , cu $k, i, j \in \{1, \dots, n\}$).

Exemplul 6.7.1. Pentru graful din Exemplul 2.6.1, prin aplicarea Algoritmului Roy-Floyd obținem matricele:

$$C^{(0)} = C = \begin{pmatrix} 0 & 15 & 10 & 5 & \infty \\ \infty & 0 & \infty & \infty & \infty \\ \infty & 5 & 0 & \infty & 5 \\ \infty & 10 & \infty & 0 & 5 \\ 5 & 5 & \infty & \infty & 0 \end{pmatrix}; \quad C^{(1)} = \begin{pmatrix} 0 & 15 & 10 & 5 & \infty \\ \infty & 0 & \infty & \infty & \infty \\ \infty & 5 & 0 & \infty & 5 \\ \infty & 10 & \infty & 0 & 5 \\ 5 & 5 & 15 & 10 & 0 \end{pmatrix}$$

(deoarece, de exemplu, $c_{53}^{(1)} = \min\{c_{53}^{(0)}, c_{51}^{(0)} + c_{13}^{(0)}\} = \min\{\infty, 5 + 10\} = 15$);

$$C^{(2)} = \begin{pmatrix} 0 & 15 & 10 & 5 & \infty \\ \infty & 0 & \infty & \infty & \infty \\ \infty & 5 & 0 & \infty & 5 \\ \infty & 10 & \infty & 0 & 5 \\ 5 & 5 & 15 & 10 & 0 \end{pmatrix}; \quad C^{(3)} = \begin{pmatrix} 0 & 15 & 10 & 5 & 15 \\ \infty & 0 & \infty & \infty & \infty \\ \infty & 5 & 0 & \infty & 5 \\ \infty & 10 & \infty & 0 & 5 \\ 5 & 5 & 15 & 10 & 0 \end{pmatrix};$$

$$C^{(4)} = \begin{pmatrix} 0 & 15 & 10 & 5 & 10 \\ \infty & 0 & \infty & \infty & \infty \\ \infty & 5 & 0 & \infty & 5 \\ \infty & 10 & \infty & 0 & 5 \\ 5 & 5 & 15 & 10 & 0 \end{pmatrix}; \quad C^{(5)} = \begin{pmatrix} 0 & 15 & 10 & 5 & 10 \\ \infty & 0 & \infty & \infty & \infty \\ 10 & 5 & 0 & 15 & 5 \\ 10 & 10 & 20 & 0 & 5 \\ 5 & 5 & 15 & 10 & 0 \end{pmatrix} = C^*$$

(matricea distanțelor minime).

Observația 6.7.2. Conform ecuațiilor (6.7.3), Algoritmul Roy-Floyd este un algoritm specific **metodei programării dinamice**.

Conform ecuațiilor (6.7.2) avem $c_{ik}^{(k)} = c_{ik}^{(k-1)}$ și $c_{kj}^{(k)} = c_{kj}^{(k-1)}$, $\forall k, i, j \in \{1, \dots, n\}$, deci pentru implementarea algoritmului putem utiliza o singură matrice $C^{(k)}$. Astfel obținem următoarea descriere în pseudocod a algoritmului.

```

ROY_FLOYD :
for  $i = \overline{1, n}$  do                                     // inițializări
┌   for  $j = \overline{1, n}$  do
└   ┌    $c_{ij}^* \leftarrow c_{ij};$ 
for  $k = \overline{1, n}$  do
┌   for  $i = \overline{1, n}$  do
└   ┌   for  $j = \overline{1, n}$  do
└   └   ┌   if  $(c_{ik}^* + c_{kj}^* < c_{ij}^*)$  then
└   └   └   ┌    $c_{ij}^* \leftarrow c_{ik}^* + c_{kj}^*;$ 
└   └   └   └

```

Considerând că funcția cost c este strict pozitivă, pentru determinarea tuturor drumurilor minime (elementare) dintre două noduri distincte x, y putem utiliza echivalența:

$$(x, z, \dots, y) = \text{drum minim} \Leftrightarrow \begin{cases} z \neq x, \\ c_{xz} + c_{zy} = c_{xy}, \end{cases}$$

unde mulțimea nodurilor este mulțimea standard $V = \{1, \dots, n\}$.

Astfel putem determina toate nodurile z ce sunt succesori direcți ai nodului x pe drumuri minime de la x la y , și continuând acest procedeu pentru subdrumurile minime dintre z și y se găsesc toate drumurile minime elementare de la x la y .

Tema 7

Metoda Branch and Bound

- FIȘIER SEPARAT!

Tema 8

Algoritmi euristici

8.1 Introducere

Algoritmii euristici (aproximativi) sunt algoritmi ce calculează în timp polinomial soluții aproximative, apropiate de soluțiile optime, pentru probleme pentru care nu dispunem de algoritmi exacți eficienți, cum ar fi **problemele NP-complete** sau **NP-dificile**.

8.2 Problema rucsacului, varianta discretă

Prezentăm în continuare un *algoritmuristic* pentru rezolvarea problemei rucsacului, în varianta discretă.

Algoritmul 8.2.1. Sub presupunerile (2.3.1) și (2.3.2), vom utiliza următoarea *strategie Greedy*:

- Ordonăm obiectele descrescător după câștigul lor unitar:

$$\frac{c_1}{g_1} \geq \frac{c_2}{g_2} \geq \dots \geq \frac{c_n}{g_n}. \quad (8.2.1)$$

- Vom alege una din următoarele două variante de încărcare a rucsacului.
- Primă variantă constă în adăugarea obiectelor în rucsac, în ordinea de mai sus și fără a sări peste obiecte, cât timp nu se depășește greutatea maximă G .

Fie $\{O_1, O_2, \dots, O_k\}$ mulțimea obiectelor adăugate, $2 \leq k \leq n-1$, deci obiectul O_{k+1} nu mai poate fi adăugat, adică

$$g_1 + g_2 + \dots + g_k \leq G < g_1 + g_2 + \dots + g_k + g_{k+1}.$$

- O a doua variantă constă în adăugarea în rucsac a obiectelor $O_{k+1}, O_k, O_{k-1}, \dots$, cât timp nu se depășește greutatea maximă G , adică a obiectelor $O_{k+1}, O_k, \dots, O_p$, unde $2 \leq p \leq k+1$ și

$$g_{k+1} + g_k + \dots + g_p \leq G < g_{k+1} + g_k + \dots + g_p + g_{p-1}.$$

- Alegem varianta cea mai bună dintre cele două:

– Dacă

$$c_1 + c_2 + \dots + c_k > c_{k+1} + c_k + \dots + c_p,$$

atunci încărcăm în rucsac obiectele $O_1, O_2, \dots, O_k$.

– În caz contrar, adică dacă

$$c_1 + c_2 + \dots + c_k < c_{k+1} + c_k + \dots + c_p,$$

atunci încărcăm în rucsac obiectele $O_{k+1}, O_k, \dots, O_p$.

Teorema 8.2.1 (de aproximare a soluției Algoritmului 8.2.1). *Fie x soluția calculată de Algoritmul 8.2.1. Atunci*

$$f(x^*) \geq f(x) > \frac{1}{2} \cdot f(x^*),$$

unde x^* este o soluție optimă a problemei.

Demonstrație. Evident, $f(x) \leq f(x^*)$.

Conform descrierii algoritmului, avem

$$f(x) = \max\{c_1 + c_2 + \dots + c_k, c_{k+1} + c_k + \dots + c_p\},$$

de unde obținem că $f(x) \geq c_1 + c_2 + \dots + c_k$ și $f(x) \geq c_{k+1}$, deci

$$2f(x) \geq c_1 + c_2 + \dots + c_k + c_{k+1}. \quad (8.2.2)$$

Fie x' soluția optimă a problemei rucsacului în varianta continuă (produsă de algoritmul Greedy 2.3.1. Din descrierea acelui algoritm, soluția x' corespunde încărcării în rucsac a obiectelor $O_1, O_2, \dots, O_k$, în întregime, și, eventual, a unei părți din obiectul O_{k+1} , deci

$$f(x') = c_1 + c_2 + \dots + c_k + x'_{k+1}c_{k+1}, \text{ cu } 0 \leq x'_{k+1} < 1,$$

de unde obținem că

$$f(x') < c_1 + c_2 + \dots + c_k + c_{k+1}. \quad (8.2.3)$$

Evident, soluția optimă x^* a variantei discrete este o soluție (nu neapărat optimă!) și pentru varianta continuă, deci

$$f(x^*) \leq f(x'). \quad (8.2.4)$$

Din inegalitățile (8.2.2), (8.2.3) și (8.2.4) rezultă că $f(x) > \frac{1}{2} \cdot f(x') \geq \frac{1}{2} \cdot f(x^*)$. $\square$

Exemplul 8.2.1. Pentru datele din Exemplul 2.3.1, aplicarea algoritmului anterior constă la compararea dintre

- alegerea obiectelor O_7, O_3, O_4, O_{10} (următorul obiect, O_1 , nu mai încapă în rucsac), de câștig (total) $50 + 40 + 20 + 33 = 143$;
- alegerea obiectelor O_1, O_{10}, O_4, O_3 (obiectul anterior, O_7 , nu mai încapă în rucsac), de câștig (total) $27 + 33 + 20 + 40 = 120$;

Prima variantă fiind mai bună, soluția produsă de algoritm este corespunzătoare încărcării obiectelor O_7, O_3, O_4 și O_{10} .

8.3 Arbori Steiner. Algoritmul Kou-Markowsky-Berman

Definiția 8.3.1. Fie (G, c) un graf ponderat, unde $G = (V, E)$ este un graf conex și $c : E \rightarrow \mathbb{R}_+$. Fie $X \subseteq V, X \neq \emptyset$.

a) Un **arbore Steiner pentru submulțimea de noduri X în graful G** este un arbore $S = (U, F)$ cu $X \subseteq U \subseteq V$ și $F \subseteq E$ (adică un subgraf al lui G ce este arbore și acoperă toate nodurile din X). Nodurile din $U \setminus X$ se numesc **nodurile Steiner** ale arborelui S .

b) Un arbore Steiner $S^* = (U^*, F^*)$ pentru X în G cu proprietatea că

$$c(S^*) = \min\{c(S) \mid S = \text{arbore Steiner pentru } X \text{ în } G\}$$

se numește **arbore Steiner de cost minim (ASM)** pentru submulțimea de noduri X în graful ponderat (G, c) .

Observația 8.3.1. Orice arbore parțial este arbore Steiner pentru orice submulțime nevidă de noduri X , dar pot exista arbori Steiner pentru X care nu conțin toate nodurile grafului, deci nu sunt arbori parțiali.

Observația 8.3.2. Dacă $X = V$, atunci arborii Steiner pentru X sunt chiar arborii parțiali ai grafului, deci în acest caz particular arborii Steiner de cost minim pentru X sunt chiar arborii parțiali de cost minim ai grafului.

Dacă $X = \{x\}$, atunci arborii Steiner pentru X sunt subarborii grafului ce conțin nodul x , deci în acest caz particular singurul arbore Steiner de cost minim pentru X este $S^* = (\{x\}, \emptyset)$.

Dacă $X = \{x_1, x_2\}$, cu $x_1 \neq x_2$, atunci arborii Steiner pentru X sunt subarborii grafului ce conțin nodurile x_1 și x_2 , deci în acest caz particular arborii Steiner de cost minim pentru X sunt chiar drumurile (lanțurile) de cost minim dintre x_1 și x_2 .

Observația 8.3.3. Spre deosebire de problema determinării unor arbori parțiali de cost minim, problema determinării unor arbori Steiner de cost minim pentru orice submulțime arbitrară de noduri este o **problemă NP-dificilă**, pentru care nu se cunosc (până în prezent) algoritmi de rezolvare cu complexitate polinomială. Dispunem totuși de numeroși **algoritmi euristici** pentru rezolvarea acestei probleme (**algoritmi aproximativi**, ce calculează în timp polinomial arbori Steiner având costul apropiat de costul minim). Un astfel de algoritm este prezentat în continuare.

Lema 8.3.1. Fie (G, c) un graf ponderat, unde $G = (V, E)$ este un graf conex și $c : E \rightarrow \mathbb{R}_+$. Fie $X \subseteq V$, $X \neq \emptyset$. Atunci există un arbore Steiner S^* de cost minim pentru submulțimea de noduri X în graful ponderat (G, c) cu proprietatea că toate nodurile terminale ale lui S^* aparțin mulțimii X .

Mai mult, dacă funcția cost este strict pozitivă (adică $c(e) > 0$, $\forall e \in E$), atunci orice arbore Steiner de cost minim pentru X în (G, c) are această proprietate.

Demonstrație. Fie $S = (U, F)$ un arbore Steiner de cost minim pentru X în (G, c) . Fie $S^* = (U^*, F^*)$ subgraful lui S obținut din arborele S prin eliminarea repetată a tuturor nodurilor $x \in V \setminus X$ ce sunt terminale în arborele curent și a muchiilor incidente cu acestea. Evident, S^* rămâne un arbore și avem $U^* \supseteq U \setminus (V \setminus X) \supseteq X$ (deoarece $U \supseteq X$) și astfel arborele S^* este de asemenea un arbore Steiner pentru X în graful G . Evident,

$$c(S^*) = c(S) - \sum_{e \in U \setminus U^*} c(e) \leq c(S),$$

deci S^* este de asemenea un arbore Steiner de cost minim pentru X în graful ponderat (G, c) (mai mult, $c(S^*) = c(S)$ și $c(e) = 0 \forall e \in U \setminus U^*$). În plus toate nodurile terminale ale lui S^* aparțin mulțimii X .

Dacă funcția cost este strict pozitivă și ar exista un arbore Steiner $S = (U, F)$ de cost minim pentru X în (G, c) ce are noduri terminale ce nu aparțin

mulțimii X , atunci arborele S^* construit ca mai sus (prin eliminarea repetată a tuturor nodurilor $x \in V \setminus X$ ce sunt terminale în arborele curent și a muchiilor incidente cu acestea) ar rămâne un arbore Steiner pentru X în (G, c) și

$$c(S^*) = c(S) - \sum_{e \in U \setminus U^*} c(e) < c(S),$$

contradicție cu alegerea lui S drept arbore Steiner de cost minim pentru X în (G, c) . Deci, în cazul unei funcții cost strict pozitive, orice arbore Steiner de cost minim pentru X are toate nodurile terminale în X . $\square$

Observația 8.3.4. Enunțul din partea a doua a lemei anterioare poate fi reformulat astfel: dacă funcția cost este strict pozitivă, atunci eventualele noduri Steiner ale unui arbore Steiner de cost minim sunt noduri neterminale (intermediare) în acest arbore.

Algoritmul 8.3.1 (Kou-Markowsky-Berman). Fie (G, c) un graf ponderat, unde $G = (V, E)$ este un graf neorientat conex, $V = \{v_1, \dots, v_n\}$ și $c : E \rightarrow \mathbb{R}_+$. Fie $X = \{x_1, \dots, x_k\} \subseteq V$.

Conform Observației 8.3.2, problema determinării unui arbore Steiner de cost minim pentru submulțimea de noduri X în graful ponderat (G, c) este trivială pentru $k = 1$, constă în determinarea unui drum de cost minim de la nodul x_1 la nodul x_2 pentru $k = 2$ (deci poate fi rezolvată utilizând Algoritmul Dijkstra), respectiv în determinarea unui arbore parțial de cost minim pentru $k = n$ (deci poate fi rezolvată utilizând algoritmi Kruskal sau Prim). Astfel putem presupune în continuare că

$$3 \leq k \leq n - 1.$$

Algoritmul are 5 pași.

- Pasul 1. Se determină graful ponderat

$$(K_k, c^*),$$

unde K_k este graful complet având mulțimea nodurilor X iar funcția cost c^* este definită prin

$$c^*([x_i, x_j]) = \text{distanța minimă dintre nodurile } x_i \text{ și } x_j$$

în graful ponderat (G, c) , $\forall x_i, x_j \in X$, $x_i \neq x_j$. Pentru calculul distanțelor minime se utilizează Algoritmul Dijkstra (sau Algoritmul Roy-Floyd).

- Pasul 2. Se determină un arbore parțial de cost minim

$$T_0 = (X, F_0)$$

al grafului ponderat (K_k, c^*) . Pentru aceasta se utilizează Algoritmul Prim (sau Algoritmul Kruskal).

- Pasul 3. Pentru fiecare muchie $e_i = [y_i, w_i] \in F_0$, $i = \overline{1, k-1}$, se determină un drum (lanț)

$$\mu_i = \text{drum de cost minim de la } y_i \text{ la } w_i \text{ în graful ponderat } (G, c).$$

Pentru aceasta se utilizează Algoritmul Dijkstra (sau Algoritmul Roy-Floyd). Fie

$$G_1 = (V_1, E_1)$$

subgraful lui G indus de muchiile drumurilor minime $\mu_1, \dots, \mu_{k-1}$, adică

$$V_1 = V(\mu_1) \cup \dots \cup V(\mu_{k-1}), \quad E_1 = E(\mu_1) \cup \dots \cup E(\mu_{k-1}),$$

unde $V(\mu_i)$ și $E(\mu_i)$ sunt mulțimea nodurilor, respectiv mulțimea muchiilor drumului (lanțului) μ_i , $\forall i \in \{1, \dots, k-1\}$.

- Pasul 4. Se determină un arbore parțial de cost minim

$$T_1 = (V_1, F_1)$$

al grafului ponderat (G_1, c) . Pentru aceasta se utilizează Algoritmul Prim (sau Algoritmul Kruskal).

- Pasul 5. Se determină subarborele

$$S = (U, F)$$

al arborelui $T_1 = (V_1, F_1)$ obținut din T_1 prin eliminarea repetată a tuturor nodurilor $x \in V_1 \setminus X$ ce sunt terminale în arborele curent și a muchiilor incidente cu acestea.

Observația 8.3.5. Deoarece arborele $T_0 = (X, F_0)$ construit la Pasul 2 este un graf conex, rezultă că și graful $G_1 = (V_1, E_1)$ construit la Pasul 3 este conex, ceea ce asigură existența arborelui parțial de cost minim $T_1 = (V_1, F_1)$ de la Pasul 4.

Teorema 8.3.1 (de aproximare a soluției Algoritmului Kou-Markowsky-Berman). În contextul Algoritmului Kou-Markowsky-Berman, $S = (U, F)$ este un arbore Steiner pentru submulțimea de noduri X în graful G și costul său verifică inegalitățile

$$c(S^*) \leq c(S) \leq \left(2 - \frac{2}{k}\right) c(S^*),$$

unde S^* este un arbore Steiner de cost minim pentru X în graful ponderat (G, c) .

Demonstrație. Din descrierea Algoritmului Kou-Markowsky-Berman deducem că arborele $T_0 = (X, F_0)$ de la Pasul 2 conține toate nodurile din X , deci orice nod $x \in X$ este extremitate pentru cel puțin o muchie $e_i \in F_0$ și astfel $x \in V(\mu_i) \subseteq V_1$ (conform Pasului 3). Deci $X \subseteq V_1$. Rezultă că arborele $T_1 = (V_1, F_1)$ de la Pasul 4 conține toate nodurile din X , deci este un arbore Steiner pentru X în graful G . Conform descrierii Pasului 5, $S = (U, F)$ este un arbore obținut din T_1 prin eventuale eliminări ale unor noduri din $V_1 \setminus X$, deci avem în continuare $X \subseteq U$ și astfel arborele S este de asemenea un arbore Steiner pentru submulțimea de noduri X în graful G .

Inegalitatea $c(S^*) \leq c(S)$ este o consecință directă a definirii lui S^* drept arbore Steiner de cost minim pentru X în graful ponderat (G, c) . Demonstrăm în continuare a doua inegalitate din enunț.

Conform lemei anterioare, putem presupune că $S^* = (U^*, F^*)$ este un arbore Steiner de cost minim pentru X în graful ponderat (G, c) cu proprietatea că toate nodurile terminale ale lui S^* aparțin mulțimii X , adică $Y^* \subseteq X$, unde $Y^* = \{x \in U^* \mid d_{S^*}(x) = 1\}$.

Fie $S^{**} = (U^*, F^{**})$ graful obținut din S^* prin dublarea fiecărei muchii, adică

$$F^{**} = \bigcup_{e \in F^*} \{e, e'\}, \text{ unde } e \text{ și } e' \text{ sunt muchii multiple, } \forall e \in F^*$$

(adică dacă $e = [x, y]$ atunci și $e' = [x, y]$). Definim

$$c(e') = c(e), \text{ pentru orice muchii multiple } e, e' \in F^{**}.$$

S^* fiind conex rezultă că și S^{**} este conex. De asemenea, avem

$$d_{S^{**}}(x) = 2d_{S^*}(x) = \text{par}, \forall x \in U^*,$$

deci conform Teoremei de caracterizare a grafurilor euleriene neorientate rezultă că graful S^{**} este eulerian. Fie C un ciclu eulerian al lui S^{**} . Evident,

$$c(C) = c(S^{**}) = 2c(S^*). \quad (8.3.1)$$

Fie $e = [x, y] \in F^*$ o muchie arbitrar fixată. Deoarece S^* este arbore rezultă că graful $S^* \setminus e = (U^*, F^* \setminus e)$ are două componente conexe, una conținându-l pe x și cealaltă pe y . Fie U_x^* și U_y^* mulțimile nodurilor celor două componente conexe, $x \in U_x^*$, $y \in U_y^*$, $U_x^* \cup U_y^* = U^*$, $U_x^* \cap U_y^* = \emptyset$. Deducem că și graful

$$S^{**} \setminus \{e, e'\} = (U^*, F^{**} \setminus \{e, e'\})$$

are două componente conexe, induse de mulțimile de noduri U_x^* , respectiv U_y^* . Deoarece S^* este arbore rezultă că ambele componente conexe ale lui $S^* \setminus e$ sunt arbori și deci fiecare conține cel puțin câte un nod terminal al lui S^* , adică

$$U_x^* \cap Y^* \neq \emptyset, U_y^* \cap Y^* \neq \emptyset. \quad (8.3.2)$$

Pentru orice nod $y \in Y^*$ avem $d_{S^*}(y) = 1$, deci $d_{S^{**}}(y) = 2$, deci ciclul eulerian C conține o singură dată fiecare nod $y \in Y^*$. Fie

$$Y^* = \{y_1, y_2, \dots, y_p\},$$

unde nodurile $y_1, y_2, \dots, y_p$ sunt scrise în ordinea apariției lor pe ciclul eulerian C , iar $p \geq 2$ (conform (8.3.2)). Deoarece $Y^* \subseteq X = \{x_1, x_2, \dots, x_k\}$, rezultă că

$$p \leq k. \quad (8.3.3)$$

Notăm cu $L_1, L_2, \dots, L_{p-1}, L_p$ porțiunile (lanțurile) din ciclul C cuprinse respectiv între y_1 și y_2 , y_2 și y_3 , $\dots$, y_{p-1} și y_p , y_p și y_1 .

C fiind ciclu eulerian al lui S^{**} , rezultă că prin eliminarea din C a oricărei perechi $\{e, e'\}$ de muchii multiple, unde $e = [x, y] \in F^*$, se obțin două lanțuri având mulțimile de noduri U_x^* , respectiv U_y^* , fiecare conținând cel puțin câte un nod din Y^* (conform (8.3.2)). Deoarece fiecare din porțiunile

$$L_1 = [y_1, \dots, y_2], L_2 = [y_2, \dots, y_3], \dots, L_{p-1} = [y_{p-1}, \dots, y_p], L_p = [y_p, \dots, y_1]$$

nu are noduri intermediare din mulțimea Y^* , deducem că aceste lanțuri nu conțin perechi $\{e, e'\}$ de muchii multiple, deci sunt lanțuri simple în graful S^{**} (deci și în arborele S^*).

Fie $q \in \{1, \dots, p\}$ astfel încât

$$c(L_q) = \max\{c(L_i) \mid i \in \{1, \dots, p\}\}$$

și fie L lanțul obținut din ciclul C prin eliminarea lanțului L_q . Avem

$$c(C) = c(L_1) + \dots + c(L_p) \leq p \cdot c(L_q) = p(c(C) - c(L)),$$

deci

$$c(L) \leq \left(1 - \frac{1}{p}\right) c(C). \quad (8.3.4)$$

Deoarece L_q este un lanț simplu și $c(e) = c(e')$ pentru orice pereche de muchii multiple $\{e, e'\} \subseteq F^{**}$, rezultă că putem presupune că lanțul L_q conține doar muchii din mulțimea $\{e' \mid e \in F^*\}$. Astfel lanțul L va conține toate muchiile $e \in F^*$, deci și toate nodurile arborelui $S^* = (U^*, F^*)$.

Cum $X \subseteq U^*$ (S^* fiind arbore Steiner pentru X în G), rezultă că lanțul L conține toate nodurile din mulțimea $X = \{x_1, x_2, \dots, x_k\}$. Putem rearanja mulțimea X sub forma

$$X = \{x_{i_1}, x_{i_2}, \dots, x_{i_k}\},$$

unde $x_{i_1}, x_{i_2}, \dots, x_{i_k}$ sunt nodurile lui X scrise în ordinea primei apariții a lor pe lanțul L .

Notăm cu $L'_{i_1}, L'_{i_2}, \dots, L'_{i_{k-1}}$ porțiunile (lanțurile) din L cuprinse respectiv între primele apariții ale nodurilor x_{i_1} și x_{i_2} , x_{i_2} și x_{i_3} , $\dots$, $x_{i_{k-1}}$ și x_{i_k} .

Din definiția funcției cost c^* (de la Pasul 1 al algoritmului), rezultă că au loc următoarele inegalități

$$\begin{aligned} c^*([x_{i_1}, x_{i_2}]) &\leq c(L'_{i_1}), \\ c^*([x_{i_2}, x_{i_3}]) &\leq c(L'_{i_2}), \\ &\dots \\ c^*([x_{i_{k-1}}, x_{i_k}]) &\leq c(L'_{i_{k-1}}), \end{aligned}$$

deci prin adunare obținem

$$c^*(T') \leq c(L'_{i_1}) + c(L'_{i_2}) + \dots + c(L'_{i_{k-1}}) \leq c(L), \quad (8.3.5)$$

unde $T' = (X, F')$, cu $F' = \{[x_{i_1}, x_{i_2}], [x_{i_2}, x_{i_3}], \dots, [x_{i_{k-1}}, x_{i_k}]\}$. Evident, T' este un arbore parțial al grafului complet K_k având mulțimea nodurilor X , deci, conform descrierii Pașilor 1 și 2 ai algoritmului, rezultă că are loc inegalitatea

$$c^*(T') \geq c^*(T_0), \quad (8.3.6)$$

deoarece T_0 este un arbore parțial de cost minim al grafului ponderat (K_k, c^*) .

Conform descrierii Pașilor 2, 3, 4 și 5 ai algoritmului, avem

$$c^*(T_0) = \sum_{i=1}^{k-1} c^*(e_i) = \sum_{i=1}^{k-1} c(\mu_i) \geq c(G_1) \geq c(T_1) \geq c(S). \quad (8.3.7)$$

Din relațiile (8.3.5), (8.3.6) și (8.3.7) obținem

$$c(L) \geq c^*(T') \geq c^*(T_0) \geq c(G_1) \geq c(T_1) \geq c(S). \quad (8.3.8)$$

Din relațiile (8.3.8) și (8.3.4) obținem

$$c(S) \leq \left(1 - \frac{1}{p}\right) c(C). \quad (8.3.9)$$

În final, conform relațiilor (8.3.9), (8.3.1) și (8.3.3) obținem

$$c(S) \leq \left(2 - \frac{2}{p}\right) c(S^*) \leq \left(2 - \frac{2}{k}\right) c(S^*). \quad (8.3.10)$$

1

Exemplul 8.3.1. Fie graful ponderat (G, c) reprezentat în Figura 8.3.1 și submulțimea de noduri $X = \{2, 3, 6, 8, 9, 11\}$, reprezentate prin pătrățele în această figură. Descriem în continuare aplicarea Algoritmului Kou-Markowsky-Berman pentru determinarea unui arbore Steiner "minim" pentru X în graful (G, c) .

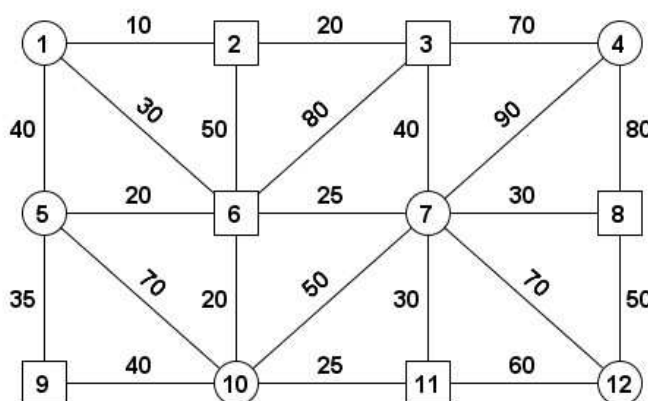


Figura 8.3.1:

La Pasul 1 se determină distanțele minime c_{ij}^* în graful ponderat (G, c) dintre oricare două noduri $i, j \in X = \{2, 3, 6, 8, 9, 11\}$, aplicând Algoritmul Dijkstra luând succesiv drept nod sursă nodurile 2, 3, 6, 8, 9. Astfel obținem graful ponderat complet (K_6, c^*) reprezentat în Figura 8.3.2.

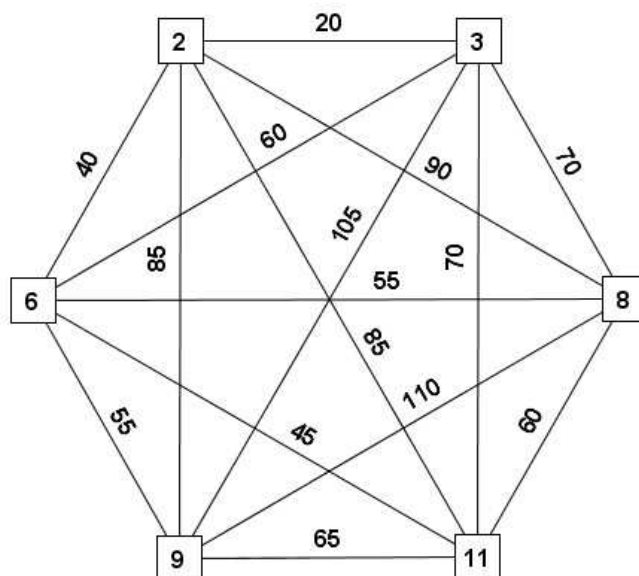


Figura 8.3.2:

La Pasul 2 se determină un arbore parțial de cost minim al grafului ponderat (K_6, c^*) . Aplicând Algoritmul Prim obținem arborele parțial de cost minim T_0 reprezentat în Figura 8.3.3.

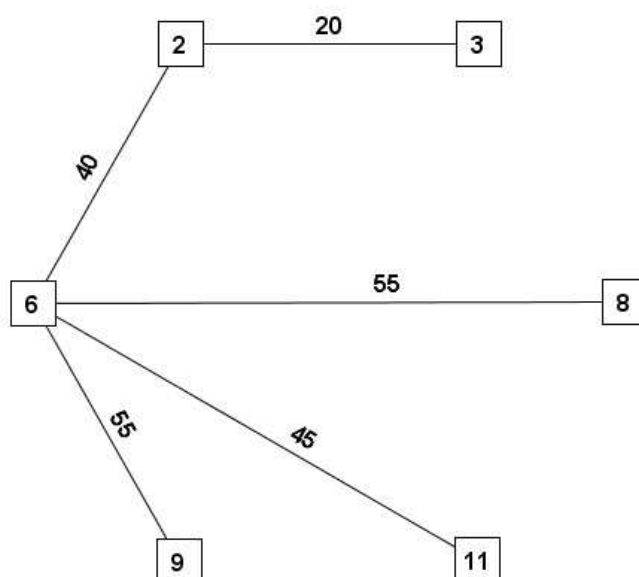


Figura 8.3.3:

La Pasul 3, pentru fiecare muchie $[i, j]$ a arborelui T_0 se determină un drum de cost minim de la i la j în graful ponderat (G, c) , aplicând Algoritmul Dijkstra. Înlocuind muchiile $[i, j]$ ale arborelui T_0 cu drumurile minime corespunzătoare, obținem subgraful lui G indus de muchiile acestor drumuri minime, și anume graful G_1 reprezentat în Figura 8.3.4.

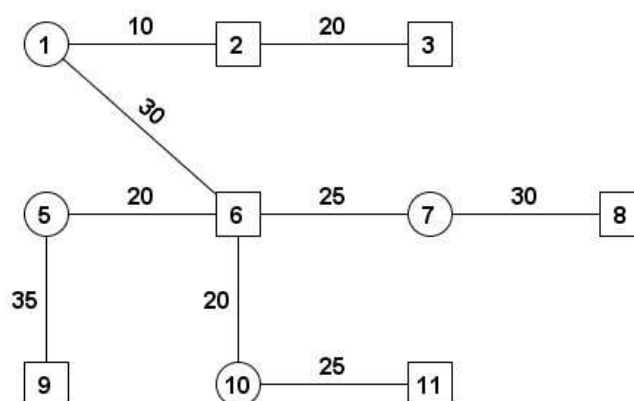


Figura 8.3.4:

La Pasul 4 se determină un arbore parțial de cost minim T_1 al subgrafului

ponderat (G_1, c) , aplicând Algoritmul Prim. Pentru exemplul de față T_1 este chiar G_1 , deoarece subgraful G_1 obținut la pasul anterior este un arbore.

La Pasul 5 se determină subarborele S al arborelui T_1 obținut din T_1 prin eliminarea repetată a tuturor eventualelor noduri terminale ce nu aparțin mulțimii $X = \{2, 3, 6, 8, 9, 11\}$. Pentru exemplul de față S este chiar T_1 , deoarece toate nodurile terminale ale arborelui T_1 obținut la pasul anterior sunt din mulțimea X .

Astfel arborele Steiner obținut prin aplicarea algoritmului este arborele S reprezentat în Figura 8.3.4. Costul acestui arborele Steiner este $c(S) = 215$.

Remarcăm că un arbore Steiner de cost minim pentru X în graful ponderat (G, c) este arborele S^* reprezentat în Figura 8.3.5, având costul $c(S^*) = 200$.

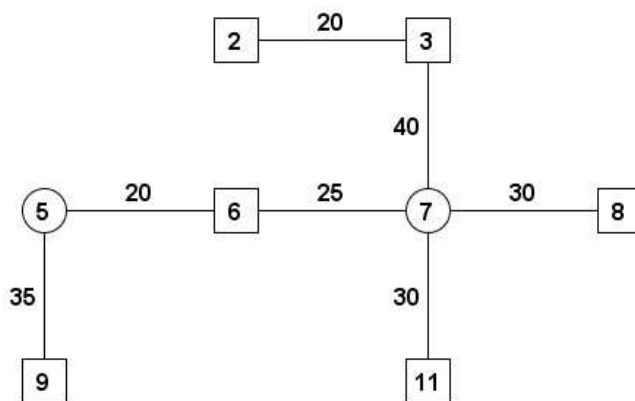


Figura 8.3.5:

Deci arborele Steiner S obținut prin aplicarea Algoritmului Kou-Markowsky-Berman nu este un arbore Steiner de cost minim.

Remarcăm de asemenea că relația (8.3.10) ne asigură că avem $c(S) \leq 1,5 \cdot c(S^*)$ (deoarece S^* are $p = 4$ noduri terminale).

Exemplul 8.3.2. Fie acum graful ponderat (G, c) reprezentat în Figura 8.3.6 și submulțimea de noduri $X = \{1, 6, 7, 12\}$ (reprezentate din nou prin pătrățele în această figură). Vom aplica din nou Algoritmul Kou-Markowsky-Berman pentru determinarea unui arbore Steiner "minim" pentru X în graful (G, c) .

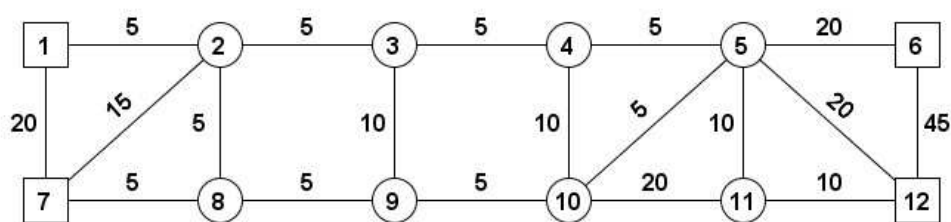


Figura 8.3.6:

La Pasul 1 se determină distanțele minime în graful ponderat (G, c) dintre oricare două noduri din X , obținând graful ponderat complet (K_4, c^*) reprezentat în Figura 8.3.7.

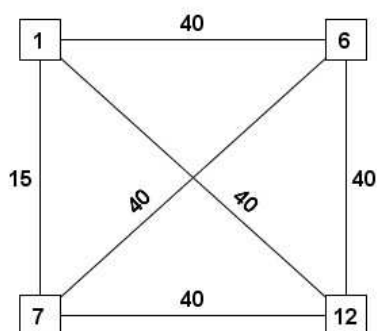


Figura 8.3.7:

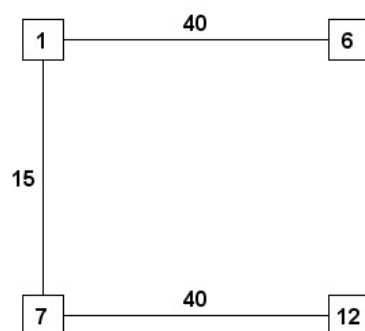


Figura 8.3.8:

La Pasul 2 se determină un arbore parțial de cost minim T_0 al grafului ponderat (K_4, c^*) . Un astfel de arbore este reprezentat în Figura 8.3.8.

La Pasul 3 se înlocuiește fiecare muchie a arborelui T_0 cu drumul minim corespunzător din graful G , obținând subgraful G_1 reprezentat în Figura 8.3.9.

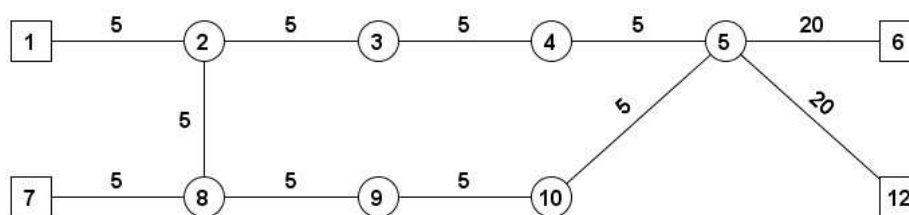


Figura 8.3.9:

La Pasul 4 se determină un arbore parțial de cost minim T_1 al subgrafului ponderat (G_1, c) . Un astfel de arbore este reprezentat în Figura 8.3.10.

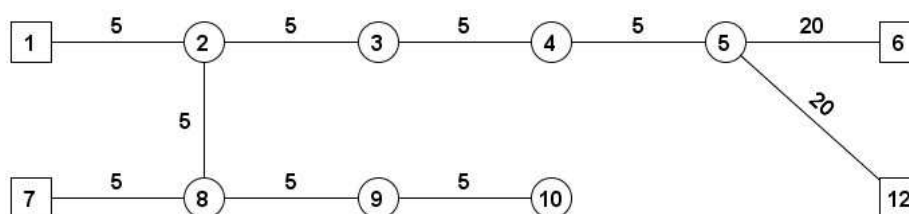


Figura 8.3.10:

La Pasul 5 se determină subarborele S al arborelui T_1 obținut prin eliminarea succesivă a nodurilor terminale 10 și 9 ce nu aparțin mulțimii X . Astfel arborele Steiner obținut prin aplicarea algoritmului este arborele S reprezentat în Figura 8.3.11. Costul acestui arborele Steiner este $c(S) = 70$.

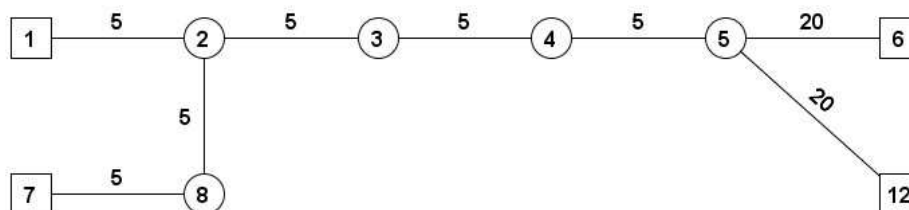


Figura 8.3.11:

Remarcăm că pentru exemplul de față arborele Steiner S obținut prin aplicarea Algoritmului Kou-Markowsky-Berman este chiar un arbore Steiner de cost minim.

Observația 8.3.6. Algoritmul Kou-Markowsky-Berman este un **algoritm euristic** bazat pe aplicarea algoritmilor Dijkstra și Prim/Kruskal, deci este specific **metodei de programare Greedy**.

Pasul 1 al Algoritmului Kou-Markowsky-Berman are complexitatea $\mathcal{O}(kn^2)$, deoarece necesită calculul distanțelor minime $c^*([x_i, x_j])$ dintre oricare două noduri din X în graful ponderat (G, c) , obiectiv realizabil prin aplicarea de $k - 1$ ori a Algoritmului Dijkstra în acest graf, luând succesiv nodurile $x_1, x_2, \dots, x_{k-1}$ drept nod sursă.

Pasul 2 al algoritmului are complexitatea $\mathcal{O}(k^2)$, deoarece necesită calculul unui arbore parțial de cost minim al grafului ponderat (K_k, c^*) , obiectiv realizabil prin aplicarea Algoritmului Prim (graful fiind complet, aplicarea Algoritmului Kruskal ar conduce la complexitatea $\mathcal{O}(k^2 \ln k)$).

Ca și Pasul 1, Pasul 3 al algoritmului are complexitatea $\mathcal{O}(kn^2)$, deoarece necesită calculul a $k - 1$ drumuri minime în graful ponderat (G, c) , obiectiv realizabil din nou prin aplicarea de $k - 1$ ori a Algoritmului Dijkstra în acest graf. De fapt aceste drumuri minime pot fi memorate chiar în momentul calculului distanțelor minime de la Pasul 1.

Pasul 4 al algoritmului are complexitatea $\mathcal{O}(n^2)$, deoarece necesită calculul unui arbore parțial de cost minim al grafului ponderat (G_1, c) având cel mult n noduri, obiectiv realizabil din nou prin aplicarea Algoritmului Prim.

Pasul 5 al algoritmului are complexitatea $\mathcal{O}(n - k)$, deoarece necesită eliminarea repetată a unor noduri terminale ce nu aparțin mulțimii X .

Rezultă că în implementarea optimă Algoritmul Kou-Markowsky-Berman are complexitatea $\mathcal{O}(kn^2)$.